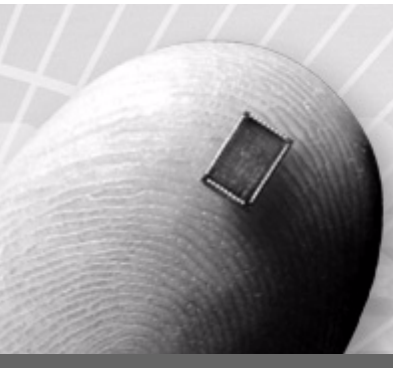
ON Semiconductor®



Firmware Reference Manual

for BELASIGNA® 300

M-20469-011

ON Semiconductor

September 2011

This document may not be redistributed or transmitted in any form.

Copyright © 2011 Semiconductor Components Industries, LLC d/b/a ON Semiconductor.

ON Semiconductor and the ON logo are registered trademarks of Semiconductor Components Industries, LLC (ON Semiconductor). BELASIGNA is a registered trademark of ON Semiconductor. All other brands, product names and company names mentioned herein may be trademarks or registered trademarks of their respective holders.

ON Semiconductor reserves the right to make changes without further notice to any products herein. ON Semiconductor makes no warranty, representation or guarantee regarding its products including suitability for any particular purpose, nor does ON Semiconductor assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in ON Semiconductor data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. ON Semiconductor does not convey any license under its patent rights nor the rights of others. ON Semiconductor products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the ON Semiconductor product could create a situation where personal injury or death may occur, unless otherwise expressly agreed to in writing by ON Semiconductor and BUYER. Should Buyer purchase or use ON Semiconductor products for any such unintended or unauthorized application, Buyer shall indemnify and hold ON Semiconductor and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that ON Semiconductor was negligent regarding the design or manufacture of the part. ON Semiconductor is an Equal Opportunity/Affirmative Action Employer. Literature is subject to all applicable copyright laws and is not for resale in any manner.

To maintain the quality of our publications, we request your comments on the accuracy, clarity, organization, and value of this document.

Address correspondence to:

Americas and Asia
ON Semiconductor
200-611 Kumpf Drive
Waterloo, Ontario, Canada N2V 1K8
Tel: +1 519 884 9696

Europe, Middle East and Africa
ON Semiconductor
Champs-Montants 12a
2074 Marin, Switzerland
Tel: +41 32 755 74 30

Website
www.onsemi.com

E-mail
dsp.support@onsemi.com

TABLE OF CONTENTS

1. Introduction 1

1.1 Purpose . . . 1
1.2 Intended Audience . . . 1
1.3 Conventions . . . 1
1.4 Manual Organization . . . 1
1.5 Further Reading . . . 2

2. Firmware Overview 3

2.1 Description . . . 3
2.2 Components . . . 3
2.2.1 Firmware Files . . . 4
2.3 SK3 Variants . . . 6
2.4 Using the Firmware . . . 7
2.4.1 ROM-Based Libraries . . . 7
2.5 Memory Map . . . 8
2.5.1 Memory Allocation . . . 8
2.5.1.1 Stack Initialization . . . 9
2.5.2 System Variables . . . 10
2.6 Firmware Naming Conventions . . . 10
2.6.1 Macro Naming . . . 10
2.6.2 Library Function Naming . . . 11
2.7 Firmware Resource Usage . . . 11
2.7.1 Register Usage . . . 11
2.7.2 Stack Usage . . . 11
2.8 Versions . . . 12
2.9 Operating Frequencies . . . 13
2.9.1 Current Operating Frequency . . . 13
2.9.2 Changing Operating Frequencies . . . 13
2.10 ADC Calibration . . . 14
2.11 Booting . . . 14
2.11.1 Boot Sequence . . . 14

2.11.2 Boot Errors 15

3. System Include Files and Macros *17*

3.1 Description 17
3.2 Hardware Definitions 17
3.2.1 Hardware Naming Conventions 17
3.2.1.1 Hardware Definitions Usage 18
3.3 Macros 18
3.3.1 Macro Naming Conventions 18
3.3.2 Macro Usage 19
3.3.2.1 Set, Get and SetReg Macros 19
3.4 System Configuration 20
3.5 Memory Management 21
3.6 Application Switching 22
3.7 Firmware Utilities 22
3.8 Peripherals and External Interfaces 24
3.9 Analog Block Configuration 27
3.10 HEAR Configuration 29

4. System Library *31*

4.1 Description 31
4.2 System Library Usage 31
4.2.1 Naming Convention 31
4.3 System Library Functions 31
4.4 System Library Macros 32
4.5 Version 32

5. Math Library *33*

5.1 Description 33
5.2 Content of the Library 33
5.3 Math Library Usage 35
5.3.1 Naming Conventions 35

5.4 Numeric Types 36
5.4.1 24-bit Fractional (frac24) 37
5.4.2 48-bit Fractional (frac48) 37
5.4.3 24-bit Mixed (m8p16) 37
5.4.4 48-bit Mixed (m24p24) 38
5.4.5 48-bit Mixed (m2p46) 38
5.4.6 24-bit Integer (int24) 38
5.4.7 24-bit Floating-Point (fl8n16) 38
5.5 Math Library Macros 38
5.5.1 Square Root 39
5.5.2 Base 2 Logarithm 40
5.5.3 Base 2 Exponent 41
5.5.4 Reciprocal 43
5.5.5 Division 45
5.5.6 Linear Interpolation 45
5.5.7 Exponential Average Filter 46
5.5.8 Attack-Release Filter 47
5.5.9 RMS to dB Conversion 48
5.5.10 dB to RMS Conversion 49
5.5.11 Power to dB Conversion 49
5.5.12 dB to Power Conversion 50
5.5.13 RMS to dB (Base 2) Conversion 51
5.5.14 dB (Base 2) to RMS Conversion 51
5.5.15 Power to dB (Base 2) Conversion 52
5.5.16 dB (Base 2) to Power Conversion 53
5.5.17 Squared Magnitude 53
5.5.18 Magnitude 54
5.5.19 Sine Function 55
5.5.20 Cosine Function 56
5.5.21 Fixed-Point to Floating-Point Conversion 56
5.5.22 Floating-Point to Fixed-Point Conversion 57
5.6 Examples 59
5.6.1 Squared Magnitude 59
5.6.2 Square Root 59
5.6.3 Cosine 60

5.6.4 Sine 61
5.6.5 Format Conversion 61
5.6.6 Logarithm 62
5.6.7 Exponential 62
5.6.8 Attack Release 62
5.6.9 Reciprocal 63
5.6.10 RMS to db10 64
5.6.11 Division 64
5.6.12 Linear Interpolation 65

6. EEPROM Library *67*

6.1 Description 67
6.2 EEPROM Library Usage 67
6.2.1 Using Dual EEPROMs 67
6.2.1.1 Hardware Configuration 68
6.2.1.2 System Variable 68
6.2.1.3 Macros and Functions 68
6.2.2 EEPROM Library Naming Conventions 68
6.3 Write Protection 69
6.4 Word Sizes 69
6.5 Cyclic Redundancy Check (CRC) 69
6.6 EEPROM Library and Dual EEPROM Library Functions . . . 70
6.7 EEPROM Library Operation 72

7. EEPROM File System *75*

7.1 Description 75
7.2 Version 75
7.3 Word Size and Addressing 75
7.4 File System Integrity 76
7.5 Security 76
7.5.1 Level 1 Security 76
7.5.2 Level 2 Security 77
7.6 File System Components 77

7.7 System Area 79
7.7.1 Boot Variables 80
7.7.2 File System Version 81
7.7.3 Application Variables 81
7.7.4 System Area Extension Vector 81
7.8 Manufacturing Area 81
7.8.1 Clock Calibration 82
7.8.2 ADC Calibration 82
7.8.3 VDDC Calibration 83
7.8.4 Manufacturing Area Extension Vector 83
7.9 Key Area 83
7.10 Bootloader 84
7.11 Applications 84
7.11.1 Application Files 84
7.11.2 Application Execution 85
7.11.3 Sections 86
7.11.4 Volatile Sections 86
7.11.5 Encrypted Sections 86

8. Program ROM *89*

8.1 Description 89
8.1.1 Versions 89
8.2 Boot ROM 89
8.2.1 Boot Initialization 89
8.2.2 Stabilization Delay 90
8.2.3 Loading the Bootloader 90
8.2.4 Boot Failures 90
8.3 Function Tables 90
8.4 File System Decryption Support 93
8.4.1 Advanced Encryption Standard (AES) Functions 93
8.4.1.1 Memory, Register and Other Resource Usage by AES Routines . 94
8.4.2 Encryption Blocks 95

9. Bootloader 97

9.1 Description 97
9.2 Installation 97
9.3 Bootloader Version 97
9.4 Memory Usage 97
9.5 Operation 97
9.6 Bootloader Errors 98

10. System Macro Reference 99

10.1 Sys_BootApp 99
10.2 Sys_Calc_Gain_Factor_Val 100
10.3 Sys_Calc_Sample_Freq_Val 101
10.4 Sys_CFX_BiasDisable 102
10.5 Sys_CFX_BiasEnable 103
10.6 Sys_CFX_CircAddrConfig 104
10.7 Sys_CFX_FlagUpdateDisable 105
10.8 Sys_CFX_FlagUpdateEnable 106
10.9 Sys_CFX_InitLoopStack 107
10.10 Sys_CFX_InitStack 108
10.11 Sys_CFX_IntDisable 109
10.12 Sys_CFX_IntEnable 110
10.13 Sys_CFX_RoundDisable 111
10.14 Sys_CFX_RoundEnable 112
10.15 Sys_CFX_SaturateDisable 113
10.16 Sys_CFX_SaturateEnable 114
10.17 Sys_CFX_SRConfig 115
10.18 Sys_CFX_StepConfig 116
10.19 Sys_Clk_Config 117
10.20 Sys_Clk_Config0 118
10.21 Sys_Clk_Config1 119
10.22 Sys_Data_fl8n16 120

10.23 Sys_Data_frac24 121
10.24 Sys_Data_frac32 122
10.25 Sys_Data_frac48 123
10.26 Sys_Data_m24p24 124
10.27 Sys_Data_m2p46 125
10.28 Sys_Data_m8p16 126
10.29 Sys_DebugPort_Get_AudioMode 127
10.30 Sys_DebugPort_Set_AccessMode 128
10.31 Sys_DebugPort_Set_Address 129
10.32 Sys_FIFO_Configure 130
10.33 Sys_fl8n16 131
10.34 Sys_frac24 132
10.35 Sys_frac32 133
10.36 Sys_frac48 134
10.37 Sys_Get_ADC_CFG 135
10.38 Sys_Get_ADC_CUR_CTRL 136
10.39 Sys_Get_ADC_GFxy_CTRL 137
10.40 Sys_Get_ANALOG_PWR_CTRL 138
10.41 Sys_Get_AppNum 139
10.42 Sys_Get_BAND_SELECT_CTRL 140
10.43 Sys_Get_ChipDebugVersion 141
10.44 Sys_Get_ChipFamily 142
10.45 Sys_Get_ChipRevision 143
10.46 Sys_Get_ChipVersion 144
10.47 Sys_Get_CLK_CTRL 145
10.48 Sys_Get_FRAC_DELAY_x_CTRL 146
10.49 Sys_Get_HEAR_BFP_BLOCK_EXP 147
10.50 Sys_Get_IN_GAIN_CTRL 148
10.51 Sys_Get_INPUT_CTRL 149
10.52 Sys_Get_LSAD_CTRL 150
10.53 Sys_Get_O_DAC_CTRL 151
10.54 Sys_Get_O_DAC_GF_CTRL 152
10.55 Sys_Get_OUTPUT_CTRL 153

10.56 Sys_Get_PSU_CFG 154
10.57 Sys_Get_SUP_COUNT_DATA 155
10.58 Sys_Get_SUP_THRSH_DATA 156
10.59 Sys_GPIO_Configure 157
10.60 Sys_GPIO_Direction 158
10.61 Sys_GPIO_Get_All_Data 159
10.62 Sys_GPIO_Get_Data 160
10.63 Sys_GPIO_IntConfig 161
10.64 Sys_GPIO_MUX_Select 162
10.65 Sys_GPIO_Set_Data 163
10.66 Sys_GPIO_Set_High 164
10.67 Sys_GPIO_Set_Low 165
10.68 Sys_GPIO_Toggle 166
10.69 Sys_GPIO_Toggle_Bits 167
10.70 Sys_HEAR_Command 168
10.71 Sys_HEAR_FunctionChain_Enable 169
10.72 Sys_HEAR_FunctionChain_Priority 170
10.73 Sys_HEAR_Pause 171
10.74 Sys_HEAR_Reset 172
10.75 Sys_HEAR_Start 173
10.76 Sys_I2C_ACK 174
10.77 Sys_I2C_Config 175
10.78 Sys_I2C_NACK 176
10.79 Sys_I2C_NACK_And_Stop 177
10.80 Sys_I2C_Pin_Config 178
10.81 Sys_I2C_Reset 179
10.82 Sys_I2C_Slave_Disable 180
10.83 Sys_I2C_Slave_Enable 181
10.84 Sys_I2C_Start_Read 182
10.85 Sys_I2C_Start_Write 183
10.86 Sys_I2C_Watchdog_Config 184
10.87 Sys_IMM_n 185
10.88 Sys_Int_Ack 186

10.89 Sys_Int_Enable 187
10.90 Sys_Int_Master_Disable 188
10.91 Sys_Int_Master_Enable 189
10.92 Sys_Int_Priority 190
10.93 Sys_IOC_FIFOConfig 191
10.94 Sys_IOC_InputConfig 192
10.95 Sys_IOC_OutputConfig 193
10.96 Sys_Log2 194
10.97 Sys_Log2_Ceil 195
10.98 Sys_Log2_Floor 196
10.99 Sys_m24p24 197
10.100 Sys_m2p46 198
10.101 Sys_m8p16 199
10.102 Sys_Mem_CopyPP 200
10.103 Sys_Mem_CopyPX 201
10.104 Sys_Mem_CopyPY 202
10.105 Sys_Mem_CopyXP 203
10.106 Sys_Mem_CopyXX 204
10.107 Sys_Mem_CopyXY 205
10.108 Sys_Mem_CopyXY_Quick 206
10.109 Sys_Mem_CopyYP 207
10.110 Sys_Mem_CopyYX 208
10.111 Sys_Mem_CopyYX_Quick 209
10.112 Sys_Mem_CopyYY 210
10.113 Sys_Mem_FillP 211
10.114 Sys_Mem_FillX 212
10.115 Sys_Mem_FillY 213
10.116 Sys_Mem_Priority 214
10.117 Sys_Mem_Reserve 215
10.118 Sys_Mem_Reserve_Init 216
10.119 Sys_Mem_Reserve_Init_Default 217
10.120 Sys_Mem_Start_Absolute_Segment 218
10.121 Sys_Mem_Start_Reloc_Segment 219

10.122 Sys_Mem_Start_Segment 220
10.123 Sys_Output_Disable 221
10.124 Sys_Output_Enable 222
10.125 Sys_Output_Mute 223
10.126 Sys_Output_Unmute 224
10.127 Sys_PCM_Config 225
10.128 Sys_PCM_Disable 226
10.129 Sys_PCM_Enable 227
10.130 Sys_Pop_Reg_1 228
10.131 Sys_Pop_Reg_10 229
10.132 Sys_Pop_Reg_2 230
10.133 Sys_Pop_Reg_3 231
10.134 Sys_Pop_Reg_4 232
10.135 Sys_Pop_Reg_5 233
10.136 Sys_Pop_Reg_6 234
10.137 Sys_Pop_Reg_7 235
10.138 Sys_Pop_Reg_8 236
10.139 Sys_Pop_Reg_9 237
10.140 Sys_Pop_Reg_Long_1 238
10.141 Sys_Pop_Reg_Long_10 239
10.142 Sys_Pop_Reg_Long_2 240
10.143 Sys_Pop_Reg_Long_3 241
10.144 Sys_Pop_Reg_Long_4 242
10.145 Sys_Pop_Reg_Long_5 243
10.146 Sys_Pop_Reg_Long_6 244
10.147 Sys_Pop_Reg_Long_7 245
10.148 Sys_Pop_Reg_Long_8 246
10.149 Sys_Pop_Reg_Long_9 247
10.150 Sys_Power2 248
10.151 Sys_Push_Reg_1 249
10.152 Sys_Push_Reg_10 250
10.153 Sys_Push_Reg_2 251
10.154 Sys_Push_Reg_3 252

10.155 Sys_Push_Reg_4 253
10.156 Sys_Push_Reg_5 254
10.157 Sys_Push_Reg_6 255
10.158 Sys_Push_Reg_7 256
10.159 Sys_Push_Reg_8 257
10.160 Sys_Push_Reg_9 258
10.161 Sys_Push_Reg_Long_1 259
10.162 Sys_Push_Reg_Long_10 260
10.163 Sys_Push_Reg_Long_2 261
10.164 Sys_Push_Reg_Long_3 262
10.165 Sys_Push_Reg_Long_4 263
10.166 Sys_Push_Reg_Long_5 264
10.167 Sys_Push_Reg_Long_6 265
10.168 Sys_Push_Reg_Long_7 266
10.169 Sys_Push_Reg_Long_8 267
10.170 Sys_Push_Reg_Long_9 268
10.171 Sys_Reg_fl8n16 269
10.172 Sys_Reg_frac24 270
10.173 Sys_Reg_frac32 271
10.174 Sys_Reg_frac48 272
10.175 Sys_Reg_m24p24 273
10.176 Sys_Reg_m2p46 274
10.177 Sys_Reg_m8p16 275
10.178 Sys_Restore_HEAR_BFP 276
10.179 Sys_Save_HEAR_BFP 277
10.180 Sys_Set_ADC_CFG 278
10.181 Sys_Set_ADC_CUR_CTRL 279
10.182 Sys_Set_ADC_GFxy_CTRL 280
10.183 Sys_Set_ANALOG_PWR_CTRL 281
10.184 Sys_Set_BAND_SELECT_CTRL 282
10.185 Sys_Set_CLK_CTRL 283
10.186 Sys_Set_Decimation_Gainxy 284
10.187 Sys_Set_FRAC_DELAY_x_CTRL 285

10.188 Sys_Set_HEAR_BFP 286
10.189 Sys_Set_HEAR_FunctionChain_Vector 287
10.190 Sys_Set_IN_GAIN_CTRL 288
10.191 Sys_Set_IN_GAIN_CTRL_NoCalibration 289
10.192 Sys_Set_INPUT_CTRL 290
10.193 Sys_Set_LSAD_CTRL 291
10.194 Sys_Set_O_DAC_CTRL 292
10.195 Sys_Set_O_DAC_GF_CTRL 293
10.196 Sys_Set_OUTPUT_CTRL 294
10.197 Sys_Set_PSU_CFG 295
10.198 Sys_Set_SUP_THRSH_DATA 296
10.199 Sys_SetReg_ADC_CUR_CTRL 297
10.200 Sys_SetReg_ADC_GFxy_CTRL 298
10.201 Sys_SetReg_ANALOG_PWR_CTRL 299
10.202 Sys_SetReg_BAND_SELECT_CTRL 300
10.203 Sys_SetReg_CLK_CTRL 301
10.204 Sys_SetReg_FRAC_DELAY_x_CTRL 302
10.205 Sys_SetReg_HEAR_BFP_BLOCK_EXP 303
10.206 Sys_SetReg_IN_GAIN_CTRL 304
10.207 Sys_SetReg_IN_GAIN_CTRL_NoCalibration 305
10.208 Sys_SetReg_INPUT_CTRL 306
10.209 Sys_SetReg_LSAD_CTRL 307
10.210 Sys_SetReg_O_DAC_CTRL 308
10.211 Sys_SetReg_O_DAC_GF_CTRL 309
10.212 Sys_SetReg_OUTPUT_CTRL 310
10.213 Sys_SetReg_PSU_CFG 311
10.214 Sys_SetReg_SUP_THRSH_DATA 312
10.215 Sys_SPI_CS_High 313
10.216 Sys_SPI_CS_Low 314
10.217 Sys_SPI_Disable 315
10.218 Sys_SPI_Enable 316
10.219 Sys_SPI_Read 317
10.220 Sys_SPI_WaitDone 318

10.221 Sys_SPI_Write 319
10.222 Sys_Stack_Alloc 320
10.223 Sys_Stack_Free 321
10.224 Sys_Timer1_Config 322
10.225 Sys_Timer1_Start 323
10.226 Sys_Timer1_Stop 324
10.227 Sys_Timer2_Config 325
10.228 Sys_Timer2_Start 326
10.229 Sys_Timer2_Stop 327
10.230 Sys_Timers_Start 328
10.231 Sys_Timers_Stop 329
10.232 Sys_UART_Disable 330
10.233 Sys_UART_Enable 331
10.234 Sys_Value_XY 332
10.235 Sys_Watchdog_Refresh 333
10.236 Sys_Watchdog_TimeoutConfig 334

11. System Library Reference 335

11.1 Sys_CalibrateADCs 335
11.2 Sys_Delay 336
11.3 Sys_DelayReg 337
11.4 Sys_Reset 338
11.5 Sys_SetOperatingFrequency 339
11.6 Sys_SoftPowerdownDisable 340
11.7 Sys_SoftPowerdownEnable 341
11.8 Sys_SoftPowerdownEnable_NoAnalog 342
11.9 Sys_WaitForSPIDone 343
11.10 SYSLIB_CalibrateADCs 344
11.11 SYSLIB_Delay 345
11.12 SYSLIB_Reset 346
11.13 SYSLIB_SetOperatingFrequency 347
11.14 SYSLIB_SoftPowerdownDisable 348

11.15 SYSLIB_SoftPowerdownEnable 349
11.16 SYSLIB_SoftPowerdownEnable_NoAnalog 350
11.17 SYSLIB_WaitForSPIDone 351

12. Math Library Macro Reference *353*

12.1 MATHLIB_ATTACK_RELEASE 353
12.2 MATHLIB_ATTACK_RELEASE_VEC 354
12.3 MATHLIB_CONV_fl8n16_frac48 355
12.4 MATHLIB_CONV_fl8n16_m24p24 356
12.5 MATHLIB_CONV_fl8n16_m8p16 357
12.6 MATHLIB_CONV_frac24_fl8n16 358
12.7 MATHLIB_CONV_frac48_fl8n16 359
12.8 MATHLIB_CONV_int24_fl8n16 360
12.9 MATHLIB_CONV_m24p24_fl8n16 361
12.10 MATHLIB_CONV_m8p16_fl8n16 362
12.11 MATHLIB_CONV_VEC_fl8n16_frac48 363
12.12 MATHLIB_CONV_VEC_fl8n16_m24p24 364
12.13 MATHLIB_CONV_VEC_fl8n16_m8p16 365
12.14 MATHLIB_CONV_VEC_frac24_fl8n16 366
12.15 MATHLIB_CONV_VEC_frac48_fl8n16 367
12.16 MATHLIB_CONV_VEC_int24_fl8n16 368
12.17 MATHLIB_CONV_VEC_m24p24_fl8n16 369
12.18 MATHLIB_CONV_VEC_m8p16_fl8n16 370
12.19 MATHLIB_COS_frac24_frac24 371
12.20 MATHLIB_COS_VEC_frac24_frac24 372
12.21 MATHLIB_DB10_PWR_m8p16_frac48 373
12.22 MATHLIB_DB10_PWR_VEC_m8p16_frac48 374
12.23 MATHLIB_DB10_RMS_m8p16_frac48 375
12.24 MATHLIB_DB10_RMS_VEC_m8p16_frac48 376
12.25 MATHLIB_DB2_PWR_m8p16_frac48 377
12.26 MATHLIB_DB2_PWR_VEC_m8p16_frac48 378
12.27 MATHLIB_DB2_RMS_m8p16_frac48 379

12.28 MATHLIB_DB2_RMS_VEC_m8p16_frac48 380
12.29 MATHLIB_DIVIDE_frac24 381
12.30 MATHLIB_DIVIDE_int24 382
12.31 MATHLIB_DIVIDE_m24p24 383
12.32 MATHLIB_EXP2_m8p16_frac48 384
12.33 MATHLIB_EXP2_m8p16_m8p16 385
12.34 MATHLIB_EXP2_VEC_m8p16_frac48 386
12.35 MATHLIB_EXP2_VEC_m8p16_m8p16 387
12.36 MATHLIB_EXP_AVG 388
12.37 MATHLIB_EXP_AVG_VEC 389
12.38 MATHLIB_LIN_INTERP 390
12.39 MATHLIB_LOG2_fl8n16_m8p16 391
12.40 MATHLIB_LOG2_frac24_m8p16 392
12.41 MATHLIB_LOG2_frac48_m8p16 393
12.42 MATHLIB_LOG2_m8p16_m8p16 394
12.43 MATHLIB_LOG2_VEC_fl8n16_m8p16 395
12.44 MATHLIB_LOG2_VEC_frac24_m8p16 396
12.45 MATHLIB_LOG2_VEC_frac48_m8p16 397
12.46 MATHLIB_LOG2_VEC_m8p16_m8p16 398
12.47 MATHLIB_MAG_frac24_m2p46 399
12.48 MATHLIB_MAG_VEC_frac24_m2p46 400
12.49 MATHLIB_PWR_DB10_frac48_m8p16 401
12.50 MATHLIB_PWR_DB10_VEC_frac48_m8p16 402
12.51 MATHLIB_PWR_DB2_frac48_m8p16 403
12.52 MATHLIB_PWR_DB2_VEC_frac48_m8p16 404
12.53 MATHLIB_RECIP_fl8n16_fl8n16 405
12.54 MATHLIB_RECIP_frac48_fl8n16 406
12.55 MATHLIB_RECIP_int24_fl8n16 407
12.56 MATHLIB_RECIP_m8p16_fl8n16 408
12.57 MATHLIB_RECIP_POS_fl8n16_fl8n16 409
12.58 MATHLIB_RECIP_POS_frac48_fl8n16 410
12.59 MATHLIB_RECIP_POS_int24_fl8n16 411
12.60 MATHLIB_RECIP_POS_m8p16_fl8n16 412

12.61 MATHLIB_RECIP_POS_VEC_fl8n16_fl8n16 413
12.62 MATHLIB_RECIP_POS_VEC_frac48_fl8n16 414
12.63 MATHLIB_RECIP_POS_VEC_int24_fl8n16 415
12.64 MATHLIB_RECIP_POS_VEC_m8p16_fl8n16 416
12.65 MATHLIB_RECIP_VEC_fl8n16_fl8n16 417
12.66 MATHLIB_RECIP_VEC_frac48_fl8n16 418
12.67 MATHLIB_RECIP_VEC_int24_fl8n16 419
12.68 MATHLIB_RECIP_VEC_m8p16_fl8n16 420
12.69 MATHLIB_RMS_DB10_frac48_m8p16 421
12.70 MATHLIB_RMS_DB10_VEC_frac48_m8p16 422
12.71 MATHLIB_RMS_DB2_frac48_m8p16 423
12.72 MATHLIB_RMS_DB2_VEC_frac48_m8p16 424
12.73 MATHLIB_SIN_frac24_frac24 425
12.74 MATHLIB_SIN_VEC_frac24_frac24 426
12.75 MATHLIB_SQMAG_frac24_m2p46 427
12.76 MATHLIB_SQMAG_VEC_frac24_m2p46 428
12.77 MATHLIB_SQRT_frac24_frac48 429
12.78 MATHLIB_SQRT_frac48_frac48 430
12.79 MATHLIB_SQRT_VEC_frac24_frac48 431
12.80 MATHLIB_SQRT_VEC_frac48_frac48 432

13. Math Library Function Reference 433

13.1 _MATHLIB_DIVIDE 433
13.2 _MATHLIB_EXP2_m8p16_frac48 434
13.3 _MATHLIB_LOG2_frac48_m8p16 435
13.4 _MATHLIB_RECIP_GEN_frac48_fl8n16 436
13.5 _MATHLIB_RECIP_POS_frac48_fl8n16 437
13.6 _MATHLIB_SQRT_frac48_frac48 438

14. EEPROM Library Reference 439

14.1 EELIB_Close 439
14.2 EELIB_Commit 440

14.3 EELIB_Initialize 441
14.4 EELIB_InitWriteWord 442
14.5 EELIB_LibWriteDisable 443
14.6 EELIB_LibWriteEnable 444
14.7 EELIB_PowerDown 445
14.8 EELIB_PowerUp 446
14.9 EELIB_ReadBlock_P 447
14.10 EELIB_ReadBlock_P16 448
14.11 EELIB_ReadBlock_X 449
14.12 EELIB_ReadBlock_X16 450
14.13 EELIB_ReadBlock_Y 451
14.14 EELIB_ReadNextBlock_P 452
14.15 EELIB_ReadNextBlock_P16 453
14.16 EELIB_ReadNextBlock_X 454
14.17 EELIB_ReadNextBlock_X16 455
14.18 EELIB_ReadNextBlock_Y 456
14.19 EELIB_ReadNextWord16 457
14.20 EELIB_ReadNextWord24 458
14.21 EELIB_ReadNextWord32 459
14.22 EELIB_ReadNextWord8 460
14.23 EELIB_ReadStatusRegister 461
14.24 EELIB_ReadWord16 462
14.25 EELIB_ReadWord24 463
14.26 EELIB_ReadWord32 464
14.27 EELIB_ReadWord8 465
14.28 EELIB_SelectSegment 466
14.29 EELIB_VerifyEEPROMConnection 467
14.30 EELIB_WriteBlock_Const32 468
14.31 EELIB_WriteBlock_P 469
14.32 EELIB_WriteBlock_X 470
14.33 EELIB_WriteBlock_Y 471
14.34 EELIB_WriteDisable 472
14.35 EELIB_WriteEnable 473

14.36 EELIB_WriteNextWord16 474
14.37 EELIB_WriteNextWord24 475
14.38 EELIB_WriteNextWord32 476
14.39 EELIB_WriteNextWord8 477
14.40 EELIB_WriteStatusRegister 478
14.41 EELIB_WriteWord16 479
14.42 EELIB_WriteWord24 480
14.43 EELIB_WriteWord32 481
14.44 EELIB_WriteWord8 482

15. Dual EEPROM Library Reference 483

15.1 EELIB_Close 483
15.2 EELIB_Commit 484
15.3 EELIB_Initialize 485
15.4 EELIB_InitWriteWord 486
15.5 EELIB_LibWriteDisable 487
15.6 EELIB_LibWriteEnable 488
15.7 EELIB_PowerDown 489
15.8 EELIB_PowerUp 490
15.9 EELIB_ReadBlock_P 491
15.10 EELIB_ReadBlock_P16 492
15.11 EELIB_ReadBlock_X 493
15.12 EELIB_ReadBlock_X16 494
15.13 EELIB_ReadBlock_Y 495
15.14 EELIB_ReadNextBlock_P 496
15.15 EELIB_ReadNextBlock_P16 497
15.16 EELIB_ReadNextBlock_X 498
15.17 EELIB_ReadNextBlock_X16 499
15.18 EELIB_ReadNextBlock_Y 500
15.19 EELIB_ReadNextWord16 501
15.20 EELIB_ReadNextWord24 502
15.21 EELIB_ReadNextWord32 503

15.22 EELIB_ReadNextWord8 504
15.23 EELIB_ReadStatusRegister 505
15.24 EELIB_ReadWord16 506
15.25 EELIB_ReadWord24 507
15.26 EELIB_ReadWord32 508
15.27 EELIB_ReadWord8 509
15.28 EELIB_SelectSegment 510
15.29 EELIB_VerifyEEPROMConnection 511
15.30 EELIB_WriteBlock_Const32 512
15.31 EELIB_WriteBlock_P 513
15.32 EELIB_WriteBlock_X 514
15.33 EELIB_WriteBlock_Y 515
15.34 EELIB_WriteDisable 516
15.35 EELIB_WriteEnable 517
15.36 EELIB_WriteNextWord16 518
15.37 EELIB_WriteNextWord24 519
15.38 EELIB_WriteNextWord32 520
15.39 EELIB_WriteNextWord8 521
15.40 EELIB_WriteStatusRegister 522
15.41 EELIB_WriteWord16 523
15.42 EELIB_WriteWord24 524
15.43 EELIB_WriteWord32 525
15.44 EELIB_WriteWord8 526
15.45 Sys_Dual_EEPROM_CS_Data_EEPROM 527
15.46 Sys_Dual_EEPROM_CSs_High 528
15.47 Sys_Set_Dual_EEPROM_SPI_CTRL 529
15.48 Sys_SetReg_Dual_EEPROM_Sel_EEPROM 530
15.49 Sys_SetReg_Dual_EEPROM_SPI_CTRL 531

1 Introduction

1.1 Purpose

This manual describes the BELASIGNA® 300 firmware. The firmware provides developers with a convenient software layer on which to build their applications. It is also responsible for system-level tasks such as booting the system and implementing portions of the security layer. It consists of include files, macros, libraries, ROM code and executables. This manual includes descriptions, function listings and usage examples to help you understand the firmware and its parts.

1.2 Intended Audience

This manual is intended to be used by developers who are designing and implementing applications for BelaSigna 300. Both novice and experienced developers can benefit from the use of this manual. The descriptions and code examples will help novice users to learn the system, while experienced developers may go straight to the reference chapters that describe the available functions, macros and libraries.

The manual assumes the reader has a basic understanding of CFX assembly language and the development tools as well as an understanding of the BelaSigna 300 architecture.

1.3 Conventions

Assembly code, macros, functions, defines and addresses are shown in a `monospace font`.

Optional parameters and placeholders for specific information are shown in **`<angle brackets>`**. To use an optional parameter or replace a placeholder, specify the information within the brackets; do not include the brackets themselves.

1.4 Manual Organization

The *Firmware Reference Manual for BelaSigna 300* contains the following chapters:

- **Chapter 1: Introduction** describes the purpose of this manual, the intended audience, the typographical conventions used, and gives an overview of the manual.
- **Chapter 2: Firmware Overview** provides a top level look at the firmware described in this manual, including a memory map showing what memory is reserved by the firmware.
- **Chapter 3: System Include Files and Macros** describes the hardware definitions and macros that are contained in the system include files.
- **Chapter 4: System Library** describes the functions that applications can use to control and configure the system hardware.
- **Chapter 5: Math Library** describes the math macros and functions that applications can use to perform common mathematical operations.
- **Chapter 6: EEPROM Library** describes the functions that provide access to an EEPROM connected to the BelaSigna 300 device on the SPI port.

- **Chapter 7: EEPROM File System** describes the EEPROM file system which is designed for storing applications and data.
- **Chapter 8: Program ROM** describes the functions of the Program ROM, including the boot sequence and JumpROM functionality.
- **Chapter 9: Bootloader** describes the operation of the bootloader.
- **Chapter 10: System Macro Reference** provides a description of the system macros.
- **Chapter 11: System Library Reference** provides a description of the functions and macros in the system library.
- **Chapter 12: Math Library Macro Reference** provides a description of the macros in the math library.
- **Chapter 13: Math Library Function Reference** provides a description of the functions in the math library.
- **Chapter 14: EEPROM Library Reference** provides a description of the functions in the EEPROM library.
- **Chapter 15: Dual EEPROM Library Reference** provides a description of the functions in the dual EEPROM library.

1.5 FURTHER READING

For more information, refer to the following documents:

- *Hardware Reference Manual for BelaSigna 300*
- *HEAR Configurable Accelerator Reference Manual*
- *CFX DSP Architecture Manual*
- *CFX Toolchain Reference for BelaSigna 300*
- *CFX Communication Protocols Manual*

2 Firmware Overview

2.1 DESCRIPTION

The firmware provides a layer of support between the application developer and the underlying hardware. It is a collection of files, macros, functions and libraries designed to make application development simpler, quicker and more reliable.

The firmware serves many functions, including performing system-level tasks (e.g., booting the system), implementing part of the security layer, storing and retrieving EEPROM data, and improving application readability.

By providing a layer of support between the hardware and the developer, the system firmware allows the developer to focus on application development and avoid having to unnecessarily learn details about the underlying BelaSigna 300 hardware. The firmware provides a clear, easy to understand interface to common operations—such as controlling and configuring the underlying hardware—while avoiding the use of absolute addresses and limiting the need for traditional assembly code. The functions and macros provided by the firmware have been extensively tested. One particularly important component the system firmware provides to developers is an optimized math library. All of these help developers to quickly produce error-free code.

When multiple programmers are involved in development, using the firmware leads to increased consistency, which in turn leads to increased overall robustness and correctness of code. Development teams also need not worry about memory conflicts as there is no need to define specific memory maps.

In some cases, depending on the particulars of the application, the firmware implementation may not be optimal; however, even in these situations, the firmware will serve as an example and advanced starting point for custom-developed functions and macros.

The firmware layer also encapsulates the details of the hardware such that many changes due to hardware revisions will be transparent at the application level. The encapsulation also provides a basic level of error checking of the hardware usage.

This chapter explores the system design details of the firmware, including components, memory maps, global variables, versions, operating frequencies, and the boot sequence.

2.2 COMPONENTS

The firmware consists of the following components:

Hardware Definitions Applies a layer of names and labels to the underlying hardware for ease of use.

System Macros A set of macros which are inserted inline in the application to configure and control the underlying hardware.

System Library A set of macros and callable functions that configure and control the underlying hardware.

EEPROM Library A set of callable functions that perform operations on an attached EEPROM, including initialize, read and write.

Dual EEPROM Library A set of callable functions that perform operations—such as initialize, read and write—on two attached EEPROMs. The active EEPROM is selected by using the SPI chip select signal and one GPIO signal.

Math Library A set of macros and callable functions that perform useful mathematical operations.

EEPROM File System A structure designed to store system data, application code and data tables in EEPROM.

Bootloader A system program that loads a full application (code and data) from EEPROM to RAM and executes it.

Program ROM Firmware code stored in ROM that controls system start-up, loads the bootloader to RAM, implements security functions, and reduces RAM usage by mirroring the implementation of some library code.

The programs and libraries included with the EDK are listed in Table 1.

TABLE 1: INSTALLED LOCATIONS OF PROGRAMS AND LIBRARIES

Programs and Libraries	Location
The System Library	• The files required to link the system library into projects can be found under the **<EDK_HOME>**\include\sk3 and **<EDK_HOME>**\lib\sk3 folders. • The system library source code is in **<EDK_HOME>**\source\sk3\syslib, and the compiled library is in the master subfolder.
The EEPROM Libraries	• The files required to link the EEPROM library or the dual EEPROM library into projects can be found under the **<EDK_HOME>**\include\sk3 and **<EDK_HOME>**\lib\sk3 folders. • The EEPROM library source code is in **<EDK_HOME>**\source\sk3\eeprom, and the compiled library is in the master subfolder. • The dual EEPROM library source code is in **<EDK_HOME>**\source\sk3\dual_eeprom, and the compiled library is in the master subfolder.
The Math Library	• The files required to link the Math library into projects can be found under the **<EDK_HOME>**\include\sk3 and **<EDK_HOME>**\lib\sk3 folders. • The Math library source code is in **<EDK_HOME>**\source\sk3\mathlib, and the compiled library is in the master subfolder.
The Bootloader	• The Bootloader source code is in **<EDK_HOME>**\source\sk3\bootloader, and the compiled program is in the master subfolder.

2.2.1 Firmware Files

The firmware files consist of include files (denoted with .cfi extensions), precompiled library binaries (denoted with .lib extensions) and precompiled program executables (denoted with

.abs extensions). Some precompiled libraries and programs are also included in source code format.

The firmware include files are listed in Table 2. With the exception of the library include files (sk3_syslib.cfi, sk3_eeprom.cfi, sk3_dual_eeprom.cfi, sk3_mathlib.cfi), the top-level firmware include file, sk3_sys.cfi, includes all other files. Thus, an application only needs to include sk3_sys.cfi to be able to access all of the provided hardware definitions and macros. The macros are divided into separate files based on their components. This organization is intended to assist developers in finding the implementation for specific macros and improve change control.

Table 2 also lists the firmware libraries. The firmware libraries are required only if functions from the libraries are used in application code. The application source code must include the associated firmware include file as well as link against the desired library.

TABLE 2: FIRMWARE INCLUDE FILES AND LIBRARIES

Firmware Component	Include Files	Library or Executable
Hardware Definitions	sk3_hw.cfi	-
System Macros	sk3_sys.cfi	-
System Library	sk3_syslib.cfi	sk3_sys.lib sk3_sys_cid102.lib sk3_sys_cid201.lib
EEPROM Library	sk3_eeprom.cfi	sk3_eeprom.lib sk3_eeprom_cid102.lib sk3_eeprom_cid201.lib
Dual EEPROM Library	sk3_dual_eeprom.cfi	sk3_dual_eeprom_cid102.lib sk3_dual_eeprom_cid201.lib
Math Library	sk3_math.cfi	sk3_math.lib sk3_math_cid102.lib sk3_math_cid201.lib
Bootloader	-	sk3_bootload_cid102.abs sk3_bootload_cid201.abs

TABLE 2: FIRMWARE INCLUDE FILES AND LIBRARIES (CONTINUED)

Firmware Component	Include Files	Library or Executable
Program ROM Vectors	sk3_romvect.cfi	-
System Components	sk3_sys_analog.cfi sk3_sys_cfx.cfi sk3_sys_clk.cfi sk3_sys_debugport.cfi sk3_sys_fifo.cfi sk3_sys_gpio.cfi sk3_sys_hear.cfi sk3_sys_i2c.cfi sk3_sys_int.cfi sk3_sys_ioc.cfi sk3_sys_mem.cfi sk3_sys_pcm.cfi sk3_sys_spi.cfi sk3_sys_system.cfi sk3_sys_timer.cfi sk3_sys_uart.cfi sk3_sys_utils.cfi sk3_sys_watchdog.cfi	-

2.3 SK3 VARIANTS

Each SK3 chip version might contain different hardware elements, as was the case with previous devices. Thus, when developing an application, the target chip within the base SK3 technology family must be specified in the project build settings using a chip identifier (CID). Also, CID-targeted versions of the firmware libraries and the bootloader exist, as listed in Table 2 on page 5 and explained in Section 2.4.1, “ROM-Based Libraries” on page 7.

The CID for a device is derived from combining the value of the D_CHIP_VERSION control register with the major revision component of the D_CHIP_REVISION register. See the *Hardware Reference Manual for BelaSigna 300* for more information.

When including sk3_sys.cfi for instance, specify the CID by using the symbol SK3_CID. Set this symbol to the target platform either in the source files or through the project build settings. The following example instructs the system libraries to provide the CID 102 variant of the constants:

```
#define SK3_CID 102              // Target BelaSigna 300 version 1.02.xx
#include <sk3_sys.cfi>
#include <sk3_syslib.cfi>
```

By default, new projects include the correct settings to target CID 201 for BelaSigna 300. To modify the target CID, update the build settings for your project. You can change the chip ID using build properties or preprocessor settings. See the *CFX Toolchain Reference for BelaSigna 300* for more information about build settings.

2.4 USING THE FIRMWARE

Applications must include the appropriate include files and link against the appropriate libraries to use the provided firmware. The top-level system include file, `sk3_sys.cfi`, should be included using the preprocessor `#include` directive at the beginning of each source file. Subsequently, if the application makes use of the firmware libraries, the associated include files for those libraries must be included wherever the library's functions or macros are called.

For example, an application using the firmware macros and hardware definitions, system library, EEPROM library, and math library would contain the following code:

```
#include <sk3_sys.cfi>
#include <sk3_syslib.cfi>
#include <sk3_eeprom.cfi>
#include <sk3_math.cfi>
```

Alternatively, the EEPROM library can be replaced with the dual EEPROM library: `sk3_dual_eeprom.cfi`.

The application's setting within the IDE would specify that it must link against the following libraries (or their ROM-based equivalents):

- `sk3_sys.lib`
- `sk3_math.lib`
- either `sk3_eeprom.lib` or one of the dual EEPROM libraries (`sk3_dual_eeprom_cid102.lib` or `sk3_dual_eeprom_cid201.lib`)

2.4.1 ROM-Based Libraries

The Program ROM contains implementations for some library functions. The ROM-based implementations are intended to reduce memory usage in the EEPROM and RAM. To use the ROM-based implementations, the application must link against the appropriate library. The ROM-based libraries contain a `_cid`**`xxx`** suffix in their filenames, where **`xxx`** is the chip ID (see Table 2 on page 5 for specifics). Note, however, that the dual EEPROM libraries are RAM-based libraries that follow this naming convention because they are chip-specific.

During development it is generally recommended that the standard RAM-based libraries be used. This allows full use of the IDE's debugging capabilities when using the libraries. For example, if the Math library is being used, the application would initially link against `sk3_math.lib`. After the Math library usage has been fully integrated and tested, the application can switch to linking against `sk3_math_cid`**`xxx`**`.lib`. This automatically switches from using the RAM-based implementations to using ROM-based implementations for the specified chip ID, which will reduce RAM and EEPROM usage with respect to the application footprint.

The Program ROM contains a function vector table so that the ROM-based libraries do not need to know the specific addresses of the ROM functions. The advantage of the design is that if a ROM function is relocated to a different memory location, application code that uses that function does not need to be changed or recompiled. The disadvantage is that, since the function table uses the `goto.0d` instruction to vector to the ROM functions, an application using the ROM-based libraries needs two more processor cycles per function call to execute than an otherwise identical application using the RAM-based libraries.

2.5 MEMORY MAP

The firmware reserves space in program memory, X memory and Y memory to ensure efficient operation. The reserved space is used for code, variables, error codes and other general firmware storage needs; it must not be used by user applications. To ensure this, the application must exclude the reserved areas in the linker command options. (This is the default in the IDE.) Table 3 lists the firmware-reserved memory.

TABLE 3: FIRMWARE-RESERVED MEMORY

Memory Space	Reserved Range	Usage
Program memory	0x3C00 - 0x3FFF	Bootloader, other code, encryption keys and (mirrored) interrupt vectors
X memory	0x0000 - 0x001F	System variables
Y memory	0x0000 - 0x001F	System variables

The user application can use any RAM outside of the reserved memory ranges.

2.5.1 Memory Allocation

The memory allocation model is designed for ease of use. It uses dynamic link-time allocation of code and data memories. When using this dynamic memory allocation model, you will not need to specify absolute memory addresses. It also frees you from manually tracking the memory allocation of your code, as it automatically avoids overlapping memory regions amongst multiple components. Furthermore, it abstracts the details of the memory map from you, so that you can write code and debug your application symbolically.

Several memory allocation macros are provided to support the memory allocation model on BelaSigna 300. They are located in the `sk3_sys_mem.cfi` include file and listed in Table 4.

TABLE 4: MEMORY ALLOCATION MACROS

Macro	Description	Reference Section
`Sys_Mem_Start_Segment`	Starts a new memory segment	10.122
`Sys_Mem_Start_Absolute_Segment`	Starts a new memory segment at a specific address	10.120
`Sys_Mem_Start_Reloc_Segment`	Starts a new memory segment after a specific address	10.121
`Sys_Mem_Reserve`	Reserved uninitialized memory in the current segment	10.117
`Sys_Mem_Reserve_Init_Default`	Reserve memory in the current segment and initialize with 0	10.119
`Sys_Mem_Reserve_Init`	Reserve memory in the current segment and initialize with specified value	10.118

Using these memory allocation macros, the application specifies its memory requirements at compile time through the use of several memory allocation macros. Segments are declared symbolically using the `Sys_Mem_Start_*` macros. The macros set up segments that are stored in the compiled binary object file. At link time, the segment information for all components is gathered by the linker and the application and its data are placed appropriately based on the available memory.

The IDE supports this memory allocation model by allowing symbolic memory addresses to be used. An application can therefore be debugged using symbols, so you will not need to know the actual program memory addresses at which your application, variables and functions are located. The application's build settings within the IDE indicate to the linker the available memory for the application.

Note that segments need to be defined for both program code and data. Variables should be initialized within the appropriate data segment using the `Sys_Mem_Reserve_*` macros.

An application will normally begin by declaring a data memory segment and declaring variables and buffers that will reside in that segment. The application must also allocate memory for the stack. The start of an application should then declare the main program memory segment and indicate the entry point of the application. The main entry point is specified to the linker with the `--entry` option.

The following example demonstrates a possible start to an application that uses the memory allocation macros:

```
Sys_Mem_Start_Segment( variables_segment, SYS_MEMSPACE_X )
Sys_Mem_Reserve_Init( x_buffer_ptr, 1, 0 )
Sys_Mem_Reserve_Init( x_buffer, 100, 0 )
Sys_Mem_Reserve_Init( x_count, 1, 0 )

Sys_Mem_Start_Segment( main_segment, SYS_MEMSPACE_P )
.global main
main:
    // Code starts here
```

This example allocates two variables (`x_buffer_ptr` and `x_count`) as well as a 100 word buffer (`x_buffer`). These variables and the buffer are created sequentially in the `variables_segment` in X memory. Next, the `main_segment` is created in program memory and the code starts after the `main:` label.

IMPORTANT: Note that the `Sys_Mem_Reserve_Init*` macros initialize memory only when loaded by the bootloader. After a soft reset, the memory remains allocated. If your application requires the values in the memory to be cleared out, it will have to re-initialize them.

2.5.1.1 Stack Initialization

The application is responsible for initializing and allocating space for the system stack. The same memory allocation macros used to allocate space for variables are used to allocate space for the stack. The stack can be created at a static address or it can be allocated dynamically. The following example shows how to dynamically allocate a 100-word stack and initialize it at the start of the application:

```
#define STACK_SIZE 100
Sys_Mem_Start_Segment( xstack, SYS_MEMSPACE_X )
Sys_Mem_Reserve( xstack, STACK_SIZE )

Sys_Mem_Start_Segment( main, SYS_MEMSPACE_P )
.global main
main:
    Sys_CFX_InitStack( xstack )       // Initialize the stack pointer
```

2.5.2 System Variables

The firmware contains a number of global variables located at fixed addresses. These variables are allocated in the firmware-reserved space at the base of X memory. The system variables are used for system purposes and generally should not be directly modified by user applications. You may sometimes find it useful to read the value of some of the system variables in your application code. Table 5 lists some of these variables.

TABLE 5: FIRMWARE SYSTEM VARIABLES

Variable	Address (X Memory)	Description
SYSVAR_BOOTLOADER_VER	0x0000	Bootloader version
SYSVAR_PROGROM_VER	0x0001	Program ROM version
SYSVAR_BOOTLOAD_VECT	0x0002	Bootloader address
SYSVAR_OPERATING_FREQ	0x0003	Current operating frequency
SYSVAR_APPNUM	0x0004	Application number
SYSVAR_SYS_ERRNO	0x0008	System error code
SYSVAR_GPIO_DATA_SHADOW	0x0018	Software copy of D_GPIO_DATA register value holding the most recent GPIO output settings.
SYSVAR_EE_SELECT	0x0019	Select the EEPROM to use in a dual EEPROM situation.

2.6 FIRMWARE NAMING CONVENTIONS

For clarity and ease of use, the firmware follows several naming conventions for library functions and macros.

2.6.1 Macro Naming

The firmware macros are all prefixed with `Sys_` and generally continue with the name of component they operate on. The components include the CFX and HEAR, control registers and peripherals. For instance, macros related to the CFX are prefixed with `Sys_CFX_`. If there is no specific target component, the component is omitted. The full name of the macro is intended to represent its underlying functionality.

2.6.2 Library Function Naming

The firmware libraries contain functions that are directly callable or callable through associated macros. The library functions are all prefixed with the name of the library, followed by the function name. For instance, the system library functions are prefixed with `SYSLIB_`. The library function prefixes contain the term `LIB` to emphasize that they are library functions. Table 6 lists the prefixes for each of the firmware libraries. Further details about the naming conventions for each library are available in their respective chapters.

TABLE 6: LIBRARY FUNCTION NAMING CONVENTION

Library	Filename	Function Prefix

EEPROM Library and Dual EEPROM Library	`sk3_eeprom.lib`
	`sk3_dual_eeprom_cid102.lib`
	`sk3_dual_eeprom_cid201.lib`
		`EELIB_`

System Library	`sk3_sys.lib`	`SYSLIB_`
Math Library	`sk3_math.lib`	`_MATHLIB_`

ROM-based libraries have a `_cid`**xxx** suffix on their file names (e.g., `sk3_sys_cid102.lib`) because they are chip-specific. The dual EEPROM libraries are chip-specific, but they are not ROM-based libraries. For more information on ROM-based libraries, see Section 2.4.1, “ROM-Based Libraries” on page 7.

2.7 FIRMWARE RESOURCE USAGE

2.7.1 Register Usage

The firmware specifies certain registers as scratch registers, which might be modified by a macro or function call. This ensures efficient implementation of the firmware functions and macros because using dedicated scratch registers avoids excessive use of the stack to save and restore register values. In its function header, every firmware macro and function lists the registers it modifies, including any scratch registers. If the calling function needs to preserve the content of any of the registers, the caller must take care of saving and restoring those registers.

The firmware attempts to minimize the number of register modifications where possible. Registers `y1`, `b1`, `xp7` and `yp3` are considered firmware scratch registers. Additionally, the status register (SR) is treated as a scratch register with specific bit-fields possibly being modified by a macro or function. The register modifications for each macro and function indicate which scratch registers are used. In general though, it is good practice to avoid using these registers in application code that uses the firmware as this avoids the overhead associated with saving and restoring those registers.

2.7.2 Stack Usage

The firmware makes use of the system stack. It expects the stack pointer to point to a valid stack that grows upward (i.e., increasing memory addresses). Furthermore, it assumes the memory location at `x[sp]` is not yet occupied by stack data, so it allocates the space it requires starting at the address `sp`.

2.8 VERSIONS

Version symbols are provided for each firmware component. The version symbols can be used directly or indirectly to verify the version of the components used to build an application. There are three types of version symbols available:

Define A preprocessor define containing the version information

Symbolic A compiled symbol contained within a binary library

Global Variable A global variable in memory containing the symbol

The available version information for each firmware component is listed in Table 7.

TABLE 7: FIRMWARE VERSION SYMBOLS

Component	Type	Version Symbol
Firmware includes	Define	SYS_FW_VER
System Library	Symbolic	SYSLIB_Version
EEPROM Library	Symbolic	EELIB_Version
Dual EEPROM Library	Symbolic	DUAL_EELIB_Version
Math Library	Symbolic	MATHLIB_Version
Bootloader	Global Variable	SYSVAR_BOOTLOADER_VER
Program ROM	Global Variable	SYSVAR_PROGROM_VER

The version information contains a major version, minor version and revision. The major version is updated to indicate significant changes to the component. Significant changes may involve a total redesign of the component including its interfaces and functionality. The minor version is updated to indicate minor changes or additions. This would generally indicate a change to the interfaces or underlying functionality, including different register modifications or leaving a different system state. The revision is updated to indicate a fairly insignificant change to the component. For example, the revision is updated when the interfaces and functionality of the component remains the same and some other change has occurred. This may include non-functional changes to the source code or optimizations that do not affect the calling application.

The major version, minor version and revision are contained within a 16-bit value. The version is described as Major.Minor.Revision. For example, Bootloader v1.0.0 indicates major version 1, minor version 0, and revision 0. Table 8 shows the bitfields in the version symbols.

TABLE 8: FIRMWARE VERSION BITFIELDS

Bits 15-12	Bits 11-8	Bits 7-0
Major Version	Minor Version	Revision

The version information for the Program ROM does not include a revision number, as all changes to the ROM image are considered to be at least minor version improvements. Table 9 shows the bitfields in the Program ROM version symbol.

TABLE 9: PROGRAM ROM VERSION BITFIELDS

Bits 15-8	Bits 7-0
Major Version	Minor Version

2.9 OPERATING FREQUENCIES

The firmware is designed with support for standard operating frequencies. A standard operating frequency is a valid system clock frequency that is recognized by all the firmware components requiring timing-specific code including the bootloader, EEPROM library and system functions. The operating frequency is used to determine the appropriate division factors where such factors are required. It may also be used by the user application as desired. Table 10 lists the standard BelaSigna 300 operating frequencies and their associated indices and defines. For more information on the storage of clock calibration values in EEPROM, see Section 7.8.1, “Clock Calibration” on page 82.

TABLE 10: OPERATING FREQUENCIES

Frequency Index	Symbol	Frequency
0	SYS_FREQ_UNKNOWN	Unknown operating frequency
1	SYS_FREQ_1M28	1.28 MHz
2	SYS_FREQ_1M92	1.92 MHz
3	SYS_FREQ_2M56	2.56 MHz
4	SYS_FREQ_3M84	3.84 MHz
5	SYS_FREQ_5M12	5.12 MHz

2.9.1 Current Operating Frequency

The current operating frequency is stored in the system variable SYSVAR_OPERATING_FREQ. The value is initially set by the Program ROM and subsequently by the bootloader and through Sys_SetOperatingFrequency. The file system contains specific clock calibration information for each BelaSigna 300 chip. The bootloader uses this information to set the internal oscillator to the indicated operating frequency, and stores that frequency index value in the system variable.

2.9.2 Changing Operating Frequencies

The current operating frequency can be changed using the system library function SYSLIB_SetOperatingFrequency, or the corresponding macro Sys_SetOperatingFrequency. The function takes a parameter specifying the operating frequency index. The operating frequency symbol as listed in Table 10 should be used. For example, to change the clock frequency to 2.56 MHz, execute the following code:

```
// Change operating frequency to 2.56 MHz
Sys_SetOperatingFrequency( SYS_FREQ_2M56 )
```

The SYSLIB_SetOperatingFrequency function will indicate a success or failure code in the status register.

IMPORTANT: The bootloader loads the clock calibration information from EEPROM to the firmware reserved X memory system variables block during boot. The clock frequency can only be changed through the system library function if the calibration data in RAM is present and valid.

2.10 ADC CALIBRATION

The firmware provides the capability to calibrate the Analog to Digital Converter (ADC) settings for the device using the system library function SYSLIB_CalibrateADCs, or the corresponding macro Sys_CalibrateADCs.

The SYSLIB_CalibrateADCs function and Sys_CalibrateADCs macro use the status register to indicate a success or failure code.

IMPORTANT: The bootloader loads the ADC calibration information from EEPROM to the firmware-reserved Y-memory system variables block during boot. The ADC calibration can only be changed through the system library function if the calibration data in RAM is present and valid.

2.11 BOOTING

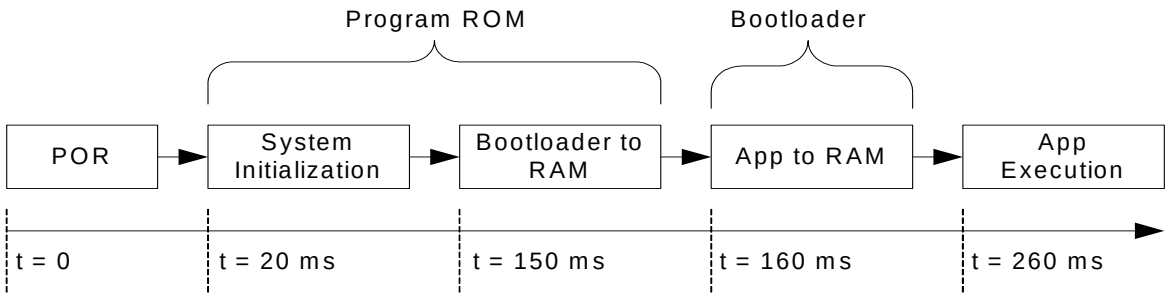
The firmware provides support for booting the system from an attached EEPROM. The user application is stored on the EEPROM. It is usually pre-loaded during manufacturing and stored along with manufacturing data such as clock calibration values. A variable in the file system indicates which application to boot if multiple applications are available on the EEPROM. For more information on the EEPROM file system see Chapter 7, “EEPROM File System” on page 75.

The Program ROM and bootloader work sequentially to load the application from EEPROM to RAM. The integrity of the application is verified while being loaded. If an error occurs during the boot process an error code is stored in memory and the system reboots after a delay.

The firmware also provides support for switching between multiple applications. This is provided through a firmware macro as well as a JumpROM function. The firmware macro Sys_BootApp can be used to allow the existing application to boot into another application. The JumpROM function can be used to allow an external system connected to the BelaSigna 300 debug port to switch between different applications. See the *CFX Communication Protocols Manual* for more information.

2.11.1 Boot Sequence

The boot process is a multistage sequence involving the reset circuitry, Program ROM and bootloader. After Power-on-Reset (POR), the reset signal to the CFX is released after a short delay and program execution begins at address 0x0000. This corresponds to the base of program memory where the Program ROM resides. Figure 1 shows the boot sequence and typical times for each stage. The example below assumes the default operating frequency is 5.12 MHz (resulting in an EEPROM read data rate of 2.56 Mbps).



NOTE: Times are typical.
Assumptions:
1) Fast boot option set
2) Bootloader size is 256 words
3) Application size is 250 kbit total

Figure 1: Boot Sequence

2.11.2 Boot Errors

During the boot process the file system and stored applications are verified for integrity. Several errors may be generated during the boot process. If an error is detected, the corresponding error code is written to the SYSVAR_SYS_ERRNO system variable. This value may be retrieved by examining memory (if possible) or through the JumpROM Get Error function. Table 11 lists the boot error codes. The Program ROM and bootloader will both detect errors. The bootloader error handler uses a logical OR to combine the error code corresponding to the boot failure with the bootloader error constant, 0x100. The bootloader error constant indicates that the error occurred in the bootloader.

TABLE 11: BOOT ERROR CODES

Error code	Symbol	Description
0x000	BOOT_ERR_NOERR	No error during boot
0x001	BOOT_ERR_BUSY	Boot currently in progress
0x002	BOOT_ERR_HEADER	Bootloader header invalid
0x003	BOOT_ERR_ALT	Alternate boot failed
0x004	BOOT_ERR_CRC	Bootloader CRC invalid
0x005	BOOT_ERR_DECRYPT	Bootloader decryption failed
0x006	BOOT_ERR_ADDR	Bootloader load address out of range
0x101	BL_ERR_SYSAREA_INVALID_CRC	System area CRC invalid
0x102	BL_ERR_MANUAREA_INVALID_CRC	Manufacturing area CRC invalid
0x103	BL_ERR_INVALID_KEY_INFO	KEY_INFO invalid
0x104	BL_ERR_INVALID_APPNUM	APPNUM invalid

TABLE 11: BOOT ERROR CODES (CONTINUED)

Error code	Symbol	Description
0x105	BL_ERR_INVALID_APPSIG	Invalid application signature
0x106	BL_ERR_APP_INVALID_SECTCOUNT	Invalid section count
0x107	BL_ERR_APP_INVALID_CRC	Application CRC invalid
0x108	BL_ERR_APP_INVALID_SECTTYPE	Application section type invalid
0x109	BL_ERR_APP_INVALID_KEY	Application decryption key invalid
0x10A	BL_ERR_APP_INVALID_SECTCRC	Application section CRC invalid
0x10B	BL_ERR_APP_INVALID_SECTSIZE	Application section size invalid
0x10C	BL_ERR_APP_INVALID_START	Application start section invalid
0x10D	BL_ERR_CORRUPT_VOLATILE	Volatile section CRC invalid
0x10E	BL_ERR_INCORRECT_FSVERSION	Incorrect file system version

3 System Include Files and Macros

3.1 DESCRIPTION

The hardware definitions and macros are contained within the system include files. Hardware definitions are used in place of absolute addresses and constants. The macros are used to perform specific system functions; they are expanded inline by the preprocessor.

3.2 HARDWARE DEFINITIONS

The hardware definitions file, `sk3_hw.cfi`, is integral to the system firmware. The hardware definitions apply a layer of names and labels to the underlying hardware so that every control register and field in the system is assigned a name and value. The name and labels are defined using the preprocessor directive `#define`, and may be used in your application code.

Using the hardware definitions file allows you to refer to system components by preprocessor symbols instead of by addresses and bit fields. This greatly improves readability and reliability of your application code. The use of hardware definitions in an application also means that some hardware changes, such as changes to addresses or field values, are transparent to your application code.

3.2.1 Hardware Naming Conventions

The hardware definitions file divides the control and configuration registers into two major categories: digital and analog registers. Digital control and configuration registers are prefixed with D_. These registers are used to control and configure the digital system components such as cores, peripherals and digital interfaces. Analog registers are prefixed with A_. These registers are used to control and configure the predominantly analog components, such as audio inputs, outputs and amplifiers.

The full register name follows the D_ or A_ prefix (i.e., D_**<REGISTERNAME>** and A_**<REGISTERNAME>**). The register name is chosen to clearly represent the underlying functionality of the register.

Each register's name is defined as a preprocessor symbol with the register's address as the symbol value. Following each register definition in the hardware definitions include file, the fields contained within that register are defined. Each field in a register is defined by a bit position definition and additionally by a mask definition for a multi-bit field. Bit position definitions are indicated by a suffix of `_POS` (i.e., **<REGISTERNAME>_<FIELD>_POS**), while bit mask definitions are indicated by a suffix of `_MASK` (i.e., **<REGISTERNAME>_<FIELD>_MASK**). A bit position definition indicates the starting bit of the corresponding field in the register. When a bit mask is applied to a register, all the bits except for those corresponding to the specified multi-bit field are zeroed. In both bit position and bit mask definitions, bit numbers start at the least significant bit (right-most bit, Bit 0).

The bit position and mask together allow for easy extraction of any field in the register. The preprocessor expression

```
(<D|A>_<REGISTERNAME> & <REGISTER_FIELD_MASK>) >> <REGISTER_FIELD_POS>
```

will extract the field value.

After all the fields for the register are defined, preprocessor symbols for all valid values for each of those fields are defined. As part of the definition, the settings are already aligned to the proper bit position of the field. This allows the preprocessor symbol to be used directly to write the setting to the register. Further, you may OR symbols together to form a full control word.

3.2.1.1 Hardware Definitions Usage

The hardware definitions file is provided as a CFX include file named `sk3_hw.cfi`. The include file may be included directly in the application source code or included as part of another include file. The file is automatically included in the top-level system firmware include file (`sk3_sys.cfi`).

Once the file has been included in the application, you may use the preprocessor symbols in place of absolute addresses and constants. For most control and configuration registers all the available settings are listed in the include file. You should OR together all the desired settings to create a control word and write the resulting word to the control register (either by writing directly to its defined address, or by using the appropriate macro).

The system macros provide an additional layer of abstraction by eliminating the need to use the register symbols directly. The Set or SetReg macros can be used to store the settings in the associated register, while the Get macros can be used to retrieve the value of a register.

3.3 MACROS

Macros provide a convenient layer of abstraction between the user application and the underlying system. They are generally targeted towards a specific system component. Instead of manually writing the code to directly modify any control register, you can simply use the macro. Other macros are provided for easy data conversion. Macros are also available to handle any required bit field manipulation and masking. For instance, if only certain fields are relevant within a control register, the other bits are left unmodified. Where possible, the macros provide a level of error checking.

The following sections describe the macro usage and syntax and include lists of all the available macros, organized by their operation.

3.3.1 Macro Naming Conventions

A macro name describes its core functionality. Macros that target a specific control register use the format:

```
<macro identifier>_<type>_<register> [(arguments)]
```

while those used to control a particular system component use the format

```
<macro identifier>_[<component>_]<operation> [(arguments)]
```

The macro identifier distinguishes the system firmware macros from user-defined macros. The `<macro identifier>` for all system firmware macros is `Sys_`.

The macro type or component follows the macro identifier. The type is one of: Set, Get, SetReg. This format is used when the target is a specific control register. When the macro is used to control a particular system component, the component is specified after the macro identifier. For instance, this format is used when the macro controls a specific peripheral. The system component is one of:

- System: Top-level system
- Mem: Memory configuration and management
- Int: Interrupt Controller
- DebugPort: Debug Port
- CFX: CFX DSP Core
- IOC: Input/Output Controller
- SPI: SPI Port
- I2C: I^2C Port
- Watchdog: Watchdog
- GPIO: General Purpose I/Os
- UART: UART Port
- Timer: Timers 1 and 2
- HEAR: HEAR Configurable Accelerator
- Output: Analog output stage blocks
- Input: Analog input stage blocks
- PCM: PCM Port
- Clk: Clock distribution
- FIFO: All FIFO buffers
- Data: Immediate data initialization

The Set, Get and SetReg macros are used to set new values and get existing values from the control registers. The SetReg macro is used to transfer a value from a CFX register. The system component macros are specific to the particular operation of that component.

Macros that don't require any arguments end after the <operation | register> field. Macros that take arguments have the arguments specified within parentheses after the name of the macro. For a complete list of the macro parameters, refer to Chapter 10, "System Macro Reference" on page 99.

3.3.2 Macro Usage

The macros are contained in several CFX include files. The top-level system include file, `sk3_sys.cfi`, includes all the subsequent include files and thus provides access to all the macros. Since all system firmware macros are implemented using the preprocessor directive #macro, you can simply use the macro names directly in your application source code. The preprocessor will expand (or "inline") each macro name to the appropriate assembly code at the point in your code where the macro name appears. See the *CFX Toolchain Reference for BelaSigna 300* for more information about the preprocessor.

3.3.2.1 Set, Get and SetReg Macros

The `Set`, `Get` and `SetReg` macros are designed for simplified access to control registers. Each control register contains a bit field mapping and associated field values in the hardware definitions include file. To execute the desired `Set`, `Get` or `SetReg` operation on the CFX, choose

the macro corresponding to the control registered to be accessed. If it is a Set or SetReg operation, you will also need to specify the desired field settings. You can OR together the field values to form the control word that will be written to the control register. For Set macros, the control word is passed to the macro directly as a macro argument. For SetReg macros, the control word is first stored in a CFX register and the name of the CFX register is passed to the macro as an argument. For Get macros, the field values of the control register will be placed in the CFX register specified as the argument to the macro.

3.4 SYSTEM CONFIGURATION

The system configuration macros relate to top-level system configuration; this includes the CFX DSP Core (Table 12), interrupts (Table 13), clocking (Table 14), debug port (Table 15), and system versions (Table 16).

TABLE 12: CFX CONFIGURATION MACROS

Macro	Description	Reference
Sys_CFX_SRConfig	Configure the CFX status register	10.17 on p. 115
Sys_CFX_BiasEnable	Clear the SR_BIAS bit to enable bias	10.5 on p. 103
Sys_CFX_BiasDisable	Set the SR_BIAS bit to disable bias	10.4 on p. 102
Sys_CFX_FlagUpdateEnable	Set the SR_FLAG_UPD bit to enable updates to SR flags	10.8 on p. 106
Sys_CFX_FlagUpdateDisable	Clear the SR_FLAG_UPD bit to disable updates to SR flags	10.7 on p. 105
Sys_CFX_IntEnable	Set the SR_IE bit to enable interrupts	10.12 on p. 110
Sys_CFX_IntDisable	Clear the SR_IE bit to disable interrupts	10.11 on p. 109
Sys_CFX_RoundEnable	Set the SR_ROUND bit to select rounding	10.14 on p. 112
Sys_CFX_RoundDisable	Clear the SR_ROUND bit to select truncation	10.13 on p. 111
Sys_CFX_SaturateEnable	Set the SR_SATURATE bit to select saturation	10.16 on p. 114
Sys_CFX_SaturateDisable	Clear the SR_SATURATE bit to select wrapping	10.15 on p. 113
Sys_CFX_StepConfig	Configure one of the CFX's step registers	10.18 on p. 116
Sys_CFX_CircAddrConfig	Configure one of the CFX's circular registers	10.6 on p. 104
Sys_CFX_InitStack	Initialize the CFX stack pointer	10.10 on p. 108
Sys_CFX_InitLoopStack	Initialize the loop stack	10.9 on p. 107

TABLE 13: INTERRUPT CONFIGURATION MACROS

Macro	Description	Reference
Sys_Int_Master_Enable	Enable all interrupts at the interrupt controller	10.91 on p. 189
Sys_Int_Master_Disable	Disable all interrupts at the interrupt controller	10.90 on p. 188
Sys_Int_Enable	Enable a set of interrupts	10.89 on p. 187
Sys_Int_Ack	Acknowledge a set of interrupts	10.88 on p. 186
Sys_Int_Priority	Configure the interrupt priority	10.92 on p. 190

TABLE 14: CLOCK CONFIGURATION MACROS

Macro	Description	Reference
Sys_Clk_Config0	Configure part of the clock configuration	10.20 on p. 118
Sys_Clk_Config1	Configure part of the clock configuration	10.21 on p. 119
Sys_Clk_Config	Configure both parts of the clock configuration	10.19 on p. 117

TABLE 15: DEBUG PORT MACROS

Macro	Description	Reference
Sys_DebugPort_Get_AudioMode	Return the audio mode setting	10.29 on p. 127
Sys_DebugPort_Set_AccessMode	Set access mode	10.30 on p. 128
Sys_DebugPort_Set_Address	Set the debug port address	10.31 on p. 129

TABLE 16: SYSTEM VERSION MACROS

Macro	Description	Reference
Sys_Get_ChipFamily	Return chip family	10.44 on p. 142
Sys_Get_ChipVersion	Return chip version	10.46 on p. 144
Sys_Get_ChipRevision	Return chip revision	10.45 on p. 143
Sys_Get_ChipDebugVersion	Return debug port version	10.43 on p. 141

3.5 MEMORY MANAGEMENT

The memory management macros are designed to facilitate the memory use paradigm. Macros are included for configuring memory segments and stack manipulation, including pushing and popping. The memory management macros are listed in Table 17.

TABLE 17: MEMORY MANAGEMENT MACROS

Macro	Description	Reference
Sys_Push_Reg_X	Store specified registers onto stack	10.151 on p. 249 to 10.160 on p. 258
Sys_Pop_Reg_X	Restore registers from stack	10.130 on p. 228 to 10.139 on p. 237
Sys_Push_Reg_Long_X	Store specified registers onto stack (long form)	10.161 on p. 259 to 10.170 on p. 268
Sys_Pop_Reg_Long_X	Restore registers from stack (long form)	10.140 on p. 238 to 10.149 on p. 247

TABLE 17: MEMORY MANAGEMENT MACROS (CONTINUED)

Macro	Description	Reference
Sys_Stack_Free	Free up previously allocated space on the stack	10.223 on p. 321
Sys_Stack_Alloc	Allocate space on the stack	10.222 on p. 320
Sys_Mem_Start_Segment	Start a memory segment	10.122 on p. 220
Sys_Mem_Start_Absolute_Segment	Start a new memory segment at a specific address	10.120 on p. 218
Sys_Mem_Start_Reloc_Segment	Start a new memory segment after a specific address	10.121 on p. 219
Sys_Mem_Reserve	Reserve uninitialized memory in the current segment	10.117 on p. 215
Sys_Mem_Reserve_Init_Default	Reserve memory in the current segment and initialize with 0	10.119 on p. 217
Sys_Mem_Reserve_Init	Reserve memory in the current segment and initialize with specified value	10.118 on p. 216
Sys_Mem_Priority	Set either the CFX or the HEAR as having higher priority for memory accesses	10.116 on p. 214

3.6 APPLICATION SWITCHING

The firmware provides a method to switch the active application by booting an application from EEPROM. The application switching macro is listed in Table 18.

TABLE 18: APPLICATION SWITCHING MACRO

Macro	Description	Reference
Sys_BootApp	Boot the specified application	10.1 on p. 99
Sys_Get_AppNum	Return the number of the currently running application (APPNUM)	10.41 on p. 139

3.7 FIRMWARE UTILITIES

The firmware utilities include convenient and efficient macros for common repetitive operations and useful data conversions. Memory copy macros are provided for copying data between the various memory spaces. Memory fill macros are provided for filling blocks of memory with data. The memory copy and fill macros are listed in Table 19. The data conversion macros are listed in Table 20.

TABLE 19: MEMORY COPY AND FILL MACROS

Macro	Description	Reference
Sys_Mem_CopyPP	Copy data from program memory to program memory	10.102 on p. 200
Sys_Mem_CopyPX	Copy data from program memory to X memory	10.103 on p. 201
Sys_Mem_CopyPY	Copy data from program memory to Y memory	10.104 on p. 202
Sys_Mem_CopyXP	Copy data from X memory to program memory	10.105 on p. 203

TABLE 19: MEMORY COPY AND FILL MACROS (CONTINUED)

Macro	Description	Reference
Sys_Mem_CopyXX	Copy data from X memory to X memory	10.106 on p. 204
Sys_Mem_CopyXY	Copy data from X memory to Y memory	10.107 on p. 205
Sys_Mem_CopyXY_Quick	Copy data from X memory to Y memory	10.108 on p. 206
Sys_Mem_CopyYP	Copy data from Y memory to program memory	10.109 on p. 207
Sys_Mem_CopyYX	Copy data from Y memory to X memory	10.110 on p. 208
Sys_Mem_CopyYX_Quick	Copy data from Y memory to X memory	10.111 on p. 209
Sys_Mem_CopyYY	Copy data from Y memory to Y memory	10.112 on p. 210
Sys_Mem_FillP	Fill a range of program memory with a value	10.113 on p. 211
Sys_Mem_FillX	Fill a range of X memory with a value	10.114 on p. 212
Sys_Mem_FillY	Fill a range of Y memory with a value	10.115 on p. 213

TABLE 20: NUMERIC TYPE CONVERSION MACROS

Macro	Description	Reference
Sys_Data_fl8n16	Create a .data directive[1] using the Sys_fl8n16 macro.	10.22 on p. 120
Sys_Data_frac24	Create a .data directive[1] using the Sys_frac24 macro	10.23 on p. 121
Sys_Data_frac32	Create a .data directive[1] using the Sys_frac32 macro	10.24 on p. 122
Sys_Data_frac48	Create a .data directive[1] using the Sys_frac48 macro	10.25 on p. 123
Sys_Data_m2p46	Create a .data directive[1] using the Sys_m2p46 macro	10.27 on p. 125
Sys_Data_m24p24	Create a .data directive[1] using the Sys_m24p24 macro	10.26 on p. 124
Sys_Data_m8p16	Create a .data directive[1] using the Sys_m8p16 macro	10.28 on p. 126
Sys_fl8n16	Convert a decimal number to <8,16> floating-point format	10.33 on p. 131
Sys_frac24	Convert a decimal number to <1,23> fixed-point format	10.34 on p. 132
Sys_frac32	Convert a decimal number to <9,23> fixed-point format	10.35 on p. 133
Sys_frac48	Convert a decimal number to <1,47> fixed-point format	10.36 on p. 134
Sys_IMM_n	Sign-extend a constant from its n LSBs, for n = 1, 2, ..., 32	10.87 on p. 185
Sys_Value_XY	Define an XY value list	10.234 on p. 332
Sys_Log2	Calculate the log base-2 for a supplied constant.	10.96 on p. 194
Sys_Log2_Ceil	Calculate the integer log base-2 (ceiling) for a supplied constant	10.97 on p. 195
Sys_Log2_Floor	Calculate the integer log base-2 (floor) for a supplied constant	10.98 on p. 196
Sys_m2p46	Convert a decimal number with up to seven digits of precision to <2,46> fixed-point format	10.100 on p. 198
Sys_m8p16	Convert a decimal number with up to seven digits of precision to <8,16> fixed-point format	10.101 on p. 199

TABLE 20: NUMERIC TYPE CONVERSION MACROS (CONTINUED)

Macro	Description	Reference
Sys_m24p24	Convert a decimal number with up to seven digits of precision to <24,24> fixed-point format	10.99 on p. 197
Sys_Power2	Calculate the power of 2 for a supplied constant.	10.150 on p. 248
Sys_Reg_fl8n16	Load a data register or high part of an accumulator with a floating-point number with an 8-bit signed exponent and a 16-bit signed mantissa using the Sys_fl8n16 macro	10.171 on p. 269
Sys_Reg_frac24	Load a data register or high part of an accumulator with a 24-bit signed immediate using the Sys_frac24 macro	10.172 on p. 270
Sys_Reg_frac32	Load the high and extension part of an accumulator with a 32-bit signed immediate using the Sys_frac32 macro	10.173 on p. 271
Sys_Reg_frac48	Load an accumulator with a 48-bit signed immediate using the Sys_frac48 macro	10.174 on p. 272
Sys_Reg_m2p46	Load an accumulator with a 48-bit signed immediate using the Sys_m2p46 macro	10.176 on p. 274
Sys_Reg_m8p16	Load a data register or high part of an accumulator with an 8-bit signed integer and a 16-bit unsigned fractional part using the Sys_m8p16 macro	10.177 on p. 275
Sys_Reg_m24p24	Load an accumulator with a 48-bit signed immediate using the Sys_m24p24 macro	10.175 on p. 273

1. For more information about .data directives, see the *CFX Toolchain Reference for BelaSigna 300*.

3.8 PERIPHERALS AND EXTERNAL INTERFACES

Macros are provided to control most of the system peripherals and external interfaces, including the watchdog timer (Table 21), UART (Table 22), SPI Port (Table 23), PCM Port (Table 24), IOC (Table 25), I^2C Port (Table 26), GPIO (Table 27), FIFO (Table 28), and timers (Table 29).

TABLE 21: WATCHDOG MACROS

Macro	Description	Reference
Sys_Watchdog_TimeoutConfig	Configure the watchdog timeout	10.236 on p. 334
Sys_Watchdog_Refresh	Refresh the watchdog timer count	10.235 on p. 333

TABLE 22: UART MACROS

Macro	Description	Reference
Sys_UART_Enable	Enable the UART	10.233 on p. 331
Sys_UART_Disable	Disable the UART	10.232 on p. 330

TABLE 23: SPI MACROS

Macro	Description	Reference
Sys_SPI_Enable	Enable the SPI interface	10.218 on p. 316
Sys_SPI_Disable	Disable the SPI interface	10.217 on p. 315
Sys_SPI_Read	Read the specified number of bits over the SPI port	10.219 on p. 317
Sys_SPI_Write	Write the specified number of bits over the SPI port	10.221 on p. 319
Sys_SPI_CS_High	Set the SPI_CS line high	10.215 on p. 313
Sys_SPI_CS_Low	Set the SPI_CS line low	10.216 on p. 314
Sys_SPI_WaitDone	Wait for the SPI operation to complete	10.220 on p. 318

TABLE 24: PCM MACROS

Macro	Description	Reference
Sys_PCM_Config	Set up the PCM interface for the given configuration	10.127 on p. 225
Sys_PCM_Enable	Enable the PCM interface (existing configuration)	10.129 on p. 227
Sys_PCM_Disable	Disable the PCM interface	10.128 on p. 226

TABLE 25: IOC MACROS

Macro	Description	Reference
Sys_IOC_InputConfig	Set up the IOC input configuration	10.94 on p. 192
Sys_IOC_OutputConfig	Set up the IOC output configuration	10.95 on p. 193
Sys_IOC_FIFOConfig	Set up the IOC FIFO configuration	10.93 on p. 191

TABLE 26: I^2C MACROS

Macro	Description	Reference
Sys_I2C_Config	Configures the I^2C interface parameters	10.77 on p. 175
Sys_I2C_Slave_Enable	Enable slave mode I^2C interface transfers	10.83 on p. 181
Sys_I2C_Slave_Disable	Disable slave mode I^2C interface transfers	10.82 on p. 180
Sys_I2C_Start_Write	Start an I^2C write transfer (master mode)	10.85 on p. 183
Sys_I2C_Start_Read	Start an I^2C read transfer (master mode)	10.84 on p. 182
Sys_I2C_ACK	Manually ACK the latest I^2C byte	10.76 on p. 174
Sys_I2C_NACK	Manually NACK the latest I^2C byte	10.78 on p. 176
Sys_I2C_NACK_And_Stop	Manually NACK the last I^2C byte and issue a stop condition (master mode)	10.79 on p. 177
Sys_I2C_Reset	Reset the general purpose I^2C interface	10.81 on p. 179

TABLE 26: I2C MACROS (CONTINUED)

Macro	Description	Reference
Sys_I2C_Pin_Config	Configure the I2C interface bus.	10.80 on p. 178
Sys_I2C_Watchdog_Config	Configure the I2C interface bus watchdog timeout	10.86 on p. 184

TABLE 27: GPIO MACROS

Macro	Description	Reference
Sys_GPIO_MUX_Select	Configure the GPIO pins for GPIO or alternate functions	10.64 on p. 162
Sys_GPIO_Configure	Configure the GPIO direction and interrupts	10.59 on p. 157
Sys_GPIO_Set_Data	Set the GPIO output to the specified value	10.65 on p. 163
Sys_GPIO_Get_Data	Get the GPIO input and store it into the specified register	10.62 on p. 160
Sys_GPIO_Get_All_Data	Get the GPIO input and store it into the specified register	10.61 on p. 159
Sys_GPIO_Toggle_Bits	Toggle the specified GPIO output pins	10.69 on p. 167
Sys_GPIO_Toggle	Toggle the specified GPIO output pin	10.68 on p. 166
Sys_GPIO_Set_High	Set the specified GPIO output value to high	10.66 on p. 164
Sys_GPIO_Set_Low	Set the specified GPIO output value to low	10.67 on p. 165
Sys_GPIO_IntConfig	Configure the GPIO interrupt	10.63 on p. 161
Sys_GPIO_Direction	Set the GPIO directions	10.60 on p. 158

TABLE 28: FIFO MACROS

Macro	Description	Reference
Sys_FIFO_Configure	Configure one of the system FIFOs	10.32 on p. 130

TABLE 29: TIMER MACROS

Macro	Description	Reference
Sys_Timer1_Config	Configure timer 1	10.224 on p. 322
Sys_Timer1_Start	Start timer 1	10.225 on p. 323
Sys_Timer1_Stop	Stop timer 1	10.226 on p. 324
Sys_Timer2_Config	Configure timer 2	10.227 on p. 325
Sys_Timer2_Start	Start timer 2	10.228 on p. 326
Sys_Timer2_Stop	Stop timer 2	10.229 on p. 327
Sys_Timers_Start	Start both timers	10.230 on p. 328
Sys_Timers_Stop	Stop both timers	10.231 on p. 329

3.9 ANALOG BLOCK CONFIGURATION

Macros related to control and configuration of the analog blocks are listed in Table 30.

TABLE 30: ANALOG REGISTER MACROS

Register	Macros	Reference
A_ADC_CFG	Sys_Set_ADC_CFG	10.180 on p. 278
	Sys_Get_ADC_CFG	10.37 on p. 135
A_ADC_CUR_CTRL	Sys_Set_ADC_CUR_CTRL	10.181 on p. 279
	Sys_SetReg_ADC_CUR_CTRL	10.199 on p. 297
	Sys_Get_ADC_CUR_CTRL	10.38 on p. 136
A_ADC_GF01_CTRL	Sys_Set_ADC_GF01_CTRL	10.182 on p. 280
A_ADC_GF23_CTRL	Sys_Set_ADC_GF23_CTRL	
	Sys_SetReg_ADC_GF23_CTRL	10.200 on p. 298
	Sys_SetReg_ADC_GF01_CTRL	
	Sys_Get_ADC_GF23_CTRL	10.39 on p. 137
	Sys_Get_ADC_GF01_CTRL	
	Sys_Set_Decimation_Gain01	10.186 on p. 284
	Sys_Set_Decimation_Gain23	
A_ANALOG_PWR_CTRL	Sys_Set_ANALOG_PWR_CTRL	10.183 on p. 281
	Sys_SetReg_ANALOG_PWR_CTRL	10.201 on p. 299
	Sys_Get_ANALOG_PWR_CTRL	10.40 on p. 138
A_BAND_SELECT_CTRL	Sys_Set_BAND_SELECT_CTRL	10.184 on p. 282
	Sys_SetReg_BAND_SELECT_CTRL	10.202 on p. 300
	Sys_Get_BAND_SELECT_CTRL	10.42 on p. 140
A_CLK_CTRL	Sys_Set_CLK_CTRL	10.185 on p. 283
	Sys_SetReg_CLK_CTRL	10.203 on p. 301
	Sys_Get_CLK_CTRL	10.47 on p. 145
A_FRAC_DELAY0_CTRL	Sys_Set_FRAC_DELAY_0_CTRL	10.187 on p. 285
A_FRAC_DELAY1_CTRL	Sys_Set_FRAC_DELAY_1_CTRL	
A_FRAC_DELAY2_CTRL	Sys_Set_FRAC_DELAY_2_CTRL	
A_FRAC_DELAY3_CTRL	Sys_Set_FRAC_DELAY_3_CTRL	
	Sys_SetReg_FRAC_DELAY_0_CTRL	10.204 on p. 302
	Sys_SetReg_FRAC_DELAY_1_CTRL	
	Sys_SetReg_FRAC_DELAY_2_CTRL	
	Sys_SetReg_FRAC_DELAY_3_CTRL	
	Sys_Get_FRAC_DELAY_0_CTRL	10.48 on p. 146
	Sys_Get_FRAC_DELAY_1_CTRL	
	Sys_Get_FRAC_DELAY_2_CTRL	
	Sys_Get_FRAC_DELAY_3_CTRL	

TABLE 30: ANALOG REGISTER MACROS (CONTINUED)

Register	Macros	Reference
A_IN_GAIN_CTRL	Sys_Set_IN_GAIN_CTRL	10.190 on p. 288
	Sys_Set_IN_GAIN_CTRL_NoCalibration	10.191 on p. 289
	Sys_SetReg_IN_GAIN_CTRL	10.206 on p. 304
	Sys_SetReg_IN_GAIN_CTRL_NoCalibration	10.207 on p. 305
	Sys_Get_IN_GAIN_CTRL	10.50 on p. 148
A_INPUT_CTRL	Sys_Set_INPUT_CTRL	10.192 on p. 290
	Sys_SetReg_INPUT_CTRL	10.208 on p. 306
	Sys_Get_INPUT_CTRL	10.51 on p. 149
A_LSAD_CTRL	Sys_Set_LSAD_CTRL	10.193 on p. 291
	Sys_SetReg_LSAD_CTRL	10.209 on p. 307
	Sys_Get_LSAD_CTRL	10.52 on p. 150
A_O_DAC_CTRL	Sys_Set_O_DAC_CTRL	10.194 on p. 292
	Sys_SetReg_O_DAC_CTRL	10.210 on p. 308
	Sys_Get_O_DAC_CTRL	10.53 on p. 151
A_O_DAC_GF_CTRL	Sys_Set_O_DAC_GF_CTRL	10.195 on p. 293
	Sys_SetReg_O_DAC_GF_CTRL	10.211 on p. 309
	Sys_Get_O_DAC_GF_CTRL	10.54 on p. 152
A_OUTPUT_CTRL	Sys_Set_OUTPUT_CTRL	10.196 on p. 294
	Sys_SetReg_OUTPUT_CTRL	10.212 on p. 310
	Sys_Get_OUTPUT_CTRL	10.55 on p. 153
A_PSU_CFG	Sys_Set_PSU_CFG	10.197 on p. 295
	Sys_SetReg_PSU_CFG	10.213 on p. 311
	Sys_Get_PSU_CFG	10.56 on p. 154
A_SUP_COUNT_DATA	Sys_Get_SUP_COUNT_DATA	10.57 on p. 155
A_SUP_THRSH_DATA	Sys_Set_SUP_THRSH_DATA	10.198 on p. 296
	Sys_SetReg_SUP_THRSH_DATA	10.214 on p. 312
	Sys_Get_SUP_THRSH_DATA	10.58 on p. 156

TABLE 31: ANALOG AUDIO SETTINGS MACROS

Macro	Description	Reference
Sys_Output_Unmute	Enable the output DAC	10.126 on p. 224
Sys_Output_Mute	Disable the output DAC	10.125 on p. 223
Sys_Output_Enable	Enable and configure the output DAC	10.124 on p. 222
Sys_Output_Disable	Disable the output DAC and clear its configuration	10.123 on p. 221
Sys_Calc_Sample_Freq_Val	Calculate sampling frequency field for `A_ADC_CFG`	10.3 on p. 101
Sys_Calc_Gain_Factor_Val	Calculate the decimation gain factor for a desired gain given an `adc_cfg_sample_freq` field	10.2 on p. 100

3.10 HEAR CONFIGURATION

Macros related to configuring the HEAR Configurable Accelerator are listed in Table 32.

TABLE 32: HEAR MACROS

Macro	Description	Reference
Sys_HEAR_Pause	Pause the HEAR	10.73 on p. 171
Sys_HEAR_Start	Start/resume the HEAR	10.75 on p. 173
Sys_HEAR_Reset	Reset the HEAR and FIFO controller (and hence the FIFO registers)	10.74 on p. 172
Sys_HEAR_Command	Issue a command to execute a HEAR function chain	10.70 on p. 168
Sys_HEAR_FunctionChain_Enable	Enable specified HEAR function chains	10.71 on p. 169
Sys_HEAR_FunctionChain_Priority	Set the priority of HEAR function chains	10.72 on p. 170
Sys_Set_HEAR_BFP	Configure the HEAR block floating point guard bits	10.188 on p. 286
Sys_Save_HEAR_BFP	Save the current HEAR block floating point configuration	10.179 on p. 277
Sys_Restore_HEAR_BFP	Restore a previously stored HEAR block floating point configuration	10.178 on p. 276
Sys_SetReg_HEAR_BFP_BLOCK_EXP	Set the value of the block floating point block exponent from a register	10.205 on p. 303
Sys_Get_HEAR_BFP_BLOCK_EXP	Store the value of the block floating point block exponent into a register	10.49 on p. 147

This page left blank for printing purposes.

4 System Library

4.1 DESCRIPTION

The firmware system library contains a set of useful functions that may be called from the user application to easily control and configure the system hardware. The functions are designed to accomplish common tasks such as resetting the system and changing the operating frequency. These functions are linked into the application.

4.2 SYSTEM LIBRARY USAGE

System library functions are contained within a compiled library. Both RAM-based and ROM-based versions are available. For more information, see Section 2.4.1, “ROM-Based Libraries” on page 7.

You must include the associated system library include file, `sk3_syslib.cfi`, in the source code. This include file contains the function declarations for the system library as well as system library macros. You must then configure your IDE to link in the appropriate compiled library.

Function parameters and return values are passed through registers. The specific registers required for each function are listed in the function reference.

4.2.1 Naming Convention

System library functions are named in mixed case preceded with a prefix of `SYSLIB_`. The operation follows the prefix.

4.3 SYSTEM LIBRARY FUNCTIONS

Table 33 lists the functions available in the system library.

TABLE 33: SYSTEM LIBRARY FUNCTIONS

Function	Description	Reference
SYSLIB_CalibrateADCs	Calibrate the DC offsets for the ADCs	11.10 on p. 344
SYSLIB_Delay	Delay the core for the specified amount of time	11.11 on p. 345
SYSLIB_Reset	Reset the system with a watchdog timeout	11.12 on p. 346
SYSLIB_SetOperatingFrequency	Set the operating frequency	11.13 on p. 347
SYSLIB_SoftPowerdownDisable	Exit from the soft power down mode	11.14 on p. 348
SYSLIB_SoftPowerdownEnable	Put the system in the soft power down mode	11.15 on p. 349

TABLE 33: SYSTEM LIBRARY FUNCTIONS

Function	Description	Reference
SYSLIB_SoftPowerdownEnable_ NoAnalog	Put the system in soft power down mode without disabling the analog blocks (recommended for VBAT > 1.3V)	11.16 on p. 350
SYSLIB_WaitForSPIDone	Wait for the SPI interface to become idle	11.17 on p. 351

4.4 SYSTEM LIBRARY MACROS

The system library macros are mostly wrappers around the system library functions. The macros provide a convenient method to use the system library functions. The macros take parameters that are subsequently set in the appropriate registers for the library function. The macro implementation can also be used as a reference on how to use the corresponding function directly. The system library macros are listed in Table 34.

TABLE 34: SYSTEM LIBRARY MACROS

Macro	Description	Reference
Sys_CalibrateADCs	Calibrate the DC offsets for the ADCs	11.1 on p. 335
Sys_Delay	Delay the core for the specified amount of time	11.2 on p. 336
Sys_DelayReg	Delay the core for the specified amount of time	11.3 on p. 337
Sys_Reset	Reset the system through a watchdog timeout	11.4 on p. 338
Sys_SetOperatingFrequency	Set the operating frequency	11.5 on p. 339
Sys_SoftPowerDownDisable	Exit soft power-down mode	11.6 on p. 340
Sys_SoftPowerDownEnable	Put the system in soft power-down mode	11.7 on p. 341
Sys_SoftPowerDownEnable_ NoAnalog	Put the system in soft power down mode without disabling the analog blocks (recommended for VBAT > 1.3V)	11.8 on p. 342
Sys_WaitForSPIDone	Wait for the SPI interface to become idle	11.9 on p. 343

4.5 VERSION

The system library version is contained within the library as a 16-bit symbol. The symbol `SYSLIB_Version` may be used directly in the application code to identify the version of the library linked with the application. See Section 2.8, “Versions” on page 12 for more information about version symbols.

5 Math Library

5.1 DESCRIPTION

This chapter describes the mathematical library deployed for the CFX processor.

The purpose of this library is to provide developers with commonly used mathematical functions that support the development of digital signal processing algorithms. This library offers optimized mathematical operations in macros and functions. Additionally, and since the source code is provided, these macros can be considered as examples for implementing other desired mathematical functions. All provided routines are optimized for speed of execution and small memory usage, within a self-contained code module. Their use is recommended, as they simplify programming by eliminating the need to write basic functions.

In the following sections, you will find:

- A list of the math library functions
- Context and usage of the functions, including a description of the naming convention used
- Descriptions of the data formats used
- Details of the functions: includes accuracy and precision of results, especially for functions using lookup table-based approximations
- Assembly code examples that make use of the math library

5.2 CONTENT OF THE LIBRARY

The library provides the mathematical and digital signal processing operations listed in Table 35.

TABLE 35: OPERATIONS INCLUDED IN THE MATH LIBRARY

Function	Operation
$y = \sqrt{x}$	Square root of *x*
$y = \log_2 x$	Base 2 logarithm of *x*
$y = 2^x$	Base 2 exponent of *x*
$y = \frac{1}{x}$	Reciprocal of *x*
$y = \frac{a}{b}$	Division *a*/*b*
$y = t0 + x*(t1 - t0)$	Linear interpolation (typically for table lookup operations)
$y(n) = a*x(n) + (1 - a)*y(n - 1)$	Exponential averaging filter (first order)

TABLE 35: OPERATIONS INCLUDED IN THE MATH LIBRARY

Function	Operation
$y(n) = a*x(n) + (1-a)*y(n-1)$ if $x(n) \geq y(n-1)$ $y(n) = b*x(n) + (1-b)*y(n-1)$ if $x(n) < y(n-1)$	Attack-release smoothing filter (first order) with *a* controlling the attack time and *b* the release time
$y = 20*\log_{10}x$	RMS to dB conversion
$y = 10^{\frac{x}{20}}$	dB to RMS conversion
$y = 10*\log_{10}x$	Power to dB conversion
$y = 10^{\frac{x}{10}}$	dB to power conversion
$y = 20*\log_{2}x$	RMS to dB (base2) conversion
$y = 2^{\frac{x}{20}}$	dB (base 2) to RMS conversion
$y = 10*\log_{2}x$	Power to dB (base 2) conversion
$y = 2^{\frac{x}{10}}$	dB (base 2) to power conversion
$y = real(x)^2 + imag(x)^2$	Squared magnitude of a complex number
$y = \sqrt{real(x)^2 + imag(x)^2}$	Magnitude of a complex number
$y = \sin(x)$	Sine function
$y = \cos(x)$	Cosine function
Fixed-point to floating-point conversion	Convert data represented in one of the fixed-point formats to floating-point format
Floating-point to fixed-point conversion	Convert data represented in floating-point format to one of the fixed-point formats.

Where required, several macros are provided for the same mathematical operation in order to support different numerical data formats (in either input or output; see Section 5.4, “Numeric Types” on page 36 for more details).

For some of the mathematical operations, different versions of the macros are provided to process either scalar or vector inputs optimally. The vector-based macros are particularly useful to process data in the frequency-domain, when the same math operation is required on every band. In such a case, data corresponding to the successive bands must be stored in a vector, where the elements of the vector occupy consecutive locations in X memory (for 24-bit data) or XY memory (for 48-bit data).

A detailed list of all the macros and functions in the library, including supported formats, calling parameters, modified registers, returned values and execution time, are provided in the following chapters:

- Chapter 12, “Math Library Macro Reference” on page 353
- Chapter 13, “Math Library Function Reference” on page 433

The library is organized into two parts: macros available from the include file `sk3_math.cfi` (or `sk3_math_ver100.cfi`) and functions stored in the source file `sk3_mathlib.cfs`. The functions—which are the core of the macros—together with the associated tables are stored in the ROM of the DSP. You are free to examine the source code, but usually direct calls to the core functions will not be needed. Using the macros will suffice in most situations.

IMPORTANT: All of the math library macros and functions modify the status register because of the operations done inside those macros and functions. Also, for vector-based macros, ensure that at least one loop is available in the loop stack. For fixed-point to floating-point conversion macros, enable Flag Updates mode.

5.3 MATH LIBRARY USAGE

To access the math functions, the application source code must include the associated system library file, `sk3_math.cfi`. This include file contains the function declarations for the math library as well as the math library macros. The project must then link against the appropriate binary library. Both RAM- and ROM-based libraries are available. See Section 2.4.1, “ROM-Based Libraries” on page 7 for more information. Both versions of the library use the same lookup tables, located in X-memory. The lookup tables are in separate sections of memory from the code that implements the library functions.

This example calculates the square root of the number 0.25, previously stored in the accumulator a0 as a 48-bit fixed-point fractional number. After execution, the 48-bit result (0.5) becomes available in `a0`.

```
// Example of math library call using a math library macro

#include <sk3_math.cfi>          // Include math library file

    load a0, 0x200000            // Load input (0.25) in a0

    MATHLIB_SQRT_frac48_frac48   // Call SQRT function for 48-bit frac
                                 //   input and output

    // Result (approx. 0.5) can be read in a0
```

5.3.1 Naming Conventions

The name of a math library macro follows the convention as described below. Note that all math library macros are prefixed with `MATHLIB_` to indicate that they are part of the math library.

`MATHLIB_<Math Op Name>_<Input data format>_<Output data format>`

In situations where the data format can be user-defined, it is omitted in the macro name. When only one data format is specified (as in the divide macros), it is the output format.

When the macro is intended for use with vector inputs, "VEC" is added after the name of the macro:

MATHLIB_**<Math Op Name>**_VEC_**<Input data format>**_**<Output data format>**

For instance, the SQRT operation is supplied in 4 library macros. One of the macros calculates the square root (SQRT) of a fractional 48-bit input value, and provides the result as a fractional 48-bit value. It would be launched as follows:

```
MATHLIB_SQRT_frac48_frac48
```

For all single-input scalar-based macros, such as the SQRT macro above, the input data is to be available in the accumulator `a0` before launching the routine. The result of the calculation is returned in the accumulator `a0`.

Another SQRT macro (named `MATHLIB_SQRT_frac24_frac48`) is a variation of the above, which expects a 24-bit fractional input. As in the previous case, it returns a 48-bit result. In this case, the input occupies the high part of the accumulator (`a0.h`) only.

For both macros, equivalent vector-based versions are also available. They are named `MATHLIB_SQRT_VEC_frac48_frac48` and `MATHLIB_SQRT_VEC_frac24_frac48` respectively. The 48-bit input version of the macros would be launched as follows:

```
MATHLIB_SQRT_VEC_frac48_frac48(XP_IN, XP_OUT, VEC_LENGTH)
```

In both cases, an input vector of data is expected at location pointed to by the pointer register `XP_IN`, and the vector length must be specified as the third macro parameter `VEC_LENGTH`. The results are returned, starting at location addressed by `XP_OUT`. All vector-based macros use this scheme. They assume input/output data in X memory (for 24-bit data) or XY memory (for 48-bit data).

5.4 NUMERIC TYPES

This section describes the different numeric types supported in the Math Library. Based on the dynamic range characteristics of the signals going through the math functions, seven different numerical formats are provided to represent data. The CFX native two's complement fixed-point format is deployed in 6 variants using either 24 or 48 bits (frac24, frac48, m8p16, m24p24, m2p46, int24). The most significant bit (MSB) is always the sign bit, and only the position of the decimal point changes among those formats. In addition, a 24-bit floating point format is introduced for operations requiring a wider dynamic range.

TABLE 36: NUMERICAL FORMATS USED IN THE MATH LIBRARY

Format	Description	Range
frac24	24-bit fractional fixed-point format	[-1, … -2^{-23}, 0, 2^{-23}, …, $1-2^{-23}$]
frac48	48-bit fractional fixed-point format	[-1, … -2^{-47}, 0, 2^{-47}, …, $1-2^{-47}$]

TABLE 36: NUMERICAL FORMATS USED IN THE MATH LIBRARY (CONTINUED)

Format	Description	Range
m8p16	24-bit mixed fixed-point format with 8-bit signed integer and 16-bit unsigned fractional	$[-128, \ldots -2^{-16}, 0, 2^{-16}, \ldots, 128-2^{-16}]$
m24p24	48-bit mixed fixed-point format with 24-bit signed integer and 24-bit unsigned fractional	$[-2^{23}, \ldots -2^{-24}, 0, 2^{-24}, \ldots, 2^{23}-2^{-24}]$
m2p46	48-bit mixed fixed-point format with 2-bit signed integer and 46-bit unsigned fractional	$[-2, \ldots -2^{-46}, 0, 2^{-46}, \ldots, 2-2^{-46}]$
int24	24-bit fixed-point integer format	$[-2^{23}, \ldots -1, 0, 1, \ldots, 2^{23} - 1]$
fl8n16	24-bit floating-point format with 8-bit signed exponent and 16-bit signed mantissa	$[-2^{127}, \ldots, -2^{-128}, \ldots, -2^{-128-15}, 0, 2^{-128-15}, \ldots, (2^{-128})*(1-2^{-15}), \ldots, (2^{127})*(1-2^{-15})]$

In the following subsections, each format is described in greater detail.

5.4.1 24-bit Fractional (frac24)

In this fractional fixed-point format, data in range [-1, 1) is represented using a 24-bit two's complement value. The MSB is the sign bit, which makes the resolution equal to 2^{-23}. As for all fractional formats, the implicit decimal point is between the sign bit and the next lesser significant bits.

5.4.2 48-bit Fractional (frac48)

In this fractional fixed-point format, data in range [-1, 1) is represented using a 48-bit two's complement value. The MSB is the sign bit, which makes the resolution equal to 2^{-47}. As for all fractional formats, the implicit decimal point is between the sign bit and the next lesser significant bits.

This format extends the fractional representation to 48 bits, for use in operations that need very high precision. Both 24-bit words are loaded in parallel when accessing XY memory. However, if the 48-bit data is interleaved in X or Y memory, more processing cycles are required, since the access to the data would be done in two steps (loading the higher 24 bits first, and then the lower 24 bits).

5.4.3 24-bit Mixed (m8p16)

In this 8.16 mixed fixed-point format, data in range [-128, 128) is represented using a 24-bit two's complement value. The 8 MSBs are dedicated to the signed integer part, while the 16 least significant bits (LSBs) store the unsigned fractional part. The fractional resolution of numbers in this format is equal to 2^{-16}.

This format is a compromise between pure fractional and pure integer representation in that it can represent numbers with both an integer and a fractional portion. This is useful for small numbers (128 as a maximum value) that need some fractional precision (up to 16-bit). This format is appropriate for representing sound levels in decibels.

5.4.4 48-bit Mixed (m24p24)

In this 24.24 mixed fixed-point format, data in range [-2^{23}, 2^{23}) is represented using a 48-bit two's complement value. The 24 MSBs are dedicated to the signed integer part, while the 24 LSBs store the unsigned fractional part. The fractional resolution of numbers in this format is equal to 2^{-24}.

This format extends the 24-bit mixed format (m8p16) to 48-bit for situations in which a larger integer portion with good fractional resolution is needed. In fact, this format is the concatenation of the 24-bit integer format to the 24-bit unsigned fractional format.

5.4.5 48-bit Mixed (m2p46)

This format is used in routines calculating the magnitude and squared magnitude of complex numbers. In those operations, where the fractional values representing the real part and imaginary part of the input are frac48 numbers, the output can reach a value of 2. Hence, the frac48 format was modified to cope with this, leading to a 2.46 mixed format. Data in range [-2, 2) can be represented as a 48-bit two's complement value. Two MSBs are dedicated to the signed integer part, while the 46 LSBs store the unsigned fractional part. The fractional resolution of numbers in this format is equal to 2^{-46}.

5.4.6 24-bit Integer (int24)

In this integer fixed-point format, data in range [-2^{23}, 2^{23}) is represented using a 24-bit two's complement value. All 24 bits are dedicated to the signed integer part with the decimal point at the right of the LSB bit. The fractional resolution of numbers in this format is 1.

This format is typically used in counting and indexing tasks, pattern matching, and error codes.

5.4.7 24-bit Floating-Point (fl8n16)

In this floating-point format, the 24 bits are split into an 8-bit signed exponent (an integer, located in the top 8 bits), and a 16-bit signed mantissa. The exponent is expressed in base 2, and it represents the number of times the mantissa should be shifted to represent the original number in fixed-point. Data in range [-2^{127}, ..., -2^{-128},..., -2^{-143}, 0, 2^{-143}, ..., 2^{-128}, ..., 2^{127}) can be represented.

For macros that output a value in fl8n16 format, the mantissa of the output is always normalized. If the mantissa is zero, the exponent is also zero.

For macros that take an input value in fl8n16 format, the mantissa of the input is not required to be normalized. Zero is represented by a mantissa of zero with any value for the exponent.

This format is particularly adapted to the large dynamic range required in operations such as the reciprocal. Support for this format is provided through the conversion routines of the math library.

5.5 MATH LIBRARY MACROS

In the following sections, the mathematical operations provided by the library are described in more detail. The associated macros are listed, and their performance is addressed.

5.5.1 Square Root

The square root of a number is calculated by decomposing the input *x* into $x = m \cdot 2^E$, where *E* is the integer exponent and *m* is the mantissa in the range [0.5, 1.0). Therefore, the square root of *x* will be calculated as:

$$\sqrt{x} = 2^{E/2} \cdot \sqrt{m}$$

for even *E*, or

$$\sqrt{x} = 2^{(E+1)/2} \cdot \sqrt{m/2}$$

for odd *E*.

In the code, *E* is calculated using the CFX log2abs instruction, and the square root of the mantissa is extracted from a lookup table using 5 MSBs of the input value as the index number. The lookup table has 32 entries where the first 16 correspond to $\sqrt{m/2}$ and the last half of the table includes values of $\sqrt{m}$. The lookup table is stored in ROM, starting at `D_LUT_SQRT_BASE`.

IMPORTANT: The input value x=0 is not allowed. An undefined result will be returned. If the value of x might be zero, the user must test the input value before launching the function.

Four macros are provided for calculating the square root, supporting scalar or vectors and 24- or 48-bit fractional fixed-point inputs. In all cases, the input must be a positive non-zero number. All output results are 48-bit fractional values.

```
// 48-bit input scalar
MATHLIB_SQRT_frac48_frac48
// 48-bit input vector
MATHLIB_SQRT_VEC_frac48_frac48(XP_IN, XP_OUT, VEC_LENGTH)
// 24-bit input scalar
MATHLIB_SQRT_frac24_frac48
// 24-bit input vector
MATHLIB_SQRT_VEC_frac24_frac48(XP_IN, XP_OUT, VEC_LENGTH)
```

All macros use the core function `_MATHLIB_SQRT_frac48_frac48`, after converting the input format if necessary.

The precision of the square root functions is the same for 24-bit and 48-bit inputs. The relative calculation error is 0.56% on average. It reaches the maximum of 1.55% for results very close to zero. The average absolute error is ±0.0035, while its maximum stands at ±0.011. This occurs at the middle of the input range, for $\sqrt{(x = 0.5)}$.

TABLE 37: PROPERTIES FOR SQRT MACROS

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_SQRT_frac48_frac48, MATHLIB_SQRT_VEC_frac48_frac48	(0, 1)	avg: 0.56% max: 1.55%	avg: ±0.0035 max: ±0.011	The input value x=0 is not valid, since an undefined result would be returned
MATHLIB_SQRT_frac24_frac48, MATHLIB_SQRT_VEC_frac24_frac48	(0, 1)	avg: 0.56% max: 1.55%	avg: ±0.0035 max: ±0.011	

5.5.2 Base 2 Logarithm

The log2(x) function is a key component of many audio algorithms because it is used as part of the conversion from signal power to signal level in dB. For level measurements, the required precision is small, since:

$$\text{Level(dB)} = 10\log_{10}x = 3.0103\log_2 x \approx 3\log_2 x$$

The base 2 logarithm of a number is calculated based on decomposing the input *x* into $x = m \cdot 2^E$, where *E* is the integer exponent and *m* is the mantissa in the range [0.5, 1.0). Therefore, the base 2 logarithm of *x* can be calculated as:

$$\log_2 x = \log_2(m \cdot 2^E) = \log_2 m + E$$

In the code, the exponent E is determined by using the CFX log2abs instruction. The base 2 logarithm of *m* is extracted from a lookup table using 5 MSBs of the input value as the index number. The lookup table has 32 entries, stored in ROM starting at `D_LUT_LOG2_BASE`.

Eight macros are provided for calculating the base 2 logarithm, supporting scalar or vector input and output data. The output format is always m8p16, while the input formats can be frac48, frac24, m8p16 or float (fl8n16). In all cases, the input must be a positive non-zero number. An undefined value is returned for zero input.

Note: The `vLinLog2` module in the HEAR microcode library performs a similar calculation. The `vLinLog2` module also returns an undefined value for a zero input, but the value returned is different from the value returned by the Math library base 2 logarithm for a zero input.

```
// 48-bit input scalar
MATHLIB_LOG2_frac48_m8p16
// 48-bit input vector
MATHLIB_LOG2_VEC_frac48_m8p16(XP_IN, XP_OUT, VEC_LENGTH)
// 24-bit input scalar
MATHLIB_LOG2_frac24_m8p16
// 24-bit input vector
MATHLIB_LOG2_VEC_frac24_m8p16(XP_IN, XP_OUT, VEC_LENGTH)
// Mixed input scalar
MATHLIB_LOG2_m8p16_m8p16
// Mixed input vector
```

```
MATHLIB_LOG2_VEC_m8p16_m8p16(XP_IN, XP_OUT, VEC_LENGTH)
// Float input scalar
MATHLIB_LOG2_fl8n16_m8p16
// Float input vector
MATHLIB_LOG2_VEC_fl8n16_m8p16(XP_IN, XP_OUT, VEC_LENGTH)
```

All macros use the core function `_MATHLIB_LOG2_frac48_m8p16`, after converting the input format to frac48 when necessary.

The average absolute error of the logarithm functions is ±0.008, except for the macro with floating-point input data which shows an average of ±0.010. The maximum reaches ±0.023 for all functions. This occurs when the input is as close to zero as possible for an input numerical format. The relative error of the logarithm functions is not meaningful, because the result of the log2 function is close to zero when the input is close to 1. In the case of float input, overflow of the result occurs when the input (which can be as small as 2^{-143} in this format) is less than 2^{-128}. All macros will provide indeterminate results in case of zero inputs.

TABLE 38: PROPERTIES FOR LOG2 MACROS

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_LOG2_frac48_m8p16, MATHLIB_LOG2_VEC_frac48_m8p16	(0, 1)	Not applicable	avg: ±0.008 max: ±0.023	Relative error is not meaningful because the log2 output can be very close to zero; the input value x=0 is not valid and will result in an undefined value being returned.
MATHLIB_LOG2_frac24_m8p16, MATHLIB_LOG2_VEC_frac24_m8p16	(0, 1)	Not applicable	avg: ±0.008 max: ±0.023	
MATHLIB_LOG2_m8p16_m8p16, MATHLIB_LOG2_VEC_m8p16_m8p16	(0, 128)	Not applicable	avg: ±0.008 max: ±0.023	
MATHLIB_LOG2_fl8n16_m8p16, MATHLIB_LOG2_VEC_fl8n16_m8p16	$[2^{-128}, 2^{127})$	Not applicable	avg: ±0.010 max: ±0.023	Overflow if input in range $[2^{-143}, 2^{-128})$; the input value x=0 is not valid and will result in an undefined value being returned.

5.5.3 Base 2 Exponent

The 2^x function is often used as part of the conversion from signal level in dB to signal power. For level measurements, the required precision is small, since:

$$\text{Power} = 10^{\text{Level}/10} = 2^{\text{Level} \cdot 0.33219}$$

The base 2 exponent of a number is calculated based on decomposing the input *x* into x = *E*+*r*, where *E* is the integer component and *r* is the fractional component. Therefore, the base 2 logarithm of *x* can be calculated as:

$$2^x = 2^E \cdot 2^r = 2^{E+1} \cdot 2^{r-1}$$

The second form is preferred, improving the precision of the calculation. The function first extracts the exponent *E*, which is the integer part of the input. Then, 2^{r-1} is approximated from a lookup table, using 5 MSBs of (*r*-1) as the index number. Finally, the approximation from the table is shifted to the left by *E*+1 bits. The lookup table has 32 entries, and is stored in ROM, starting at location `D_LUT_EXP2_BASE`.

Four macros are provided for calculating the base 2 exponent, supporting scalar or vector input and output data. The input format is always m8p16, while the output formats can be frac48 or m8p16.

```
// 48-bit output scalar
MATHLIB_EXP2_m8p16_frac48
// 48-bit output vector
MATHLIB_EXP2_VEC_m8p16_frac48(XP_IN, XP_OUT, VEC_LENGTH)
// Mixed output scalar
MATHLIB_EXP2_m8p16_m8p16
// Mixed output vector
MATHLIB_EXP2_VEC_m8p16_m8p16(XP_IN, XP_OUT, VEC_LENGTH)
```

The first two macros must have negative inputs, as they cannot return values equal to or higher than 1 because of the fractional output format. The other macros can accept inputs as high as 7, thanks to the m8p16 format. For all macros, the output format limits the representation of very small values approaching zero. Using the 48-bit fractional format, the smallest output is 2^{-47}, which makes the minimum input equal to -47. To avoid underflows, smaller inputs are internally saturated to -47 before running the function. Similarly, for m8p16 output formats, the smallest output is 2^{-16}, which limits the input to a minimum value of -16. However, the macro `MATHLIB_EXP2_m8p16_m8p16` is actually a special case, because the output results are made available on the entire 48-bit accumulator, extending the precision of the fractional part from 16 to 16+24 = 40 bits. Hence, the actual minimum output value is not 2^{-16}, but 2^{-40}, just like in a m8p40 format. The user is free to use the extended output word and extend the input data range from [-16, 7) to [-40, 7). The inputs are saturated to -40 internally to avoid underflows.

In all cases, the core function `_MATHLIB_EXP2_m8p16_frac48` is used, converting its output to the appropriate format when necessary.

The precision of the exponent functions is different for fractional input format and for m8p16 format. In the first case, the absolute error is of the order $\pm 2^{-4}$ on average, but it can reach 0.011 for inputs close to zero (results close to 1). In the second case, this error is higher. While its average is about ±0.05, it can reach up to ± 1.2 for the largest outputs close to 2^7. The relative error is not meaningful, as the output value can be very close to zero for certain inputs.

TABLE 39: PROPERTIES FOR EXP2 MACROS

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_EXP2_m8p16_frac48, MATHLIB_EXP2_VEC_m8p16_frac48	[-47, 0)	Not applicable	avg: ±0.0002 max: ±0.011	Inputs < -47 will be saturated, resulting in an output of 0
MATHLIB_EXP2_m8p16_m8p16, MATHLIB_EXP2_VEC_m8p16_m8p16	[-40, 7)	Not applicable	avg: ±0.050 max: ±1.2	Inputs < -40 will be saturated, resulting in an output of 0

5.5.4 Reciprocal

The reciprocal function is approximated by expressing the input *x* as $x = m \cdot 2^E$, where *E* is the integer exponent and *m* is the mantissa in the range [0.5, 1.0). Therefore, the reciprocal of *x* can be calculated as:

$$\frac{1}{x} = \left(\frac{1}{m}\right) \cdot 2^{-E} = \frac{1}{2m} \cdot 2^{-E+1}$$

The second form is preferred in the code because $\frac{1}{2m}$ is a fractional number. *E* is calculated using the CFX instruction log2abs, and $\frac{1}{2m}$ is extracted from a lookup table using 5 MSBs of the modified mantissa (*2m*) as the index number. Finally, the approximation from the table is shifted by -*E*+1 bits. The lookup table has 32 entries. It is stored in ROM, starting at `D_LUT_RECIP_BASE`.

16 macros are provided for calculating the reciprocal, supporting scalar and vector data for both input and output. Macros supporting scalar data use accumulator a0 for input and output, while macros supporting vector data require pointers to the input and output locations.

To take advantage of the CFX's capabilities, two sets of macros are provided: one for positive numbers only and the other for signed numbers. The macro for positive-only inputs is more computationally efficient. For both kinds of macros, several input formats (frac48, m8p16, fl8n16 and int24) are acceptable in different versions of the macros, while the output is always represented in floating-point format.

General Input

```
// 48-bit input scalar
MATHLIB_RECIP_frac48_fl8n16
// 48-bit input vector
MATHLIB_RECIP_VEC_frac48_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
// Mixed input scalar
MATHLIB_RECIP_m8p16_fl8n16
// Mixed input vector
MATHLIB_RECIP_VEC_m8p16_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
// Float input scalar
MATHLIB_RECIP_fl8n16_fl8n16
// Float input vector
MATHLIB_RECIP_VEC_fl8n16_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
// Integer input scalar
MATHLIB_RECIP_int24_fl8n16
// Integer input vector
MATHLIB_RECIP_VEC_int24_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
```

Positive Input

```
// 48-bit positive input scalar
MATHLIB_RECIP_POS_frac48_fl8n16
// 48-bit positive input vector
MATHLIB_RECIP_POS_VEC_frac48_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
```

```
// Mixed positive input scalar
MATHLIB_RECIP_POS_m8p16_fl8n16
// Mixed positive input vector
MATHLIB_RECIP_POS_VEC_m8p16_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
// Float positive input scalar
MATHLIB_RECIP_POS_fl8n16_fl8n16
// Float positive input vector
MATHLIB_RECIP_POS_VEC_fl8n16_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
// Integer positive input scalar
MATHLIB_RECIP_POS_int24_fl8n16
// Integer positive input vector
MATHLIB_RECIP_POS_VEC_int24_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
```

The core function `_MATHLIB_RECIP_GEN_frac48_fl8n16` is called by the general input macros, while `_MATHLIB_RECIP_POS_frac48_fl8n16` is called by the positive input macros. Conversion of the input to the appropriate format is performed when necessary.

The relative error of the reciprocal functions is 0.56% on average, except for the function with floating-point input data which has an average around 0.7%. The maximum relative error is smaller than 1.55% for all functions. Absolute errors can become huge in case of very small inputs (large results). Overflow occurs with floating-point input formats, when the input is in the range $(-2^{-127}, 2^{-127})$. All macros will provide indeterminate results in case of zero inputs.

TABLE 40: PROPERTIES FOR RECIP MACROS

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_RECIP_frac48_fl8n16, MATHLIB_RECIP_VEC_frac48_fl8n16	[-1, 1)	avg: 0.56% max: 1.55%	Not applicable	Results and absolute errors are huge for inputs close to 0
MATHLIB_RECIP_m8p16_fl8n16, MATHLIB_RECIP_VEC_m8p16_fl8n16	[-128, 128)	avg: 0.56% max: 1.55%	Not applicable	
MATHLIB_RECIP_int24_fl8n16, MATHLIB_RECIP_VEC_int24_fl8n16	$[-2^{23}, 2^{23})$	avg: 0.56% max: 1.55%	Not applicable	
MATHLIB_RECIP_fl8n16_fl8n16, MATHLIB_RECIP_VEC_fl8n16_fl8n16	$[-2^{127}, -2^{-127}]$ + $[2^{-127}, 2^{127})$	avg: 0.76% max: 1.55%	Not applicable	Overflow if input in range $(-2^{-127}, 2^{-127})$
MATHLIB_RECIP_POS_frac48_fl8n16, MATHLIB_RECIP_POS_VEC_frac48_fl8n16	(0, 1)	avg: 0.56% max: 1.55%	Not applicable	Results and absolute errors are huge for inputs close to 0
MATHLIB_RECIP_POS_m8p16_fl8n16, MATHLIB_RECIP_POS_VEC_m8p16_fl8n16	(0, 128)	avg: 0.56% max: 1.55%	Not applicable	
MATHLIB_RECIP_POS_int24_fl8n16, MATHLIB_RECIP_POS_VEC_int24_fl8n16	$(0, 2^{23})$	avg: 0.56% max: 1.55%	Not applicable	
MATHLIB_RECIP_POS_fl8n16_fl8n16, MATHLIB_RECIP_POS_VEC_fl8n16_fl8n16	$[2^{-127}, 2^{127})$	avg: 0.70% max: 1.55%	Not applicable	Overflow if input smaller than 2^{-127}

5.5.5 Division

The DIVIDE macros are implemented based on the division-step instruction of the CFX. These macros use the division-step instruction consecutively as many times as needed (depending on the number of bits in the data format).

Three macros are provided for division. Considering the heavy calculation load in such an operation, it is assumed that the division operation will not be run iteratively. Therefore, only scalar inputs are supported. Any kind of 24-bit user-defined fixed-point format is acceptable in input, while the output is either 24-bit integer, 48-bit mixed or 24-bit fractional.

```
MATHLIB_DIVIDE_int24            // Integer output scalars
MATHLIB_DIVIDE_m24p24           // Mixed output scalars
MATHLIB_DIVIDE_frac24           // Frac output scalars
```

All divide macros call the core function `_MATHLIB_DIVIDE`. They expect positive dividend and divisor numbers, represented in the same fixed-point format. The dividend is expected in location `a0.h`, while the divisor must be placed in `x0`. For `MATHLIB_DIVIDE_int24` and `MATHLIB_DIVIDE_frac24`, the quotient is returned in the lower part of the accumulator, `a0.l`. `MATHLIB_DIVIDE_m24p24`, on the other hand, returns the 48-bit quotient in `a0`. In the case of `MATHLIB_DIVIDE_int24`, the remainder of the division can be optionally determined by executing the instruction `add a0, x0, lt` after the macro.

The relative error is as small as 0.0016% and 0.00063% for the macros providing mixed and fractional outputs, respectively. This represents absolute errors of order $\pm 10^{-7}$ and $\pm 10^{-8}$ respectively. The error is zero for the int24 version of the macro, while the precision of the remainder reaches $\pm 10^{-16}$.

TABLE 41: PROPERTIES FOR DIVISION MACROS

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_DIVIDE_int24	User-defined, positive	0%	0	Remainder error smaller than 10^{-16}
MATHLIB_DIVIDE_m24p24	User-defined, positive	avg: 1.67×10^{-5}% max: 1.60×10^{-3}%	avg: $\pm 3.0 \times 10^{-8}$ max: $\pm 6.0 \times 10^{-8}$	Relative error and absolute error provided here assume fractional input data
MATHLIB_DIVIDE_frac24	User-defined, positive	avg: 2.1×10^{-5}% max: 6.3×10^{-4}%	avg: $\pm 5.7 \times 10^{-8}$ max: $\pm 1.2 \times 10^{-7}$	

5.5.6 Linear Interpolation

Based on a fractional ratio *x*, this macro performs the linear interpolation of two values *t0* and *t1*:

$$y = t0 + x \cdot (t1 - t0)$$

When x=0.5, the middle point between *t0* and *t1* is returned (arithmetic average).

The primary application of this function is to extract a value from a lookup table, when the input to the table is situated between two consecutive indexes. The ratio *x* is the ratio of the distance between the first value and the input value to the distance between the first value and the next

value in the lookup table. (It is assumed that the first index is associated to the element *t0*, while the next index is associated to *t1*).

A single macro is provided for the linear interpolation. Its input parameter `XP_IN` is a pointer that addresses the location where *t0* is located in X memory. The value *t1* must be stored in the next memory word, and the 24-bit ratio *x* is expected in the high part of the accumulator (`a0.h`):

```
MATHLIB_LIN_INTERP(XP_IN)
```

The numerical format of the table elements *t0* and *t1* can be any fixed-point format. However, (*t1-t0*) must not exceed the range of the format used. Normally, when using a fractional format, (*t1-t0*) must be in range [-1, 1). This means that the table must contain smoothly organized elements (which is generally the case). *x* is a 24-bit fractional, usually in the range [0,1), although negative values are supported as well.

The relative error measured on the interpolation output can reach 0.0003% at maximum, which makes the absolute error smaller than $\pm 1.2 \times 10^{-7}$.

TABLE 42: PROPERTIES FOR THE LINEAR INTERPOLATION MACROS

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_LIN_INTERP	User-defined	avg: 1.9×10^{-5}% max: 3×10^{-4}%	avg: $\pm 5.9 \times 10^{-8}$ max: $\pm 1.2 \times 10^{-7}$	Relative error and absolute error provided here assume fractional input data

5.5.7 Exponential Average Filter

The first order exponential averaging filter can be expressed as the following difference equation:

$$y(n) = a \cdot x(n) + (1 - a) \cdot y(n - 1) = a \cdot (x(n) - y(n - 1)) + y(n - 1)$$

In actual implementation, the second form is used, where a is the coefficient of the filter and *x(n)* is the current input sample. *y(n)* and *y(n-1)* are the current and previous output samples respectively.

Two macros are provided for the exponential averaging operation:

```
MATHLIB_EXP_AVG(XP_OUT, XP_COEFF)                          // input scalar
MATHLIB_EXP_AVG_VEC(XP_IN, XP_OUT, XP_COEFF, VEC_LENGTH)   // input vector
```

The first macro expects a 24-bit scalar value in `a0.h` for the input *x(n)*, while the second accepts a vector in which the first element is addressed by `XP_IN`. In both cases, `XP_OUT` is a pointer to the previous output *y(n-1)*, and `XP_COEFF` is the pointer to the coefficient *a*. For the vector macro, there must be `VEC_LENGTH` number of values stored in memory starting from locations `XP_OUT` and `XP_COEFF`, specifying one previous output y(n-1) and one filter coefficient a for each input vector element. This allows multiple averaging filters to be applied independently to different input data. The result of the filtering is returned in `a0.h` for the scalar macro, and in the locations addressed by `XP_OUT` when using the vector macro.

The format of the input *x(n)* (and consequently of the output *y(n)*) can be any type of 24-bit fixed-point numerical representation. The filter coefficient(s) must be positive frac24 numbers. It is assumed that *(x(n)-y(n-1))* does not exceed the range of the format used. For example, *(x(n)-y(n-1))* must be in the range [-1, 1) for frac24 inputs.

The relative error measured on the filtered output can reach 0.056% at maximum, which makes the absolute error smaller than $\pm 1.2 \times 10^{-7}$.

TABLE 43: PROPERTIES FOR THE EXPONENTIAL AVERAGE MACROS

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_EXP_AVG, MATHLIB_EXP_AVG_VEC	User-defined	avg: 7.9×10^{-5}% max: 5.6×10^{-2}%	avg: $\pm 5.9 \times 10^{-8}$ max: $\pm 1.2 \times 10^{-7}$	Relative error and absolute error provided here assume fractional input data

5.5.8 Attack-Release Filter

First order attack-release filters are widely used in audio processing, for the smoothing of frame-by-frame sequences (like signal energy). The expression for the output of the filter *y(n)* is as follows:

$$y(n) = coeff \cdot x(n) + (1 - coeff) \cdot y(n-1) = coeff \cdot (x(n) - y(n-1)) + y(n-1)$$

where:

coeff = *a* if $x(n) \geq y(n-1)$

coeff = *b* if $x(n) < y(n-1)$.

In this expression, *x(n)* is the input sample, *y(n-1)* is the previous output, while *a* and *b* are the coefficient of the filter, used in case of attack or release respectively.

Two macros are provided for the attack-release filtering operation:

```
// input scalar
MATHLIB_ATTACK_RELEASE(XP_OUT, XP_COEFF)
// input vector
MATHLIB_ATTACK_RELEASE_VEC(XP_IN, XP_OUT, XP_COEFF, VEC_LENGTH)
```

The first macro expects a 24-bit scalar value in `a0.h` for the input *x(n)*. `XP_OUT` is a pointer to the previous output *y(n-1)*, and `XP_COEFF` is the pointer to the filter coefficient *a*. This coefficient must be immediately followed by coefficient *b*. The result of the filtering is returned. The second macro accepts a vector of inputs *x(n)*, which first element is addressed by `XP_IN`. There must be `VEC_LENGTH` values stored in memory starting from location `XP_OUT` (one previous output *y(n-1*) per input vector element), and 2*`VEC_LENGTH` values stored in memory starting from location `XP_COEFF` (one coefficient *a* followed by one coefficient *b* per input vector element). The outputs *y(n)* are stored at the locations addressed by `XP_OUT`.

The format of the input *x(n)* (and consequently of the output *y(n)*) can be any type of 24-bit fixed-point numerical representation. The filter coefficients must be positive frac24 numbers. It is assumed that *(x(n)-y(n-1))* does not exceed the range of the used format. For example, *(x(n)-y(n-1))* must be in range [-1, 1) for frac24 inputs.

The relative error measured on the filtered output can reach 0.07% at maximum, which makes the absolute error smaller than $\pm 1.2 \times 10^{-7}$.

TABLE 44: PROPERTIES FOR THE ATTACK-RELEASE MACROS

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_ATTACK_RELEASE, MATHLIB_ATTACK_RELEASE_VEC	User-defined	avg: 7.3×10^{-5}% max: 7.0×10^{-2}%	avg: $\pm 5.9 \times 10^{-8}$ max: $\pm 1.2 \times 10^{-7}$	Relative error and absolute error provided here assume fractional input data

5.5.9 RMS to dB Conversion

RMS (Root-Mean-Square) to dB conversions are often required in audio processing, for calculating signal energy levels. The expression for the conversion is as follows:

$$y = 20 \cdot \log_{10} x = 20 \cdot \log_{10} 2 \cdot \log_2 x$$

In the implementation, the second form is used, taking advantage of the available log2 function.

Two macros are provided for the conversion operation. Only non-zero positive input numbers in the range (0,1) are supported. Scalar or vector inputs must be expressed in frac48, while the output is returned in m8p16 format, well-adapted for representing numbers in decibel levels:

```
// input scalar
MATHLIB_RMS_DB10_frac48_m8p16
// input vector
MATHLIB_RMS_DB10_VEC_frac48_m8p16(XP_IN, XP_OUT, VEC_LENGTH)
```

The core function `_MATHLIB_LOG2_frac48_m8p16` is called by the macros to calculate the base 2 logarithm involved.

As the output format (m8p16) cannot represent values smaller than -128, overflow of the result returned in `a0.h` will occur when the input is less than $10^{(-128)/20}$. Within the valid input range, the absolute error of the conversion is ±0.050 on average, and always lower than ±0.133 dB. The relative error is not meaningful because the results of the logarithm can be very close to zero.

TABLE 45: PROPERTIES FOR THE RMS TO DB CONVERSION MACROS

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_RMS_DB10_frac48_m8p16, MATHLIB_RMS_DB10_VEC_frac48_m8p16	$[10^{-128/20}, 1)$	Not applicable	avg: ±0.050 max: ±0.133	Overflow if input smaller than $10^{-128/20}$

5.5.10 dB to RMS Conversion

The expression for the dB to RMS (Root-Mean-Square) conversion is as follows:

$$y = 10^{\frac{x}{20}} = 2^{\frac{x}{20 \cdot \log_{10}2}}$$

In the implementation, the second form is used, taking advantage of the available exp2 function.

Two macros are provided for the conversion operation supporting scalar or vector inputs. The input level is expressed in m8p16 format.

```
// input scalar
MATHLIB_DB10_RMS_m8p16_frac48
// input vector
MATHLIB_DB10_RMS_VEC_m8p16_frac48(XP_IN, XP_OUT, VEC_LENGTH)
```

The core function `_MATHLIB_EXP2_m8p16_frac48` is called by the macros to calculate the exponent involved.

As the output format (frac48) cannot represent values equal to or higher than 1, the input data must be negative. Overflow occurs if the input is zero (since $2^0 = 1$), unless saturation mode is set. Within the valid input range, the maximum absolute error of the conversion is lower than ±0.011. Highest absolute errors occur for outputs very close to 1. The relative error is smaller than 1.1%.

TABLE 46: PROPERTIES FOR THE DB TO RMS CONVERSION MACROS

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_DB10_RMS_m8p16_frac48, MATHLIB_DB10_RMS_VEC_m8p16_frac48	[-128, 0)	avg: 0.5% max: 1.1%	avg: $\pm 4.2 \times 10^{-4}$ max: ±0.011	Overflow / saturation if input is zero or positive

5.5.11 Power to dB Conversion

Power to dB conversions are also widely needed in audio processing for calculating signal energy levels. The expression for the conversion is as follows:

$$y = 10 \cdot \log_{10}x = 10 \cdot \log_{10}2 \cdot \log_{2}x$$

In the implementation, the second form is used, taking advantage of the available log2 function.

Two macros are provided for the conversion operation. Only non-zero positive input numbers in the range (0,1) are supported. Scalar or vector inputs must be expressed in frac48, while the output is returned in m8p16 format:

```
// input scalar
MATHLIB_PWR_DB10_frac48_m8p16
```

```
// input vector
MATHLIB_PWR_DB10_VEC_frac48_m8p16(XP_IN, XP_OUT, VEC_LENGTH)
```

The core function `_MATHLIB_LOG2_frac48_m8p16` is called by the macros to calculate the logarithm involved.

As the output format (m8p16) cannot represent values smaller than -128, overflow of the result returned in a0.h will occur when the input is less than $10^{-128/10}$. Within the valid input range, the absolute error of the conversion is ±0.024 on average, and always lower than ±0.067 decibels. The relative error is not meaningful because the results of the logarithm can be very close to zero.

TABLE 47: PROPERTIES FOR THE POWER TO DB CONVERSION MACROS

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_PWR_DB10_frac48_m8p16, MATHLIB_PWR_DB10_VEC_frac48_m8p16	$[10^{-128/10}, 1)$	Not applicable	avg: ±0.024 max: ±0.067	Overflow if input smaller than $10^{-128/10}$

5.5.12 dB to Power Conversion

The expression for the conversion from decibel to power is as follows:

$$y = 10^{\frac{x}{10}} = 2^{\frac{x}{10 \cdot \log_{10} 2}}$$

In the implementation, the second form is used, taking advantage of the available exp2 function.

Two macros are provided for the conversion operation supporting scalar or vector inputs. The input level is expressed in m8p16 format, well-adapted for representing numbers in decibel levels. The output is in frac48.

```
// input scalar
MATHLIB_DB10_PWR_m8p16_frac48
// input vector
MATHLIB_DB10_PWR_VEC_m8p16_frac48(XP_IN, XP_OUT, VEC_LENGTH)
```

The core function `_MATHLIB_EXP2_m8p16_frac48` is called by the macros to calculate the exponent involved.

As the output format (frac48) cannot represent values equal or higher than 1, the input data must be negative. Overflow occurs if the input is zero (since $2^0 = 1$), unless saturation mode is set. Within the valid input range, the maximum absolute error of the conversion is lower than ±0.011. Highest absolute errors occur for outputs very close to 1. The relative error is smaller than 5%.

TABLE 48: PROPERTIES FOR THE DB TO POWER CONVERSION MACROS

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_DB10_PWR_m8p16_frac48, MATHLIB_DB10_PWR_VEC_m8p16_frac48	[-128, 0)	avg: 0.59% max: 5.0%	avg: $\pm 2.3 \times 10^{-4}$ max: ±0.011	Overflow / saturation if input is zero or positive

5.5.13 RMS to dB (Base 2) Conversion

The expression for the RMS to dB (base 2) conversion is as follows:

$$y = 20 \cdot \log_2 x$$

Two macros are provided for the conversion operation. Only non-zero positive input numbers in the range (0,1) are supported. Scalar or vector inputs must be expressed in frac48, while the output is returned in m8p16 format:

```
// input scalar
MATHLIB_RMS_DB2_frac48_m8p16
// input vector
MATHLIB_RMS_DB2_VEC_frac48_m8p16(XP_IN, XP_OUT, VEC_LENGTH)
```

The core function `_MATHLIB_LOG2_frac48_m8p16` is called by the macros to calculate the logarithm involved.

As the output format (m8p16) cannot represent values smaller than -128, overflow of the result returned in `a0.h` will occur when the input is less than $2^{-128/20}$. Within the valid input range, the absolute error of the conversion is ±0.157 on average, and always lower than ±0.442. The relative error is not meaningful because the results of the logarithm can be very close to zero.

TABLE 49: PROPERTIES FOR THE RMS TO DB (BASE 2) CONVERSION MACROS

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_RMS_DB2_frac48_m8p16, MATHLIB_RMS_DB2_VEC_frac48_m8p16	$[2^{-128/20}, 1)$	Not applicable	avg: ±0.157 max: ±0.442	Overflow if input smaller than $2^{-128/20}$

5.5.14 dB (Base 2) to RMS Conversion

The expression for the conversion from base 2 decibels to RMS (Root-Mean-Square) is as follows:

$$y = 2^{\frac{x}{20}}$$

Two macros are provided for the conversion operation supporting scalar or vector inputs. The input level is expressed in m8p16 format. The output is in frac48.

```
// input scalar
MATHLIB_DB2_RMS_m8p16_frac48
// input vector
MATHLIB_DB2_RMS_VEC_m8p16_frac48(XP_IN, XP_OUT, VEC_LENGTH)
```

The core function `_MATHLIB_EXP2_m8p16_frac48` is called by the macros to calculate the exponent involved.

As the output format (frac48) cannot represent values equal or higher than 1, the input data must be negative. Overflow occurs if the input is zero (since $2^0 = 1$), unless saturation mode is set. Within the valid input range, the maximum absolute error of the conversion is lower than ±0.011. Highest absolute errors occur for outputs very close to 1. The relative error is smaller than 1.1%.

TABLE 50: PROPERTIES FOR THE DB (BASE 2) TO RMS CONVERSION MACROS

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_DB2_RMS_m8p16_frac48, MATHLIB_DB2_RMS_VEC_m8p16_frac48	[-128, 0)	avg: 0.54% max: 1.1%	avg: ±0.0012 max: ±0.011	Overflow / saturation if input is non-negative

5.5.15 Power to dB (Base 2) Conversion

The expression for the power to dB (base 2) conversion is as follows:

$$y = 10 \cdot \log_2 x$$

Two macros are provided for the conversion operation. Only non-zero positive input numbers in the range (0,1) are supported. Scalar or vector inputs must be expressed in frac48, while the output is returned in m8p16 format:

```
// input scalar
MATHLIB_PWR_DB2_frac48_m8p16
// input vector
MATHLIB_PWR_DB2_VEC_frac48_m8p16(XP_IN, XP_OUT, VEC_LENGTH)
```

The core function `_MATHLIB_LOG2_frac48_m8p16` is called by the macros to calculate the logarithm involved.

As the output format (m8p16) cannot represent values smaller than -128, overflow of the result returned in `a0.h` will occur when the input is less than $2^{-128/10}$. Within the valid input range, the absolute error of the conversion is ±0.077 on average, and always lower than ±0.22. The relative error is not meaningful because the results of the logarithm can be very close to zero.

TABLE 51: PROPERTIES FOR THE POWER TO DB (BASE 2) CONVERSION MACROS

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_PWR_DB2_frac48_m8p16, MATHLIB_PWR_DB2_VEC_frac48_m8p16	$[2^{-128/10}, 1)$	Not applicable	avg: ±0.077 max: ±0.22	Overflow if input smaller than $2^{-128/10}$

5.5.16 dB (Base 2) to Power Conversion

The expression for the base 2 decibels to power conversion is as follows:

$$y = 2^{\frac{x}{10}}$$

Two macros are provided for the conversion operation supporting scalar or vector inputs. The input level is expressed in m8p16 format, well-adapted to decibels. The output is in frac48.

```
// input scalar
MATHLIB_DB2_PWR_m8p16_frac48
// input vector
MATHLIB_DB2_PWR_VEC_m8p16_frac48(XP_IN, XP_OUT, VEC_LENGTH)
```

The core function `_MATHLIB_EXP2_m8p16_frac48` is called by the macros to calculate the exponent involved.

As the output format (frac48) cannot represent values equal or higher than 1, the input data must be negative. Overflow occurs if the input is zero (since $2^0 = 1$), unless saturation mode is set. Within the valid input range, the maximum absolute error of the conversion is lower than ±0.011. Highest absolute errors occur for outputs very close to 1. The relative error is smaller than 1.1%.

TABLE 52: PROPERTIES FOR THE DB (BASE 2) TO POWER CONVERSION MACROS

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_DB2_PWR_m8p16_frac48, MATHLIB_DB2_PWR_VEC_m8p16_frac48	[-128, 0)	avg: 0.54% max: 1.1%	avg: $\pm 6.6 \times 10^{-4}$ max: ±0.011	Overflow / saturation if input is positive

5.5.17 Squared Magnitude

Calculation of the squared magnitude *y* of a complex number *x* is often needed, notably after a frequency-domain analysis. The operation is:

$$y = \text{real}(x)^2 + \text{imag}(x)^2$$

Two macros are provided for calculating the squared magnitude, one for scalar and the other for vector inputs. Both macros expect 24-bit fractional input data:

```
// input scalar
MATHLIB_SQMAG_frac24_m2p46
// input vector
MATHLIB_SQMAG_VEC_frac24_m2p46(XP_IN, XP_OUT, VEC_LENGTH)
```

The format of the output results (m2p46, see Section 5.4, “Numeric Types” on page 36) was specially defined for this operation, since the squared magnitude of fractional complex numbers is between 0 and 2. Using such a format, the calculation is performed without any quantization error. Since the input has an unsigned precision of only 23 bits and the output uses 46 bits to represent the fractional part of the result, the absolute error is zero.

TABLE 53: PROPERTIES FOR THE SQMAG MACROS

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_SQMAG_frac24_m2p46, MATHLIB_SQMAG_VEC_frac24_m2p46	[-1, 1)	0%	0	Overflow occurs for input = -1-1j unless saturation mode is set

Note: As a special case, the squared magnitude of the number -1-1j (that can be represented as a 24-bit fractional) is equal to 2. Unfortunately, 2 just overflows the m2p46 format, and the result would be -2. Hence, saturation mode should be used to avoid this special situation.

5.5.18 Magnitude

The expression to calculate the magnitude *y* of a complex number *x* is as follows:

$$y = \sqrt{\text{real}(x)^2 + \text{imag}(x)^2}$$

Two macros are provided for calculating the magnitude, one for scalar and the other for vector inputs. Both macros expect 24-bit fractional input data:

```
// input scalar
MATHLIB_MAG_frac24_m2p46
// input vector
MATHLIB_MAG_VEC_frac24_m2p46(XP_IN, XP_OUT, VEC_LENGTH)
```

The format of the output results (m2p46, see Section 5.4, “Numeric Types” on page 36) was specially defined for this operation, since the magnitude of fractional complex numbers can be higher than 1.

The macro uses the core SQRT function named `_MATHLIB_SQRT_frac48_frac48`. As this function uses a table lookup for the determination of the square root, the magnitude result is an approximation. The relative error of the approximation is 0.57% on average, and 1.55% at

maximum. The average absolute error is ±0.0044, while its maximum reaches ±0.022 for the highest output value.

IMPORTANT: The input value x=0+0j is not allowed because the SQRT function does not support 0 inputs. An undefined result would be returned. If necessary, the user must test the input value before launching the function.

IMPORTANT: The MAG function calls the SQMAG function, so the input value x=-1-j causes the same issue for MAG as it does for SQMAG. Specifically, it will overflow if saturation mode is not set.

TABLE 54: PROPERTIES FOR THE MAG MACROS

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_MAG_frac24_m2p46, MATHLIB_MAG_VEC_frac24_m2p46	[-1, 1)	avg: 0.57% max: 1.55%	avg: ±0.0044 max: ±0.022	The input value 0+0j is not allowed; the input value -1-j will cause overflow unless saturation mode is set

5.5.19 Sine Function

The sine function:

$$y = \sin(x)$$

is implemented by means of a table lookup. The supported input and output formats are 24-bit fractional. Both scalar and vector versions are implemented. Input data are expected in normalized radian units, where the radian range [-π, +π) is re-mapped to the fractional range [-1, 1).

```
// input scalar
MATHLIB_SIN_frac24_frac24
// input vector
MATHLIB_SIN_VEC_frac24_frac24(XP_IN, XP_OUT, VEC_LENGTH)
```

The average absolute error of the sine function is ±0.004, while its maximum reaches ±0.013. This occurs for outputs very close to zero. The relative error is not meaningful, as results can be very close to zero.

TABLE 55: PROPERTIES FOR THE SIN MACROS

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_SIN_frac24_frac24, MATHLIB_SIN_VEC_frac24_frac24	[-1, 1)	Not applicable	avg: ±0.004 max: ±0.013	The relative error is not meaningful, as results can be very close to zero

5.5.20 Cosine Function

The cosine function:

$$y = \cos(x)$$

is implemented by means of a table lookup. The supported input and output formats are 24-bit fractional. Both scalar and vector versions are implemented. Input data are expected in normalized radian units, where the radian range [-π, +π) is re-mapped to the fractional range [-1, 1).

```
// input scalar
MATHLIB_COS_frac24_frac24
// input vector
MATHLIB_COS_VEC_frac24_frac24(XP_IN, XP_OUT, VEC_LENGTH)
```

The average absolute error of the cosine function is ±0.004, while its maximum reaches ±0.013 This occurs for outputs very close to zero. The relative error is not meaningful, as results can become very close to zero.

TABLE 56: PROPERTIES FOR THE COS MACROS

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_COS_frac24_frac24, MATHLIB_COS_VEC_frac24_frac24	[-1, 1)	Not applicable	avg: ±0.004 max: ±0.013	The relative error is not meaningful, as results can be very close to zero

5.5.21 Fixed-Point to Floating-Point Conversion

Several conversion routines are provided to convert data represented in one of the fixed-point formats (frac24, frac48, m8p16, m24p24, int24) to the floating-point format fl8n16. Such routines are particularly useful when the programmer wishes to use one of the library routines with floating-point input data. All the provided conversion routines make use of the log2abs instruction available in the CFX.

IMPORTANT: For fixed-point to floating-point conversion macros, enable Flag Updates mode.

The following macros are available, depending on the input format and the form of the input (scalar or vector):

```
// mixed 24.24 input scalar
MATHLIB_CONV_m24p24_fl8n16
// mixed 24.24 input vector
MATHLIB_CONV_VEC_m24p24_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
// mixed 8.16 input scalar
MATHLIB_CONV_m8p16_fl8n16
// mixed 8.16 input vector
MATHLIB_CONV_VEC_m8p16_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
// integer input scalar
MATHLIB_CONV_int24_fl8n16
// integer input vector
MATHLIB_CONV_VEC_int24_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
// frac48 input scalar
MATHLIB_CONV_frac48_fl8n16
// frac48 input vector
MATHLIB_CONV_VEC_frac48_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
// frac24 input scalar
MATHLIB_CONV_frac24_fl8n16
// frac24 input vector
MATHLIB_CONV_VEC_frac24_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)
```

All the conversions provide good precision as the relative error is maintained below 0.006%.

TABLE 57: PROPERTIES FOR FIXED-POINT TO FLOATING-POINT CONVERSION MACROS

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_CONV_m24p24_fl8n16, MATHLIB_CONV_VEC_m24p24_fl8n16	$[-2^{23}, 2^{23})$	avg: 0.002% max: 0.006%	avg: ±84.7 max: ±255	Large absolute errors are caused by large input values
MATHLIB_CONV_m8p16_fl8n16, MATHLIB_CONV_VEC_m8p16_fl8n16	[-128, 128)	avg: 0.002% max: 0.006%	avg: ±0.001 max: ±0.004	
MATHLIB_CONV_int24_fl8n16, MATHLIB_CONV_VEC_int24_fl8n16	$[-2^{23}, 2^{23})$	avg: 0.002% max: 0.006%	avg: ±74.4 max: ±255	Large absolute errors are caused by large input values
MATHLIB_CONV_frac48_fl8n16, MATHLIB_CONV_VEC_frac48_fl8n16	[-1, 1)	avg: 0.002% max: 0.006%	avg: $\pm 1.1 \times 10^{-5}$ max: $\pm 3.1 \times 10^{-5}$	
MATHLIB_CONV_frac24_fl8n16, MATHLIB_CONV_VEC_frac24_fl8n16	[-1, 1)	avg: 0.002% max: 0.006%	avg: $\pm 1.1 \times 10^{-5}$ max: $\pm 3.1 \times 10^{-5}$	

5.5.22 Floating-Point to Fixed-Point Conversion

Several conversion routines are provided to convert floating-point data to one of the fixed-point formats (m24p24, frac48, m8p16). Such routines are particularly useful when floating-point results are returned by a library routine (for example, the reciprocal operation) and need to be converted to fixed-point format for a subsequent ALU operation. The following macros are available, depending on the input format and the form of the input (scalar or vector):

```
// mixed 8.16 output scalar
MATHLIB_CONV_fl8n16_m8p16
// mixed 8.16 output vector
MATHLIB_CONV_VEC_fl8n16_m8p16(XP_IN, XP_OUT, VEC_LENGTH)
// frac48 output scalar
MATHLIB_CONV_fl8n16_frac48
// frac48 output vector
MATHLIB_CONV_VEC_fl8n16_frac48(XP_IN, XP_OUT, VEC_LENGTH)
// mixed 24.24 output scalar
MATHLIB_CONV_fl8n16_m24p24
// mixed 24.24 output vector
MATHLIB_CONV_VEC_fl8n16_m24p24(XP_IN, XP_OUT, VEC_LENGTH)
```

When converting from floating-point to fixed-point format, the range of the resulting data is obviously reduced compared to the range of the input. Hence, one must be careful not to overflow the target format. Typically, the absolute value of the input must not be higher than the following for conversions:

- 128 for the m8p16 format
- 2^{23} for the m24p24 format
- 1 for the frac48 format

Similarly, one must be aware that very small input values approaching zero will be quantized to the precision of the fixed-point format, which increases the conversion error for these very small input values. To avoid the quantization effects, the absolute value of the input must not be lower than the following for conversions:

- 2^{-16} for the m8p16 format
- 2^{-24} for the m24p24 format
- 2^{-47} for the frac48 format

If an input is below the lower bound in any of these cases, the output of the conversion macro will be zero. Furthermore, for any floating-point input that has an exponent less than -56, the output of the macro is undefined.

TABLE 58: PROPERTIES FOR FLOATING-POINT TO FIXED-POINT CONVERSION MACROS

Macro Name	Input Range	Relative Error	Absolute Error	Comment
MATHLIB_CONV_fl8n16_m8p16, MATHLIB_CONV_VEC_fl8n16_m8p16	[-128, 128)	Not applicable	avg: $\pm 2.8 \times 10^{-6}$ max: $\pm 1.6 \times 10^{-5}$	Relative error is not meaningful because the floating-point input can be very close to zero compared to the output format
MATHLIB_CONV_fl8n16_frac48, MATHLIB_CONV_VEC_fl8n16_frac48	[-1, 1)	avg: 8.2×10^{-16}% max: 2.3×10^{-14}%	avg: $\pm 1.5 \times 10^{-22}$ max: $\pm 6.8 \times 10^{-21}$	
MATHLIB_CONV_fl8n16_m24p24, MATHLIB_CONV_VEC_fl8n16_m24p24	[-2^{23}, 2^{23})	avg: 6.5×10^{-16}% max: 2.3×10^{-14}%	avg: $\pm 2.2 \times 10^{-22}$ max: $\pm 1.4 \times 10^{-20}$	

5.6 EXAMPLES

This section shows typical code examples that use the math library macros and the different numerical formats. The numerical results given for these examples were generated with the rounding modes in the SR set for biased rounding.

5.6.1 Squared Magnitude

The first example calculates the squared magnitude of the complex number 0.5 + *j**0.5. The macro which expects a single complex input would be typically used in that situation. Its name is MATHLIB_SQMAG_frac24_m2p46. The complex input number must be made of a 24-bit fractional real part and a 24-bit fractional imaginary part. Both components must be available as single words in consecutive locations of the X memory, the first of it being addressed by a register (xp0). Hence, the first task of the code is to place both real and imaginary parts in memory, and set the pointer accordingly. Then, the macro can be called after setting the saturation mode.

```
Sys_Reg_frac24(x0, 5000000)      // Load real part (0.5) into x0
Sys_Reg_frac24(x1, 5000000)      // Load imaginary part (0.5) into x1
load xp0, 0x1000                 // Set address register xp0 to location
                                 //   X:0x1000
nop ||| store x[xp0++], x0       // Place real part into the location
                                 //   addressed by xp0
nop ||| store x[xp0--], x1       // Place imaginary part into the next
                                 //   memory location
Sys_CFX_SaturateEnable           // Set saturation mode
MATHLIB_SQMAG_frac24_m2p46(xp0)  // Call the macro
Sys_CFX_SaturateDisable          // Unset saturation mode
```

After the macro execution, the result for the squared magnitude (which is 0.5) is returned into the accumulator in the format m2p46. Hence the accumulator holds 0x200000|000000, which corresponds to the expected value divided by 2 if interpreted in the usual double fractional format. The possible next steps in the calculation should take account of this special format. A left shift might be possible later to restore the scale, but care should be taken about the data range. In this example, the overflow protection is set before the macro call (and reset afterwards) in order to saturate the result of the calculation in the special case where the input is -1-j.

5.6.2 Square Root

This example demonstrates calculating the square roots of the elements of a vector using the macro MATHLIB_SQRT_VEC_frac24_frac48. The input vector is made of 4 fractional values [0.01, 0.25, 0.5, 1] that are converted to 24-bit fractional form using Sys_Reg_frac24. The code below places those four values in X memory (from location X:0x1000 as controlled by the input pointer xp1), and runs the macro with its parameters. Since the outputs are 48-bit numbers, the output register xp2 must point somewhere in the XY memory. The register xp0 is used internally by the macro, so it cannot be used as an input / output parameter.

```
load xp1, 0x1000                 // Set address register xp1 to location
                                 //   X:0x1000
Sys_Reg_frac24(x0,   100000)     // Load first element into x0
nop ||| store x[xp1++], x0       // Move it to memory (first location)
Sys_Reg_frac24(x0,  2500000)     // Load second element into x0
nop ||| store x[xp1++], x0       // Move it to memory (second location)
Sys_Reg_frac24(x0,  5000000)     // Load third element into x0
```

```
nop ||| store x[xp1++], x0       // Move it to memory (third location)
Sys_Reg_frac24(x0, 10000000)     // Load fourth element into x0
nop ||| store x[xp1], x0         // Move it to memory (fourth location)
load xp1, 0x1000                 // Set register xp1 to the first element
                                 //   in the input vector
load xp2, 0x500                  // Set register xp2 to the output
                                 //   location XY:0x500
MATHLIB_SQRT_VEC_frac24_frac48(xp1, xp2, 4)    // Call the macro
```

After execution of this code, the result vector can now be read from XY memory, at location 0x500. The output values are [0x0CCE66400000, 0x40FE08000000, 0x5BE9BB000000, 0x7EFEFE000000] or [0.10005, 0.50775, 0.71807, 0.99216]. Their precision is not perfect because of the approximation (table lookup) used in the calculation, but they effectively correspond to the expected results [0.1, 0.5, 0.70711, 1].

You can use the SQRT macros for integer input as well. The integer input must be represented in one of the fixed-point numeric formats that has integer bits, such as m24p24, so that the SQRT macro's input value is in the fractional range. Also, since the fractional formats have a single integer bit while the mixed formats have an even number of integer bits, the operations are simpler if the input value is pre-scaled by an odd power of two.

The following example calculates the square root of 25. The input is stored in m24p24 format and then pre-scaled by 2, so the square root produced by MATHLIB_SQRT_frac24_frac48 is scaled by 2^{11} when interpreted as a fraction. Dividing by 2^{12} produces the result in m24p24 format.

```
load a0, 25                      // 25 as m24p24 or 25/(2^23) as frac24
shll a0, 1 ||| nop               // a0 is now 25/(2^22) in frac24 format
MATHLIB_SQRT_frac24_frac48       // Get result 5/(2^11) as frac48 in a0
shra a0, 12 ||| nop              // Divide result by 2^12 to get 5/(2^23)
                                 //   as frac24, or 5 in m24p24 format
```

The result in a0 after the division (the shift right) is 0x0000050CBC90, which represents 5.04975.

5.6.3 Cosine

The cosine of the angle $\pi/4$ is calculated in this example, using the scalar macro MATHLIB_COS_frac24_frac24. The input angle is expected in a0, and it must be normalized by π (that is with the range $[-\pi, \pi)$ being mapped to [-1, 1)). Hence, an original angle expressed in radians should be normalized before running the macro. Typically, this can be done as shown below. In the present case, the original angle $\pi/4$ is smaller than one, so it can be represented as a fractional. If the original radian angle is higher than 1, then it would be represented using a mixed format, and the normalization procedure should take this into account (see Section 5.6.4, "Sine" for the sine calculation).

```
Sys_Reg_frac24(x0, 31415927 / 4) // The input value (pi/4) is made
                                 //   available in x0 as a fractional
Sys_Reg_frac24(x1,  3183099)     // Load the normalization factor 1/pi in
                                 //   x1 as a fractional
mul a0, x0, x1 ||| nop           // Perform the product and get the
                                 //   normalized value in a0
nop                             // Delay, waiting for 2nd cycle of multiply
```

Here the normalized angle 1/4 (0x200000) is obtained in a0, and the cosine macro can be run, providing its fractional result (0.70711 or 0x5A827A) in a0 :

```
MATHLIB_COS_frac24_frac24        // Calculate cosine
```

5.6.4 Sine

This example shows the calculation of sin(-π/2) using the macro MATHLIB_SIN_frac24_frac24. Since the input angle is larger than 1, it is originally represented with the mixed format m4p20, where 4 bits are allocated to the signed integer part, and 20 bits to the unsigned fractional part (which is always positive). The frac24 format is m1p23, so the value of -π/2 in m4p20 format is computed using Sys_Reg_frac24 to convert the desired value divided by 8, or -π/16. The code is shown below:

```
Sys_Reg_frac24(x0, -31415927/16) // -pi/16 in frac24 is -pi/2 in m4p20
Sys_Reg_frac24(x1,   3183099)    // This is 1/pi in fractional format
mul a0, x0, x1 ||| nop           // Perform the product and get the
                                 //   normalized value -1/2 (0xF80000 as
                                 //   m4p20) in a0
nop                              // Delay, waiting for 2nd cycle of mul
shll a0, (4-1) ||| nop           // Convert the normalized angle -1/2 to
                                 //   fractional (a0 becomes 0xC00000)
MATHLIB_SIN_frac24_frac24        // Run the macro, which returns 0x800001
                                 //   (that is, about -1) in a0
```

5.6.5 Format Conversion

These examples demonstrate two types of format conversion. The first one illustrates a conversion from the 24-bit fractional fixed-point format (frac24) to the floating-point format (fl8n16). The vector macro MATHLIB_CONV_VEC_frac24_fl8n16 is used, with a 4-element input vector made of the values [0.5, 0.25, -1, -0.5]. After conversion, the outputs are [0x004000, 0xFF4000, 0x008000, 0xFF8000]. The pointer registers xp0 and xp1 are used to address the input and output memory locations, respectively, in X memory. The code is shown below:

```
load xp0, 0x1000                 // Set address register xp0 to location
                                 //   X:0x1000
Sys_Reg_frac24(x0,   5000000)    // Load first element into x0
nop ||| store x[xp0++], x0       // Move it to memory (first location)
Sys_Reg_frac24(x0,   2500000)    // Load second element into x0
nop ||| store x[xp0++], x0       // Move it to memory (second location)
Sys_Reg_frac24(x0, -10000000)    // Load third element into x0
nop ||| store x[xp0++], x0       // Move it to memory (third location)
Sys_Reg_frac24(x0,  -5000000)    // Load fourth element into x0
nop ||| store x[xp0], x0         // Move it to memory (fourth location)
load xp0, 0x1000                 // Set register xp0 to the first element
                                 //   in the input vector
load xp1, 0x1010                 // Set register xp1 to the output
                                 //   location X:0x1010
MATHLIB_CONV_VEC_frac24_fl8n16(xp0, xp1, 4)    // Call the macro
```

In the second example, let's perform the opposite conversion, but this time, a 48-bit fractional result is desired. The macro MATHLIB_CONV_VEC_fl8n16_frac48 is used as follows. The input register is xp1, and the output one is xp2, pointing at a 48-bit XY memory location.

```
load xp1, 0x1010                // Set register xp1 to the first element
                                //   in the input vector
load xp2, 0x500                 // Set register xp2 to the output
                                //   location XY:0x500
MATHLIB_CONV_VEC_fl8n16_frac48(xp1, xp2, 4)    // Call the macro
```

The returned output vector is [0x400000000000, 0x200000000000, 0x800000000000, 0xC00000000000], as expected.

5.6.6 Logarithm

This example considers the floating-point vector generated in the example above, and calculates its logarithm in base 2 using the macro MATHLIB_LOG2_VEC_fl8n16_m8p16. Only the positive values 0x004000 (0.5) and 0xFF4000 (0.25) are processed, addressed by xp1. The output is obtained at the location addressed by xp2 in X memory, and the vector length is set to 2:

```
load xp1, 0x1010                // Set register xp1 to the first element
                                //   in the input vector
load xp2, 0x1020                // Set register xp2 to the output
                                //   location X:0x1020
MATHLIB_LOG2_VEC_fl8n16_m8p16(xp1, xp2, 2)    // Call the macro
```

The resulting vector, expressed in mixed format m8p16, is [0xFF05BA, 0xFE05BA]. In this vector, the first number represents -1+0.0224 = -0.9776 since 0xFF is -1 (signed integer part) and 0x05BA is 0.0224 (unsigned 16-bit fractional value). The result is an approximation because of the table used to estimate the logarithm (log2(0.5) = -1). Similarly, the second number in the vector is -2+0.2224 = -1.9776 since 0xFE is -2.

5.6.7 Exponential

Using the macro MATHLIB_EXP2_VEC_m8p16_m8p16, this example calculates the base 2 exponent of an input vector. The input vector is the result from the previous example, [0xFF05BA, 0xFE05BA], so that the logarithm operation will be reversed (expecting [0.5, 0.25]). However, the result of the operation is now expressed in the mixed format m8p16. The input vector is addressed by xp1, while the output uses xp2. The call is as follows:

```
load xp1, 0x1020                // Set register xp1 to the first
                                //   element in the input vector
load xp2, 0x1030                // Set register xp2 to the output
                                //   location X:0x1030
MATHLIB_EXP2_VEC_m8p16_m8p16(xp1, xp2, 2)    // Call the macro
```

The resulting vector, expressed in mixed format m8p16, is [0x008165, 0x0040B2] = [0.5054, 0.2527].

5.6.8 Attack Release

An attack-release filtering operation (*y(n) = coeff*x(n) + (1-coeff)*y(n-1)* with *coeff = alpha* or *beta*) is illustrated in this example, assuming a 4-channel parallel process (4-element input and output vectors). The input vector is addressed by xp0, while xp1 points to the output. The macro MATHLIB_ATTACK_RELEASE_VEC is used, with xp2 pointing to the memory location that holds the value of alpha for the first channel. Note that alpha and beta must be written as consecutive pairs (beta following alpha), one pair per channel. In this example, they are written from location

X:0xFF0, and the same pairs are written 4 times since the same alpha and beta values are used in every channel. Assuming an initial input vector *x(n)* = [0x7FFFFF, 0x400000, 0x200000, 0x100000] available in location X:0x1000, the first call to the macro would be as follows, requiring some initialization steps (output vector y(n-1) and filter parameters alpha and beta):

```
// Initialize y(n-1) to zero for all channels
loop.2d 4, Init_Output_Label     // Set loop to repeat 4 times
load xp1, 0x1010                 // [ds] Set pointer xp1 to X:0x1010
nop                              // [ds]
Init_Output_Label:
    nop ||| clr x[xp1++]         // Clear the previous output y(n-1)

// Initialize alpha
loop.2d 4, Alpha_Init_Label      // Set loop to repeat 4 times
Sys_Reg_frac24(x0, 9843750)      // [ds] Set alpha to 0.984375
load xp2, 0x0FF0                 // [ds] Set pointer xp2 to X:0x0FF0
                                 //        (first alpha)
Alpha_Init_Label:
    nop ||| store x[xp2+=2], x0  // Set each channel's alpha in memory

// Initialize beta
loop.2d 4, Beta_Init_Label       // Set loop to repeat 4 times
Sys_Reg_frac24(x0, 7500000)      // [ds] Set beta to 0.75
load xp2, 0x0FF1                 // [ds] Set pointer xp2 to X:0x0FF1
                                 //        (first beta)
Beta_Init_Label:
    nop ||| store x[xp2+=2], x0  // Set each channel's beta in memory

// Set address registers and run macro
load xp0, 0x1000                 // Set pointer xp0: input vector x(n)
load xp1, 0x1010                 // Set pointer xp1: output vector y(n-1)
load xp2, 0x0FF0                 // Set pointer xp2: alpha and beta
MATHLIB_ATTACK_RELEASE_VEC(xp0, xp1, xp2, 4)    // Call the macro
```

The output found at location xp1 would be [0x7DFFFF, 0x3F0000, 0x1F8000, 0x0FC000]. Later, a subsequent call to the same macro with the new input vector [0x100000, 0x200000, 0x400000, 0x7FFFFF] would be simply:

```
// Set address registers and run macro
load xp0, 0x1000                 // Set pointer xp0: input vector x(n)
load xp1, 0x1010                 // Set pointer xp1: output vector y(n-1)
load xp2, 0x0FF0                 // Set pointer xp2: alpha and beta
MATHLIB_ATTACK_RELEASE_VEC(xp0, xp1, xp2, 4)    // Call the macro
```

In this particular case, since the values in the first two channels are now decreasing (release), beta is used in the calculation. On the other hand, the other two channels are in attack, and alpha is used. The output vector y(n) becomes [0x2B8000, 0x27C000, 0x3F7E00, 0x7E3EFF].

5.6.9 Reciprocal

Using the macro MATHLIB_RECIP_VEC_int24_fl8n16, this example calculates the reciprocal of the integer input vector [4, -2]. The input vector is addressed by xp1, while the output uses xp2. The call is as follows:

```
// Create 24-bit integer input vector [-4, 2] at location X:0x1000
store x[0x1000], 4
store x[0x1001], -2

// Set address registers and run macro
load xp1, 0x1000                // Set register xp1 to the first element
                                //   in the input vector
load xp2, 0x1010                // Set register xp2 to the output
                                //   location X:0x1010
MATHLIB_RECIP_VEC_int24_fl8n16(xp1, xp2, 4)    // Call the macro
```

The resulting vector, expressed in floating-point format, is [0xFE7E08, 0xFF81F8]. The first result is made of the signed exponent 0xFE = -2 and signed fractional mantissa 0x7E08 = 0.9846, which turns into the number 0.9846 * 2^{-2} = 0.2462. The second result is made of the signed exponent 0xFF = -1 and signed mantissa 0x81F8 = -0.9846. This represents -0.9846 * 2^{-1} = -0.4923. Since the reciprocal operation uses lookup tables, these results are approximations. A conversion routine, such as `MATHLIB_CONV_VEC_fl8n16_frac48`, can be used afterwards to convert the result to the desired format.

5.6.10 RMS to db10

This example shows the conversion from RMS values to decibels, using the macro MATHLIB_RMS_DB10_frac48_m8p16. This particular case converts the value 0.5 into 20*log10(0.5) = -6.02. After running the macro, the 48-bit input value in the accumulator is replaced with the result (0xFA1D34 = -6 + 0.114 = -5.89), expressed in the mixed format m8p16. The result is an approximation since the calculation of the logarithm in base 2 is performed by means of a lookup table.

```
Sys_Reg_frac24(a0, 5000000)     // Load accumulator with 0.5 in frac48
                                //   format; the bottom 24 bits are zero
MATHLIB_RMS_DB10_frac48_m8p16   // Call the macro
```

5.6.11 Division

In this example, the use of the divide macros is demonstrated. The macro MATHLIB_DIVIDE_frac24 first calculates the ratio 0.125 / 0.5 = 0.25. The dividend is expected in a0.h, while the divisor must be available in register x0. The quotient 0x200000 is returned into a0.l. The code is as follows:

```
Sys_Reg_frac24(a0, 1250000)     // Load accumulator with dividend 0.125
Sys_Reg_frac24(x0, 5000000)     // Load register x0 with divisor 0.5
MATHLIB_DIVIDE_frac24           // Call the macro
```

Please note that this macro expects a dividend smaller than the divisor (since the returned value is fractional). If this is not the case, the other macro, `MATHLIB_DIVIDE_m24p24`, must be used, as shown below. Here, calculation of the division 0.5 / 0.125 = 4 is performed. The result (4) is returned into the accumulator a0 in the form of the mixed m24p24 0x000004|000000.

```
Sys_Reg_frac24(a0, 5000000)     // Load accumulator with dividend 0.5
Sys_Reg_frac24(x0, 1250000)     // Load register x0 with divisor 0.125
MATHLIB_DIVIDE_m24p24           // Call the macro
```

Finally, another macro can be used to divide integers: MATHLIB_DIVIDE_int24. This macro provides the integer part of the result in a0.l. The example below illustrates the division 5/2:

```
load a0, 5                      // Load accumulator with dividend 5
load x0, 2                      // Load register x0 with divisor 2
MATHLIB_DIVIDE_int24            // Call the macro
```

The value 2 is returned into a0.l. Additionally, the instruction

```
add a0, x0, lt ||| nop
```

can be run after the macro. This is the final step to compute, in a0.h, the remainder of the division after multiplication by the divisor. The quotient in a0.l is not modified. Thus a0.h would get the value 1 in this example.

5.6.12 Linear Interpolation

This example illustrates the behavior of the linear interpolation macro MATHLIB_LIN_INTERP. The linear interpolation is performed between two data points available in two consecutive memory words. An address register (xp0 in this particular case) is expected to contain the address of the first point. The positive interpolation ratio is placed into a0 before running the macro. In the following code, the data points to be interpolated are t0 = 0.125 (0x100000) and t1 = 0.625 (0x500000), while the interpolation ratio is x = 0.75 (0x600000). The result is t0 + x*(t1-t0) = 0.5 (0x400000), returned into a0.

```
load xp0, 0x1000                // Set address register
Sys_Reg_frac24(x0, 1250000)     // Load 0.125 into x0
nop ||| store x[xp0++], x0      // Move it to memory
Sys_Reg_frac24(x0, 6250000)     // Load 0.625 into x0
nop ||| store x[xp0--], x0      // Move it to memory
Sys_Reg_frac24(a0, 7500000)     // Load 0.75 into a0
MATHLIB_LIN_INTERP(xp0)         // Run macro
```

This page left blank for printing purposes.

6 EEPROM Library

6.1 DESCRIPTION

The EEPROM library is provided as part of the firmware. The EEPROM library provides access to an EEPROM connected to the BelaSigna 300 device through the SPI port. The library is designed to work with the following EEPROM types:

1. Microchip 25AA256 and 25AA512
2. Atmel AT25256
3. ON Semiconductor CAT25256
4. ON Semiconductor custom E2M2 ASIC

These EEPROMs are 256 kbit or 512 kbit serial EEPROMs with related command sets. The EEPROMs support individual word writes (they are not limited to page writes).

The EEPROM library provides high-level functions for configuring, reading from and writing to the EEPROM. Low-level functions are internal to the EEPROM library and are not intended to be used by the user application.

6.2 EEPROM LIBRARY USAGE

EEPROM library functions are contained within a compiled library. Both RAM-based and ROM-based versions are available. For more information, see Section 2.4.1, “ROM-Based Libraries” on page 7.

You must include the associated system library include file, `sk3_eeprom.cfi` or `sk3_dual_eeprom.cfi`, in the source code. This include file contains the function declarations for the EEPROM library as well as associated macros. You must then configure your IDE to link in the appropriate compiled library.

The EEPROM library functions are designed to be called from within a user application. The user application initializes the EEPROM library and executes the desired functions. Parameters are passed to the EEPROM functions using registers. Return codes, if any, are returned to the calling application mostly through registers. The function interfaces are listed in the EEPROM function reference.

The library is designed to be used while interrupts are enabled or disabled. That is, the EEPROM library functions are interruptible. The EEPROM library itself does not use interrupts. This design allows the calling application to handle any other system tasks, such as signal processing, within ISRs during EEPROM operations.

6.2.1 Using Dual EEPROMs

If you have two EEPROMs, you can use the dual EEPROM library to perform operations on each of them. To distinguish one EEPROM from another, we use the following names:

Code EEPROM The EEPROM selected by using the standard SPI interface chip select pin

Data EEPROM The EEPROM selected by using a GPIO that has been assigned as a second SPI chip select pad for the BelaSigna 300 device

6.2.1.1 Hardware Configuration

To configure a dual-EEPROM system, connect the SPI clock, and serial input and serial output signals to both EEPROMs. Connect the SPI chip select signal to the chip select line of the code EEPROM, and connect GPIO 0 to the chip select line of the data EEPROM. For proper communications with the data EEPROM, GPIO 0 must operate at the same IO voltage level as the SPI interface (typically 1.8 V or higher). To obtain this higher voltage, you can supply the BelaSigna 300 IOs with 1.8 V, or externally level translate GPIO 0 to the required voltage level.

This hardware configuration can be demonstrated using the BelaSigna 300 Evaluation and Development Board, by connecting pins 1, 2 of the chip select header (SPI chip select for the code EEPROM, labelled `CS`) and connecting pin 4 of the chip select header (SPI chip select for the data EEPROM) with pin 1 of the GPIO header (GPIO 0). Because the development board does not include level translation, you must also supply VBAT with 1.8 V so that the IO voltage at GPIO 0 will be 1.8 V.

6.2.1.2 System Variable

To select an EEPROM, use the `SYSVAR_EE_SELECT` system variable. Set the `EE_SELECT` field to `EE_SELECT_CODE_EEPROM` to select the code EEPROM, or to `EE_SELECT_DATA_EEPROM` to select the data EEPROM.

If a function is designed to deselect an EEPROM, it deselects both EEPROMs. If the function is supposed to select an EEPROM, it checks the value of the `SYSVAR_EE_SELECT` system variable to determine which EEPROM to select.

6.2.1.3 Macros and Functions

All of the functions in the dual EEPROM library have the same names as the functions in the regular EEPROM library, even though some functions have been adapted to support two separate EEPROMs. As a result, you can only link to either the single EEPROM library or the dual EEPROM library, but not both.

All EEPROM operations after an `EELIB_Initialize` call operate on the same EEPROM. If you want operations to occur on the other EEPROM, you have to re-call the `EELIB_Initialize` function with the appropriate input. If you want to write to the selected EEPROM, you must also call the `EELIB_LibWriteEnable` function to re-enable writing.

6.2.2 EEPROM Library Naming Conventions

EEPROM library functions are named in mixed case preceded with a prefix of `EELIB_`. The operation follows the prefix. The operation specifies either a read operation, a write operation or another operation. Other operations include the library initialization, setup and configuration, as well as EEPROM write enabling and disabling.

Several functions have associated with them alternate functions that perform the same core operation but with different word sizes. These alternate functions are differentiated by the word size (in bits) following the function name. For example, `EELIB_ReadBlock_X` operates with the default word size (24 bits in this case) whereas `EELIB_ReadBlock_X16` operates with a word size of 16 bits.

6.3 WRITE PROTECTION

A mechanism provides protection against accidental EEPROM corruption or erasure: the EEPROM device has built-in block write protection (BWP). To allow the EEPROM library to write to the EEPROM, the application must specifically enable writing in the library. This is accomplished by calling the EEPROM library function `EELIB_LibWriteEnable` with the appropriate parameter. The parameter is the EEPROM write enable opcode (`AT_WREN`).

IMPORTANT: To ensure maximum protection against accidental EEPROM corruption or erasure, the parameter passed into the EELIB_LibWriteEnable function should not be stored in memory. Instead, the value should be passed into the device from an external system, where possible.

6.4 WORD SIZES

The EEPROM library supports multiple word sizes. The following word sizes are supported: 8 bits (single byte), 16 bits (two bytes), 24 bits (three bytes) and 32 bits (four bytes). This design facilitates multiple word sizes on BelaSigna 300. **The default word size for the EEPROM library is 16 bits**.

Functions that operate exclusively on the EEPROM use a 16-bit word size (unless otherwise indicated). Functions that operate through a specific memory space use that memory space's word size as the default. For example, `EELIB_ReadBlock_P` operates on program memory and thus uses a 32-bit word size.

Functions that use a smaller word size than the nominal word size for a particular memory space operate on the lower significant bits of the nominal word. When reading from the EEPROM to a memory space or register, any bits that are not set by the read are cleared to 0.

IMPORTANT: User applications should ensure that single-word EEPROM writes of any word size are contained within the boundaries of a single 64-byte EEPROM page. This affects all `EELIB_WriteWord` and `EELIB_WriteNextWord` function calls. If a user application calls these functions, and the number of bytes written by any one function call or sequence of function calls exceeds the bytes remaining in the currently selected EEPROM page, the excess bytes will wrap to the beginning of the EEPROM page buffer and be written to the beginning of the currently selected EEPROM page.

6.5 CYCLIC REDUNDANCY CHECK (CRC)

The EEPROM library provides a convenient and robust method to validate data read from or written to the EEPROM. All the EEPROM read and write functions automatically update a 16-bit CRC value. The initial CRC value must be set by the user application. The CRC value is stored in the D_CRC_VALUE register. The application can use this value to write the final CRC value of a block of data or to verify data when it is read from the EEPROM.

The initial value for the CRC should be 0xFFFF. The *Hardware Reference Manual for BelaSigna 300* has more information about the CRC.

6.6 EEPROM LIBRARY AND DUAL EEPROM LIBRARY FUNCTIONS

The EEPROM library and Dual EEPROM library functions are listed in Table 59.

TABLE 59: EEPROM LIBRARY AND DUAL EEPROM LIBRARY FUNCTIONS

Function	Description	EEPROM Library Reference	Dual EEPROM Library Reference
EELIB_Initialize	Initialize the EEPROM library	14.3 on p. 441	15.3 on p. 485
EELIB_Close	Close the EEPROM library	14.1 on p. 439	15.1 on p. 483
EELIB_LibWriteDisable	Write disable the EEPROM library	14.5 on p. 443	15.5 on p. 487
EELIB_LibWriteEnable	Write enable the EEPROM library	14.6 on p. 444	15.6 on p. 488
EELIB_WriteDisable	Write disable the EEPROM	14.34 on p. 472	15.34 on p. 516
EELIB_WriteEnable	Write enable the EEPROM	14.35 on p. 473	15.35 on p. 517
EELIB_ReadStatusRegister	Read the EEPROM status register	14.23 on p. 461	15.23 on p. 505
EELIB_WriteStatusRegister	Write the EEPROM status register	14.40 on p. 478	15.40 on p. 522
EELIB_VerifyEEPROMConnection	Verify that an EEPROM is connected to the SPI port	14.29 on p. 467	15.29 on p. 511
EELIB_PowerDown[1]	Attempt to place the EEPROM in power down mode	14.7 on p. 445	15.7 on p. 489
EELIB_PowerUp[1]	Place an EEPROM, that was in power down mode, into power up mode	14.8 on p. 446	15.8 on p. 490
EELIB_SelectSegment[1]	Select EEPROM segment 0 or 1 (address bit 17), *applicable to ON Semiconductor custom E2M2 ASIC only*	14.28 on p. 466	15.28 on p. 510
EELIB_ReadWord8	Initialize the EEPROM and read a word (either 8-bit, 16-bit, 24-bit, or 32-bit)	14.27 on p. 465	15.27 on p. 509
EELIB_ReadWord16		14.24 on p. 462	15.24 on p. 506
EELIB_ReadWord24		14.25 on p. 463	15.25 on p. 507
EELIB_ReadWord32		14.26 on p. 464	15.26 on p. 508
EELIB_ReadNextWord8	Read a word from the queued EEPROM (either 8-bit, 16-bit, 24-bit, or 32-bit)	14.22 on p. 460	15.22 on p. 504
EELIB_ReadNextWord16		14.19 on p. 457	15.19 on p. 501
EELIB_ReadNextWord24		14.20 on p. 458	15.20 on p. 502
EELIB_ReadNextWord32		14.21 on p. 459	15.21 on p. 503
EELIB_ReadBlock_P	Read a block of data from EEPROM to PMEM (32-bit or 16-bit)	14.9 on p. 447	15.9 on p. 491
EELIB_ReadBlock_P16		14.10 on p. 448	15.10 on p. 492
EELIB_ReadBlock_X	Read a block of data from EEPROM to XMEM (24-bit or 16-bit)	14.11 on p. 449	15.11 on p. 493
EELIB_ReadBlock_X16		14.12 on p. 450	15.12 on p. 494
EELIB_ReadBlock_Y	Read a block of data from EEPROM to YMEM (24-bit)	14.13 on p. 451	15.13 on p. 495

TABLE 59: EEPROM LIBRARY AND DUAL EEPROM LIBRARY FUNCTIONS (CONTINUED)

Function	Description	EEPROM Library Reference	Dual EEPROM Library Reference
EELIB_ReadNextBlock_P EELIB_ReadNextBlock_P16	Read the next block of data from a queued EEPROM to PMEM (32-bit or 16-bit)	14.14 on p. 452 14.15 on p. 453	15.14 on p. 496 15.15 on p. 497
EELIB_ReadNextBlock_X EELIB_ReadNextBlocK_X16	Read the next block of data from a queued EEPROM to XMEM (24-bit or 16-bit)	14.16 on p. 454 14.17 on p. 455	15.16 on p. 498 15.17 on p. 499
EELIB_ReadNextBlock_Y	Read the next block of data (24-bit) from a queued EEPROM to YMEM	14.18 on p. 456	15.18 on p. 500
EELIB_InitWriteWord	Initialize the EEPROM for writing at the specified address	14.4 on p. 442	15.4 on p. 486
EELIB_WriteWord8 EELIB_WriteWord16 EELIB_WriteWord24 EELIB_WriteWord32	Initialize the EEPROM for writing and send one word (8-bit, 16-bit, 24-bit, or 32-bit)	14.44 on p. 482 14.41 on p. 479 14.42 on p. 480 14.43 on p. 481	15.44 on p. 526 15.41 on p. 523 15.42 on p. 524 15.43 on p. 525
EELIB_WriteNextWord8 EELIB_WriteNextWord16 EELIB_WriteNextWord24 EELIB_WriteNextWord32	Write the next word to the EEPROM (8-bit, 16-bit, 24-bit, or 32-bit)	14.39 on p. 477 14.36 on p. 474 14.37 on p. 475 14.38 on p. 476	15.39 on p. 521 15.36 on p. 518 15.37 on p. 519 15.38 on p. 520
EELIB_WriteBlock_P	Write a block of data (32-bit) from PMEM to EEPROM	14.31 on p. 469	15.31 on p. 513
EELIB_WriteBlock_X	Write a block of data (24-bit) from XMEM to EEPROM	14.32 on p. 470	15.32 on p. 514
EELIB_WriteBlock_Y	Write a block of data (24-bit) from YMEM to EEPROM	14.33 on p. 471	15.33 on p. 515
EELIB_WriteBlock_Const32	Write a block of data from one PMEM word to EEPROM	14.30 on p. 468	15.30 on p. 512
EELIB_Commit	Commit a block of data that has been written to the EEPROM from the EEPROM's internal buffer to its non-volatile memory	14.2 on p. 440	15.2 on p. 484
Sys_Dual_EEPROM_CS_Data_EEPROM	Set the GPIO chip select to low if SYSVAR_EE_SELECT == EE_SELECT_DATA_EEPROM (== 1)	None	15.45 on p. 527
Sys_Dual_EEPROM_CSs_High	Set chip select high on both EEPROMs.	None	15.46 on p. 528
Sys_Set_Dual_EEPROM_SPI_CTRL	Set the `SPI_CTRL` register bits. Select the code EEPROM if SYSVAR_EE_SELECT = EE_SELECT_CODE_EEPROM.	None	15.47 on p. 529

TABLE 59: EEPROM LIBRARY AND DUAL EEPROM LIBRARY FUNCTIONS (CONTINUED)

Function	Description	EEPROM Library Reference	Dual EEPROM Library Reference
Sys_SetReg_Dual_EEPROM_Sel_EEPROM	Set the selected EEPROM to value in `reg`.	None	15.48 on p. 530
Sys_SetReg_Dual_EEPROM_SPI_CTRL	Set the `SPI_CTRL` register bits based on a register. Select the code EEPROM if SYSVAR_EE_SELECT = EE_SELECT_CODE_EEPROM.	None	15.49 on p. 531

1. Not available in the ROM-based library.

6.7 EEPROM LIBRARY OPERATION

The EEPROM library supports block read, word read, and status register read operations. Figure 2 illustrates how the EEPROM library can be used to perform a read operation or multiple read operations.

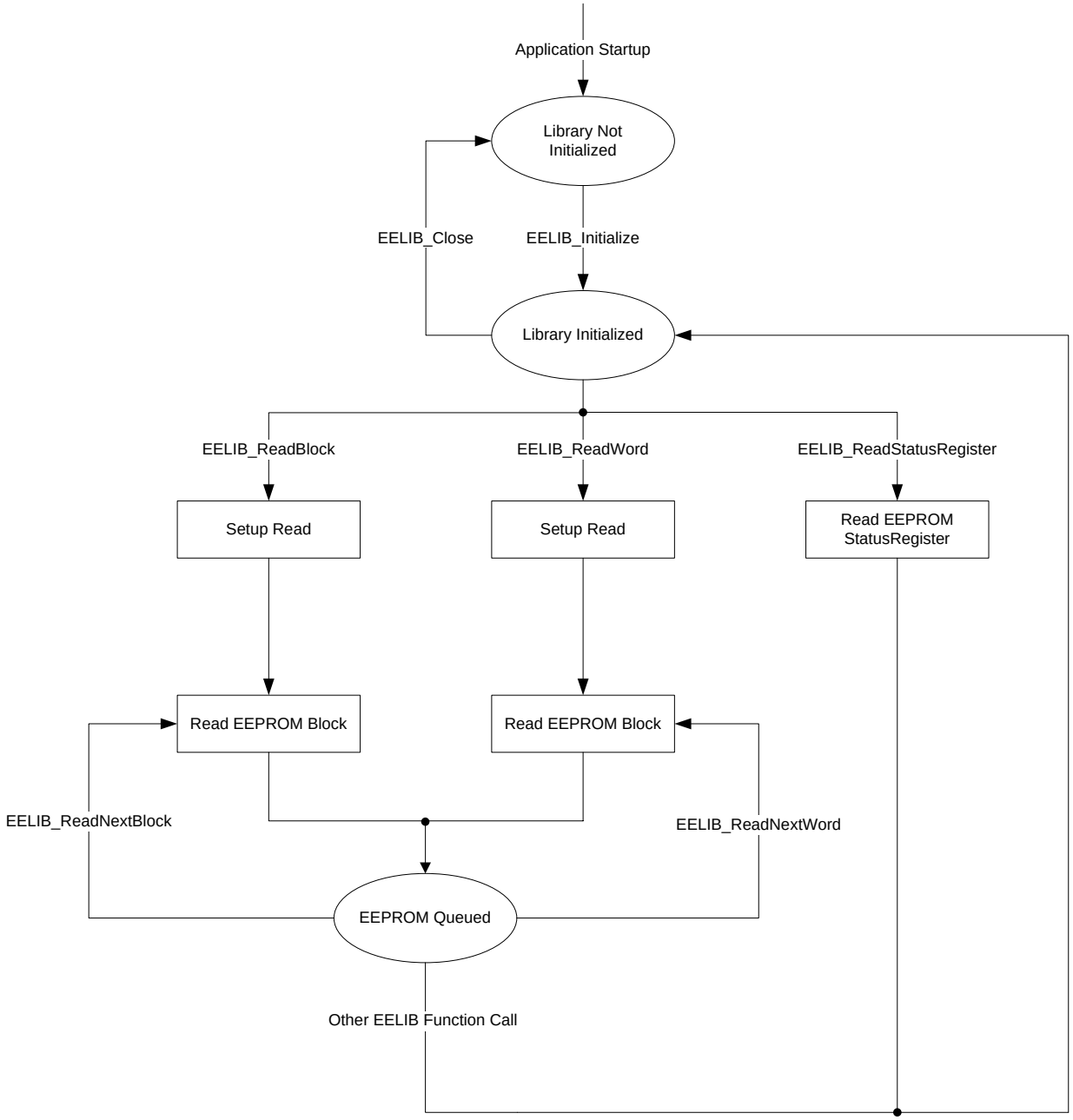


Figure 2: EEPROM Library Read Operation

The EEPROM library supports block write, word write, and status register write operations. Figure 3 illustrates how the EEPROM library can be used to perform a write operation or multiple write operations.



Figure 3: EEPROM Library Write Operation

7 EEPROM File System

7.1 DESCRIPTION

The file system securely and efficiently stores system data, system code, application code and application data in non-volatile memory in an EEPROM. System data includes specific clock calibration settings, manufacturing information and application selection. System code is the bootloader. Having multiple stored applications with shared data and code areas is supported. Security is provided through AES encryption.

The file system is designed for a non-volatile memory (EEPROM) attached through the SPI port. The EEPROM must provide byte read and write capability. The EEPROM sizes supported are 128 kbit, 256 kbit and 512 kbit.

7.2 VERSION

The file system version is identified with a 16-bit identifier. The identifier format is shown in Table 60.

TABLE 60: FILE SYSTEM VERSION

Bits 15-8	Bits 7-0
Major Version	Minor Version

Minor changes to the file system will increase the lower byte of the file system version. Major changes to the file system will increase the upper byte of the file system version.

The EEPROM contains the file system version variable in the system area. The value stored in the EEPROM identifies the format of the accompanying data. The value is checked by the bootloader to ensure compatibility between the installed bootloader and filesystem. If the bootloader does not support the corresponding file system version, an error is generated.

The file system format described here corresponds to file system version 1.2.

7.3 WORD SIZE AND ADDRESSING

The file system supports multiple word sizes depending on the type of data. The default word size is 16 bits. System data stored in the EEPROM always consists of 16-bit words. The EEPROM library supports file system word sizes by providing functions to read or write 8-bit, 16-bit, 24-bit or 32-bit words.

Block reads and writes generally require the same word size as the memory space the function is trying to read from or write to. Data words that are destined to the CFX X memory and Y memory are 24 bits wide. Data words that are destined for CFX program memory are 32 bits wide. To support these various widths, several block read and write macros are provided.

All EEPROM addressing is byte based. File system address vectors—which are used to point to the start of sections and application files in the system—correspond to absolute byte addresses on the EEPROM. Address vectors are system data and thus use 16-bit words. A single address vector provides an addressable range between 0x0000 and 0xFFFF. A 256 kbit EEPROM has 32768 addressable locations (0x0000 to 0x7FFF) and a 512 kbit EEPROM has 65536 addressable locations (0x0000 to 0xFFFF).

7.4 FILE SYSTEM INTEGRITY

Each application file, code section, and data section is always accompanied by a 16-bit CRC value. The CRC value helps to ensure file system integrity. The CRC value corresponds to the CRC-CCITT value calculated over the preceding data. The CRC-CCITT parameters are listed in Table 61.

TABLE 61: FILE SYSTEM CRC PARAMETERS

CRC Parameter	Value
Bits	16
Polynomial	x^16 + x^12 + x^5 + 1 (0x1021)
Initial Value	0xFFFF
Final XOR Value	0x0000 (No change due to XOR)

Hardware support is available for calculating the CRC values that accompany each section; see the *Hardware Reference Manual for BelaSigna 300* for more details.

In addition to the CRC value accompanying each section, the file system integrity is ensured by verifying that certain integer and address values are within specified ranges.

7.5 SECURITY

The file system provides optional security through the use of AES encryption. Encrypted sections are unreadable unless the shared user key is known. The user key is stored in the file system. For more information on encrypted sections see Section 7.11.5, “Encrypted Sections” on page 86.

7.5.1 Level 1 Security

Level 1 security is recommended when the device package does not physically prevent direct access to the EEPROM. In this configuration, an external system may read the user key data directly. Level 1 security provides a two-stage encryption process to secure the application code. To prevent unauthorized decryption of the section data, a user key (i.e., one created by the developer) is used to encrypt the section data. This key is, in turn, encrypted with a secret shared key. This shared key is assigned by ON Semiconductor and is unique for each customer. The shared key is used to encrypt the user key and bootloader. The index to the shared key is set in the BOOT_INFO variable in the system area.

During boot, the bootloader and user key are decrypted with the shared key which is only accessible by the Program ROM. The shared key cannot be accessed by user applications and hence the shared key remains secret.

Once decrypted, the user key is used by the bootloader to decrypt encrypted sections of the application. The bootloader itself is encrypted (with the shared key) in order to keep the user key secret.

IMPORTANT: Shared keys are acquired from ON Semiconductor. For more information please contact your ON Semiconductor representative.

7.5.2 Level 2 Security

Level 2 security is suitable when the device package physically restricts direct access to the EEPROM. In this configuration, the user key is stored in the EEPROM in plain text. The user key is used by the bootloader to decrypt all encrypted sections. Since the EEPROM is not externally accessible, any attempts to read the user key data must go through the debug port. By default, the debug port is in restricted mode, which prevents access to the content of the EEPROM from an external device. Switching the debug port out of restricted mode requires that the contents of the key area be wiped.

Once the user key has been wiped the remaining data can not be deciphered since the user key is unavailable. In this mode the system will not boot properly and the user key must be rewritten to the EEPROM in order to boot the EEPROM application.

7.6 FILE SYSTEM COMPONENTS

The file system consists of a number of different components. The components are listed below.

- System area
- Manufacturing area
- Key area
- Application files
- Code sections
- Data sections

Together, these components form the file system. The system area and manufacturing area contain high-level application information as well as system-specific information. The application files describe the applications stored on the EEPROM. Application files contain pointers to their respective code and data sections. Code and data sections may be shared amongst different applications.

Code sections are blocks of data consisting of 32-bit words destined for program memory. Data sections are blocks of either 24-bit or 32-bit words destined for either X memory, Y memory or program memory. The words in the system area, manufacturing area, key area and application files are all 16-bit.

The system area, manufacturing area and key area addresses are fixed in the file system. The file system map is shown in Table 62. The application files, code sections and data sections may be placed at any available location in the EEPROM. The address vectors for the application files are stored in the system area.

TABLE 62: FILE SYSTEM MAP

EEPROM Byte Address	Size (Bytes)	Component
E:0x0000-E:0x001F	32	System area
E:0x0020-E:0x007F	96	Manufacturing area
E:0x0080-E:0x00FF	128	Key area
E:0x0100-E:0x7FFF	32512	Available (256 kbit EEPROM)
E:0x0100-E:0xFFFF	65280	Available (512 kbit EEPROM)

The total size of the reserved EEPROM space is 256 bytes (from 0x0000 to 0x00FF). User code and data and the bootloader are free to use any address above 0x00FF. Generally, the bootloader will be placed at address 0x0100 (the first available address).

An overview of the file system is displayed in Figure 4.

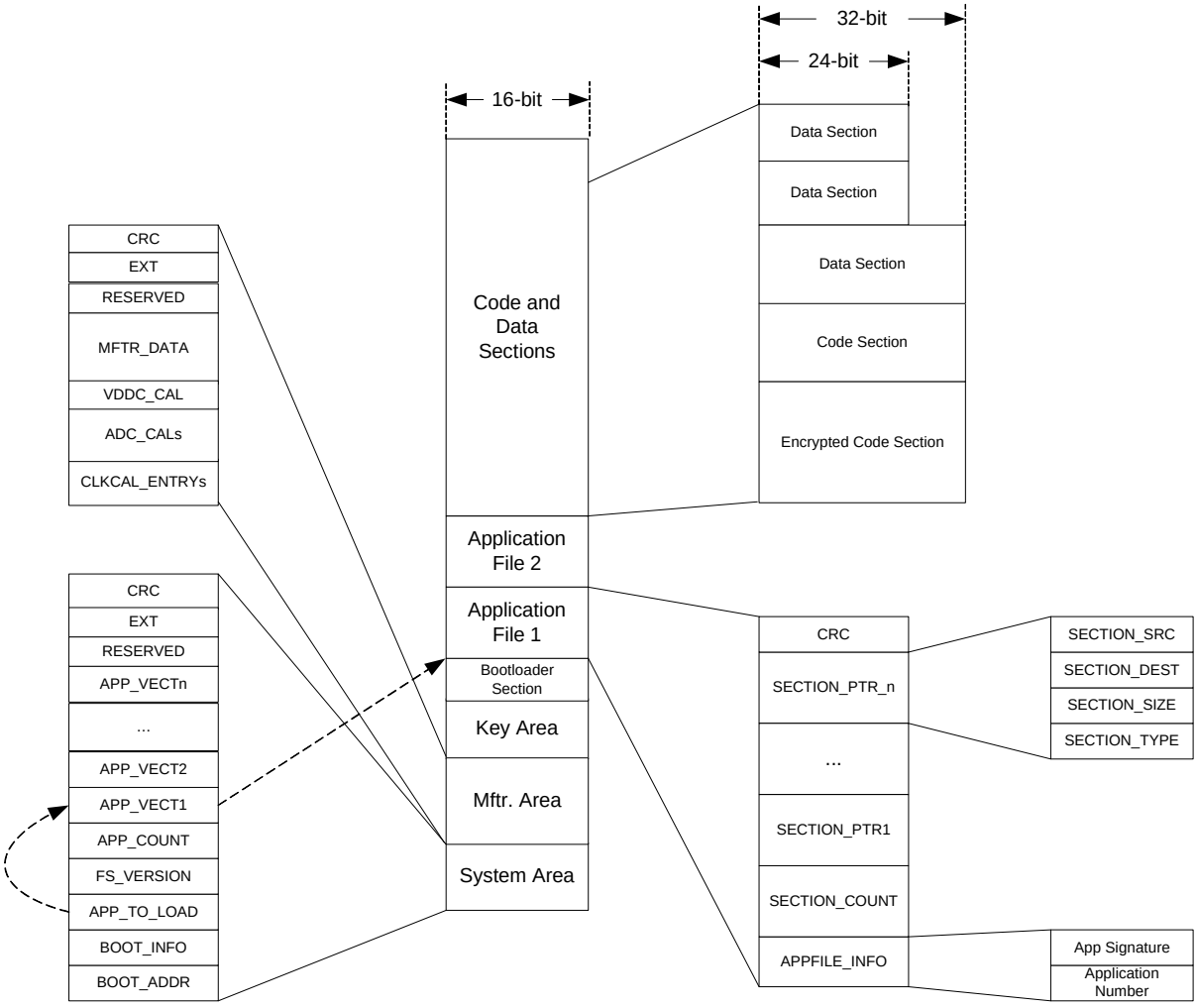


Figure 4: File System Overview

7.7 SYSTEM AREA

The system area contains system variables and address vectors. The variables specify the file system version, boot speed, encryption key index, and information on the stored applications. The address vectors indicate where in the EEPROM the application files and the bootloader are stored. Table 63 lists the entries in the system area.

TABLE 63: SYSTEM AREA

EEPROM Byte Address	Size (Bytes)	System Variable	Description
E:0x0000	2	BOOT_ADDR	Address vector for bootloader
E:0x0002	2	BOOT_INFO	Bit 15: EEPROM boot speed Bits 7-0: Encryption key index
E:0x0004	2	APP_TO_LOAD	Application # to load during boot
E:0x0006	2	FS_VERSION	File system version
E:0x0008	2	APP_COUNT	Number of applications in EEPROM
E:0x000A-E:0x0019	16	APP_VECT1 … APP_VECTn	Application vectors
E:0x001A	2	Reserved	Reserved
E:0x001C	2	SYS_AREA_EXT	System Area Extension vector
E:0x001E	2	SYS_AREA_CRC	CRC of system area

7.7.1 Boot Variables

The system area contains several words required for booting the system. The address of the bootloader is stored as the first word of the system area. BOOT_ADDR specifies the byte address of the bootloader. The boot ROM code uses this address to load the bootloader from EEPROM to RAM at system start-up. The bootloader code is organized as a special code section; it may also be encrypted.

The boot speed and encryption key index are stored in the BOOT_INFO word. The high bit (bit 15) specifies whether the bootloader should be read with fast or slow SPI clock. By default, the EEPROM is accessed with a slow SPI clock. After reading BOOT_INFO, the SPI clock speed can be increased to speed up overall booting speed. Since the on-chip oscillator is not calibrated until later in the boot process, the typical SPI speed may vary. Table 64 describes the settings for the BOOT_INFO speed bit.

TABLE 64: BOOT_INFO VALUES

Bit 15	Description	Typical SPI speed
0	Slow SPI speed	123 kbps
1	Fast SPI speed	980 kbps

The low 8 bits of BOOT_INFO specify whether the bootloader code section and user key are encrypted. A value of 0 indicates that the code section and user key are not encrypted. Otherwise, the value indicates the key index of the secret shared key to use for decryption. For more information on file system security see Section 7.5, “Security” on page 76.

7.7.2 File System Version

The system area includes the FS_VERSION word that indicates the file system version of the data in the EEPROM. This value may be used by the firmware to distinguish between various file system formats. See Section 7.2, “Version” on page 75 for more information.

7.7.3 Application Variables

The system area contains application variables that store information about the applications stored in the EEPROM. These variables are used by the bootloader to load the applications from EEPROM to RAM.

Applications are identified by sequential integer values. The first application stored in the EEPROM is application 1. The second application is application 2, and so forth.

The APP_COUNT variable specifies the total number of applications. The APP_TO_LOAD variable specifies the default application to boot by its application number. Each application stored has an associated APP_VECT in the system area. The APP_VECTn variables contain pointers to the associated application files stored in the EEPROM. For example, application 1 would contain an application file, and a pointer to that application file would be stored at APP_VECT1 (E:0x000A). Application 2 would contain a different application file from application 1. A pointer to application 2’s application file would be stored at APP_VECT2 (E:0x000C).

The system area supports a maximum of 8 separate applications. For more information on applications, see Section 7.11, “Applications” on page 84.

7.7.4 System Area Extension Vector

The system area includes a reserved vector at SYS_AREA_EXT (E:0x001C) for possible future extensions of the system area. This reserved vector is not used for file system version 1.1, and for this file system version, must be left as 0x0000.

7.8 MANUFACTURING AREA

The manufacturing area contains clock calibration information and other manufacturing specific data. This data is usually set during manufacturing and it should not be modified by users or applications. The clock calibration information is loaded from EEPROM to RAM during the normal boot procedure by the bootloader. Table 65 lists the entries in the manufacturing area.

TABLE 65: MANUFACTURING AREA

EEPROM Byte Address	Size (Bytes)	System Variable	Description
E:0x0020	2	CLKCAL_DEFAULT	Default clock calibration entry
E:0x0022-E:0x002F	14	CLKCAL_ENTRY1 … CLKCAL_ENTRY7	Additional clock calibration entries
E:0x0030-E:0x0037	8	ADC0_CALn	ADC0 calibration entries
E:0x0038-E:0x003F	8	ADC1_CALn	ADC1 calibration entries

TABLE 65: MANUFACTURING AREA

EEPROM Byte Address	Size (Bytes)	System Variable	Description
E:0x0040-E:0x0047	8	ADC2_CALn	ADC2 calibration entries
E:0x0048-E:0x004F	8	ADC3_CALn	ADC3 calibration entries
E:0x0050	1	ADC_CAL_VALID	Valid marker for ADC calibration entries
E:0x0051	1	VDDC_CAL	VDDC calibration
E:0x0052-E:0x0079	40	MFTR_DATA	Manufacturing data
E:0x007A	2	Reserved	Reserved
E:0x007C	2	MFTR_EXT	Manufacturing Area Extension
E:0x007E	2	MFTR_AREA_CRC	CRC of manufacturing area

IMPORTANT: For Ezairo 5900 hybrid packages produced by ON Semiconductor, the Manufacturing Area is programmed during hybrid production testing and must not be modified. For other packages, this same structure is also recommended because the standard firmware libraries use the data organization stored in this area for system calibration.

7.8.1 Clock Calibration

The manufacturing area contains eight 16-bit words to store clock calibration data. The clock calibration data is specific to individual chips. During manufacturing the clock calibration values for all valid operating frequencies are determined and stored in the appropriate area. The first clock calibration entry is the default entry and is used by the bootloader to initially calibrate the clock. Unused clock calibration entries are set to 0x0000.

Each calibration entry includes the value to write to the `A_CLK_CTRL` register. The value contains both the coarse and fine settings for the oscillator. The calibration entry also includes the corresponding operating frequency. The operating frequency is denoted with a value corresponding to a SYS_FREQ_* define. The format for each clock calibration entry is shown in Table 66.

TABLE 66: CLOCK CALIBRATION ENTRY FORMAT

Bits 15-12	Bits 11-8	Bits 7-0
Operating Frequency (SYS_FREQ_*)	SYSCLK value	CCR_DATA value

For more information on operating frequencies and clock calibration, see Section 2.9, “Operating Frequencies” on page 13.

7.8.2 ADC Calibration

The manufacturing area contains eight ADC calibration settings for each ADC, corresponding to each of the eight pre-amplifier configurations.

The bootloader loads this calibration table from non-volatile memory to firmware reserved Y-memory where it is used by Sys_CalibrateADCs. The calibration table is considered valid if bit 7 of the byte containing the valid marker for ADC calibration entries (loaded with the rest of the table) is set.

7.8.3 VDDC Calibration

The manufacturing area contains a one-byte VDDC calibration setting. The bootloader uses this value to configure the VDDC setting in `A_PSU_CFG` if the valid marker in bit 7 of this byte is set.

7.8.4 Manufacturing Area Extension Vector

The manufacturing area includes a reserved vector at MFTR_EXT (E:0x007C) for possible future extensions of the manufacturing area. This reserved vector is not used for file system version 1.1, and for this file system, must be left as 0x0000.

7.9 KEY AREA

The key area is reserved exclusively to store user encryption keys and other secure data. The keys are used by the bootloader to decrypt encrypted code sections in user applications. For increased security the keys themselves may be encrypted using the ROM-based shared key. For more information on file system security see Section 7.5, “Security” on page 76. Table 67 lists the entries in the key area.

TABLE 67: KEY AREA

EEPROM Byte Address	Size (Bytes)	System Variable	Description
E:0x0080	2	KEY_INFO	User key information
E:0x0082-E:0x00A1	32	KEY_DATA[n]	User encryption keys
E:0x00A2-E:0x00FF	94	RESERVED	Reserved

The KEY_INFO variable indicates whether KEY_DATA is encrypted. Table 68 lists the possible KEY_INFO values.

TABLE 68: KEY_INFO VALUES

KEY_INFO	Description
0	No user key
1	Encrypted user key
2	Plain-text user key

The key area is bounded on a 128 byte page boundary and is 128 bytes in size. This is required for compatibility with some 512 kbit EEPROMs that only support page writes.

7.10 BOOTLOADER

The bootloader is a special code section stored in the EEPROM. The bootloader code is automatically loaded from EEPROM to RAM at boot time by the boot ROM. The bootloader can be placed at any available location in the EEPROM. Generally, the bootloader is placed at E:0x0100 (the first available address after the reserved areas). The BOOT_VECT word in the system area points to the first word of the bootloader section. The bootloader consists of a special format single code section. Table 69 lists the format of the section. The bootloader section contains the bootloader size, destination address, data, and a CRC. The data may be encrypted.

TABLE 69: BOOTLOADER SECTION

Byte Offset	Bootloader Variable	Description
0	BOOT_SIZE	Size of the bootloader data (in 32-bit words)
2	BOOT_DEST	Destination address
4	Data	Bootloader data
4 + 4 * BOOT_SIZE	CRC	CRC of the bootloader section

For more information on the bootloader, see Chapter 9, “Bootloader” on page 97.

7.11 APPLICATIONS

The file system may store up to 8 different applications. Each application consists of an application vector, application file, and sections. The application vector is stored in the system area and contains a pointer to the associated application file. The application file contains section pointers to the associated sections. The application sections may contain code or data and are loaded into either X memory, Y memory or program memory.

Multiple applications may share section data. This reduces overall EEPROM usage as it allows common parameters and code to be shared between different applications.

7.11.1 Application Files

The application file describes each of the sections associated with a particular application. The sections may be code or data destined for X memory, Y memory or program memory. Each section is described in the application file by a section pointer. The application file is made up of a header and multiple section pointers. The header includes a signature, application number and a count indicating the total number of sections in the application. The section pointers are continuous in the application file. The application file contains a CRC calculated based on all the preceding words in the application file.

TABLE 70: APPLICATION FILE FORMAT

Byte Offset	Variable	Description
0	APPFILE_INFO	Application number and signature
2	SECTION_COUNT	Number of sections in the application

TABLE 70: APPLICATION FILE FORMAT

Byte Offset	Variable	Description
4	SECTION_PTR[n]	Section pointers
4 + 8 * SECTION_COUNT	CRC	CRC of the application file

The APPFILE_INFO word contains two fields. The high byte of the word contains an application signature. The application signature value is a constant value of 0xAA. The application signature is used to further verify file system integrity. The low byte of APPFILE_INFO is the application number.

Applications are limited to a maximum of 256 sections. An application specifying a SECTION_COUNT exceeding 256 is considered invalid.

Table 71 lists the contents of each section pointer in the application file.

TABLE 71: SECTION POINTER FORMAT

Byte Offset	Variable	Description
0	SECTION_TYPE	Indicates the type of data
2	SECTION_SIZE	Size of the section (in words)
4	SECTION_DEST	Destination address for the section data
6	SECTION_SRC	EEPROM address of the section data

The section type describes the type of data associated with the section. The section types are listed in Table 72. The size of the section is specified in the native word size depending on the section type.

TABLE 72: SECTION TYPES

SECTION_TYPE	Identifier	Word Size	Description
0	STD_X	24-bit	Data destined for X memory. The CRC must be valid.
1	VOL_X	24-bit	Data destined for X memory. The CRC is calculated but its validity is not critical.
2	STD_Y	24-bit	Data destined for Y memory. The CRC must be valid.
3	VOL_Y	24-bit	Data destined for Y memory. The CRC is calculated but its validity is not critical.
4	STD_P	32-bit	Data destined for program memory. The CRC must be valid.
5	ENC_P	32-bit	Data destined for program memory. The data is encrypted with the user key. The CRC must be valid.

7.11.2 Application Execution

During the boot process, the final step of the bootloader is to execute the loaded application. The bootloader selects the start address of the application using the first program memory section

pointer in the application file. Hence, when organizing an application in the file system the first program memory section pointer must correspond to the main entry point of the application.

7.11.3 Sections

The section pointer indicates the EEPROM address for the section data. The section pointer contains all of the information about the data. The section data itself only contains the data to be loaded into XMEM, YMEM, or PMEM, and a CRC. Table 73 shows the section format.

The size of the data in words is specified in the SECTION_SIZE word of the section pointer. The size in EEPROM of the section data is determined based on the size in words and the word size. The word size is based on the destination memory space.

The last word of the section contains a 16-bit CRC value. The CRC value is a CRC calculated over the actual EEPROM data.

TABLE 73: SECTION FORMAT

Byte Offset	Description
0	Section Data
SECTION_SIZE * WORD_SIZE	CRC of section

7.11.4 Volatile Sections

The file system has provisions for data sections that may have an incorrect CRC. The volatile section type indicates a non-critical error should be generated when the CRC does not match the data. A non-critical error does not prevent the rest of the application from being loaded and subsequently executed. This section type is useful for parameters that may be updated in the field. If, during a programming cycle, the EEPROM loses power or otherwise does not program correctly, the data being written may become corrupt. In this case, the bootloader will detect the incorrect CRC on the subsequent boot. For non-volatile sections, an error will be generated and the bootloader will not start the application. For volatile sections, an error is generated but the bootloader will start the application. The application must check the error code and contain provisions for re-initializing the corrupt section.

7.11.5 Encrypted Sections

The file system supports sections that are encrypted. Encrypted sections are denoted in the section pointer with the ENC_P section type. The encryption key used for encrypted sections is stored in the key area. During the boot process the key is read from EEPROM and used to decrypt all encrypted sections.

Encrypted sections have two associated CRCs. First, the section data CRC is calculated based on the encrypted data stored in the EEPROM. This CRC must be verified before the data is decrypted. Further, the plain-text data contains an additional CRC used to verify the plain-text data after decryption.

Encrypted sections are only valid for code and data destined for program memory. **If encrypted X memory or Y memory data is desired, the data must be encoded into a program memory data section and later manually copied by the application into the appropriate memory space.**

Encrypted sections must be padded to 128 bits. Since encrypted sections are always destined for program memory, this corresponds to four 32-bit words. The last word of the plain-text data is the plain-text CRC. The decryption is done in place in program memory.

IMPORTANT: A program memory block will be padded with up to 3 additional words if the section is encrypted. This is required because AES encryption and decryption operate on 128-bit (four 32-bit word) data blocks. Take care to ensure that the added program memory words do not overwrite other program segments in program memory with the extra padding when the data is being loaded from the EEPROM. One possible way to ensure this is to space out the program memory sections by three or more words to account for this padding. Alternatively, the section pointers in the application file can be set up in increasing order based on the destination address to ensure that even if sections overlap, only the padding data is overwritten.

This page left blank for printing purposes.

8 Program ROM

8.1 DESCRIPTION

The Program ROM is a ROM mapped into program memory. It contains various elements of firmware code. The primary purpose of the Program ROM is to facilitate booting the system, but it also contains firmware code required for security. In addition, many of the firmware libraries are duplicated in the Program ROM so that ROM-based functions can be used instead of RAM-based ones, reducing the firmware footprint in the RAM and EEPROM spaces. The Program ROM provides access to the ROM instances of the firmware code, internally through function tables and externally through the debug port's JumpROM facility. For more information about the JumpROM facility and file system security, see the *CFX Communication Protocols Manual*.

8.1.1 Versions

The Program ROM version is a 16-bit symbol that can be read from the `SYSVAR_PROGROM_VER` variable. This value can be used directly in the application code to identify the version of the ROM available on your device. An external application or device can also read the ROM version from the `SYSVAR_PROGROM_VER` variable or through the JumpROM command. See Section 2.8, "Versions" on page 12 for more information about version symbols. Table 74 lists the versions of the Program ROM referenced by this manual.

TABLE 74: PROGRAM ROM VERSIONS

Device	Version
BelaSigna 300 version 1.02.xx	4.00
BelaSigna 300 version 2.01.xx	4.00

8.2 BOOT ROM

After Power-on-Reset (POR), program execution begins in the Program ROM. The start of the Program ROM contains the boot ROM code.

8.2.1 Boot Initialization

The boot ROM performs basic system initialization required for booting the system. The system initialization consists of the following:

- CFX status register reset
- CFX loop stack reset
- CFX stack pointer initialized
- Watchdog timeout set to maximum (nominally 3.2 seconds)
- GPIO pins configured for GPIO input
- Operating frequency set to default
- Enabling power supply to analog components (if disabled)

- Clocking configured: selects the internal oscillator for all BelaSigna 300 devices with CID 102; external oscillator for BelaSigna 300 devices with CID 201
- Firmware global variables initialized

8.2.2 Stabilization Delay

After the system initialization, the boot ROM implements a power stabilization delay. This delay is intended to prevent spurious system resets due to voltage transients when connecting a battery. The power stabilization delay is nominally 168 ms (minimum 100 ms, maximum 260 ms). If desired, the application may introduce an additional delay before starting audio output.

8.2.3 Loading the Bootloader

The boot ROM proceeds to load the bootloader from EEPROM. During the boot process the boot ROM checks various file system parameters and the bootloader CRC to verify data integrity. If the bootloader is encrypted, the boot ROM executes the appropriate decryption functions in the Program ROM. The selected application (APP_TO_BOOT) is also loaded from EEPROM and stored in RAM. The boot ROM finally releases control of the system to the loaded bootloader.

8.2.4 Boot Failures

A boot failure may occur if the file system does not contain a valid bootloader image. In the case of a failed boot, the boot ROM implements a delay and proceeds to reboot the system. The system error code indicates the particular error. The delay before resetting the system is nominally 1685 ms (minimum 1000 ms, maximum 2600 ms). See Section 2.11.2, “Boot Errors” on page 15 for more information on boot errors.

For security, the BelaSigna 300 debug port is initially in restricted mode. Program ROM functions are provided to wipe the EEPROM key area and then unlock the device. However, for convenience, the boot ROM will automatically attempt to unlock the device in the special case when the value of the first word of the EEPROM is 0xFFFF. In that case, the boot ROM will check the contents of the key area. If the key area has previously been wiped, the device is automatically unlocked. This feature allows developers to easily connect to an un-initialized device for development.

8.3 FUNCTION TABLES

The Program ROM contains the implementation of many firmware functions. Most of the functions are available in firmware libraries. To reduce EEPROM and RAM usage, specialized versions of the libraries are available that make use of the code in the Program ROM. See Section 2.4.1, “ROM-Based Libraries” on page 7 for more information on ROM-based libraries.

To avoid dependencies on the particular addresses of the Program ROM functions, the Program ROM contains a function table consisting of `goto.0d` vectors for each of the embedded functions. The specialized ROM-based libraries use the function table instead of absolute addresses.

In the function table, each function is assigned a function number. The function number is used to calculate an offset into the function table. The firmware include file, `sk3_romvect.cfi` defines the function table entries for available Program ROM functions. A list of the function table vectors that are available is in Table 75.

TABLE 75: PROGRAM ROM FUNCTIONS IN THE FUNCTION TABLE

Function Name	Function Number	Notes
ROMVECT_PopLR_Return	0	-
ROMVECT_SystemInitialize	1	-
ROMVECT_BootAppEntry	2	Used for JumpROM BootApp
ROMVECT_PushLoopsToStack	3	Push all the hardware loops to the stack
ROMVECT_PopLoopsFromStack	4	Pop all the hardware loops from the stack
ROMVECT_AESInitialize	5	See the *CFX Communication Protocols Manual*
ROMVECT_AESExpandKey	6	See the *CFX Communication Protocols Manual*
ROMVECT_AESDecrypt	7	See the *CFX Communication Protocols Manual*
ROMVECT_AESCleanUp	8	See the *CFX Communication Protocols Manual*
ROMVECT_SYSLIB_Reset	9	Implements SYSLIB_Reset
ROMVECT_SYSLIB_Delay	10	Implements SYSLIB_Delay
ROMVECT_SYSLIB_DelayBase	11	-
ROMVECT_SYSLIB_WaitForSPIDone	12	Implements SYSLIB_WaitForSPIDone
ROMVECT_SYSLIB_SetOperatingFrequency	13	Implements SYSLIB_SetOperatingFrequency
ROMVECT_SYSLIB_SoftPowerdownEnable	14	Implements SYSLIB_SoftPowerdownEnable
ROMVECT_SYSLIB_SoftPowerdownDisable	15	Implements SYSLIB_SoftPowerdownDisable
ROMVECT_SYSLIB_CalibrateADCs	69	Implements SYSLIB_CalibrateADCs; *not applicable to Ezairo 5900 version 1.01.xx*
ROMVECT_MATHLIB_SQRT_frac48_frac48	16	Implements MATHLIB_SQRT_frac48_frac48
ROMVECT_MATHLIB_LOG2_frac48_m8p16	17	Implements MATHLIB_LOG2_frac48_m8p16
ROMVECT_MATHLIB_LOG2_frac48_m8p16_2	70	Implements MATHLIB_LOG2_frac48_m8p16
ROMVECT_MATHLIB_EXP2_m8p16_frac48	18	Implements MATHLIB_EXP2_m8p16_frac48
ROMVECT_MATHLIB_RECIP_POS_frac48_fl8n16	19	Implements MATHLIB_RECIP_POS_frac48_fl8n16
ROMVECT_MATHLIB_RECIP_GEN_frac48_fl8n16	20	Implements MATHLIB_RECIP_GEN_frac48_fl8n16
ROMVECT_MATHLIB_DIVIDE	21	Implements MATHLIB_DIVIDE
ROMVECT_MATHLIB_DIVIDE_LOOP_START	22	-
ROMVECT_EELIB_Initialize	23	Implements EELIB_Initialize
ROMVECT_EELIB_Close	24	Implements EELIB_Close
ROMVECT_EELIB_VerifyEEPROMConnection	25	Implements EELIB_VerifyEEPROMConnection
ROMVECT_EELIB_LibWriteEnable	26	Implements EELIB_LibWriteEnable
ROMVECT_EELIB_LibWriteDisable	27	Implements EELIB_LibWriteDisable
ROMVECT_EELIB_WriteDisable	28	Implements EELIB_WriteDisable

TABLE 75: PROGRAM ROM FUNCTIONS IN THE FUNCTION TABLE

Function Name	Function Number	Notes
ROMVECT_EELIB_WriteEnable	29	Implements EELIB_WriteEnable
ROMVECT_EELIB_ReadStatusRegister	30	Implements EELIB_ReadStatusRegister
ROMVECT_EELIB_WriteStatusRegister	31	Implements EELIB_WriteStatusRegister
ROMVECT_EELIB_ReadWord16	32	Implements EELIB_ReadWord16
ROMVECT_EELIB_ReadNextWord16	33	Implements EELIB_ReadNextWord16
ROMVECT_EELIB_ReadWord8	34	Implements EELIB_ReadWord8
ROMVECT_EELIB_ReadNextWord8	35	Implements EELIB_ReadNextWord8
ROMVECT_EELIB_ReadWord24	36	Implements EELIB_ReadWord24
ROMVECT_EELIB_ReadNextWord24	37	Implements EELIB_ReadNextWord24
ROMVECT_EELIB_ReadWord32	38	Implements EELIB_ReadWord32
ROMVECT_EELIB_ReadNextWord32	39	Implements EELIB_ReadNextWord32
ROMVECT_EELIB_InitWriteWord	40	Implements EELIB_InitWriteWord
ROMVECT_EELIB_WriteWord16	41	Implements EELIB_WriteWord16
ROMVECT_EELIB_WriteNextWord16	42	Implements EELIB_WriteNextWord16
ROMVECT_EELIB_WriteWord8	43	Implements EELIB_WriteWord8
ROMVECT_EELIB_WriteNextWord8	44	Implements EELIB_WriteNextWord8
ROMVECT_EELIB_WriteWord24	45	Implements EELIB_WriteWord24
ROMVECT_EELIB_WriteNextWord24	46	Implements EELIB_WriteNextWord24
ROMVECT_EELIB_WriteWord32	47	Implements EELIB_WriteWord32, *EEPROM library version 1.1.1*
ROMVECT_EELIB_WriteNextWord32	48	Implements EELIB_WriteNextWord32, *EEPROM library version 1.1.1*
ROMVECT_EELIB_WriteBlock_P	49	Implements EELIB_WriteBlock_P, *EEPROM library version 1.1.1*
ROMVECT_EELIB_WriteBlock_X	50	Implements EELIB_WriteBlock_X, *EEPROM library version 1.1.1*
ROMVECT_EELIB_WriteBlock_Y	51	Implements EELIB_WriteBlock_Y, *EEPROM library version 1.1.1*
ROMVECT_EELIB_WriteBlock_Const32	52	Implements EELIB_WriteBlock_Const32, *EEPROM library version 1.1.1*
ROMVECT_EELIB_WriteBlock	53	-
ROMVECT_EELIB_CommitAndWrite	54	-
ROMVECT_EELIB_ReadBlock_P	55	Implements EELIB_ReadBlock_P
ROMVECT_EELIB_ReadBlock_P16	56	Implements EELIB_ReadBlock_P16
ROMVECT_EELIB_ReadNextBlock_P	57	Implements EELIB_ReadNextBlock_P
ROMVECT_EELIB_ReadNextBlock_P16	58	Implements EELIB_ReadNextBlock_P16

TABLE 75: PROGRAM ROM FUNCTIONS IN THE FUNCTION TABLE

Function Name	Function Number	Notes
ROMVECT_EELIB_ReadBlock_X	59	Implements EELIB_ReadBlock_X
ROMVECT_EELIB_ReadBlock_X16	60	Implements EELIB_ReadBlock_X16
ROMVECT_EELIB_ReadNextBlock_X	61	Implements EELIB_ReadNextBlock_X
ROMVECT_EELIB_ReadNextBlock_X16	62	Implements EELIB_ReadNextBlock_X16
ROMVECT_EELIB_ReadBlock_Y	63	Implements EELIB_ReadBlock_Y
ROMVECT_EELIB_ReadNextBlock_Y	64	Implements EELIB_ReadNextBlock_Y
ROMVECT_EELIB_Commit	65	Implements EELIB_Commit
ROMVECT_EELIB_Init24BitSPIWrite	66	-
ROMVECT_EELIB_Init8BitSPIWrite	67	-
ROMVECT_EELIB_InitSPIWriteOperation	68	-

8.4 FILE SYSTEM DECRYPTION SUPPORT

BelaSigna 300 supports the use of encrypted data in the non-volatile memory to provide protection to software intellectual property for copyright and related purposes. A key component of this support is provided through the AES function set provided by the Program ROM. The Program ROM uses these functions to decrypt AES encrypted data blocks in P memory for Level 1 security (see Section 7.5.1, “Level 1 Security” on page 76), and the bootloader uses these functions for encrypted application sections (see Section 7.11.5, “Encrypted Sections” on page 86).

For a complete view of all of the security components provided in the system, see the file system security section of the *CFX Communication Protocols Manual*.

8.4.1 Advanced Encryption Standard (AES) Functions

The ROM-based implementation of the AES standard contains a function set that can be used to decrypt AES pages. The functions provided are as follows:

AESInitialize Load the look-up tables required for a standard table-driven implementation of the AES standard

AESExpandKey Expand a 128-bit AES key to the full expanded key—includes both the encryption form (needed for the expansion of the decryption key), and the decryption form (needed for decrypting blocks of AES encrypted data)

AESDecrypt Decrypt one or more blocks of contiguous data using the specified expanded key

AESCleanup Clear the keys from memory—use this before the application switches the access mode to unrestricted, if it is important to maintain the security of the current key

8.4.1.1 Memory, Register and Other Resource Usage by AES Routines

The AES routines that are provided in the Program ROM for BelaSigna 300 use a standard table-driven implementation for both expansion of a provided AES key and the decryption of any blocks of encrypted data. A list of the memory sections and use of these memory sections by the AES routines is provided in Table 76.

TABLE 76: AES MEMORY USAGE

Memory	Start Address	End Address	Used for
X	0xA000	0xA5FF	AES S-Box, Inverted S-Box and other lookup tables
P	0x3F80	0x3FD4	Full AES key expansion
P	Block start address	Block end address	Temporary space for decryption (blocks are decrypted in place)

The resources used by the AES routines have been selected to allow encrypted sections to be loaded at any point during the loading of an application without concern for previously loaded values.

AES routines are not designed to save and restore any registers that they use, as they generally are not intended to be used during normal program execution. A list of register modifications can be found in Table 77.

TABLE 77: AES FUNCTION SUMMARY

Function Vector	Inputs	Outputs	Register Mods
ROMVECT_AESInitialize	-	Loaded AES lookup tables	x0, y0, a0, a1, xp0, xp1, xp2, xp3, yp0, yp1, yp2, yp3, xstep0, xstep1, xstep2, xstep3
ROMVECT_AESExpandKey	yp1 - Key pointer	Expanded key	x0, x1, y0, y1, a0, a1, b0, b1, xp0, xp1, yp0, yp2, xstep0, xstep1, xstep2, ystep1, xcirc0
ROMVECT_AESDecryrpt	yp0 - Pointer to encrypted data yp1 - Key pointer x0 - Number of encrypted blocks	Decrypted data	x1, y0, y1, a0, a1, b0, b1, xp0, xp1, xp2, yp2, yp3, xstep0, ystep1, ystep2, ystep3, xcirc0, ycirc1, ycirc2, ycirc3
ROMVECT_AESCleanUp	yp1 - Key pointer	-	b0, yp1, ystep1

All AES routines are blocking routines that execute on the CFX DSP core, but do not prevent interruption through the interrupt controller. No part of the AES routines are processed on the HEAR coprocessor.

8.4.2 Encryption Blocks

AES encryption is a block-based encryption and decryption scheme that operates on 128-bit data blocks. The implementation used on BelaSigna 300 can be used to decrypt data to be loaded to P memory in 4-word blocks. P memory sections that are not a multiple of 4 words in length are padded out to 128-bits before they are encrypted.

CAUTION: P memory sections that have been encrypted are decrypted in place, so it is possible for the padding to the 128-bit block size to overlap with the segment that is loaded to the immediately following P memory address.

To ensure that padded P memory data does not result in code or data being lost in P memory due to the padding overlapping with a subsequent function, take one of the following approaches:

- Load P memory sections from the section with the lowest starting address to the section with the highest starting address.
- Space P memory sections that are padded so that the padding does not overlap with any other sections.

This page left blank for printing purposes.

9 Bootloader

9.1 DESCRIPTION

The bootloader is a small system program that is included with the firmware. The bootloader's purpose is to load full applications from EEPROM to RAM (providing decryption as required) and to verify the integrity of the file system. This corresponds to the second stage of the two-stage boot process. The bootloader itself is stored in EEPROM and is loaded to RAM by the Program ROM. The bootloader is also responsible for loading the clock calibration values from EEPROM to RAM and setting the clock to the default operating frequency.

9.2 INSTALLATION

The bootloader is distributed in both source and binary forms. The binary form is a standard `.abs` file, while the source form is a standard IDE project. The binary form is generally used when creating an EEPROM image to boot an application.

To use the bootloader, it must be loaded into the EEPROM. The bootloader may be located at any EEPROM address after the Key Area of the file system. By default, the bootloader is stored in EEPROM at E:0x0100. The System Area at the base of the file system contains a vector indicating the location of the bootloader in EEPROM.

9.3 BOOTLOADER VERSION

The bootloader contains a 16-bit version identifier. The version identifier is stored in the binary image and is loaded to RAM during the bootloader operation. For more information see Section 2.8, "Versions" on page 12.

9.4 MEMORY USAGE

The bootloader executes out of program memory once it has been loaded by the Program ROM. By default, the bootloader itself is loaded into reserved program memory. The bootloader also may use the reserved areas in X memory, Y memory and program memory. For more information see Section 2.5, "Memory Map" on page 8.

9.5 OPERATION

The bootloader loads a full application from the EEPROM to RAM. Multiple applications can be stored on the EEPROM. The bootloader will select a single application as specified through a system variable. This system variable is loaded by the Program ROM from the EEPROM during boot. The applications must be stored in the file system format described in Chapter 7, "EEPROM File System" on page 75. The bootloader verifies the integrity of the selected application and all system areas.

The bootloader begins operation by performing initialization. After the bootloader initialization, the bootloader process is as follows:

- Verify the System area
- Verify the Manufacturing area
- Load clock calibration values and set the default operating frequency
- Load and decrypt the user key, if required
- Load specified application
- Verify application
- Decrypt application sections, if required
- Execute the application

The verification steps are performed using CRCs and range checks. If a CRC or range check fails, the bootloader executes its error handler. In the case of an invalid CRC for a volatile section, the boot process may continue (See Chapter 7, “Volatile Sections” on page 86 for more information). For all other errors the boot process is terminated.

9.6 BOOTLOADER ERRORS

When a bootloader error occurs, the boot process is terminated and an error handler is executed. The bootloader error handler sets a system variable indicating the error that occurred. The error handler implements a delay and then proceeds to reboot the system. The delay is nominally 3000 ms. The actual delay may vary depending on the default clock frequency and whether the default operating frequency was modified by the bootloader.

The bootloader error handler uses a logical OR to combine the error code corresponding to the boot failure with the bootloader error constant, 0x100. The resulting code is stored in a system variable. The bootloader error constant indicates that the error occurred in the bootloader. For more information on boot errors, including a list of error codes, see Section 2.11.2, “Boot Errors” on page 15.

10 System Macro Reference

This reference chapter presents a detailed description of all the macros defined in the system include files. For each macro, it describes calling parameters, modified registers, returned values and execution time.

10.1 SYS_BOOTAPP

Boot the specified application

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_BootApp( reg )`
Source File	sk3_sys_system.cfi
Description	Boot the specified application. The boot ROM is used to load the bootloader and application from EEPROM. The APPNUM argument overrides the value set in the file system. If APPNUM is invalid, the system will reboot and the default application will be used.
Inputs	reg = Register specifying application number to boot (APPNUM)
Outputs	None
Register Modifications	Not applicable because the application that calls Sys_BootApp is terminated
Assumptions	None
Clock Cycles	Not applicable

Example	`// Load x0 with the number of the application we'd like to run (2)`
	`// and reboot into that application using the boot ROM.`
	`load x0, 2`
	`Sys_BootApp( x0 )`

10.2 SYS_CALC_GAIN_FACTOR_VAL

Calculate the decimation gain factor for a desired gain given an adc_cfg_sample_freq field

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Calc_Gain_Factor_Val( adc_cfg_sample_freq, gain )`
Source File	sk3_sys_analog.cfi
Description	Calculate the decimation gain factor for a desired gain given an adc_cfg_sample_freq field. The developer must pre-multiply the gain by 10^7 before passing it in. The resulting gain factor produces the closest gain possible without going over the specified parameter.
Inputs	adc_cfg_sample_freq = Sampling frequency field in A_ADC_CFG gain = Desired gain multiplied by 10^7. Can be positive or negative. Range is dependent on adc_cfg_sample_freq
Outputs	SYS_CALC_GF = Define containing the rounded value for the desired gain factor
Register Modifications	None
Assumptions	adc_cfg_sample_freq, gain are within valid ranges
Clock Cycles	0 (evaluated during compile)

Example	`// Calculate the gain factor value that will produce a unity gain`
	`// in both channel 0 and channel 1 given ADC_CFG_SAMPLE_FREQ`
	`// NOTE: Gain factor value is #set in SYS_CALC_GF`
	`#define ADC_CFG_SAMPLE_FREQ 0x2`
	`#define UNITY_GAIN 10000000`
	
	`Sys_Calc_Gain_Factor_Val( ADC_CFG_SAMPLE_FREQ, UNITY_GAIN )`
	`Sys_Set_ADC_GF01_CTRL( SYS_CALC_GF, SYS_CALC_GF )`

10.3 SYS_CALC_SAMPLE_FREQ_VAL

Calculate sampling frequency field for A_ADC_CFG

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Calc_Sample_Freq_Val( slowclk, sampling_freq )`
Source File	sk3_sys_analog.cfi
Description	Calculate sampling frequency field for A_ADC_CFG
Inputs	slowclk = Slow clock frequency in Hz sampling_freq = Desired sampling frequency in Hz
Outputs	SYS_CALC_SF = Define containing the rounded value for the sampling frequency field in A_ADC_CFG
Register Modifications	None
Assumptions	slowclk, sampling_freq are within valid ranges
Clock Cycles	0 (evaluated during compile)

Example	`// Set the sampling frequency to 16 kHz with a 1.28 MHz slow clock`
	`// NOTE: Value for ADC_CFG_SAMPLE_FREQ field in ADC_CFG is #set in`
	`//  SYS_CALC_SF`
	`Sys_Calc_Sample_Freq_Val( 1280000, 16000 )`
	`Sys_Set_ADC_CFG( SYS_CALC_SF )`
	
	`// Set the sampling frequency to 24 kHz with a 1.92 MHz slow clock`
	`Sys_Calc_Sample_Freq_Val( 1920000, 24000 )`
	`Sys_Set_ADC_CFG( SYS_CALC_SF )`

10.4 SYS_CFX_BIASDISABLE

Set the SR_BIAS bit to disable bias

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_CFX_BiasDisable`
Source File	sk3_sys_cfx.cfi
Description	Set the SR_BIAS bit to disable bias
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Set the SR_BIAS bit to disable bias when rounding/truncating` `Sys_CFX_BiasDisable`

10.5 SYS_CFX_BIASENABLE

Clear the SR_BIAS bit to enable bias

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_CFX_BiasEnable`
Source File	sk3_sys_cfx.cfi
Description	Clear the SR_BIAS bit to enable bias
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Clear the SR_BIAS bit to enable bias when rounding/truncating` `Sys_CFX_BiasEnable`

10.6 SYS_CFX_CIRCADDRCONFIG

Configure one of the CFX's circular registers

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_CFX_CircAddrConfig( reg, config )`
Source File	sk3_sys_cfx.cfi
Description	Configure one of the CFX's circular registers
Inputs	reg = Circular register to configure config = Two-part parameter consisting of CIRC_CIRCULAR / CIRC_BIT_REVERSE 14-bit size of buffer (value between 0 and 16383)
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Configure xcirc0 to support a circular buffer of 8 words.`
	`Sys_CFX_CircAddrConfig( xcirc0, ( CIRC_CIRCULAR | 8 ) )`
	
	`// Configure ycirc0 to support bit-reversed addressing on a`
	`// 128-word buffer`
	`Sys_CFX_CircAddrConfig( ycirc0, ( CIRC_BIT_REVERSE | 128 ) )`

10.7 SYS_CFX_FLAGUPDATEDISABLE

Clear the SR_FLAG_UPD bit to disable updates to SR flags

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_CFX_FlagUpdateDisable`
Source File	sk3_sys_cfx.cfi
Description	Clear the SR_FLAG_UPD bit to disable updates to SR flags
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Clear the SR_FLAG_UPD bit to disable flag updates on arithmetic`
	`// operations`
	`Sys_CFX_FlagUpdateDisable`

10.8 SYS_CFX_FLAGUPDATEENABLE

Set the SR_FLAG_UPD bit to enable updates to SR flags

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_CFX_FlagUpdateEnable`
Source File	sk3_sys_cfx.cfi
Description	Set the SR_FLAG_UPD bit to enable updates to SR flags
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Set the SR_FLAG_UPD bit to enable flag updates on arithmetic`
	`// operations`
	`Sys_CFX_FlagUpdateEnable`

10.9 SYS_CFX_INITLOOPSTACK

Initialize the loop stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_CFX_InitLoopStack`
Source File	sk3_sys_cfx.cfi
Description	Initialize the loop stack
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	4
Example	`// Initialize the loop stack, clearing any existing hardware loops` `Sys_CFX_InitLoopStack`

10.10 SYS_CFX_INITSTACK

Initialize the stack pointer

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_CFX_InitStack( stack_base_addr )`
Source File	sk3_sys_cfx.cfi
Description	Initialize the stack pointer
Inputs	stack_base_addr = Base address of stack
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Initialize stack pointer to 0x1000 in X memory` `Sys_CFX_InitStack( 0x1000 )`

10.11 SYS_CFX_INTDISABLE

Clear the SR_IE bit to disable interrupts

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_CFX_IntDisable`
Source File	sk3_sys_cfx.cfi
Description	Clear the SR_IE bit to disable interrupts
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Clear the SR_IE bit to disable interrupts` `Sys_CFX_IntDisable`

10.12 SYS_CFX_INTENABLE

Set the SR_IE bit to enable interrupts

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_CFX_IntEnable`
Source File	sk3_sys_cfx.cfi
Description	Set the SR_IE bit to enable interrupts
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	3
Example	`// Set the SR_IE bit to enable interrupts` `Sys_CFX_IntEnable`

10.13 SYS_CFX_ROUNDDISABLE

Clear the SR_ROUND bit to select truncation

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_CFX_RoundDisable`
Source File	sk3_sys_cfx.cfi
Description	Clear the SR_ROUND bit to select truncation
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Clear the SR_ROUND bit to enable truncation of data when copied` `Sys_CFX_RoundDisable`

10.14 SYS_CFX_ROUNDENABLE

Set the SR_ROUND bit to select rounding

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_CFX_RoundEnable`
Source File	sk3_sys_cfx.cfi
Description	Set the SR_ROUND bit to select rounding
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Set the SR_ROUND bit to enable rounding of data when copied` `Sys_CFX_RoundEnable`

10.15 SYS_CFX_SATURATEDISABLE

Clear the SR_SATURATE bit to select wrapping

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_CFX_SaturateDisable`
Source File	sk3_sys_cfx.cfi
Description	Clear the SR_SATURATE bit to select wrapping
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Disable saturation` `Sys_CFX_SaturateDisable`

10.16 SYS_CFX_SATURATEENABLE

Set the SR_SATURATE bit to select saturation

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_CFX_SaturateEnable`
Source File	sk3_sys_cfx.cfi
Description	Set the SR_SATURATE bit to select saturation
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Enable saturation` `Sys_CFX_SaturateEnable`

10.17 SYS_CFX_SRCONFIG

Configure the CFX status register

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_CFX_SRConfig( arg )`
Source File	sk3_sys_cfx.cfi
Description	Configure the CFX status register
Inputs	arg = Five-part parameter consisting of SR_ROUND_[ENABLE | DISABLE], SR_BIAS_[ENABLE | DISABLE], SR_INT_[ENABLE | DISABLE], SR_SATURATE_[ENABLE | DISABLE], SR_FLAG_UPD_[ENABLE | DISABLE]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Configure the core (through the status register) for:`
	`// - Rounding enabled`
	`// - Bias disabled`
	`// - Interrupts enabled`
	`// - Saturation disabled`
	`// - Flag updates enabled`
	`Sys_CFX_SRConfig( SR_ROUND_ENABLE |\`
	`SR_BIAS_DISABLE |\`
	`SR_INT_ENABLE |\`
	`SR_SATURATE_DISABLE |\`
	`SR_FLAG_UPD_ENABLE )`

10.18 SYS_CFX_STEPCONFIG

Configure one of the CFX's step registers

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_CFX_StepConfig( reg, config )`
Source File	sk3_sys_cfx.cfi
Description	Configure one of the CFX's step registers
Inputs	reg = Step register to configure config = 16-bit signed two's complement value to load into reg
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Configure xstep0 to provide a step size of 2` `Sys_CFX_StepConfig( xstep0, 2 )`

10.19 SYS_CLK_CONFIG

Configure both parts of the clock configuration

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Clk_Config( arg0, arg1 )`
Source File	sk3_sys_clk.cfi
Description	Configure both parts of the clock configuration
Inputs	arg0 = Data to be used by clock configuration 0 arg1 = Data to be used by clock configuration 1
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	4

Example	`// Configure the clock distribution:`
	`// - Select the default configuration for SYSCLK with no external`
	`//   clock supplied`
	`// - Do not share cycles between the cores but configure slow clock`
	`//   as 1/8 of the system clock (640kHz for a 5.12MHz system clock)`
	`Sys_Clk_Config( SYSCLK_SEL_DEFAULT | EXTCLK_DISABLED, \`
	`DBL_ACCESS_MODE_DISABLE |  SLOWCLK_PRESCALE_8 )`

10.20 SYS_CLK_CONFIG0

Configure part of the clock configuration

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Clk_Config0( arg )`
Source File	sk3_sys_clk.cfi
Description	Configure part of the clock configuration
Inputs	arg = Data to be used in the clock configuration
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	2

Example	`// Enable an external clock input (scaled by 4) and the default`
	`// HEARCLK configuration`
	`Sys_Clk_Config0( SYSCLK_SEL_DEFAULT | HEARCLK_PRESCALE_1_000 | \`
	`EXTCLK_IN_ENABLE | EXTCLK_PRESCALE_4 )`

10.21 SYS_CLK_CONFIG1

Configure part of the clock configuration

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Clk_Config1( arg )`
Source File	sk3_sys_clk.cfi
Description	Configure part of the clock configuration
Inputs	arg = Data to be used in the clock configuration
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	2

Example	`// Given a system clock of 5.12MHz:`
	`// - Select a slow clock of 1.28MHz`
	`// - Select a user clock of 10.0kHz`
	`// - Output user clock on the PCM frame pad`
	`// - No sharing of cycles between the cores`
	`Sys_Clk_Config1( PCM_CLK_PAD_AS_INPUT | PCM_FRAME_PAD_SEL_UCLK | \`
	`DBL_ACCESS_MODE_DISABLE | \`
	`SLOWCLK_PRESCALE_4 | UCLK_PRESCALE_512 )`

10.22 SYS_DATA_FL8N16

Create a .data directive using the Sys_fl8n16 macro

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Data_fl8n16( const )`
Source File	sk3_sys_utils.cfi
Description	Create a .data directive using the Sys_fl8n16 macro.
Inputs	const = Constant to be passed to Sys_fl8n16
Outputs	Sys_Data_fl8n16 = .data directive with floating-point approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^63
Clock Cycles	0 (evaluated during compile)

Example	`// Load 7.125 to memory as a <8,16> floating point`
	`Data_Value_fl8n16:`
	`Sys_Data_fl8n16( 71250000 )`

10.23 SYS_DATA_FRAC24

Create a .data directive using the Sys_frac24 macro

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Data_frac24( const )`
Source File	sk3_sys_utils.cfi
Description	Create a .data directive using the Sys_frac24 macro.
Inputs	const = Constant to be passed to Sys_frac24
Outputs	Sys_Data_frac24 = .data directive with fractional approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^46
Clock Cycles	0 (evaluated during compile)

Example	`// Load 0.125 to memory as a <1,23> fractional value`
	`Data_Value_frac24:`
	`Sys_Data_frac24( 1250000 )`

10.24 SYS_DATA_FRAC32

Create a .data directive using the Sys_frac32 macro

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Data_frac32( const )`
Source File	sk3_sys_utils.cfi
Description	Create a .data directive using the Sys_frac32 macro. *Note:* The frac32 data format is layout-compatible with the frac24 format.
Inputs	const = Constant to be passed to Sys_frac32
Outputs	Sys_Data_frac32 = .data directive with <9,23> fixed-point approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^46
Clock Cycles	0 (evaluated during compile)

Example	`// Load 64.0 to memory as a <9,23> fixed-point value`
	`Data_Value_frac32:`
	`Sys_Data_frac32( 640000000 )`

10.25 SYS_DATA_FRAC48

Create a .data directive using the Sys_frac48 macro

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Data_frac48( const )`
Source File	sk3_sys_utils.cfi
Description	Create a .data directive using the Sys_frac48 macro.
Inputs	const = Constant to be passed to Sys_frac48
Outputs	Sys_Data_frac48 = .data directive with <1,47> fractional approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^24
Clock Cycles	0 (evaluated during compile)

Example	`// Load 0.125 to memory as a <1,47> fractional value`
	`Data_Value_frac48:`
	`Sys_Data_frac48( 1250000 )`

10.26 SYS_DATA_M24P24

Create a .data directive using the Sys_m24p24 macro

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Data_m24p24( const )`
Source File	sk3_sys_utils.cfi
Description	Create a .data directive using the Sys_m24p24 macro.
Inputs	const = Constant to be passed to Sys_m24p24
Outputs	Sys_Data_m24p24 = .data directive with <24,24> fixed-point approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^62
Clock Cycles	0 (evaluated during compile)

Example	`// Load 7.125 to memory as a <24,24> fixed-point value`
	`Data_Value_m24p24:`
	`Sys_Data_m24p24( 71250000 )`

10.27 SYS_DATA_M2P46

Create a .data directive using the Sys_m2p46 macro

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Data_m2p46( const )`
Source File	sk3_sys_utils.cfi
Description	Create a .data directive using the Sys_m2p46 macro.
Inputs	const = Constant to be passed to Sys_m2p46
Outputs	Sys_Data_m2p46 = .data directive with <2,46> fixed-point approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^40
Clock Cycles	0 (evaluated during compile)

Example	`// Load 1.125 to memory as a <2,46> fixed-point value`
	`Data_Value_m2p46:`
	`Sys_Data_m2p46( 11250000 )`

10.28 SYS_DATA_M8P16

Create a .data directive using the Sys_m8p16 macro

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Data_m8p16( const )`
Source File	sk3_sys_utils.cfi
Description	Create a .data directive using the Sys_m8p16 macro.
Inputs	const = Constant to be passed to Sys_m8p16
Outputs	Sys_Data_m8p16 = .data directive with fixed-point approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^53
Clock Cycles	0 (evaluated during compile)

Example	`// Load 7.125 to memory as a <8,16> fixed-point value`
	`Data_Value_m8p16:`
	`Sys_Data_m8p16( 71250000 )`

10.29 SYS_DEBUGPORT_GET_AUDIOMODE

Return the debug port audio mode setting

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_DebugPort_Get_AudioMode( reg )`
Source File	sk3_sys_debugport.cfi
Description	Return the debug port audio mode setting
Inputs	reg = Data register the audio mode setting is returned through
Outputs	reg = Value of D_DEBUG_CFG register's AUDIO_MODE bit
Register Modifications	None
Assumptions	Result will be compared with AUDIO_STREAM_DISABLED or AUDIO_STREAM_ENABLED and hence no shift is required
Clock Cycles	2
Example	`// Load x1 with the current debug port audio mode setting` `Sys_DebugPort_Get_AudioMode( x1 )`

10.30 SYS_DEBUGPORT_SET_ACCESSMODE

Set the debug port access mode

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_DebugPort_Set_AccessMode( mode )`
Source File	sk3_sys_debugport.cfi
Description	Set the debug port access mode
Inputs	mode = ACCESS_[RESTRICTED | UNRESTRICTED]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Unrestrict access through the debug port` `Sys_DebugPort_Set_AccessMode( ACCESS_UNRESTRICTED )`

10.31 SYS_DEBUGPORT_SET_ADDRESS

Set the debug port internally-programmable address

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_DebugPort_Set_Address( addr )`
Source File	sk3_sys_debugport.cfi
Description	Set the debug port internally-programmable address
Inputs	addr = 7-bit I2C internally-programmable address for debug port
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Assign 0x50 to the debug port internally-programmable I2C address` `Sys_DebugPort_Set_Address( 0x50 )`

10.32 SYS_FIFO_CONFIGURE

Configure one of the system FIFOs for use within an application

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_FIFO_Configure( fifo_name, start, fifo_size, block_size, base_ptr, ioblock_ptr )`
Source File	sk3_sys_fifo.cfi
Description	Configure one of the system FIFOs for use within an application
Inputs	fifo_name = One of: SYS_FIFO_A0, SYS_FIFO_A1, SYS_FIFO_A2, SYS_FIFO_A3, SYS_FIFO_B0, SYS_FIFO_B1, SYS_FIFO_B2, SYS_FIFO_B3 start = Start offset for the FIFO fifo_size = Size of the FIFO block_size = Size of a block in the FIFO base_ptr = Initial value for the BASE pointer ioblock_ptr = Initial value for the IOBLOCK pointer
Outputs	None
Register Modifications	y1
Assumptions	The FIFO start offset is a multiple of 32 The FIFO size is a multiple of 32 The FIFO block size is a multiple of 2 and is less than 1023 The base and IO block pointers point to locations between FIFO start and FIFO start + FIFO size.
Clock Cycles	13 cycles maximum, saving 1 cycle for each of start, fifo_size, start + fifo_size, block_size, base_ptr, and ioblock_ptr that can be sign-extended from 8 bits

Example	`#define BLOCK_SIZE 8`
	
	`#define FIFO_A0_START 0 // The base of FIFO A0 relative to AMEM`
	`#define FIFO_A0_LENGTH 32`
	`#define FIFO_A0_END FIFO_A0_START + FIFO_A0_LENGTH`
	`#define FIFO_A0_BASE_PTR 0`
	`#define FIFO_A0_IOBLOCK_PTR 0`
	`#define FIFO_A0_BLOCK_SIZE BLOCK_SIZE`
	
	`Sys_FIFO_Configure( SYS_FIFO_A0, \`
	`FIFO_A0_START, \`
	`FIFO_A0_LENGTH, \`
	`FIFO_A0_BLOCK_SIZE, \`
	`FIFO_A0_BASE_PTR, \`
	`FIFO_A0_IOBLOCK_PTR )`

10.33 SYS_FL8N16

Convert a decimal number with up to seven digits of precision to floating-point format with an 8-bit signed exponent and a 16-bit signed mantissa

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_fl8n16( const )`
Source File	sk3_sys_utils.cfi
Description	Convert a decimal number with up to seven digits of precision to floating-point format with an 8-bit signed exponent and a 16-bit signed mantissa. The developer must pre-multiply the decimal number by 10^7 before passing it to the macro so that the macro receives an integer. For example, 7.125 becomes 71250000. The 10^7 multiplier provides 23 fractional bits of accuracy, although the 16-bit mantissa only stores 15 bits of precision, and this includes the value of any non-fractional part of the input. Although the valid range for the floating-point format is approximately -2^127 to 2^127, the whole-number input format for this macro and the 64-bit arithmetic of the preprocessor constrain the pre-multiplied inputs to have magnitude less than 2^63 or about 9 x 10^18 (representing numbers up to about 9 x 10^11). Furthermore, the 7-digit fractional part of the decimal input number constrains the inputs to have magnitude of 2^-23 or greater if they are non-zero. The input number therefore cannot exceed the limits of the floating-point format. This macro is useful for filling data registers through an immediate load and for defining floating-point data to be loaded from XMEM or YMEM.
Inputs	const = Constant to convert to floating-point format, multiplied by 10^7
Outputs	Sys_fl8n16 = Floating-point approximation (8-bit signed exponent and 16-bit signed mantissa) of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^63
Clock Cycles	0 (evaluated during compile)

Example	`// Set VALUE to 25.125 converted to floating-point format with 8-bit`
	`// signed exponent and 16-bit signed mantissa and then store this`
	`// to a0.`
	`#set VALUE Sys_fl8n16( 251250000 )`
	`load a0, VALUE`

10.34 SYS_FRAC24

Convert a decimal number with up to seven digits of precision to <1,23> fractional format

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_frac24( const )`
Source File	sk3_sys_utils.cfi
Description	Convert a decimal number with up to seven digits of precision to <1,23> fractional format. The developer must pre-multiply the decimal number by 10^7 before passing it to the macro so that the macro receives an integer. For example, 0.125 becomes 1250000. The valid input range (after pre-multiplication) for <1,23> fractional numbers is -10000000 to +9999999. Numbers outside this range are saturated to the closest valid value, but the results are undefined for inputs with magnitude greater than about 2^46 or 70 x 10^12 (representing numbers greater than about 7000000.0). This macro is useful for filling data registers through an immediate load and for defining fractional data to be loaded from XMEM or YMEM.
Inputs	const = Constant to convert to <1,23>, multiplied by 10^7
Outputs	Sys_frac24 = <1,23> fractional approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^46
Clock Cycles	0 (evaluated during compile)

Example	`// Set VALUE to 0.125 converted to the <1,23> fractional format`
	`// and then load this to a0.`
	`#set VALUE Sys_frac24( 1250000 )`
	`load a0, VALUE`

10.35 SYS_FRAC32

Convert a decimal number with up to seven digits of precision to <9,23> fixed-point format

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_frac32( const )`
Source File	sk3_sys_utils.cfi
Description	Convert a decimal number with up to seven digits of precision to <9,23> fixed-point format. The developer must pre-multiply the decimal number by 10^7 before passing it to the macro so that the macro receives an integer. For example, 7.125 becomes 71250000. The valid input range (after pre-multiplication) for <9,23> fixed-point numbers is -2560000000 to +2559999999. Numbers outside this range are saturated to the closest valid value, but the results are undefined for inputs with magnitude greater than about 2^46 or 70 x 10^12 (representing numbers greater than about 7000000.0). This macro is useful for filling an accumulator's high and extension parts using two immediate loads and for defining <9,23> data to be loaded from PMEM. *Note:* The frac32 data format is layout-compatible with the frac24 format.
Inputs	const = Constant to convert to <9,23>, multiplied by 10^7
Outputs	Sys_frac32 = <9,23> fixed-point approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^46
Clock Cycles	0 (evaluated during compile)

Example	`// Set VALUE to 12.5 converted to the <9,23> fixed-point format`
	`// and then store this in P memory at Direct_Data_Value_frac32.`
	`Sys_Mem_Start_Segment( Frac32_Table, SYS_MEMSPACE_P )`
	`#set VALUE Sys_frac32( 125000000 )`
	`Direct_Data_Value_frac32:`
	`.data VALUE`

10.36 SYS_FRAC48

Convert a decimal number with up to seven digits of precision to <1,47> fractional format

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_frac48( const )`
Source File	sk3_sys_utils.cfi
Description	Convert a decimal number with up to seven digits of precision to <1,47> fractional format. The developer must pre-multiply the decimal number by 10^7 before passing it to the macro so that the macro receives an integer. For example, 0.125 becomes 1250000. The valid input range (after pre-multiplication) for <1,47> fractional numbers is -10000000 to +9999999. Numbers outside this range are saturated to the closest valid value, but the results are undefined for inputs with magnitude greater than about 2^24 or 16000000 (representing numbers greater than about 1.6). *Note:* This macro places the number in a <1,47> field, but provides only <1,45> accuracy. The bottom 2 bits of an unsaturated result are always zero. This macro is useful for filling accumulators through a pair of immediate loads and for defining <1,47> fractional data to be loaded from XMEM, YMEM, or XYMEM.
Inputs	const = Constant to convert to <1,47>, multiplied by 10^7
Outputs	Sys_frac48 = <1,47> fractional approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^24
Clock Cycles	0 (evaluated during compile)

Example	`// Set VALUE to 0.125 converted to the <1,47> fractional format`
	`// and then store the two halves of the 48-bit value in memory at`
	`// Direct_Data_Value_frac48.`
	`#set VALUE Sys_frac48( 1250000 )`
	`#set VAL_H VALUE >> 24`
	`#set VAL_L Sys_IMM_24(VALUE)`
	`Direct_Data_Value_frac48:`
	`.data VAL_H, VAL_L`

10.37 SYS_GET_ADC_CFG

Get the value of control register A_ADC_CFG

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Get_ADC_CFG( reg )`
Source File	sk3_sys_analog.cfi
Description	Get the value of control register A_ADC_CFG
Inputs	reg = Register to load with the A_ADC_CFG value
Outputs	reg = Value of A_ADC_CFG register
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the current contents of register A_ADC_CFG into CFX data`
	`// register x0.`
	`Sys_Get_ADC_CFG( x0 )`

10.38 SYS_GET_ADC_CUR_CTRL

Get the value of control register A_ADC_CUR_CTRL

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Get_ADC_CUR_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Get the value of control register A_ADC_CUR_CTRL
Inputs	reg = Register to load with the A_ADC_CUR_CTRL value
Outputs	reg = Value of A_ADC_CUR_CTRL register
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the current contents of register A_ADC_CUR_CTRL`
	`// into CFX data register b0.`
	`Sys_Get_ADC_CUR_CTRL( b0 )`

10.39 SYS_GET_ADC_GFXY_CTRL

Get the value of control register A_ADC_GFxy_CTRL

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Get_ADC_GFxy_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Get the value of control register A_ADC_GFxy_CTRL
Inputs	reg = Register to load with the A_ADC_GFxy_CTRL value
Outputs	reg = Value of A_ADC_GFxy_CTRL register
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the current contents of register A_ADC_GF01_CTRL`
	`// into CFX data register x1`
	`Sys_Get_ADC_GF01_CTRL( x1 )`

10.40 SYS_GET_ANALOG_PWR_CTRL

Get the value of control register A_ANALOG_PWR_CTRL

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Get_ANALOG_PWR_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Get the value of control register A_ANALOG_PWR_CTRL
Inputs	reg = Register to load with the A_ANALOG_PWR_CTRL value
Outputs	reg = Value of A_ANALOG_PWR_CTRL register
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the current contents of register A_ANALOG_PWR_CTRL`
	`// into CFX data register y1`
	`Sys_Get_ANALOG_PWR_CTRL( y1 )`

10.41 SYS_GET_APPNUM

Return the number of the currently running application (APPNUM)

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Get_AppNum( reg )`
Source File	sk3_sys_system.cfi
Description	Return the number of the currently running application (APPNUM)
Inputs	reg = Register to load with current APPNUM
Outputs	reg = Current APPNUM
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load x0 with the number of the currently running application.`
	`// This value is loaded by the boot ROM when booting the device.`
	`Sys_Get_AppNum( x0 )`

10.42 SYS_GET_BAND_SELECT_CTRL

Get the value of control register A_BAND_SELECT_CTRL

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Get_BAND_SELECT_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Get the value of control register A_BAND_SELECT_CTRL
Inputs	reg = Register to load with the A_BAND_SELECT_CTRL value
Outputs	reg = Value of A_BAND_SELECT_CTRL register
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the current contents of register A_BAND_SELECT_CTRL`
	`// into CFX data register b0`
	`Sys_Get_BAND_SELECT_CTRL( b0 )`

10.43 SYS_GET_CHIPDEBUGVERSION

Return debug port version

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Get_ChipDebugVersion( reg )`
Source File	sk3_sys_system.cfi
Description	Return debug port version
Inputs	reg = Register to load with the debug version value
Outputs	reg = Value of D_DEBUG_VERSION register
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load x0 with the debug port protocol version number for this chip` `Sys_Get_ChipDebugVersion( x0 )`

10.44 SYS_GET_CHIPFAMILY

Return chip family

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Get_ChipFamily( reg )`
Source File	sk3_sys_system.cfi
Description	Return chip family
Inputs	reg = Register to load with the chip family value
Outputs	reg = Value of D_CHIP_FAMILY register
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load x0 with the technology family for this chip` `Sys_Get_ChipFamily( x0 )`

10.45 SYS_GET_CHIPREVISION

Return chip revision

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Get_ChipRevision( reg )`
Source File	sk3_sys_system.cfi
Description	Return chip revision
Inputs	reg = Register to load with the chip revision value
Outputs	reg = Value of D_CHIP_REVISION register
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load x0 with the exact revision for this chip` `Sys_Get_ChipRevision( x0 )`

10.46 SYS_GET_CHIPVERSION

Return chip version

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Get_ChipVersion( reg )`
Source File	sk3_sys_system.cfi
Description	Return chip version
Inputs	reg = Register to load with the chip version value
Outputs	reg = Value of D_CHIP_VERSION register
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Load x0 with the version for this chip` `Sys_Get_ChipVersion( x0 )`

10.47 SYS_GET_CLK_CTRL

Get the value of control register A_CLK_CTRL

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Get_CLK_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Get the value of control register A_CLK_CTRL
Inputs	reg = Register to load with the A_CLK_CTRL value
Outputs	reg = Value of A_CLK_CTRL register
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the current contents of register A_CLK_CTRL into CFX`
	`// data register x0`
	`Sys_Get_CLK_CTRL( x0 )`

10.48 SYS_GET_FRAC_DELAY_X_CTRL

Get the value of control register A_FRAC_DELAY_x_CTRL

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Get_FRAC_DELAY_x_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Get the value of control register A_FRAC_DELAY_x_CTRL
Inputs	reg = Register to load with the A_FRAC_DELAY_x_CTRL value
Outputs	reg = Value of A_FRAC_DELAY_x_CTRL register
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the current contents of register A_FRAC_DELAY_1_CTRL`
	`// into CFX data register b0`
	`Sys_Get_FRAC_DELAY_1_CTRL( b0 )`

10.49 SYS_GET_HEAR_BFP_BLOCK_EXP

Store the value of the block floating point block exponent into a register

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Get_HEAR_BFP_BLOCK_EXP( reg )`
Source File	sk3_sys_hear.cfi
Description	Store the value of the block floating point block exponent into a register.
Inputs	reg = Register to load with the HEAR BFP block floating point block exponent
Outputs	reg = HEAR BFP block floating point block exponent
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Store the current value of the block floating point exponent used`
	`// on the HEAR Configurable Accelerator to x0.`
	`Sys_Get_HEAR_BFP_BLOCK_EXP( x0 )`

10.50 SYS_GET_IN_GAIN_CTRL

Get the value of control register A_IN_GAIN_CTRL

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Get_IN_GAIN_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Get the value of control register A_IN_GAIN_CTRL
Inputs	reg = Register to load with the A_IN_GAIN_CTRL value
Outputs	reg = Value of A_IN_GAIN_CTRL register
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the current contents of register A_IN_GAIN_CTRL into CFX`
	`// data register x1`
	`Sys_Get_IN_GAIN_CTRL( x1 )`

10.51 SYS_GET_INPUT_CTRL

Get the value of control register A_INPUT_CTRL

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Get_INPUT_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Get the value of control register A_INPUT_CTRL
Inputs	reg = Register to load with the A_INPUT_CTRL value
Outputs	reg = Value of A_INPUT_CTRL register
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the current contents of register A_INPUT_CTRL into CFX`
	`// data register a1`
	`Sys_Get_INPUT_CTRL( a1 )`

10.52 SYS_GET_LSAD_CTRL

Get the value of control register A_LSAD_CTRL

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Get_LSAD_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Get the value of control register A_LSAD_CTRL
Inputs	reg = Register to load with the A_LSAD_CTRL value
Outputs	reg = Value of A_LSAD_CTRL register
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the current contents of register A_LSAD_CTRL into CFX`
	`// data register a1`
	`Sys_Get_LSAD_CTRL( a1 )`

10.53 SYS_GET_O_DAC_CTRL

Get the value of control register A_O_DAC_CTRL

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Get_O_DAC_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Get the value of control register A_O_DAC_CTRL
Inputs	reg = Register to load with the A_O_DAC_CTRL value
Outputs	reg = Value of A_O_DAC_CTRL register
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the current contents of register A_O_DAC_CTRL into CFX`
	`// data register a0`
	`Sys_Get_O_DAC_CTRL( a0 )`

10.54 SYS_GET_O_DAC_GF_CTRL

Get the value of control register A_O_DAC_GF_CTRL

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Get_O_DAC_GF_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Get the value of control register A_O_DAC_GF_CTRL
Inputs	reg = Register to load with the A_O_DAC_GF_CTRL value
Outputs	reg = Value of A_O_DAC_GF_CTRL register
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the current contents of register A_O_DAC_GF_CTRL into CFX`
	`// data register y1`
	`Sys_Get_O_DAC_GF_CTRL( y1 )`

10.55 SYS_GET_OUTPUT_CTRL

Get the value of control register A_OUTPUT_CTRL

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Get_OUTPUT_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Get the value of control register A_OUTPUT_CTRL
Inputs	reg = Register to load with the A_OUTPUT_CTRL value
Outputs	reg = Value of A_OUTPUT_CTRL register
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the current contents of register A_OUTPUT_CTRL into CFX`
	`// data register y1`
	`Sys_Get_OUTPUT_CTRL( y1 )`

10.56 SYS_GET_PSU_CFG

Get the value of configuration register A_PSU_CFG [Macro for BelaSigna 300 only]

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Get_PSU_CFG( reg )`
Source File	sk3_sys_analog.cfi
Description	Get the value of configuration register A_PSU_CFG [Macro for BelaSigna 300 only]
Inputs	reg = Register to load with the A_PSU_CFG value
Outputs	reg = Value of A_PSU_CFG register
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the current contents of register A_PSU_CFG into CFX`
	`// data register b0`
	`Sys_Get_PSU_CFG( b0 )`

10.57 SYS_GET_SUP_COUNT_DATA

Get the value of control register A_SUP_COUNT_DATA

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Get_SUP_COUNT_DATA( reg )`
Source File	sk3_sys_analog.cfi
Description	Get the value of control register A_SUP_COUNT_DATA
Inputs	reg = Register to load with the A_SUP_COUNT_DATA value
Outputs	reg = Value of A_SUP_COUNT_DATA register
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the current contents of register A_SUP_COUNT_DATA into`
	`// CFX data register x0`
	`Sys_Get_SUP_COUNT_DATA( x0 )`

10.58 SYS_GET_SUP_THRSH_DATA

Get the value of control register A_SUP_THRSH_DATA

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Get_SUP_THRSH_DATA( reg )`
Source File	sk3_sys_analog.cfi
Description	Get the value of control register A_SUP_THRSH_DATA
Inputs	reg = Register to load with the A_SUP_THRSH_DATA value
Outputs	reg = Value of A_SUP_THRSH_DATA register
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the current contents of register A_SUP_THRSH_DATA into`
	`// CFX data register x1`
	`Sys_Get_SUP_THRSH_DATA( x1 )`

10.59 SYS_GPIO_CONFIGURE

Configure the GPIO direction, interrupts, pull-up resistor and digital input enable signal configuration

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_GPIO_Configure( gpio_cfg )`
Source File	sk3_sys_gpio.cfi
Description	Configure the GPIO direction, interrupts, pull-up resistor and digital input enable signal configuration.
Inputs	gpio_cfg = GPIO configuration. Use GPIO_INT_SRC_*, GPIO_INT_EVENT_*, GPIO*_[INPUT | OUTPUT], GPIO*_PULLUP_[ENABLE | DISABLE], GPIO*_DIGITAL_INPUT_[ENABLE | DISABLE]
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if gpio_cfg can be sign-extended from 8 bits; 2 otherwise

Example	`// Configure GPIOs 0, 1, 2, and 3 as inputs and configure`
	`// GPIO4 as an output. Also, generate an interrupt when a`
	`// rising edge is detected on GPIO3.`
	`Sys_GPIO_Configure( GPIO0_INPUT | GPIO1_INPUT | GPIO2_INPUT |\`
	`GPIO3_INPUT | GPIO4_OUTPUT |\`
	`GPIO_INT_SRC_GPIO3 |\`
	`GPIO_INT_EVENT_RISING_EDGE )`

10.60 SYS_GPIO_DIRECTION

Set the GPIO directions

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_GPIO_Direction( gpio_io )`
Source File	sk3_sys_gpio.cfi
Description	Set the GPIO directions
Inputs	gpio_io = Bitfield for direction (1 - Input, 0 - Output). Use GPIOx_[INPUT | OUTPUT].
Outputs	None
Register Modifications	y1, b1
Assumptions	None
Clock Cycles	6

Example	`// Configure GPIO's 0, 1, 2, and 3 as inputs and configure`
	`// GPIO4 as an output. Leaves the GPIO interrupt configuration`
	`// intact.`
	`Sys_GPIO_Direction( GPIO0_INPUT | GPIO1_INPUT | GPIO2_INPUT |\`
	`GPIO3_INPUT | GPIO4_OUTPUT )`

10.61 SYS_GPIO_GET_ALL_DATA

Get the GPIO input and store it in the specified register

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_GPIO_Get_All_Data( data_reg )`
Source File	sk3_sys_gpio.cfi
Description	Get the GPIO input and store it in the specified register. The returned data reflects the physical data on the GPIO input pins and the values that the user application stored to the shadow register for GPIO output pins for BelaSigna 300 and Ezairo 5900 version 1.02.xx. For all other versions, the returned data reflects the physical data on the GPIO pins for all configurations.
Inputs	data_reg = Data register to load the GPIO data to (cannot be y1)
Outputs	data_reg = Current GPIO input data, output shadow values
Register Modifications	y1
Assumptions	None
Clock Cycles	8 for SK3_CID 102, 201; 1 otherwise

Example	`// Read the value from the GPIO inputs, combine it with the output`
	`// data in the GPIO shadow register and store it in x1`
	`Sys_GPIO_Get_All_Data( x1 )`

10.62 SYS_GPIO_GET_DATA

Get the GPIO input and store it in the specified register

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_GPIO_Get_Data( reg )`
Source File	sk3_sys_gpio.cfi
Description	Get the GPIO input and store it in the specified register. The returned data reflects the physical data on the GPIO pins for Ezairo 5900 version 1.01.xx and Ezairo 6200 version 3.01.xx. The returned data reflects the physical data on the GPIO input pins with 0 being returned for GPIO output pins for Ezairo 5900 version 1.02.xx and BelaSigna 300.
Inputs	reg = Register to load the GPIO data to
Outputs	reg = Current GPIO input data
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Read the value from the GPIO inputs and store it in x1` `Sys_GPIO_Get_Data( x1 )`

10.63 SYS_GPIO_INTCONFIG

Configure the GPIO interrupt

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_GPIO_IntConfig( gpio_int )`
Source File	sk3_sys_gpio.cfi
Description	Configure the GPIO interrupt
Inputs	gpio_int = GPIO interrupt configuration. Use GPIO_INT_SRC_* and GPIO_INT_EVENT_*
Outputs	None
Register Modifications	y1, b1
Assumptions	None
Clock Cycles	6

Example	`// Configure GPIO3 to generate an interrupt when a rising edge`
	`// is detected. Leaves the GPIO direction configuration intact.`
	`Sys_GPIO_IntConfig( GPIO_INT_SRC_GPIO3 | \`
	`GPIO_INT_EVENT_RISING_EDGE )`

10.64 SYS_GPIO_MUX_SELECT

Configure the GPIO pins for GPIO or alternate functions

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_GPIO_MUX_Select( gpio_mux )`
Source File	sk3_sys_gpio.cfi
Description	Configure the GPIO pins for GPIO or alternate functions
Inputs	gpio_mux = GPIO pin functions. Use GPIO*_SELECT_*
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if gpio_mux can be sign-extended from 8 bits; 2 otherwise

Example	`// Configure GPIO pins 0, 1 and 2 as GPIOs.`
	`// Configure GPIO pins 3 and 4 as LSAD inputs.`
	`Sys_GPIO_MUX_Select( GPIO0_SELECT_GPIO | GPIO1_SELECT_GPIO |\`
	`GPIO2_SELECT_GPIO | GPIO3_SELECT_LSAD |\`
	`GPIO4_SELECT_LSAD )`

10.65 SYS_GPIO_SET_DATA

Set the GPIO output (and shadow register) to the specified value

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_GPIO_Set_Data( gpio_data )`
Source File	sk3_sys_gpio.cfi
Description	Set the GPIO output (and shadow register) to the specified value
Inputs	gpio_data = GPIO pin output data
Outputs	None
Register Modifications	None
Assumptions	GPIO direction is set to output
Clock Cycles	2 for SK3_CID 102, 201; 1 otherwise
Example	`// Set the GPIO output data to 0xA.` `Sys_GPIO_Set_Data( 0xA )`

10.66 SYS_GPIO_SET_HIGH

Set the specified GPIO output value to high

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_GPIO_Set_High( gpio_pin )`
Source File	sk3_sys_gpio.cfi
Description	Set the specified GPIO output value to high
Inputs	gpio_pin = GPIO pin to set high
Outputs	None
Register Modifications	y1
Assumptions	GPIO direction for specified pin is set to output
Clock Cycles	4 for SK3_CID 102, 201; 3 otherwise
Example	`// Set GPIO2 output to high.` `Sys_GPIO_Set_High( 2 )`

10.67 SYS_GPIO_SET_LOW

Set the specified GPIO output value to low

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_GPIO_Set_Low( gpio_pin )`
Source File	sk3_sys_gpio.cfi
Description	Set the specified GPIO output value to low
Inputs	gpio_pin = GPIO pin to set low
Outputs	None
Register Modifications	y1
Assumptions	GPIO direction for specified pin is set to output
Clock Cycles	4 for SK3_CID 102, 201; 3 otherwise
Example	`// Set GPIO2 output to low.` `Sys_GPIO_Set_Low( 2 )`

10.68 SYS_GPIO_TOGGLE

Toggle the specified GPIO output pin

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_GPIO_Toggle( gpio_pin )`
Source File	sk3_sys_gpio.cfi
Description	Toggle the specified GPIO output pin
Inputs	gpio_pin = GPIO pin to toggle
Outputs	None
Register Modifications	y1
Assumptions	GPIO direction for specified pin is set to output
Clock Cycles	4 for SK3_CID 102, 201; 3 otherwise
Example	`// Toggle GPIO2 output (low becomes high or high becomes low).` `Sys_GPIO_Toggle( 2 )`

10.69 SYS_GPIO_TOGGLE_BITS

Toggle the specified GPIO output pins

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_GPIO_Toggle_Bits( gpio_bitfield )`
Source File	sk3_sys_gpio.cfi
Description	Toggle the specified GPIO output pins
Inputs	gpio_bitfield = GPIO pins to toggle (1 - toggle, 0 - do not toggle)
Outputs	None
Register Modifications	y1, b1
Assumptions	GPIO direction for specified pins are set to output
Clock Cycles	5 for SK3_CID 102, 201; 4 otherwise

Example	`// Toggle GPIO2 and GPIO0 outputs (low becomes high`
	`// or high becomes low). Leaves the values on GPIO4, 3,`
	`// and 1 intact.`
	`Sys_GPIO_Toggle( 0x5 )`

10.70 SYS_HEAR_COMMAND

Issue a command to execute a HEAR function chain

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_HEAR_Command( cmd )`
Source File	sk3_sys_hear.cfi
Description	Issue a command to execute a HEAR function chain.
Inputs	cmd = Command to issue to the HEAR
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Issue a command to the HEAR to trigger function chain 1` `Sys_HEAR_Command( 1 )`

10.71 SYS_HEAR_FUNCTIONCHAIN_ENABLE

Enable specified HEAR function chains

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_HEAR_FunctionChain_Enable( function_ebl )`
Source File	sk3_sys_hear.cfi
Description	Enable specified HEAR function chains
Inputs	function_ebl = HEAR function chains to enable
Outputs	None
Register Modifications	y1
Assumptions	Previous function chain settings are no longer needed, as they are overwritten with this macro.
Clock Cycles	1 if function_ebl can be sign-extended from 8 bits; 2 otherwise

Example	`// Enable the function chains triggered by the HEAR Configurable`
	`// Accelerator commands 0, 1 and 2 as well as FIFO A0 and B0.`
	`Sys_HEAR_FunctionChain_Enable( HEAR_FC_EBL_CMD0 |`
	`HEAR_FC_EBL_CMD1 |`
	`HEAR_FC_EBL_CMD2 |`
	`HEAR_FC_EBL_FIFO_A0 |`
	`HEAR_FC_EBL_FIFO_B0 )`

10.72 SYS_HEAR_FUNCTIONCHAIN_PRIORITY

Set the priority of HEAR Function chains

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_HEAR_FunctionChain_Priority( function_prio )`
Source File	sk3_sys_hear.cfi
Description	Set the priority of HEAR Function chains
Inputs	function_prio = HEAR function chains priority setting
Outputs	None
Register Modifications	y1
Assumptions	Previous HEAR function chain priority settings are no longer needed as they are overwritten with this macro
Clock Cycles	1 if function_prio can be sign-extended from 8 bits; 2 otherwise

Example	`// Assign high priority to the function chains triggered by`
	`// command 2 and FIFO A0. The remaining function chains default`
	`// to normal priority.`
	`Sys_HEAR_FunctionChain_Priority( HEAR_FC_HIGHPRIO_CMD2 | \`
	`HEAR_FC_HIGHPRIO_FIFO_A0 )`

10.73 SYS_HEAR_PAUSE

Pause the HEAR Configurable Accelerator

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_HEAR_Pause`
Source File	sk3_sys_hear.cfi
Description	Pause the HEAR Configurable Accelerator
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Pause the HEAR Configurable Accelerator` `Sys_HEAR_Pause`

10.74 SYS_HEAR_RESET

Reset the HEAR Configurable Accelerator and the FIFO controllers

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_HEAR_Reset`
Source File	sk3_sys_hear.cfi
Description	Reset the HEAR Configurable Accelerator and the FIFO controllers
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Reset the HEAR Configurable Accelerator` `Sys_HEAR_Reset`

10.75 SYS_HEAR_START

Start (or resume) the HEAR Configurable Accelerator

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_HEAR_Start`
Source File	sk3_sys_hear.cfi
Description	Start (or resume) the HEAR Configurable Accelerator
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Start the HEAR Configurable Accelerator` `Sys_HEAR_Start`

10.76 SYS_I2C_ACK

Manually ACK the latest I²C byte

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_I2C_ACK`
Source File	sk3_sys_i2c.cfi
Description	Manually ACK the latest I²C byte.
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Manually ACK the latest I2C byte` `Sys_I2C_ACK`

10.77 SYS_I2C_CONFIG

Enable the I²C interface with the specified interface configuration

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_I2C_Config( cfg )`
Source File	sk3_sys_i2c.cfi
Description	Enable the I²C interface with the specified interface configuration. *Note:* Enabling master mode disables slave mode and similarly, enabling slave mode disables master mode.
Inputs	cfg = Combination of I2C_CFG_SLAVE_[ENABLE | DISABLE], I2C_CFG_AUTO_ACK_[ENABLE | DISABLE], I2C_CFG_STOP_INT_[ENABLE | DISABLE], I2C_MASTER_PRESCALE_x, and an I2C_CFG_SLAVE_ID setting
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if cfg can be sign-extended from 8 bits; 2 otherwise

Example	`// Enable the I2C slave and set the address to 0x10. Also enable`
	`// auto-acknowledge mode.`
	`Sys_I2C_Config( I2C_CFG_SLAVE_ENABLE | \`
	`I2C_CFG_AUTO_ACK_ENABLE | \`
	`(0x10 << I2C_CFG_SLAVE_ID_POS) )`

10.78 SYS_I2C_NACK

Manually NACK the latest I²C byte

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_I2C_NACK`
Source File	sk3_sys_i2c.cfi
Description	Manually NACK the latest I²C byte.
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Manually NACK the latest I2C byte` `Sys_I2C_NACK`

10.79 SYS_I2C_NACK_AND_STOP

Manually NACK the last I²C byte and issue a stop condition (Applies to I²C master mode transfers only)

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_I2C_NACK_And_Stop`
Source File	sk3_sys_i2c.cfi
Description	Manually NACK the last I²C byte and issue a stop condition (Applies to I²C master mode transfers only)
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Manually NACK the latest I2C byte and issue a stop condition` `Sys_I2C_NACK_And_Stop`

10.80 SYS_I2C_PIN_CONFIG

Configure the I²C interface bus

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_I2C_Pin_Config( cfg )`
Source File	sk3_sys_i2c.cfi
Description	Configure the I²C interface bus
Inputs	cfg = Combination of I2C_PIN_CFG_FILTER_[ENABLE | DISABLE], I2C_PIN_CFG_PULL_UP_[250K | 1K], and I2C_PIN_CFG_PULLUP_[ENABLE | DISABLE]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Configure the I2C bus behavior`
	`Sys_I2C_Pin_Config( I2C_PIN_CFG_FILTER_ENABLE | \`
	`I2C_PIN_CFG_PULLUP_1K | \`
	`I2C_PIN_CFG_PULLUP_ENABLE )`

10.81 SYS_I2C_RESET

Reset the general purpose I²C interface

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_I2C_Reset`
Source File	sk3_sys_i2c.cfi
Description	Reset the general purpose I²C interface.
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Reset the general purpose I2C interface.` `Sys_I2C_Reset`

10.82 SYS_I2C_SLAVE_DISABLE

Disable slave mode I²C interface transfers (no configuration changes)

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_I2C_Slave_Disable`
Source File	sk3_sys_i2c.cfi
Description	Disable slave mode I²C interface transfers (no configuration changes)
Inputs	None
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	3
Example	`// Disable the I2C interface slave mode, preventing slave transfers` `Sys_I2C_Slave_Disable`

10.83 SYS_I2C_SLAVE_ENABLE

Enable slave mode I²C interface transfers (no configuration changes)

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_I2C_Slave_Enable`
Source File	sk3_sys_i2c.cfi
Description	Enable slave mode I²C interface transfers (no configuration changes)
Inputs	None
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	3
Example	`// Enable the I2C interface for slave mode transfers` `Sys_I2C_Slave_Enable`

10.84 SYS_I2C_START_READ

Start an I²C read transfer (Applies to I²C master mode transfers only)

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_I2C_Start_Read( addr )`
Source File	sk3_sys_i2c.cfi
Description	Start an I²C read transfer (Applies to I²C master mode transfers only)
Inputs	addr = Address of remote I2C slave device
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Start an I2C master mode transfer read from a device`
	`// connected to the bus at I2C_ADDRESS.`
	`Sys_I2C_Start_Read( I2C_ADDRESS )`

10.85 SYS_I2C_START_WRITE

Start an I²C write transfer (Applies to I²C master mode transfers only)

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_I2C_Start_Write( addr )`
Source File	sk3_sys_i2c.cfi
Description	Start an I²C write transfer (Applies to I²C master mode transfers only)
Inputs	addr = Address of remote I2C slave device
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Start an I2C master mode transfer writing to any devices`
	`// connected to the bus at I2C_ADDRESS.`
	`Sys_I2C_Start_Write( I2C_ADDRESS )`

10.86 SYS_I2C_WATCHDOG_CONFIG

Configure the I²C interface bus watchdog timeout

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_I2C_Watchdog_Config( cfg )`
Source File	sk3_sys_i2c.cfi
Description	Configure the I²C interface bus watchdog timeout
Inputs	cfg = I2C_WATCHDOG_TIMEOUT_[250M | 500M | 1000M | 2000M | 4000M | DISABLED]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Configure the I2C bus watchdog timeout for 1 second` `Sys_I2C_Watchdog_Config( I2C_WATCHDOG_TIMEOUT_1000M )`

10.87 SYS_IMM_N

Sign-extend a constant expression from its n LSBs, for n = 1, 2, ..., 32

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_IMM_n( const )`
Source File	sk3_sys_utils.cfi
Description	Sign-extend a constant expression from its n LSBs, for n = 1, 2, ..., 32. The result is in the range of a signed n-bit number so it fits into an n-bit (or larger) signed immediate field.
Inputs	const = Constant expression to convert
Outputs	Sys_IMM_n = Constant expression with the same n LSBs as the input, but sign-extended from bit n-1 (where the LSB is bit 0) so that the result is in the signed n-bit range
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)

Example	`// Sign-extend a 16-bit initializer.  Since all bits above bit 7`
	`// are the same as bit 7, this value can be stored using the`
	`// signed 8-bit immediate field of the store-immediate instruction.`
	`store p[D_CRC_VALUE], Sys_IMM_16( 0xFFFF )`
	
	`// Load 0.125 (converted to the <1,47> fractional format) into a0.`
	`// Sys_IMM_24 extracts the low 24 bits of the 48-bit number as a`
	`// signed 24-bit value to store into a0.l.`
	`#set VALUE Sys_frac48( 1250000 )`
	`#set VAL_H VALUE >> 24`
	`#set VAL_L Sys_IMM_24( VALUE )`
	`load a0, VAL_H`
	`load a0.l, VAL_L`
	
	`// Test whether a constant expression in a macro parameter can be`
	`// stored into a 12-bit control register using a store-immediate`
	`// instruction (signed 8-bit immediate field).`
	`#if Sys_IMM_8( param ) == Sys_IMM_12( param )`
	`    // Bits 7-11 are all the same so store-immediate can be used`
	`    store p[CONTROL_REG], Sys_IMM_8( param )`
	`#else`
	`    // Value cannot be represented in 8 bits, so use a register`
	`    load y1, (param)`
	`    store p[CONTROL_REG], y1`
	`#endif`

10.88 SYS_INT_ACK

Acknowledge a set of interrupts

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Int_Ack( int_ack )`
Source File	sk3_sys_int.cfi
Description	Acknowledge a set of interrupts
Inputs	int_ack = Interrupts to acknowledge
Outputs	None
Register Modifications	b1
Assumptions	Previous interrupt settings are no longer needed, as they are overwritten with this macro
Clock Cycles	1 if int_ack can be sign-extended from 8 bits; 2 if int_ack can be sign-extended from 24 bits; 3 otherwise
Example	`// Manually acknowledge timer 1 and PCM interrupts` `Sys_Int_Ack( INT_ACK_TIMER_1 | INT_ACK_PCM )`

10.89 SYS_INT_ENABLE

Enable a set of interrupts

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Int_Enable( int_ebl )`
Source File	sk3_sys_int.cfi
Description	Enable a set of interrupts
Inputs	int_ebl = Interrupts to enable
Outputs	None
Register Modifications	b1
Assumptions	Previous interrupt settings are no longer needed, as they are overwritten with this macro
Clock Cycles	1 if int_ebl can be sign-extended from 8 bits; 2 if int_ebl can be sign-extended from 24 bits; 3 otherwise

Example	`// Enable interrupts for watchdog, timer 1, and FIFO A0.`
	`// NOTE: Master interrupts and core interrupts must be enabled to`
	`//  receive interrupts`
	`Sys_Int_Enable( INT_EBL_MASTER | INT_EBL_WATCHDOG | \`
	`  INT_EBL_TIMER_1 | INT_EBL_FIFO_A0 )`

10.90 SYS_INT_MASTER_DISABLE

Disable all interrupts at the interrupt controller

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Int_Master_Disable`
Source File	sk3_sys_int.cfi
Description	Disable all interrupts at the interrupt controller
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`Critical_Section_Start:`
	`// Disables all interrupts at the interrupt controller level`
	`Sys_Int_Master_Disable`
	
	`// Do processing with interrupts disabled`
	`Critical_Section_End:`
	
	`// Re-enable previously disabled interrupts`
	`Sys_Int_Master_Enable`

10.91 SYS_INT_MASTER_ENABLE

Enable all interrupts at the interrupt controller

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Int_Master_Enable`
Source File	sk3_sys_int.cfi
Description	Enable all interrupts at the interrupt controller
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`Critical_Section_Start2:`
	`// Disables all interrupts at the interrupt controller level`
	`Sys_Int_Master_Disable`
	
	`// Do processing with interrupts disabled`
	`Critical_Section_End2:`
	
	`// Re-enable previously disabled interrupts`
	`Sys_Int_Master_Enable`

10.92 SYS_INT_PRIORITY

Configure the interrupt priority

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Int_Priority( int_prio )`
Source File	sk3_sys_int.cfi
Description	Configure the interrupt priority
Inputs	int_prio = Interrupt priority configuration
Outputs	None
Register Modifications	b1
Assumptions	Previous interrupt settings are no longer needed, as they are overwritten with this macro
Clock Cycles	1 if int_prio can be sign-extended from 8 bits; 2 if int_prio can be sign-extended from 24 bits; 3 otherwise

Example	`// Set the FIFO A0 interrupt and HEAR 0 interrupt as having highest`
	`// priority`
	`Sys_Int_Priority( INT_HIGHPRIO_FIFO_A0 | INT_HIGHPRIO_HEAR_0 )`

10.93 SYS_IOC_FIFOCONFIG

Set up the IOC FIFO configuration

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_IOC_FIFOConfig( arg )`
Source File	sk3_sys_ioc.cfi
Description	Set up the IOC FIFO configuration
Inputs	arg = Combination of IOC_FIFO_CFG_*_DBL_ACC, IOC_FIFO_CFG_DAC_[SATURATE | UNSATURATE], and IOC_FIFO_CFG_*_SHIFT_[RIGHT | LEFT]
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if arg can be sign-extended from 8 bits; 2 otherwise

Example	`// DAC, ADC0, ADC1 - shifted right`
	`// Enable double access`
	`#define IOC_FIFO_CFG_VAL ( IOC_FIFO_CFG_DAC_SHIFT_RIGHT | \`
	`IOC_FIFO_CFG_ADC0_SHIFT_RIGHT | \`
	`IOC_FIFO_CFG_ADC1_SHIFT_RIGHT | \`
	`IOC_FIFO_CFG_ADC2_SHIFT_RIGHT | \`
	`IOC_FIFO_CFG_ADC3_SHIFT_RIGHT | \`
	`IOC_FIFO_CFG_FB0_DBL_ACC_EN | \`
	`IOC_FIFO_CFG_FB1_DBL_ACC_EN | \`
	`IOC_FIFO_CFG_FB2_DBL_ACC_EN | \`
	`IOC_FIFO_CFG_FB3_DBL_ACC_EN | \`
	`IOC_FIFO_CFG_FA0_DBL_ACC_EN | \`
	`IOC_FIFO_CFG_FA1_DBL_ACC_EN | \`
	`IOC_FIFO_CFG_FA2_DBL_ACC_EN | \`
	`IOC_FIFO_CFG_FA3_DBL_ACC_EN )`
	`Sys_IOC_FIFOConfig( IOC_FIFO_CFG_VAL )`

10.94 SYS_IOC_INPUTCONFIG

Set up the IOC input configuration

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_IOC_InputConfig( arg )`
Source File	sk3_sys_ioc.cfi
Description	Set up the IOC input configuration
Inputs	arg = Combination of IOC_INPUT_CFG_ADC0_*, IOC_INPUT_CFG_ADC1_*, IOC_INPUT_CFG_ADC2_*, IOC_INPUT_CFG_ADC3_*, IOC_INPUT_CFG_PCM0_*, IOC_INPUT_CFG_PCM1_*
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if arg can be sign-extended from 8 bits; 2 otherwise

Example	`// Connect ADC 0 to FIFO A0 and both PCM channels to FIFO A1`
	`#define IOC_INPUT_CFG_VAL ( IOC_INPUT_CFG_ADC0_FA0 |  \`
	`IOC_INPUT_CFG_ADC1_NONE |  \`
	`IOC_INPUT_CFG_ADC2_NONE |  \`
	`IOC_INPUT_CFG_ADC3_NONE |  \`
	`IOC_INPUT_CFG_PCM0_FA1 |  \`
	`IOC_INPUT_CFG_PCM1_FA1 )`
	
	`// Configure how the IOC handles input data`
	`Sys_IOC_InputConfig( IOC_INPUT_CFG_VAL )`

10.95 SYS_IOC_OUTPUTCONFIG

Set up the IOC output configuration

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_IOC_OutputConfig( arg )`
Source File	sk3_sys_ioc.cfi
Description	Set up the IOC output configuration
Inputs	arg = Combination of IOC_OUTPUT_CFG_PCM_SYNC_ADCDAC_[EN | DIS], IOC_OUTPUT_CFG_PCM_DBL_ACC_[EN | DIS], IOC_OUTPUT_CFG_PCM_STEREO_[EN | DIS], IOC_OUTPUT_CFG_DAC_DBL_ACC_[EN | DIS], IOC_OUTPUT_CFG_PCM1_*, IOC_OUTPUT_CFG_PCM0_*, IOC_OUTPUT_CFG_DAC_*
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if arg can be sign-extended from 8 bits; 2 otherwise

Example	`// Connect DAC to FIFO B0. Enable double access mode.`
	`#define IOC_OUTPUT_CFG_VAL ( IOC_OUTPUT_CFG_DAC_FB0 | \`
	`IOC_OUTPUT_CFG_PCM0_NONE | \`
	`IOC_OUTPUT_CFG_PCM1_NONE | \`
	`IOC_OUTPUT_CFG_PCM_STEREO_DIS | \`
	`IOC_OUTPUT_CFG_PCM_DBL_ACC_DIS | \`
	`IOC_OUTPUT_CFG_DAC_DBL_ACC_EN )`
	`Sys_IOC_OutputConfig( IOC_OUTPUT_CFG_VAL )`

10.96 SYS_LOG2

Calculate the integer log base-2 (ceiling, always positive) for a supplied constant

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Log2( const )`
Source File	sk3_sys_utils.cfi
Description	Calculate the integer log base-2 (ceiling, always positive) for a supplied constant. The minimum value for the constant is 1. The maximum value is 0x1FFFFFFFFF when the macro is evaluated by the preprocessor (in #set or #if) and 0x7FFFFFF when the macro is evaluated by the assembler (in an instruction or a directive).
Inputs	const = Calculate the integer log base-2 for this value
Outputs	Sys_Log2 = Approximation of log2(const), rounded up to the nearest integer value and always positive - Sys_Log2(1) is 1, not 0.
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)
Example	`// Load the value of log base-2 of 0x1000 to x0 (x0 = 0xC)` `load x0, Sys_Log2( 0x1000 )`

10.97 SYS_LOG2_CEIL

Calculate the integer log base-2 (ceiling) for a supplied constant

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Log2_Ceil( const )`
Source File	sk3_sys_utils.cfi
Description	Calculate the integer log base-2 (ceiling) for a supplied constant. The minimum value for the constant is 1. The maximum value is 0x1FFFFFFFFF when the macro is evaluated by the preprocessor (in #set or #if) and 0x7FFFFFF when the macro is evaluated by the assembler (in an instruction or a directive).
Inputs	const = Calculate ceiling(log base-2) for this value
Outputs	Sys_Log2_Ceil = log2(const), rounded up to the nearest integer value.
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)

Example	`// Load the value of log base-2 (ceiling) of 0x1000 to x0 (x0 = 0xC)`
	`load x0, Sys_Log2_Ceil( 0x1000 )`
	
	`// Load the value of log base-2 (ceiling) of 0x1001 to x1 (x1 = 0xD)`
	`load x1, Sys_Log2_Ceil( 0x1001 )`

10.98 SYS_LOG2_FLOOR

Calculate the integer log base-2 (floor) for a supplied constant

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Log2_Floor( const )`
Source File	sk3_sys_utils.cfi
Description	Calculate the integer log base-2 (floor) for a supplied constant. The minimum value for the constant is 0. The maximum value is 0x1FFFFFFFFF when the macro is evaluated by the preprocessor (in #set or #if) and 0x7FFFFFF when the macro is evaluated by the assembler (in an instruction or a directive).
Inputs	const = Calculate floor(log base-2) for this value
Outputs	Sys_Log2_Floor = log2(const), rounded down to the nearest integer value. When the input is 0, the output is -1.
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)

Example	`// Load the value of log base-2 (floor) of 0x1000 to x0 (x0 = 0xC)`
	`load x0, Sys_Log2_Floor( 0x1000 )`
	
	`// Load the value of log base-2 (floor) of 0x1001 to x1 (x1 = 0xC)`
	`load x1, Sys_Log2_Floor( 0x1001 )`

10.99 SYS_M24P24

Convert a decimal number with up to seven digits of precision to <24,24> fixed-point format

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_m24p24( const )`
Source File	sk3_sys_utils.cfi
Description	Convert a decimal number with up to seven digits of precision to <24,24> fixed-point format. The developer must pre-multiply the decimal number by 10^7 before passing it to the macro so that the macro receives an integer. For example, 7.125 becomes 71250000. The 10^7 multiplier provides 23 fractional bits of accuracy. The valid input range (after pre-multiplication) for <24,24> fixed-point numbers is -83886080000000 to +83886079999999. Numbers outside this range are saturated to the closest valid value, but the results are undefined for inputs with magnitude greater than about 2^62 or 4 x 10^18 (representing numbers greater than about 4 x 10^11). This macro is useful for filling accumulators through a pair of immediate loads and for defining <24,24> data to be loaded from XMEM, YMEM, or XYMEM.
Inputs	const = Constant to convert to <24,24>, multiplied by 10^7
Outputs	Sys_m24p24 = <24,24> fixed-point approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^62
Clock Cycles	0 (evaluated during compile)

Example	`// Set VALUE to 7.125 converted to the <24,24> fixed-point format`
	`// and then store the two halves of the 48-bit value in memory at`
	`// Direct_Data_Value_m24p24.`
	`#set VALUE Sys_m24p24( 71250000 )`
	`#set VAL_H VALUE >> 24`
	`#set VAL_L Sys_IMM_24(VALUE)`
	`Direct_Data_Value_m24p24:`
	`.data VAL_H, VAL_L`

10.100 SYS_M2P46

Convert a decimal number with up to seven digits of precision to <2,46> fixed-point format

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_m2p46( const )`
Source File	sk3_sys_utils.cfi
Description	Convert a decimal number with up to seven digits of precision to <2,46> fixed-point format. The developer must pre-multiply the decimal number by 10^7 before passing it to the macro so that the macro receives an integer. For example, 1.125 becomes 11250000. The 10^7 multiplier provides 23 fractional bits of accuracy. The valid input range (after pre-multiplication) for <2,46> fixed-point numbers is -20000000 to +19999999. Numbers outside this range are saturated to the closest valid value, but the results are undefined for inputs with magnitude greater than about 2^40 or 10^12 (representing numbers greater than about 100000.0). *Note:* This macro places the number in a <2,46> field, but provides only <2,45> accuracy. The bottom bit of an unsaturated result is always zero. This macro is useful for filling accumulators through a pair of immediate loads and for defining <2,46> data to be loaded from XMEM, YMEM, or XYMEM.
Inputs	const = Constant to convert to <2,46>, multiplied by 10^7
Outputs	Sys_m2p46 = <2,46> fixed-point approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^40
Clock Cycles	0 (evaluated during compile)

Example	`// Set VALUE to 1.125 converted to the <2,46> fixed-point format`
	`// and then store the two halves of the 48-bit value in memory at`
	`// Direct_Data_Value_m2p46.`
	`#set VALUE Sys_m2p46( 11250000 )`
	`#set VAL_H VALUE >> 24`
	`#set VAL_L Sys_IMM_24(VALUE)`
	`Direct_Data_Value_m2p46:`
	`.data VAL_H, VAL_L`

10.101 SYS_M8P16

Convert a decimal number with up to seven digits of precision to <8,16> fixed-point format

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_m8p16( const )`
Source File	sk3_sys_utils.cfi
Description	Convert a decimal number with up to seven digits of precision to <8,16> fixed-point format. The developer must pre-multiply the decimal number by 10^7 before passing it to the macro so that the macro receives an integer. For example, 7.125 becomes 71250000. The 10^7 multiplier provides 23 fractional bits of accuracy, although the <8,16> format only stores 16 fractional bits. The valid input range (after pre-multiplication) for <8,16> fixed-point numbers is -1280000000 to +1279999999. Numbers outside this range are saturated to the closest valid value, but the results are undefined for inputs with magnitude greater than about 2^53 or 9 x 10^15 (representing numbers greater than about 900000000.0). This macro is useful for filling data registers through an immediate load and for defining <8,16> data to be loaded from XMEM or YMEM.
Inputs	const = Constant to convert to <8,16>, multiplied by 10^7
Outputs	Sys_m8p16 = <8,16> fixed-point approximation of the supplied constant
Register Modifications	None
Assumptions	Input magnitude less than 2^53
Clock Cycles	0 (evaluated during compile)

Example	`// Set VALUE to 7.125 converted to the <8,16> fixed-point format`
	`// and then store this to a0.`
	`#set VALUE Sys_m8p16( 71250000 )`
	`load a0, VALUE`

10.102 SYS_MEM_COPYPP

Copy data from program memory to program memory

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Mem_CopyPP( src, dest, len )`
Source File	sk3_sys_mem.cfi
Description	Copy data from program memory to program memory
Inputs	src = Source of data dest = Destination for data len = Number of words to copy (minimum 1, maximum 4096)
Outputs	None
Register Modifications	b1, yp2, yp3
Assumptions	The hardware loop stack is not full. The destination data block starts below the source data block or is located above the source data block.
Clock Cycles	3 + 2 * len

Example	`// Copy 100 words from program memory at address 0x1500 to`
	`// program memory at address 0x1600.`
	`Sys_Mem_CopyPP( 0x1500, 0x1600, 100 )`

10.103 SYS_MEM_COPYPX

Copy data from program memory to X memory

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Mem_CopyPX( src, dest, len )`
Source File	sk3_sys_mem.cfi
Description	Copy data from program memory to X memory. Data passes through the saturation unit.
Inputs	src = Source of data dest = Destination for data len = Number of words to copy (minimum 1, maximum 4096)
Outputs	None
Register Modifications	b1, yp3, xp7
Assumptions	The hardware loop stack is not full
Clock Cycles	3 + 2 * len

Example	`// Copy 100 words from program memory at address 0x1500 to`
	`// X memory at address 0x1000.`
	`Sys_Mem_CopyPX( 0x1500, 0x1000, 100 )`

10.104 SYS_MEM_COPYPY

Copy data from program memory to Y memory

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Mem_CopyPY( src, dest, len )`
Source File	sk3_sys_mem.cfi
Description	Copy data from program memory to Y memory. Data passes through the saturation unit.
Inputs	src = Source of data dest = Destination for data len = Number of words to copy (minimum 1, maximum 4096)
Outputs	None
Register Modifications	b1, yp2, yp3
Assumptions	The hardware loop stack is not full
Clock Cycles	3 + 2 * len

Example	`// Copy 100 words from program memory at address 0x1500 to`
	`// Y memory at address 0x0200.`
	`Sys_Mem_CopyPY( 0x1500, 0x0200, 100 )`

10.105 SYS_MEM_COPYXP

Copy data from X memory to program memory

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Mem_CopyXP( src, dest, len )`
Source File	sk3_sys_mem.cfi
Description	Copy data from X memory to program memory. Data is sign-extended when copied.
Inputs	src = Source of data dest = Destination for data len = Number of words to copy (minimum 1, maximum 4096)
Outputs	None
Register Modifications	b1, yp3, xp7
Assumptions	The hardware loop stack is not full
Clock Cycles	3 + 2 * len

Example	`// Copy 100 words from X memory at address 0x1000 to`
	`// program memory at address 0x1500.`
	`Sys_Mem_CopyXP( 0x1000, 0x1500, 100 )`

10.106 SYS_MEM_COPYXX

Copy data from X memory to X memory

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Mem_CopyXX( src, dest, len )`
Source File	sk3_sys_mem.cfi
Description	Copy data from X memory to X memory.
Inputs	src = Source of data dest = Destination for data len = Number of words to copy (minimum 1, maximum 4096)
Outputs	None
Register Modifications	y1, xp6, xp7
Assumptions	The hardware loop stack is not full. The destination data block starts below the source data block or is located above the source data block.
Clock Cycles	3 + 2 * len

Example	`// Copy 100 words from X memory at address 0x1000 to`
	`// X memory at address 0x1100.`
	`Sys_Mem_CopyXX( 0x1000, 0x1100, 100 )`

10.107 SYS_MEM_COPYXY

Copy data from X memory to Y memory

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Mem_CopyXY( src, dest, len )`
Source File	sk3_sys_mem.cfi
Description	Copy data from X memory to Y memory.
Inputs	src = Source of data dest = Destination for data len = Number of words to copy (minimum 1, maximum 4096)
Outputs	None
Register Modifications	y1, xp7, yp3
Assumptions	The hardware loop stack is not full
Clock Cycles	3 + 2 * len

Example	`// Copy 100 words from X memory at address 0x1000 to`
	`// Y memory at address 0x0200. NOTE: this macro has a`
	`// small P memory footprint, but uses 3+200 cycles.`
	`Sys_Mem_CopyXY( 0x1000, 0x0200, 100 )`

10.108 SYS_MEM_COPYXY_QUICK

Copy data from X memory to Y memory

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Mem_CopyXY_Quick( src, dest, len )`
Source File	sk3_sys_mem.cfi
Description	Copy data from X memory to Y memory. A dual-move instruction allows one word to be copied per clock cycle. This implementation has a more efficient run time for len>6, at the cost of 7 extra P-memory words.
Inputs	src = Source of data dest = Destination for data len = Number of words to copy (minimum 3, maximum 4098)
Outputs	None
Register Modifications	b1, y1, yp3
Assumptions	The hardware loop stack is not full
Clock Cycles	9 + len

Example	`// Copy 100 words from X memory at address 0x1000 to`
	`// Y memory at address 0x0200.  NOTE: 7 extra P memory`
	`// words are required, but only 9+100 cycles are needed.`
	`Sys_Mem_CopyXY_Quick( 0x1000, 0x0200, 100 )`

10.109 SYS_MEM_COPYYP

Copy data from Y memory to program memory

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Mem_CopyYP( src, dest, len )`
Source File	sk3_sys_mem.cfi
Description	Copy data from Y memory to program memory. Data is sign-extended when copied.
Inputs	src = Source of data dest = Destination for data len = Number of words to copy (minimum 1, maximum 4096)
Outputs	None
Register Modifications	b1, yp2, yp3
Assumptions	The hardware loop stack is not full
Clock Cycles	3 + 2 * len

Example	`// Copy 100 words from Y memory at address 0x0200 to`
	`// program memory at address 0x1500.`
	`Sys_Mem_CopyYP( 0x0200, 0x1500, 100 )`

10.110 SYS_MEM_COPYYX

Copy data from Y memory to X memory

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Mem_CopyYX( src, dest, len )`
Source File	sk3_sys_mem.cfi
Description	Copy data from Y memory to X memory.
Inputs	src = Source of data dest = Destination for data len = Number of words to copy (minimum 1, maximum 4096)
Outputs	None
Register Modifications	y1, xp7, yp3
Assumptions	The hardware loop stack is not full
Clock Cycles	3 + 2 * len

Example	`// Copy 100 words from Y memory at address 0x0200 to`
	`// X memory at address 0x1000. NOTE: this macro has a`
	`// small P memory footprint, but uses 3+200 cycles.`
	`Sys_Mem_CopyYX( 0x0200, 0x1000, 100 )`

10.111 SYS_MEM_COPYYX_QUICK

Copy data from Y memory to X memory

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Mem_CopyYX_Quick( src, dest, len )`
Source File	sk3_sys_mem.cfi
Description	Copy data from Y memory to X memory. A dual-move instruction allows one word to be copied per clock cycle. This implementation has a more efficient run time for len>6, at the cost of 7 extra P-memory words.
Inputs	src = Source of data dest = Destination for data len = Number of words to copy (minimum 3, maximum 4098)
Outputs	None
Register Modifications	b1, y1, yp3
Assumptions	The hardware loop stack is not full
Clock Cycles	9 + len

Example	`// Copy 100 words from Y memory at address 0x0200 to`
	`// X memory at address 0x1000. NOTE: 7 extra P memory`
	`// words are required, but only 9+100 cycles are needed.`
	`Sys_Mem_CopyYX_Quick( 0x0200, 0x1000, 100 )`

10.112 SYS_MEM_COPYYY

Copy data from Y memory to Y memory

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Mem_CopyYY( src, dest, len )`
Source File	sk3_sys_mem.cfi
Description	Copy data from Y memory to Y memory.
Inputs	src = Source of data dest = Destination for data len = Number of words to copy (minimum 1, maximum 4096)
Outputs	None
Register Modifications	y1, yp2, yp3
Assumptions	The hardware loop stack is not full. The destination data block starts below the source data block or is located above the source data block.
Clock Cycles	3 + 2 * len

Example	`// Copy 100 words from Y memory at address 0x0200 to`
	`// Y memory at address 0x0300.`
	`Sys_Mem_CopyYY( 0x0200, 0x0300, 100 )`

10.113 SYS_MEM_FILLP

Fill a range of program memory with a value

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Mem_FillP( value, addr, len )`
Source File	sk3_sys_mem.cfi
Description	Fill a range of program memory with a value.
Inputs	value = Value to be written to memory addr = Destination address len = Number of words to fill (minimum 1, maximum 4096)
Outputs	None
Register Modifications	b1, yp3
Assumptions	The hardware loop stack is not full
Clock Cycles	4 + len

Example	`// Fill program memory starting at address 0x1500 for 100 words with`
	`// the value 0x12345678`
	`Sys_Mem_FillP( 0x12345678, 0x1500, 100 )`

10.114 SYS_MEM_FILLX

Fill a range of X memory with a value

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Mem_FillX( value, addr, len )`
Source File	sk3_sys_mem.cfi
Description	Fill a range of X memory with a value.
Inputs	value = Value to be written to memory addr = Destination address len = Number of words to fill (minimum 1, maximum 4096)
Outputs	None
Register Modifications	y1, xp7
Assumptions	The hardware loop stack is not full
Clock Cycles	3 + len

Example	`// Fill X memory starting at address 0x1500 for 100 words with`
	`// the value 0x123456`
	`Sys_Mem_FillX( 0x123456, 0x1500, 100 )`

10.115 SYS_MEM_FILLY

Fill a range of Y memory with a value

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Mem_FillY( value, addr, len )`
Source File	sk3_sys_mem.cfi
Description	Fill a range of Y memory with a value.
Inputs	value = Value to be written to memory addr = Destination address len = Number of words to fill (minimum 1, maximum 4096)
Outputs	None
Register Modifications	y1, yp3
Assumptions	The hardware loop stack is not full
Clock Cycles	3 + len

Example	`// Fill Y memory starting at address 0x0200 for 100 words with`
	`// the value 0x123456`
	`Sys_Mem_FillP( 0x123456, 0x0200, 100 )`

10.116 SYS_MEM_PRIORITY

Set either the CFX or the HEAR as having higher priority when accessing shared memories

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Mem_Priority( arg )`
Source File	sk3_sys_mem.cfi
Description	Set either the CFX or the HEAR as having higher priority when accessing shared memories.
Inputs	arg = Either SYSTEM_PRIORITY_CFX or SYSTEM_PRIORITY_HEAR
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Select the CFX DSP core as having priority access to memory in`
	`// case of a memory access conflict with the HEAR core`
	`Sys_Mem_Priority( SYSTEM_PRIORITY_CFX )`

10.117 SYS_MEM_RESERVE

Reserve uninitialized memory in the current memory segment

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Mem_Reserve( name, size )`
Source File	sk3_sys_mem.cfi
Description	Reserve uninitialized memory in the current memory segment
Inputs	name = Global label to give to the reserved memory size = Amount of uninitialized memory to reserve
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)

Example	`// Reserve a block of 20 uninitialized words in X memory.`
	`Sys_Mem_Start_Segment( Table_Seg_Uninit, SYS_MEMSPACE_X )`
	`Sys_Mem_Reserve( Table_Uninitialized, 20 )`

10.118 SYS_MEM_RESERVE_INIT

Reserve memory in the current memory segment and initialize with the specified value

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Mem_Reserve_Init( name, size, value )`
Source File	sk3_sys_mem.cfi
Description	Reserve memory in the current memory segment and initialize with the specified value
Inputs	name = Global label to give to the reserved memory size = Amount of initialized memory to reserve value = Data value to store into the reserved memory; to initialize XY-memory use the Sys_Value_XY macro
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)

Example	`// Reserve a block of 20 words in X memory. Initialize the memory to`
	`// 0x123456.`
	`Sys_Mem_Start_Segment( Table_Seg, SYS_MEMSPACE_X )`
	`Sys_Mem_Reserve_Init( Table_Initialized, 20, 0x123456 )`

10.119 SYS_MEM_RESERVE_INIT_DEFAULT

Reserve memory in the current memory segment and initialize with 0

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Mem_Reserve_Init_Default( name, size )`
Source File	sk3_sys_mem.cfi
Description	Reserve memory in the current memory segment and initialize with 0
Inputs	name = Global label to give to the reserved memory size = Amount of initialized-to-zero memory to reserve
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)

Example	`// Reserve a block of 20 words in X memory, initialized to 0`
	`Sys_Mem_Start_Segment( Table_Seg_Default, SYS_MEMSPACE_X )`
	`Sys_Mem_Reserve_Init_Default( Table_Default, 20 )`

10.120 SYS_MEM_START_ABSOLUTE_SEGMENT

Start an absolute memory segment

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Mem_Start_Absolute_Segment( name, mem_space, addr )`
Source File	sk3_sys_mem.cfi
Description	Start an absolute memory segment
Inputs	name = Name for the memory segment mem_space = Memory space for the segment addr = Absolute starting address for this segment
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)

Example	`// Create a segment for data at a known location in XMEM`
	`#define TABLE_START_ADDR 0x0200`
	`Sys_Mem_Start_Absolute_Segment( Table_A, SYS_MEMSPACE_X, \`
	`TABLE_START_ADDR )`
	`.global Table_A_Data`
	`Table_A_Data:`
	`.data 0x000000`
	`.data 0x000010`
	`.data 0x000012`
	`.data 0x0000FF`
	
	`// Create a segment for code at a known location in PMEM`
	`#define START_CODE_ADDR 0x1200`
	`Sys_Mem_Start_Absolute_Segment( Start_Code, SYS_MEMSPACE_P, \`
	`START_CODE_ADDR )`
	`Start_Code:`
	`nop >>> nop`

10.121 SYS_MEM_START_RELOC_SEGMENT

Start a relocatable memory segment

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Mem_Start_Reloc_Segment( name, mem_space, addr )`
Source File	sk3_sys_mem.cfi
Description	Start a relocatable memory segment
Inputs	name = Name for the memory segment mem_space = Memory space for the segment addr = Base (lower-bound) starting address for this segment
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)

Example	`// Create a relocatable segment for data at some address above`
	`// the specified base`
	`#define TABLE_START_BASE_ADDR 0x0200`
	`Sys_Mem_Start_Reloc_Segment( Table_B, SYS_MEMSPACE_X, \`
	`TABLE_START_BASE_ADDR )`
	`.global Table_B_Data`
	`Table_B_Data:`
	`.data 0x000000`
	`.data 0x000010`
	`.data 0x000012`
	`.data 0x0000FF`
	
	`// Create a relocatable code segment above address 0x1200`
	`#define CODE_BASE 0x1200`
	`Sys_Mem_Start_Reloc_Segment( Reloc_Code, SYS_MEMSPACE_P, \`
	`CODE_BASE )`
	`.global Reloc_Code`
	`Reloc_Code:`
	`nop >>> nop`

10.122 SYS_MEM_START_SEGMENT

Start a relocatable memory segment

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Mem_Start_Segment( name, mem_space )`
Source File	sk3_sys_mem.cfi
Description	Start a relocatable memory segment
Inputs	name = Name for the memory segment mem_space = Memory space for the segment
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)

Example	`// Create a segment for the start of the application`
	`Sys_Mem_Start_Segment( main, SYS_MEMSPACE_P )`
	`.global main`
	`main:`
	`  Sys_Watchdog_Refresh`
	`  goto.0d main`
	
	`// Create a segment for a data table in X memory`
	`Sys_Mem_Start_Segment( datatable, SYS_MEMSPACE_X )`
	`.global datatable`
	`datatable:`
	`.data 0x000001`
	`.data 0x000001`
	`.data 0x000002`
	`.data 0x000003`
	`.data 0x000005`

10.123 SYS_OUTPUT_DISABLE

Disable the output DA converter and clear its configuration

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Output_Disable`
Source File	sk3_sys_analog.cfi
Description	Disable the output DA converter and clear its configuration
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	Existing output configuration no longer required
Clock Cycles	1
Example	`// Disable the output` `Sys_Output_Disable`

10.124 SYS_OUTPUT_ENABLE

Enable and configure the output DA converter

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Output_Enable( arg )`
Source File	sk3_sys_analog.cfi
Description	Enable and configure the output DA converter
Inputs	arg = Configuration for the analog output control register of OD0_HIGHPOWER_[ENABLE | DISABLE]
Outputs	None
Register Modifications	None
Assumptions	Existing output configuration no longer required
Clock Cycles	1
Example	`// Enable the output with high power disabled` `Sys_Output_Enable( OD0_HIGHPOWER_DISABLE )`

10.125 SYS_OUTPUT_MUTE

Disable the output DA converter, maintaining all other settings

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Output_Mute`
Source File	sk3_sys_analog.cfi
Description	Disable the output DA converter, maintaining all other settings
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	3
Example	`// Mute the output` `Sys_Output_Mute`

10.126 SYS_OUTPUT_UNMUTE

Enable the output DA converter, maintaining all other settings

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Output_Unmute`
Source File	sk3_sys_analog.cfi
Description	Enable the output DA converter, maintaining all other settings
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	3
Example	`// Unmute the output` `Sys_Output_Unmute`

10.127 SYS_PCM_CONFIG

Set up the PCM interface for the given configuration

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_PCM_Config( pcm_cfg )`
Source File	sk3_sys_pcm.cfi
Description	Set up the PCM interface for the given configuration
Inputs	pcm_cfg = Configuration consisting of: PCM_[MASTER | SLAVE], PCM_[ENABLE | DISABLE], PCM_SUBFRAME_[ENABLE | DISABLE], PCM_BIT_ORDER_[LSB | MSB]_FIRST, PCM_FRAME_ALIGN_[LAST | FIRST]_BIT, PCM_FRAME_WIDTH_[SHORT | LONG], PCM_RX_INT_[DISABLE | ENABLE], PCM_TX_INT_[DISABLE | ENABLE], PCM_DATA_MGT_[IOC | CFX], PCM_MULTIWORD_x, PCM_WORD_SIZE_x, PCM_SHIFT_RX_DATA_[LEFT | RIGHT], PCM_FIELD_TX_DATA_[MSB | LSB], PCM_TX_DATA_[SATURATED | UNSATURATED], PCM_[ENABLE | DISABLE], PCM_CLK_FILTER_[DISABLED | ENABLED
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	1 if pcm_cfg can be sign-extended from 8 bits; 2 otherwise

Example	`// Configure the PCM interface as a slave receiver. The transfer is`
	`// configured for:`
	`// - 16 bits per word (sent MSB first), with received data placed in`
	`//   the lower 16 bits of the PCM data register.`
	`// - Pulse frame signal aligned with the first bit every 2 words`
	`#define cfg ( PCM_SLAVE | PCM_ENABLE | PCM_SUBFRAME_DISABLE | \`
	`PCM_BIT_ORDER_MSB_FIRST | \`
	`PCM_FRAME_ALIGN_FIRST_BIT | \`
	`PCM_FRAME_WIDTH_SHORT | PCM_RX_INT_ENABLE | \`
	`PCM_TX_INT_DISABLE | PCM_DATA_MGT_CFX | \`
	`PCM_MULTIWORD_2 | PCM_WORD_SIZE_16 | \`
	`PCM_SHIFT_RX_DATA_RIGHT | \`
	`PCM_FIELD_TX_DATA_LSB | \`
	`PCM_TX_DATA_UNSATURATED )`
	`Sys_PCM_Config( cfg )`

10.128 SYS_PCM_DISABLE

Disable the PCM interface by clearing the PCM_ENABLE_POS bit (no configuration changes)

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_PCM_Disable`
Source File	sk3_sys_pcm.cfi
Description	Disable the PCM interface by clearing the PCM_ENABLE_POS bit (no configuration changes)
Inputs	None
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	3
Example	`// Disable the PCM interface` `Sys_PCM_Disable`

10.129 SYS_PCM_ENABLE

Enable the PCM interface by setting the PCM_ENABLE_POS bit (no configuration changes)

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_PCM_Enable`
Source File	sk3_sys_pcm.cfi
Description	Enable the PCM interface by setting the PCM_ENABLE_POS bit (no configuration changes)
Inputs	None
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	3
Example	`// Enable the PCM interface` `Sys_PCM_Enable`

10.130 SYS_POP_REG_1

Restore specified register from stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Pop_Reg_1( reg1 )`
Source File	sk3_sys_utils.cfi
Description	Restore specified register from stack.
Inputs	reg1 = Register to load from stack location [sp-1]
Outputs	None
Register Modifications	sp (sp = sp - 1)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to pop
Clock Cycles	2
Example	`Sys_Pop_Reg_1( a0.e )`

10.131 SYS_POP_REG_10

Restore 10 specified registers from stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Pop_Reg_10( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8, reg9, reg10 )`
Source File	sk3_sys_utils.cfi
Description	Restore 10 specified registers from stack.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5] reg6 = Register to load from stack location [sp-6] reg7 = Register to load from stack location [sp-7] reg8 = Register to load from stack location [sp-8] reg9 = Register to load from stack location [sp-9] reg10 = Register to load from stack location [sp-10]
Outputs	None
Register Modifications	sp (sp = sp - 10)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to pop
Clock Cycles	11
Example	`Sys_Pop_Reg_10( a0.e, a0.h, a0.l, xp0, xp1, b0.e, b0.h, b0.l, \` `yp0, x0 )`

10.132 SYS_POP_REG_2

Restore 2 specified registers from stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Pop_Reg_2( reg1, reg2 )`
Source File	sk3_sys_utils.cfi
Description	Restore 2 specified registers from stack.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2]
Outputs	None
Register Modifications	sp (sp = sp - 2)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to pop
Clock Cycles	3
Example	`Sys_Pop_Reg_2( a0.e, a0.h )`

10.133 SYS_POP_REG_3

Restore 3 specified registers from stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Pop_Reg_3( reg1, reg2, reg3 )`
Source File	sk3_sys_utils.cfi
Description	Restore 3 specified registers from stack.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3]
Outputs	None
Register Modifications	sp (sp = sp - 3)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to pop
Clock Cycles	4
Example	`Sys_Pop_Reg_3( a0.e, a0.h, a0.l )`

10.134 SYS_POP_REG_4

Restore 4 specified registers from stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Pop_Reg_4( reg1, reg2, reg3, reg4 )`
Source File	sk3_sys_utils.cfi
Description	Restore 4 specified registers from stack.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4]
Outputs	None
Register Modifications	sp (sp = sp - 4)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to pop
Clock Cycles	5
Example	`Sys_Pop_Reg_4( a0.e, a0.h, a0.l, xp0 )`

10.135 SYS_POP_REG_5

Restore 5 specified registers from stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Pop_Reg_5( reg1, reg2, reg3, reg4, reg5 )`
Source File	sk3_sys_utils.cfi
Description	Restore 5 specified registers from stack.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5]
Outputs	None
Register Modifications	sp (sp = sp - 5)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to pop
Clock Cycles	6
Example	`Sys_Pop_Reg_5( a0.e, a0.h, a0.l, xp0, xp1 )`

10.136 SYS_POP_REG_6

Restore 6 specified registers from stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Pop_Reg_6( reg1, reg2, reg3, reg4, reg5, reg6 )`
Source File	sk3_sys_utils.cfi
Description	Restore 6 specified registers from stack.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5] reg6 = Register to load from stack location [sp-6]
Outputs	None
Register Modifications	sp (sp = sp - 6)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to pop
Clock Cycles	7
Example	`Sys_Pop_Reg_6( a0.e, a0.h, a0.l, xp0, xp1, b0.e )`

10.137 SYS_POP_REG_7

Restore 7 specified registers from stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Pop_Reg_7( reg1, reg2, reg3, reg4, reg5, reg6, reg7 )`
Source File	sk3_sys_utils.cfi
Description	Restore 7 specified registers from stack.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5] reg6 = Register to load from stack location [sp-6] reg7 = Register to load from stack location [sp-7]
Outputs	None
Register Modifications	sp (sp = sp - 7)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to pop
Clock Cycles	8
Example	`Sys_Pop_Reg_7( a0.e, a0.h, a0.l, xp0, xp1, b0.e, b0.h )`

10.138 SYS_POP_REG_8

Restore 8 specified registers from stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Pop_Reg_8( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8 )`
Source File	sk3_sys_utils.cfi
Description	Restore 8 specified registers from stack.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5] reg6 = Register to load from stack location [sp-6] reg7 = Register to load from stack location [sp-7] reg8 = Register to load from stack location [sp-8]
Outputs	None
Register Modifications	sp (sp = sp - 8)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to pop
Clock Cycles	9
Example	`Sys_Pop_Reg_8( a0.e, a0.h, a0.l, xp0, xp1, b0.e, b0.h, b0.l )`

10.139 SYS_POP_REG_9

Restore 9 specified registers from stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Pop_Reg_9( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8, reg9 )`
Source File	sk3_sys_utils.cfi
Description	Restore 9 specified registers from stack.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5] reg6 = Register to load from stack location [sp-6] reg7 = Register to load from stack location [sp-7] reg8 = Register to load from stack location [sp-8] reg9 = Register to load from stack location [sp-9]
Outputs	None
Register Modifications	sp (sp = sp - 9)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to pop
Clock Cycles	10
Example	`Sys_Pop_Reg_9( a0.e, a0.h, a0.l, xp0, xp1, b0.e, b0.h, b0.l, yp0 )`

10.140 SYS_POP_REG_LONG_1

Restore specified register from the stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Pop_Reg_Long_1( reg1 )`
Source File	sk3_sys_utils.cfi
Description	Restore specified register from the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Pop_Reg_1 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to load from stack location [sp-1]
Outputs	None
Register Modifications	sp (sp = sp - 1)
Assumptions	None
Clock Cycles	2
Example	`Sys_Pop_Reg_Long_1( xstep0 )`

10.141 SYS_POP_REG_LONG_10

Restore 10 specified registers from the stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Pop_Reg_Long_10( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8, reg9, reg10 )`
Source File	sk3_sys_utils.cfi
Description	Restore 10 specified registers from the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Pop_Reg_10 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5] reg6 = Register to load from stack location [sp-6] reg7 = Register to load from stack location [sp-7] reg8 = Register to load from stack location [sp-8] reg9 = Register to load from stack location [sp-9] reg10 = Register to load from stack location [sp-10]
Outputs	None
Register Modifications	sp (sp = sp - 10)
Assumptions	None
Clock Cycles	11
Example	`Sys_Pop_Reg_Long_10( xstep0, xcirc0, xstep1, xcirc1, xstep2, \` `xstep3, xstep4, xstep5, xstep6, xstep7 )`

10.142 SYS_POP_REG_LONG_2

Restore 2 specified registers from the stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Pop_Reg_Long_2( reg1, reg2 )`
Source File	sk3_sys_utils.cfi
Description	Restore 2 specified registers from the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Pop_Reg_2 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2]
Outputs	None
Register Modifications	sp (sp = sp - 2)
Assumptions	None
Clock Cycles	3
Example	`Sys_Pop_Reg_Long_2( xstep0, xcirc0 )`

10.143 SYS_POP_REG_LONG_3

Restore 3 specified registers from the stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Pop_Reg_Long_3( reg1, reg2, reg3 )`
Source File	sk3_sys_utils.cfi
Description	Restore 3 specified registers from the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Pop_Reg_3 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3]
Outputs	None
Register Modifications	sp (sp = sp - 3)
Assumptions	None
Clock Cycles	4
Example	`Sys_Pop_Reg_Long_3( xstep0, xcirc0, xstep1 )`

10.144 SYS_POP_REG_LONG_4

Restore 4 specified registers from the stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Pop_Reg_Long_4( reg1, reg2, reg3, reg4 )`
Source File	sk3_sys_utils.cfi
Description	Restore 4 specified registers from the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Pop_Reg_4 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4]
Outputs	None
Register Modifications	sp (sp = sp - 4)
Assumptions	None
Clock Cycles	5
Example	`Sys_Pop_Reg_Long_4( xstep0, xcirc0, xstep1, xcirc1 )`

10.145 SYS_POP_REG_LONG_5

Restore 5 specified registers from the stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Pop_Reg_Long_5( reg1, reg2, reg3, reg4, reg5 )`
Source File	sk3_sys_utils.cfi
Description	Restore 5 specified registers from the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Pop_Reg_5 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5]
Outputs	None
Register Modifications	sp (sp = sp - 5)
Assumptions	None
Clock Cycles	6
Example	`Sys_Pop_Reg_Long_5( xstep0, xcirc0, xstep1, xcirc1, xstep2 )`

10.146 SYS_POP_REG_LONG_6

Restore 6 specified registers from the stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Pop_Reg_Long_6( reg1, reg2, reg3, reg4, reg5, reg6 )`
Source File	sk3_sys_utils.cfi
Description	Restore 6 specified registers from the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Pop_Reg_6 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5] reg6 = Register to load from stack location [sp-6]
Outputs	None
Register Modifications	sp (sp = sp - 6)
Assumptions	None
Clock Cycles	7
Example	`Sys_Pop_Reg_Long_6( xstep0, xcirc0, xstep1, xcirc1, xstep2, \` `xstep3 )`

10.147 SYS_POP_REG_LONG_7

Restore 7 specified registers from the stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Pop_Reg_Long_7( reg1, reg2, reg3, reg4, reg5, reg6, reg7 )`
Source File	sk3_sys_utils.cfi
Description	Restore 7 specified registers from the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Pop_Reg_7 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5] reg6 = Register to load from stack location [sp-6] reg7 = Register to load from stack location [sp-7]
Outputs	None
Register Modifications	sp (sp = sp - 7)
Assumptions	None
Clock Cycles	8
Example	`Sys_Pop_Reg_Long_7( xstep0, xcirc0, xstep1, xcirc1, xstep2, \` `xstep3, xstep4 )`

10.148 SYS_POP_REG_LONG_8

Restore 8 specified registers from the stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Pop_Reg_Long_8( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8 )`
Source File	sk3_sys_utils.cfi
Description	Restore 8 specified registers from the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Pop_Reg_8 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5] reg6 = Register to load from stack location [sp-6] reg7 = Register to load from stack location [sp-7] reg8 = Register to load from stack location [sp-8]
Outputs	None
Register Modifications	sp (sp = sp - 8)
Assumptions	None
Clock Cycles	9
Example	`Sys_Pop_Reg_Long_8( xstep0, xcirc0, xstep1, xcirc1, xstep2, \` `xstep3, xstep4, xstep5 )`

10.149 SYS_POP_REG_LONG_9

Restore 9 specified registers from the stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Pop_Reg_Long_9( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8, reg9 )`
Source File	sk3_sys_utils.cfi
Description	Restore 9 specified registers from the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Pop_Reg_9 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to load from stack location [sp-1] reg2 = Register to load from stack location [sp-2] reg3 = Register to load from stack location [sp-3] reg4 = Register to load from stack location [sp-4] reg5 = Register to load from stack location [sp-5] reg6 = Register to load from stack location [sp-6] reg7 = Register to load from stack location [sp-7] reg8 = Register to load from stack location [sp-8] reg9 = Register to load from stack location [sp-9]
Outputs	None
Register Modifications	sp (sp = sp - 9)
Assumptions	None
Clock Cycles	10
Example	`Sys_Pop_Reg_Long_9( xstep0, xcirc0, xstep1, xcirc1, xstep2, \` `xstep3, xstep4, xstep5, xstep6 )`

10.150 SYS_POWER2

Calculate the power of 2 for a supplied constant

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Power2( const )`
Source File	sk3_sys_utils.cfi
Description	Calculate the power of 2 for a supplied constant.
Inputs	const = Exponent
Outputs	Sys_Power2 = Value of 2^const
Register Modifications	None
Assumptions	Constant is a non-negative integer
Clock Cycles	0 (evaluated during compile)
Example	`// Load 2^3 to x0` `load x0, Sys_Power2( 3 )`

10.151 SYS_PUSH_REG_1

Store specified register onto stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Push_Reg_1( reg1 )`
Source File	sk3_sys_utils.cfi
Description	Store specified register onto stack
Inputs	reg1 = Register to store at stack location [sp-1]
Outputs	None
Register Modifications	sp (sp = sp + 1)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to push
Clock Cycles	2
Example	`Sys_Push_Reg_1( a0.e )`

10.152 SYS_PUSH_REG_10

Store 10 specified registers onto stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Push_Reg_10( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8, reg9, reg10 )`
Source File	sk3_sys_utils.cfi
Description	Store 10 specified registers onto stack
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5] reg6 = Register to store at stack location [sp-6] reg7 = Register to store at stack location [sp-7] reg8 = Register to store at stack location [sp-8] reg9 = Register to store at stack location [sp-9] reg10 = Register to store at stack location [sp-10]
Outputs	None
Register Modifications	sp (sp = sp + 10)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to push
Clock Cycles	11
Example	`Sys_Push_Reg_10( a0.e, a0.h, a0.l, xp0, xp1, b0.e, b0.h, b0.l, \` `yp0, x0 )`

10.153 SYS_PUSH_REG_2

Store 2 specified registers onto stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Push_Reg_2( reg1, reg2 )`
Source File	sk3_sys_utils.cfi
Description	Store 2 specified registers onto stack
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2]
Outputs	None
Register Modifications	sp (sp = sp + 2)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to push
Clock Cycles	3
Example	`Sys_Push_Reg_2( a0.e, a0.h )`

10.154 SYS_PUSH_REG_3

Store 3 specified registers onto stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Push_Reg_3( reg1, reg2, reg3 )`
Source File	sk3_sys_utils.cfi
Description	Store 3 specified registers onto stack
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3]
Outputs	None
Register Modifications	sp (sp = sp + 3)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to push
Clock Cycles	4
Example	`Sys_Push_Reg_3( a0.e, a0.h, a0.l )`

10.155 SYS_PUSH_REG_4

Store 4 specified register onto stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Push_Reg_4( reg1, reg2, reg3, reg4 )`
Source File	sk3_sys_utils.cfi
Description	Store 4 specified register onto stack
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4]
Outputs	None
Register Modifications	sp (sp = sp + 4)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to push
Clock Cycles	5
Example	`Sys_Push_Reg_4( a0.e, a0.h, a0.l, xp0 )`

10.156 SYS_PUSH_REG_5

Store 5 specified registers onto stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Push_Reg_5( reg1, reg2, reg3, reg4, reg5 )`
Source File	sk3_sys_utils.cfi
Description	Store 5 specified registers onto stack
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5]
Outputs	None
Register Modifications	sp (sp = sp + 5)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to push
Clock Cycles	6
Example	`Sys_Push_Reg_5( a0.e, a0.h, a0.l, xp0, xp1 )`

10.157 SYS_PUSH_REG_6

Store 6 specified register onto stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Push_Reg_6( reg1, reg2, reg3, reg4, reg5, reg6 )`
Source File	sk3_sys_utils.cfi
Description	Store 6 specified register onto stack
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5] reg6 = Register to store at stack location [sp-6]
Outputs	None
Register Modifications	sp (sp = sp + 6)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to push
Clock Cycles	7
Example	`Sys_Push_Reg_6( a0.e, a0.h, a0.l, xp0, xp1, b0.e )`

10.158 SYS_PUSH_REG_7

Store 7 specified registers onto stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Push_Reg_7( reg1, reg2, reg3, reg4, reg5, reg6, reg7 )`
Source File	sk3_sys_utils.cfi
Description	Store 7 specified registers onto stack
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5] reg6 = Register to store at stack location [sp-6] reg7 = Register to store at stack location [sp-7]
Outputs	None
Register Modifications	sp (sp = sp + 7)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to push
Clock Cycles	8
Example	`Sys_Push_Reg_7( a0.e, a0.h, a0.l, xp0, xp1, b0.e, b0.h )`

10.159 SYS_PUSH_REG_8

Store 8 specified registers onto stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Push_Reg_8( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8 )`
Source File	sk3_sys_utils.cfi
Description	Store 8 specified registers onto stack
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5] reg6 = Register to store at stack location [sp-6] reg7 = Register to store at stack location [sp-7] reg8 = Register to store at stack location [sp-8]
Outputs	None
Register Modifications	sp (sp = sp + 8)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to push
Clock Cycles	9
Example	`Sys_Push_Reg_8( a0.e, a0.h, a0.l, xp0, xp1, b0.e, b0.h, b0.l )`

10.160 SYS_PUSH_REG_9

Store 9 specified register onto stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Push_Reg_9( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8, reg9 )`
Source File	sk3_sys_utils.cfi
Description	Store 9 specified register onto stack
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5] reg6 = Register to store at stack location [sp-6] reg7 = Register to store at stack location [sp-7] reg8 = Register to store at stack location [sp-8] reg9 = Register to store at stack location [sp-9]
Outputs	None
Register Modifications	sp (sp = sp + 9)
Assumptions	No xstep, xcirc, ystep or ycirc registers are included in the list of registers to push
Clock Cycles	10
Example	`Sys_Push_Reg_9( a0.e, a0.h, a0.l, xp0, xp1, b0.e, b0.h, b0.l, yp0 )`

10.161 SYS_PUSH_REG_LONG_1

Store specified register onto the stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Push_Reg_Long_1( reg1 )`
Source File	sk3_sys_utils.cfi
Description	Store specified register onto the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Push_Reg_1 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to store at stack location [sp-1]
Outputs	None
Register Modifications	sp (sp = sp + 1)
Assumptions	None
Clock Cycles	2
Example	`Sys_Push_Reg_Long_1( xstep0 )`

10.162 SYS_PUSH_REG_LONG_10

Store 10 specified registers onto the stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Push_Reg_Long_10( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8, reg9, reg10 )`
Source File	sk3_sys_utils.cfi
Description	Store 10 specified registers onto the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Push_Reg_10 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5] reg6 = Register to store at stack location [sp-6] reg7 = Register to store at stack location [sp-7] reg8 = Register to store at stack location [sp-8] reg9 = Register to store at stack location [sp-9] reg10 = Register to store at stack location [sp-10]
Outputs	None
Register Modifications	sp (sp = sp + 10)
Assumptions	None
Clock Cycles	11
Example	`Sys_Push_Reg_Long_10( xstep0, xcirc0, xstep1, xcirc1, xstep2, \` `xstep3, xstep4, xstep5, xstep6, xstep7 )`

10.163 SYS_PUSH_REG_LONG_2

Store 2 specified registers onto the stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Push_Reg_Long_2( reg1, reg2 )`
Source File	sk3_sys_utils.cfi
Description	Store 2 specified registers onto the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Push_Reg_2 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2]
Outputs	None
Register Modifications	sp (sp = sp + 2)
Assumptions	None
Clock Cycles	3
Example	`Sys_Push_Reg_Long_2( xstep0, xcirc0 )`

10.164 SYS_PUSH_REG_LONG_3

Store 3 specified registers onto the stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Push_Reg_Long_3( reg1, reg2, reg3 )`
Source File	sk3_sys_utils.cfi
Description	Store 3 specified registers onto the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Push_Reg_3 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3]
Outputs	None
Register Modifications	sp (sp = sp + 3)
Assumptions	None
Clock Cycles	4
Example	`Sys_Push_Reg_Long_3( xstep0, xcirc0, xstep1 )`

10.165 SYS_PUSH_REG_LONG_4

Store 4 specified registers onto the stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Push_Reg_Long_4( reg1, reg2, reg3, reg4 )`
Source File	sk3_sys_utils.cfi
Description	Store 4 specified registers onto the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Push_Reg_4 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4]
Outputs	None
Register Modifications	sp (sp = sp + 4)
Assumptions	None
Clock Cycles	5
Example	`Sys_Push_Reg_Long_4( xstep0, xcirc0, xstep1, xcirc1 )`

10.166 SYS_PUSH_REG_LONG_5

Store 5 specified registers onto the stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Push_Reg_Long_5( reg1, reg2, reg3, reg4, reg5 )`
Source File	sk3_sys_utils.cfi
Description	Store 5 specified registers onto the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Push_Reg_5 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5]
Outputs	None
Register Modifications	sp (sp = sp + 5)
Assumptions	None
Clock Cycles	6
Example	`Sys_Push_Reg_Long_5( xstep0, xcirc0, xstep1, xcirc1, xstep2 )`

10.167 SYS_PUSH_REG_LONG_6

Store 6 specified registers onto the stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Push_Reg_Long_6( reg1, reg2, reg3, reg4, reg5, reg6 )`
Source File	sk3_sys_utils.cfi
Description	Store 6 specified registers onto the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Push_Reg_6 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5] reg6 = Register to store at stack location [sp-6]
Outputs	None
Register Modifications	sp (sp = sp + 6)
Assumptions	None
Clock Cycles	7
Example	`Sys_Push_Reg_Long_6( xstep0, xcirc0, xstep1, xcirc1, xstep2, \` `xstep3 )`

10.168 SYS_PUSH_REG_LONG_7

Store 7 specified registers onto the stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Push_Reg_Long_7( reg1, reg2, reg3, reg4, reg5, reg6, reg7 )`
Source File	sk3_sys_utils.cfi
Description	Store 7 specified registers onto the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Push_Reg_7 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5] reg6 = Register to store at stack location [sp-6] reg7 = Register to store at stack location [sp-7]
Outputs	None
Register Modifications	sp (sp = sp + 7)
Assumptions	None
Clock Cycles	8
Example	`Sys_Push_Reg_Long_7( xstep0, xcirc0, xstep1, xcirc1, xstep2, \` `xstep3, xstep4 )`

10.169 SYS_PUSH_REG_LONG_8

Store 8 specified registers onto the stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Push_Reg_Long_8( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8 )`
Source File	sk3_sys_utils.cfi
Description	Store 8 specified registers onto the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Push_Reg_8 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5] reg6 = Register to store at stack location [sp-6] reg7 = Register to store at stack location [sp-7] reg8 = Register to store at stack location [sp-8]
Outputs	None
Register Modifications	sp (sp = sp + 8)
Assumptions	None
Clock Cycles	9
Example	`Sys_Push_Reg_Long_8( xstep0, xcirc0, xstep1, xcirc1, xstep2, \` `xstep3, xstep4, xstep5 )`

10.170 SYS_PUSH_REG_LONG_9

Store 9 specified registers onto the stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Push_Reg_Long_9( reg1, reg2, reg3, reg4, reg5, reg6, reg7, reg8, reg9 )`
Source File	sk3_sys_utils.cfi
Description	Store 9 specified registers onto the stack. Note that this macro uses the long instruction form and as a result is not as efficient as the regular Sys_Push_Reg_9 macro in terms of memory usage. However, this macro is required for xstep, xcirc, ystep and ycirc registers.
Inputs	reg1 = Register to store at stack location [sp-1] reg2 = Register to store at stack location [sp-2] reg3 = Register to store at stack location [sp-3] reg4 = Register to store at stack location [sp-4] reg5 = Register to store at stack location [sp-5] reg6 = Register to store at stack location [sp-6] reg7 = Register to store at stack location [sp-7] reg8 = Register to store at stack location [sp-8] reg9 = Register to store at stack location [sp-9]
Outputs	None
Register Modifications	sp (sp = sp + 9)
Assumptions	None
Clock Cycles	10
Example	`Sys_Push_Reg_Long_9( xstep0, xcirc0, xstep1, xcirc1, xstep2, \` `xstep3, xstep4, xstep5, xstep6 )`

10.171 SYS_REG_FL8N16

Load a data register or high part of an accumulator with a floating-point number with an 8-bit signed exponent and a 16-bit signed mantissa using the Sys_fl8n16 macro

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Reg_fl8n16( reg, const )`
Source File	sk3_sys_utils.cfi
Description	Load a data register or high part of an accumulator with a floating-point number with an 8-bit signed exponent and a 16-bit signed mantissa using the Sys_fl8n16 macro.
Inputs	reg = Destination register const = Constant to convert with Sys_fl8n16
Outputs	reg = Floating-point approximation of the supplied constant
Register Modifications	None
Assumptions	Constant parameter has magnitude less than 2^63
Clock Cycles	1
Example	`// Load 7.125 to register x1 as a <8,16> floating-point value` `Sys_Reg_fl8n16( x1, 71250000 )`

10.172 SYS_REG_FRAC24

Load a data register or high part of an accumulator with a 24-bit signed immediate using the Sys_frac24 macro

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Reg_frac24( reg, const )`
Source File	sk3_sys_utils.cfi
Description	Load a data register or high part of an accumulator with a 24-bit signed immediate using the Sys_frac24 macro.
Inputs	reg = Destination register const = Constant to convert with Sys_frac24
Outputs	reg = Fractional approximation of the supplied constant
Register Modifications	None
Assumptions	Constant parameter has magnitude less than 2^46
Clock Cycles	1
Example	`// Load 0.125 to register x1 as a <1,23> fractional value` `Sys_Reg_frac24( x1, 1250000 )`

10.173 SYS_REG_FRAC32

Load the high and extension part of an accumulator with a 32-bit signed immediate using the Sys_frac32 macro

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Reg_frac32( accum, const )`
Source File	sk3_sys_utils.cfi
Description	Load the high and extension part of an accumulator with a 32-bit signed immediate using the Sys_frac32 macro. *Note:* This macro clears the low segment of the specified accumulator. *Note:* The frac32 data format is layout-compatible with the frac24 format.
Inputs	accum = Destination accumulator const = Constant to convert with Sys_frac32
Outputs	accum = <9,23> fixed-point approximation of the supplied constant
Register Modifications	None
Assumptions	Constant parameter has magnitude less than 2^46
Clock Cycles	2
Example	`// Load 7.125 to accumulator a1.e, a1.h as a <9,23> fractional value` `Sys_Reg_frac32( a1, 71250000 )`

10.174 SYS_REG_FRAC48

Load an accumulator with a 48-bit signed immediate using the Sys_frac48 macro

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Reg_frac48( accum, const )`
Source File	sk3_sys_utils.cfi
Description	Load an accumulator with a 48-bit signed immediate using the Sys_frac48 macro.
Inputs	accum = Destination accumulator const = Constant to convert with Sys_frac48
Outputs	accum = Fractional approximation of the supplied constant
Register Modifications	None
Assumptions	Constant parameter has magnitude less than 2^24
Clock Cycles	2
Example	`// Load 0.125 to accumulator a1 as a <1,47> fractional value` `Sys_Reg_frac48( a1, 1250000 )`

10.175 SYS_REG_M24P24

Load an accumulator with a 48-bit signed immediate using the Sys_m24p24 macro

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Reg_m24p24( accum, const )`
Source File	sk3_sys_utils.cfi
Description	Load an accumulator with a 48-bit signed immediate using the Sys_m24p24 macro.
Inputs	accum = Destination accumulator const = Constant to convert with Sys_m24p24
Outputs	accum = <24,24> fixed-point approximation of the supplied constant
Register Modifications	None
Assumptions	Constant parameter has magnitude less than 2^62
Clock Cycles	2
Example	`// Load 7.125 to accumulator a1 as a <24,24> fixed-point value` `Sys_Reg_m24p24( a1, 71250000 )`

10.176 SYS_REG_M2P46

Load an accumulator with a 48-bit signed immediate using the Sys_m2p46 macro

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Reg_m2p46( accum, const )`
Source File	sk3_sys_utils.cfi
Description	Load an accumulator with a 48-bit signed immediate using the Sys_m2p46 macro.
Inputs	accum = Destination accumulator const = Constant to convert with Sys_m2p46
Outputs	accum = <2,46> fixed-point approximation of the supplied constant
Register Modifications	None
Assumptions	Constant parameter has magnitude less than 2^40
Clock Cycles	2
Example	`// Load 1.125 to accumulator a1 as a <2,46> fixed-point value` `Sys_Reg_m2p46( a1, 11250000 )`

10.177 SYS_REG_M8P16

Load a data register or high part of an accumulator with an 8-bit signed integer and a 16-bit unsigned fractional part using the Sys_m8p16 macro

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Reg_m8p16( reg, const )`
Source File	sk3_sys_utils.cfi
Description	Load a data register or high part of an accumulator with an 8-bit signed integer and a 16-bit unsigned fractional part using the Sys_m8p16 macro.
Inputs	reg = Destination register const = Constant to convert with Sys_m8p16
Outputs	reg = Fixed-point approximation of the supplied constant
Register Modifications	None
Assumptions	Constant parameter has magnitude less than 2^53
Clock Cycles	1
Example	`// Load 7.125 to register x1 as a <8,16> fixed-point value` `Sys_Reg_m8p16( x1, 71250000 )`

10.178 SYS_RESTORE_HEAR_BFP

Configure the HEAR block floating point using a stored value

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Restore_HEAR_BFP( memspace, bfp_addr )`
Source File	sk3_sys_hear.cfi
Description	Configure the HEAR block floating point using a stored value
Inputs	mem_space = Memory space to load from; use SYS_MEMSPACE_P, SYS_MEMSPACE_X or SYS_MEMSPACE_Y bfp_addr = Memory location that contains a configuration setting for the HEAR BFP unit
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	2
Example	`// Restore a previously saved HEAR block floating point configuration` `Sys_Restore_HEAR_BFP( SYS_MEMSPACE_X, X_ALGO0_BFP_CONFIG )`

10.179 SYS_SAVE_HEAR_BFP

Store the current HEAR block floating point configuration

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Save_HEAR_BFP( memspace, bfp_addr )`
Source File	sk3_sys_hear.cfi
Description	Store the current HEAR block floating point configuration
Inputs	mem_space = Memory space to store to; use SYS_MEMSPACE_P, SYS_MEMSPACE_X or SYS_MEMSPACE_Y bfp_addr = Memory location where a configuration setting for the HEAR BFP unit will be stored
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	2

Example	`// Configure the block floating point exponent to a normalized`
	`// reference setting`
	`load x0, NORM_REFERENCE_BLOCK_EXP`
	`Sys_SetReg_HEAR_BFP_BLOCK_EXP(x0)`
	
	`// Save the new HEAR block floating point configuration so it can`
	`// be restored later (insert a NOP to avoid reading the previous`
	`// D_HEAR_BFP value due to a buffered write of D_HEAR_BFP_BLOCK_EXP)`
	`nop`
	`Sys_Save_HEAR_BFP( SYS_MEMSPACE_X, X_ALGO0_BFP_CONFIG )`

10.180 SYS_SET_ADC_CFG

Set the control register A_ADC_CFG

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Set_ADC_CFG( control_word )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_ADC_CFG
Inputs	control_word = Fourteen-part parameter consisting of ADC3_DC_REMOVE_*, ADC2_DC_REMOVE_*, ADC1_DC_REMOVE_*, ADC0_DC_REMOVE_*, ADC_HIGH_FREQ_[ENABLE | DISABLE], LPF3_IN_[BYPASS | ACTIVE_20KHZ], LPF2_IN_[BYPASS | ACTIVE_20KHZ], LPF1_IN_[BYPASS | ACTIVE_20KHZ], LPF0_IN_[BYPASS | ACTIVE_20KHZ], LINEOUT3_[ENABLE | DISABLE], LINEOUT2_[ENABLE | DISABLE], LINEOUT1_[ENABLE | DISABLE], LINEOUT0_[ENABLE | DISABLE] and Sample Frequency (6-bit value)
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	2

Example	`// Configure the ADC as follows:`
	`//  - Line out 0 = Enabled`
	`//  - Line out 1,2,3 = Disabled`
	`//  - ADC0 DC Removal filter cut-off = 5Hz`
	`//  - ADC1 DC Removal filter cut-off = 20Hz`
	`//  - ADC2 DC Removal filter cut-off = 10Hz`
	`//  - ADC3 DC Removal filter cut-off = 5Hz`
	`//  - Input LPF0 = 20kHz`
	`//  - Input LPF1 = 20kHz`
	`//  - Input LPF2 = 20kHz`
	`//  - Input LPF3 = Bypassed`
	`//  - Input stage frequency mode = High Frequency`
	`//  - Set the sampling frequency = 0x02 (16kHz)`
	`//  (Assumes 1.28 MHz slow clock frequency)`
	`Sys_Set_ADC_CFG( LINE_OUT0_ENABLE | LINE_OUT1_DISABLE | \`
	`LINE_OUT2_DISABLE | LINE_OUT3_DISABLE | \`
	`ADC0_DC_REMOVE_CUTOFF_5HZ | \`
	`ADC1_DC_REMOVE_CUTOFF_20HZ | \`
	`ADC2_DC_REMOVE_CUTOFF_10HZ | \`
	`ADC3_DC_REMOVE_CUTOFF_5HZ | \`
	`LPF0_IN_ACTIVE_20KHZ | LPF1_IN_ACTIVE_20KHZ | \`
	`LPF2_IN_ACTIVE_20KHZ | LPF3_IN_BYPASS | \`
	`ADC_HIGH_FREQ_ENABLE | 0x02 )`

10.181 SYS_SET_ADC_CUR_CTRL

Set the control register A_ADC_CUR_CTRL

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Set_ADC_CUR_CTRL( control_word )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_ADC_CUR_CTRL
Inputs	control_word = A four-part parameter, consisting of ADC0_CURRENT_*, ADC1_CURRENT_*, ADC2_CURRENT_*, ADC3_CURRENT_*
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if control_word can be sign-extended from 8 bits; 2 otherwise

Example	`// Configure the ADC currents as follows:`
	`//  - ADC0 current = 1.3 x Inom`
	`//  - ADC1 current = 1.0 x Inom`
	`//  - ADC2 current = 2.0 x Inom`
	`//  - ADC3 current = 0.6 x Inom`
	`Sys_Set_ADC_CUR_CTRL( ADC0_CURRENT_1_3X | ADC1_CURRENT_1_0X | \`
	`ADC2_CURRENT_2_0X | ADC3_CURRENT_0_6X )`

10.182 SYS_SET_ADC_GFXY_CTRL

Set the control register A_ADC_GFxy_CTRL (where xy is 01 or 23)

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Set_ADC_GFxy_CTRL( GFx, GFy )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_ADC_GFxy_CTRL (where xy is 01 or 23)
Inputs	GFx = Gain factor (12-bit value) for ADC x GFy = Gain factor (12-bit value) for ADC y
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	2

Example	`// Select unity gain for all input channels (assumes system is using`
	`// the default sampling frequency setting)`
	`Sys_Set_ADC_GF01_CTRL( 0x665, 0x665 )`
	`Sys_Set_ADC_GF23_CTRL( 0x665, 0x665 )`

10.183 SYS_SET_ANALOG_PWR_CTRL

Set the control register A_ANALOG_PWR_CTRL

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Set_ANALOG_PWR_CTRL( control_word )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_ANALOG_PWR_CTRL
Inputs	control_word = ANALOG_[ENABLE | DISABLE]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Place the system in a soft-power down state by selecting the`
	`// standby clock and disabling the analog system components`
	`Sys_Clk_Config0( SYSCLK_SEL_STANDBY )`
	`Sys_Set_ANALOG_PWR_CTRL( ANALOG_DISABLE )`

10.184 SYS_SET_BAND_SELECT_CTRL

Set the control register A_BAND_SELECT_CTRL

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Set_BAND_SELECT_CTRL( control_word )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_BAND_SELECT_CTRL
Inputs	control_word = A two-part parameter consisting of BAND_SELECT_ADC3_* and BAND_SELECT_ADC2_*. NOTE: The defined BAND_SELECT_ADC* bands are for FS=16kHz.
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Configure the band 56kHz to 64kHz from ADC2`
	`// and the band 48kHz to 56kHz from ADC3 to be`
	`// filtered by the decimation filter`
	`Sys_Set_BAND_SELECT_CTRL( BAND_SELECT_ADC2_56K_TO_64K | \`
	`BAND_SELECT_ADC3_48K_TO_56K )`

10.185 SYS_SET_CLK_CTRL

Set the control register A_CLK_CTRL

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Set_CLK_CTRL( control_word )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_CLK_CTRL
Inputs	control_word = A two-part parameter consisting of SYSCLK_* and an 8-bit CCR value
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if control_word can be sign-extended from 8 bits; 2 otherwise

Example	`// Set the system clock to 1.57MHz (nominally), with a CCR value`
	`// of 0xE7`
	`Sys_Set_CLK_CTRL( SYSCLK_1M59 | 0xE7 )`

10.186 SYS_SET_DECIMATION_GAINXY

Calculate and set the control register A_ADC_GFxy_CTRL for the desired gains given the configured adc_cfg_sample_freq (where xy is 01 or 23)

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Set_Decimation_Gainxy( adc_cfg_sample_freq, GAINx, GAINy )`
Source File	sk3_sys_analog.cfi
Description	Calculate and set the control register A_ADC_GFxy_CTRL for the desired gains given the configured adc_cfg_sample_freq (where xy is 01 or 23)
Inputs	adc_cfg_sample_freq = Sampling frequency field in A_ADC_CFG GAINx = Gain for ADC x (pre-multiplied by 10^7) GAINy = Gain for ADC y (pre-multiplied by 10^7)
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	2

Example	`// Configure the decimation gain factors to produce the desired gains`
	`// assuming that ADC_CFG_SAMPLE_FREQ is set to 0x2 (16 kHz)`
	`#define ADC_CFG_SAMPLE_FREQ 0x2`
	`#define GAIN0 10000000 // Gain of ~ 1.0`
	`#define GAIN1 8000000 // Gain of ~ 0.8`
	`#define GAIN2 -10000000 // Gain of ~ -1.0`
	`#define GAIN3 -8000000 // Gain of ~ -0.8`
	
	`Sys_Set_Decimation_Gain01( ADC_CFG_SAMPLE_FREQ, GAIN0, GAIN1 )`
	`Sys_Set_Decimation_Gain23( ADC_CFG_SAMPLE_FREQ, GAIN2, GAIN3 )`

10.187 SYS_SET_FRAC_DELAY_X_CTRL

Set the control register A_FRAC_DELAY_x_CTRL for x = 0, 1, 2, 3

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Set_FRAC_DELAY_x_CTRL( control_word )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_FRAC_DELAY_x_CTRL for x = 0, 1, 2, 3
Inputs	control_word = A two-part parameter consisting of Integer Delay Component (ADCx_INTEGER_DELAY_*) and Fractional Delay Component (6-bit value)
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	2

Example	`// Delay sampling on channel 1 by half of a sampling period`
	`Sys_Set_FRAC_DELAY_1_CTRL( ADC1_INTEGER_DELAY_4 | \`
	`( 0 << FRAC_DELAY_FRACTIONAL_POS ) )`
	`// Delay sampling on channel 2 by 3 slow clock cycles`
	`Sys_Set_FRAC_DELAY_2_CTRL( ADC2_INTEGER_DELAY_0 | \`
	`( 3 << FRAC_DELAY_FRACTIONAL_POS ) )`

10.188 SYS_SET_HEAR_BFP

Configure the HEAR block floating point guard bits

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Set_HEAR_BFP( bfp )`
Source File	sk3_sys_hear.cfi
Description	Configure the HEAR block floating point guard bits
Inputs	bfp = Two-part parameter consisting of HEAR_BFP_FMD_[ENABLE | DISABLE] and HEAR_BFP_FORMAT_xyBITS
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if bfp can be sign-extended from 8 bits; 2 otherwise

Example	`// Configure the HEAR block floating point guard bits allowing 0 or 1`
	`// shifts and disabling full magnitude detection.`
	`Sys_Set_HEAR_BFP( HEAR_BFP_FORMAT_23BITS | HEAR_BFP_FMD_DISABLE )`

10.189 SYS_SET_HEAR_FUNCTIONCHAIN_VECTOR

Set a function chain vector to the specified function chain entry point

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Set_HEAR_FunctionChain_Vector( fc_pos, fc_entry )`
Source File	sk3_sys_hear.cfi
Description	Set a function chain vector to the specified function chain entry point
Inputs	fc_pos = HEAR function chain vector to set. Use HEAR_FC_*_POS fc_entry - HEAR function chain entry point. Use the _entry symbols generated by the HCT.
Outputs	None
Register Modifications	y1
Assumptions	Previous HEAR function chain vector is no longer needed as it is overwritten by this macro
Clock Cycles	2

Example	`// Set the function chain vector for command 2 to`
	`// a previously unbound function chain.`
	`Sys_Set_HEAR_FunctionChain_Vector( HEAR_FC_CMD2_POS,  \`
	`unbound_function_chain_entry )`

10.190 SYS_SET_IN_GAIN_CTRL

Set the control register A_IN_GAIN_CTRL and calibrate the DC offsets for the ADCs

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Set_IN_GAIN_CTRL( control_word )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_IN_GAIN_CTRL and calibrate the DC offsets for the ADCs. Calibration is based on the ADC calibration table and the new ADC preamp gain settings.
Inputs	control_word = A four-part parameter consisting of PGAIN3_*, PGAIN2_*, PGAIN1_* and PGAIN0_*
Outputs	sr[SR_X_ZERO] = If 0, ADCs calibrated. If 1, could not calibrate.
Register Modifications	yp0, yp1, a0, a1, b1, x0, x1, y1, ystep1
Assumptions	ADC calibration table loaded
Clock Cycles	Not available

Example	`// Configure the input stage gain control as follows:`
	`//  - the preamp0 gain is 15dB`
	`//  - the preamp1 gain is 30dB`
	`//  - the preamp2 gain is 27dB`
	`//  - the preamp3 gain is 24dB`
	`Sys_Set_IN_GAIN_CTRL( PGAIN0_15DB | PGAIN1_30DB | PGAIN2_27DB | \`
	`PGAIN3_24DB )`

10.191 SYS_SET_IN_GAIN_CTRL_NOCALIBRATION

Set the control register A_IN_GAIN_CTRL

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Set_IN_GAIN_CTRL_NoCalibration( control_word )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_IN_GAIN_CTRL. This is only recommended for configurations that do not have an ADC calibration table.
Inputs	control_word = A four-part parameter consisting of PGAIN3_*, PGAIN2_*, PGAIN1_* and PGAIN0_*
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if control_word can be sign-extended from 8 bits; 2 otherwise

Example	`// Configure the input stage gain control as follows:`
	`//  - the preamp0 gain is 15dB`
	`//  - the preamp1 gain is 30dB`
	`//  - the preamp2 gain is 27dB`
	`//  - the preamp3 gain is 24dB`
	`// NOTE: For optimal noise performance, invoke Sys_CalibrateADCs`
	`// after this macro`
	`Sys_Set_IN_GAIN_CTRL_NoCalibration( PGAIN0_15DB | PGAIN1_30DB | \`
	`PGAIN2_27DB | PGAIN3_24DB )`

10.192 SYS_SET_INPUT_CTRL

Set the control register A_INPUT_CTRL

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Set_INPUT_CTRL( control_word )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_INPUT_CTRL
Inputs	control_word = An eight-part parameter consisting of ADC3_SEL_*, ADC3_[ENABLE | DISABLE], ADC2_SEL_*, ADC2_[ENABLE | DISABLE], ADC1_SEL_*, ADC1_[ENABLE | DISABLE], ADC0_SEL_* and ADC0_[ENABLE | DISABLE]
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if control_word can be sign-extended from 8 bits; 2 otherwise

Example	`// Configure the ADC input configuration as follows:`
	`//  - route AI3 into ADC0`
	`//  - route AI0 into ADC1`
	`//  - route AI2 into ADC2`
	`//  - route AI1 into ADC3`
	`//  - disable ADC0, ADC1, ADC2`
	`//  - enable ADC3`
	`Sys_Set_INPUT_CTRL( ADC0_SEL_AI3 | ADC1_SEL_AI0 | ADC2_SEL_AI2 | \`
	`ADC3_SEL_AI1 | ADC0_DISABLE | ADC1_DISABLE | \`
	`ADC2_DISABLE | ADC3_ENABLE )`

10.193 SYS_SET_LSAD_CTRL

Set the control register A_LSAD_CTRL

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Set_LSAD_CTRL( control_word )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_LSAD_CTRL
Inputs	control_word = An eight-part parameter consisting of LSAD_CTRL_VBAT_MODE_[TRIMMER | AUDIO_OUT], LSAD_CTRL_VDBL_MODE_[TRIMMER | AUDIO_OUT], LSAD_CTRL_CH4_MODE_[TRIMMER | AUDIO_OUT], LSAD_CTRL_CH3_MODE_[TRIMMER | AUDIO_OUT], LSAD_CTRL_CH2_MODE_[TRIMMER | AUDIO_OUT], LSAD_CTRL_CH1_MODE_[TRIMMER | AUDIO_OUT], LSAD_CTRL_CH0_MODE_[TRIMMER | AUDIO_OUT] and LSAD_CTRL_[DISABLE | PRESCALE_*]
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if control_word can be sign-extended from 8 bits; 2 otherwise

Example	`// Configure the LSAD with the following settings:`
	`//  - Channel 0-3: Configure to audio out mode`
	`//  - Channel 4: Configure to trimmer mode`
	`//  - VBAT, VDBL: Configure to audio out mode`
	`//  - Configure the LSAD sampling frequency to`
	`//  have a prescale of 320`
	`Sys_Set_LSAD_CTRL( LSAD_CTRL_CH0_MODE_AUDIO_OUT | \`
	`LSAD_CTRL_CH1_MODE_AUDIO_OUT | \`
	`LSAD_CTRL_CH2_MODE_AUDIO_OUT | \`
	`LSAD_CTRL_CH3_MODE_AUDIO_OUT | \`
	`LSAD_CTRL_CH4_MODE_TRIMMER | \`
	`LSAD_CTRL_VBAT_MODE_AUDIO_OUT | \`
	`LSAD_CTRL_VDDC_MODE_AUDIO_OUT | \`
	`LSAD_CTRL_PRESCALE_320 )`

10.194 SYS_SET_O_DAC_CTRL

Set the control register O_DAC_CTRL

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Set_O_DAC_CTRL( control_word )`
Source File	sk3_sys_analog.cfi
Description	Set the control register O_DAC_CTRL
Inputs	control_word = A single parameter consisting of OSR_PRESCALE_*
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	1 if control_word can be sign-extended from 8 bits; 2 otherwise
Example	`// Configure the DAC oversampling rate to oversample by 2` `Sys_Set_O_DAC_CTRL( OSR_PRESCALE_2 )`

10.195 SYS_SET_O_DAC_GF_CTRL

Set the control register O_DAC_GF_CTRL

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Set_O_DAC_GF_CTRL( control_word )`
Source File	sk3_sys_analog.cfi
Description	Set the control register O_DAC_GF_CTRL
Inputs	control_word = A three-part parameter consisting of O_DAC_GF_CTRL_[ENABLE | DISABLE], O_DAC_GF_CTRL_BY_GPIO4 and Gain Factor for DA (12-bit value)
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	2

Example	`// Enable the DAC Gain Factor Control, and set the gain factor`
	`// to a value of 0x350 to obtain less than unity gain across the`
	`// interpolation filter.`
	`Sys_Set_O_DAC_GF_CTRL( O_DAC_GF_CTRL_ENABLE | 0x350 )`

10.196 SYS_SET_OUTPUT_CTRL

Set the control register OUTPUT_CTRL

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Set_OUTPUT_CTRL( control_word )`
Source File	sk3_sys_analog.cfi
Description	Set the control register OUTPUT_CTRL
Inputs	control_word = A two-part parameter consisting of OD0_HIGHPOWER_[ENABLE | DISABLE] and OD0_[ENABLE | DISABLE]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Configure the output stage to enable the DAC, but not the`
	`// high-power output.`
	`Sys_Set_OUTPUT_CTRL( OD0_ENABLE | OD0_HIGHPOWER_DISABLE )`

10.197 SYS_SET_PSU_CFG

Set the configuration register A_PSU_CFG [Macro for BelaSigna 300 only]

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Set_PSU_CFG( cfg_word )`
Source File	sk3_sys_analog.cfi
Description	Set the configuration register A_PSU_CFG [Macro for BelaSigna 300 only]
Inputs	cfg_word = Configuration values for band-gap and VDDC power supply regulator levels
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Power supply configuration to use the default VDDC and band-gap`
	`// voltage regulator setup`
	`Sys_Set_PSU_CFG( VDDC_LEVEL_111_VOLT | BAND_GAP_LEVEL_100_VOLT )`

10.198 SYS_SET_SUP_THRSH_DATA

Set the control register A_SUP_THRSH_DATA

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Set_SUP_THRSH_DATA( control_word )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_SUP_THRSH_DATA
Inputs	control_word = Supply threshold for SUP_COUNT (8-bit value)
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Set the battery monitor threshold to 0.95 volts` `Sys_Set_SUP_THRSH_DATA( 0x79 )`

10.199 SYS_SETREG_ADC_CUR_CTRL

Set the control register A_ADC_CUR_CTRL from a specified register

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SetReg_ADC_CUR_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_ADC_CUR_CTRL from a specified register
Inputs	reg = Register containing the configuration for A_ADC_CUR_CTRL
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the following settings into register CFX register x1:`
	`//  - ADC0 current = 1.3 x Inom`
	`//  - ADC1 current = 1.0 x Inom`
	`//  - ADC2 current = 2.0 x Inom`
	`//  - ADC3 current = 0.6 x Inom`
	`load x1, ( ADC0_CURRENT_1_3X | ADC1_CURRENT_1_0X | \`
	`  ADC2_CURRENT_2_0X | ADC3_CURRENT_0_6X )`
	
	`// Save the settings in CFX register x1 to A_ADC_CUR_CTRL`
	`Sys_SetReg_ADC_CUR_CTRL( x1 )`

10.200 SYS_SETREG_ADC_GFXY_CTRL

Set the control register A_ADC_GFxy_CTRL (where xy is 01 or 23) from a specified register

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SetReg_ADC_GFxy_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_ADC_GFxy_CTRL (where xy is 01 or 23) from a specified register
Inputs	reg = Register containing the configuration for A_ADC_GFxy_CTRL
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load x0 with the unity gain for the decimation filters and then`
	`// load this setting to all of the gain factors.`
	`load x0, (0x665 << ADC_GF_CTRL_ADC0_GF_POS) | \`
	`(0x665 << ADC_GF_CTRL_ADC1_GF_POS)`
	`Sys_SetReg_ADC_GF01_CTRL( x0 )`
	`Sys_SetReg_ADC_GF23_CTRL( x0 )`

10.201 SYS_SETREG_ANALOG_PWR_CTRL

Set the control register A_ANALOG_PWR_CTRL from a specified register

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SetReg_ANALOG_PWR_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_ANALOG_PWR_CTRL from a specified register
Inputs	reg = Register to load A_ANALOG_PWR_CTRL from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load x0 with ANALOG_ENABLE, then load this to A_ANALOG_PWR_CTRL`
	`load x0, ANALOG_ENABLE`
	`Sys_SetReg_ANALOG_PWR_CTRL( x0 )`

10.202 SYS_SETREG_BAND_SELECT_CTRL

Set the control register A_BAND_SELECT_CTRL from a specified register

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SetReg_BAND_SELECT_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_BAND_SELECT_CTRL from a specified register
Inputs	reg = Register to load A_BAND_SELECT_CTRL from. NOTE: The defined BAND_SELECT_ADC* bands are for FS=16kHz.
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the following settings into CFX register y1:`
	`//  - band 56kHz to 64kHz from ADC2 and the band`
	`//  48kHz to 56kHz from ADC3 to be filtered by`
	`//  the decimation filter`
	`load y1, ( BAND_SELECT_ADC2_56K_TO_64K | \`
	`  BAND_SELECT_ADC3_48K_TO_56K )`
	
	`// Save the settings in CFX register y1 to A_BAND_SELECT_CTRL`
	`Sys_SetReg_BAND_SELECT_CTRL( y1 )`

10.203 SYS_SETREG_CLK_CTRL

Set the control register A_CLK_CTRL from a specified register

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SetReg_CLK_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_CLK_CTRL from a specified register
Inputs	reg = Register to load A_CLK_CTRL from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the following settings into CFX register x1:`
	`//  - set the system clock = 1.59MHz (nominally)`
	`//  - CCR calibration value of 0xE7`
	`load x1, ( SYSCLK_1M59 | 0xE7 )`
	
	`// Save the settings in CFX register x1 to A_CLK_CTRL`
	`Sys_SetReg_CLK_CTRL( x1 )`

10.204 SYS_SETREG_FRAC_DELAY_X_CTRL

Set the control register A_FRAC_DELAY_x_CTRL (for x = 0, 1, 2, 3) from a specified register

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SetReg_FRAC_DELAY_x_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_FRAC_DELAY_x_CTRL (for x = 0, 1, 2, 3) from a specified register
Inputs	reg = Register to load A_FRAC_DELAY_x_CTRL from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load x0 with the setting required to introduce a delay of one half`
	`// of the sampling period.`
	`load x0, ( ADC0_INTEGER_DELAY_4 | \`
	`( 0 << FRAC_DELAY_FRACTIONAL_POS ) )`
	`// Delay sampling on channels 0 and 3 by half of a sampling period`
	`// (NOTE: ADC0_INTEGER_DELAY_4 is equivalent to ADC3_INTEGER_DELAY_4)`
	`Sys_SetReg_FRAC_DELAY_0_CTRL( x0 )`
	`Sys_SetReg_FRAC_DELAY_3_CTRL( x0 )`

10.205 SYS_SETREG_HEAR_BFP_BLOCK_EXP

Set the value of the block floating point block exponent from a register

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SetReg_HEAR_BFP_BLOCK_EXP( reg )`
Source File	sk3_sys_hear.cfi
Description	Set the value of the block floating point block exponent from a register.
Inputs	reg = Register containing the value to be written to the block floating point block exponent
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`#define X_BFP_EXPONENT  0x100`
	
	`// Load x0 from X_BFP_EXPONENT`
	`load x0, x[ X_BFP_EXPONENT ]`
	
	`// Store the value loaded into x0 into the block floating point`
	`// exponent used by the HEAR.`
	`Sys_SetReg_HEAR_BFP_BLOCK_EXP( x0 )`

10.206 SYS_SETREG_IN_GAIN_CTRL

Set the control register A_IN_GAIN_CTRL and calibrate the DC offsets for the ADCs

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SetReg_IN_GAIN_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_IN_GAIN_CTRL and calibrate the DC offsets for the ADCs. Calibration is based on the ADC calibration table and the new ADC preamp gain settings.
Inputs	reg = Register to load A_IN_GAIN_CTRL from
Outputs	sr[SR_X_ZERO] = If 0, ADCs calibrated. If 1, could not calibrate.
Register Modifications	yp0, yp1, a0, a1, b1, x0, x1, y1, ystep1
Assumptions	ADC calibration table loaded
Clock Cycles	Not available

Example	`// Load the following settings into CFX register x1:`
	`//  - the preamp0 gain is 15dB`
	`//  - the preamp1 gain is 30dB`
	`//  - the preamp2 gain is 27dB`
	`//  - the preamp3 gain is 24dB`
	`load x1, ( PGAIN0_15DB | PGAIN1_30DB | PGAIN2_27DB | PGAIN3_24DB )`
	
	`// Save the settings in CFX register x1 to A_IN_GAIN_CTRL`
	`Sys_SetReg_IN_GAIN_CTRL( x1 )`

10.207 SYS_SETREG_IN_GAIN_CTRL_NOCALIBRATION

Set the control register A_IN_GAIN_CTRL from a specified register

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SetReg_IN_GAIN_CTRL_NoCalibration( reg )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_IN_GAIN_CTRL from a specified register. This is only recommended for configurations that do not have an ADC calibration table.
Inputs	reg = Register to load A_IN_GAIN_CTRL from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the following settings into CFX register x1:`
	`//  - the preamp0 gain is 15dB`
	`//  - the preamp1 gain is 30dB`
	`//  - the preamp2 gain is 27dB`
	`//  - the preamp3 gain is 24dB`
	`load x1, ( PGAIN0_15DB | PGAIN1_30DB | PGAIN2_27DB | PGAIN3_24DB )`
	
	`// Save the settings in CFX register x1 to A_IN_GAIN_CTRL`
	`// NOTE: For optimal noise performance, invoke Sys_CalibrateADCs`
	`// after this macro`
	`Sys_SetReg_IN_GAIN_CTRL_NoCalibration( x1 )`

10.208 SYS_SETREG_INPUT_CTRL

Set the control register A_INPUT_CTRL from a specified register

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SetReg_INPUT_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_INPUT_CTRL from a specified register
Inputs	reg = Register to load A_INPUT_CTRL from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the following settings into CFX register y0:`
	`//  - route AI3 into ADC0`
	`//  - route AI1 into ADC1`
	`//  - route AI2 into ADC2`
	`//  - route AI1 into ADC3`
	`//  - disable ADC0, ADC1, ADC2`
	`//  - enable ADC3`
	`load y0, ( ADC0_SEL_AI3 | ADC1_SEL_AI1 | ADC2_SEL_AI2 | \`
	`  ADC3_SEL_AI1 | ADC0_DISABLE | ADC1_DISABLE | \`
	`  ADC2_DISABLE | ADC3_ENABLE )`
	
	`// Save the settings in CFX register y0 to A_INPUT_CTRL`
	`Sys_SetReg_INPUT_CTRL( y0 )`

10.209 SYS_SETREG_LSAD_CTRL

Set the control register A_LSAD_CTRL from a specified register

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SetReg_LSAD_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_LSAD_CTRL from a specified register
Inputs	reg = Register to load A_LSAD_CTRL from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the following settings into CFX register y0:`
	`//  - Channel 0-3: Configure to audio out mode`
	`//  - Channel 4: Configure to trimmer mode`
	`//  - VBAT, VDBL: Configure to audio out mode`
	`//  - Configure the LSAD sampling frequency to`
	`//  have a prescale of 320`
	`load y0, ( LSAD_CTRL_CH0_MODE_AUDIO_OUT | \`
	`LSAD_CTRL_CH1_MODE_AUDIO_OUT | \`
	`LSAD_CTRL_CH2_MODE_AUDIO_OUT | \`
	`LSAD_CTRL_CH3_MODE_AUDIO_OUT | \`
	`LSAD_CTRL_CH4_MODE_TRIMMER | \`
	`LSAD_CTRL_VBAT_MODE_AUDIO_OUT | \`
	`LSAD_CTRL_VDDC_MODE_AUDIO_OUT | \`
	`LSAD_CTRL_PRESCALE_320 )`
	
	`// Save the settings in CFX register y0 to A_LSAD_CTRL`
	`Sys_SetReg_LSAD_CTRL( y0 )`

10.210 SYS_SETREG_O_DAC_CTRL

Set the control register A_O_DAC_CTRL from a specified register

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SetReg_O_DAC_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_O_DAC_CTRL from a specified register
Inputs	reg = Register to load A_O_DAC_CTRL from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load x0 with the setting for the desired oversampling rate`
	`load x0, OSR_PRESCALE_1`
	`// Configure the DAC oversampling rate using the value loaded into x0`
	`Sys_SetReg_O_DAC_CTRL( x0 )`

10.211 SYS_SETREG_O_DAC_GF_CTRL

Set the control register A_O_DAC_GF_CTRL from a specified register

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SetReg_O_DAC_GF_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_O_DAC_GF_CTRL from a specified register
Inputs	reg = Register to load A_O_DAC_GF_CTRL from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load x0 with settings to enable the DAC Gain Factor control, and`
	`// set the gain factor to a value of 0x400 (to obtain unity gain`
	`// across the interpolation filter).`
	`load x0, ( O_DAC_GF_CTRL_ENABLE | 0x400 )`
	`Sys_SetReg_O_DAC_GF_CTRL( x0 )`

10.212 SYS_SETREG_OUTPUT_CTRL

Set the control register A_OUTPUT_CTRL from a specified register

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SetReg_OUTPUT_CTRL( reg )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_OUTPUT_CTRL from a specified register
Inputs	reg = Register to load A_OUTPUT_CTRL from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the output settings for the system to x0, configuring the`
	`// output stage to enable the DAC, but not the high-power output.`
	`load x0, ( OD0_ENABLE | OD0_HIGHPOWER_DISABLE )`
	`// Save the settings in CFX register x0 to A_OUTPUT_CTRL`
	`Sys_SetReg_OUTPUT_CTRL( x0 )`

10.213 SYS_SETREG_PSU_CFG

Set the configuration register A_PSU_CFG from a specified register [Macro for BelaSigna 300 only]

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SetReg_PSU_CFG( reg )`
Source File	sk3_sys_analog.cfi
Description	Set the configuration register A_PSU_CFG from a specified register [Macro for BelaSigna 300 only]
Inputs	reg = Register to load A_PSU_CFG from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Reload the default setting for the power supply configuration`
	`// using x0.`
	`load x0, (VDDC_LEVEL_111_VOLT | BAND_GAP_LEVEL_100_VOLT)`
	`Sys_SetReg_PSU_CFG( x0 )`

10.214 SYS_SETREG_SUP_THRSH_DATA

Set the control register A_SUP_THRSH_DATA from a specified register

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SetReg_SUP_THRSH_DATA( reg )`
Source File	sk3_sys_analog.cfi
Description	Set the control register A_SUP_THRSH_DATA from a specified register
Inputs	reg = Register to load A_SUP_THRSH_DATA from
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Load the new setting for the A_SUP_THRSH_DATA register to x0,`
	`// and set the battery monitor threshold to 1.05 volts.`
	`load x0, 0x85`
	`Sys_SetReg_SUP_THRSH_DATA( x0 )`

10.215 SYS_SPI_CS_HIGH

Set the SPI_CS line high

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SPI_CS_High`
Source File	sk3_sys_spi.cfi
Description	Set the SPI_CS line high
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	SPI port enabled
Clock Cycles	1
Example	`// Set the SPI_CS line high` `Sys_SPI_CS_High`

10.216 SYS_SPI_CS_LOW

Set the SPI_CS line low

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SPI_CS_Low`
Source File	sk3_sys_spi.cfi
Description	Set the SPI_CS line low
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	SPI port enabled
Clock Cycles	1
Example	`// Set the SPI_CS line low` `Sys_SPI_CS_Low`

10.217 SYS_SPI_DISABLE

Disable the SPI interface

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SPI_Disable`
Source File	sk3_sys_spi.cfi
Description	Disable the SPI interface
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Disable the SPI interface` `Sys_SPI_Disable`

10.218 SYS_SPI_ENABLE

Enable the SPI interface using the specified options

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SPI_Enable( spi_cfg )`
Source File	sk3_sys_spi.cfi
Description	Enable the SPI interface using the specified options
Inputs	spi_cfg = SPI configuration. Use: SPI_CLK_PRESCALE_*, SPI_PORT_AUTO_[ENABLE | DISABLE], SPI_CLKPOL_[NORMAL | INVERSE], SPI_PULLUP_[ENABLE | DISABLE]
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Enable the SPI interface in auto mode, with a normal polarity`
	`// clock prescaled by 4`
	`Sys_SPI_Enable( SPI_CLK_PRESCALE_4 | SPI_PORT_AUTO_ENABLE | \`
	`SPI_CLKPOL_NORMAL | SPI_PULLUP_ENABLE )`

10.219 SYS_SPI_READ

Read the specified number of bits over the SPI port

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SPI_Read( bits )`
Source File	sk3_sys_spi.cfi
Description	Read the specified number of bits over the SPI port. *Note:* Caller must wait until SPI operation is complete before starting another SPI operation.
Inputs	bits = Number of bits to receive (Between 1-32) Optional: OR in SPI_CS_HIGH to force CS = 1
Outputs	None
Register Modifications	None
Assumptions	SPI port enabled
Clock Cycles	1

Example	`// Initiate a read of 32 bits from the SPI port (another SPI`
	`// operation cannot be initiated until this has completed).`
	`Sys_SPI_Read( 32 )`

10.220 SYS_SPI_WAITDONE

Wait for the SPI operation to complete; exit when the SPI port is no longer busy

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SPI_WaitDone`
Source File	sk3_sys_spi.cfi
Description	Wait for the SPI operation to complete; exit when the SPI port is no longer busy
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	SPI port enabled. Valid stack pointer
Clock Cycles	Depends on SYSCLK, SPI prescale and number of bits

Example	`// Read 16 bits from the SPI port`
	`Sys_SPI_Read( 16 )`
	`// Block and wait for the 16 bits to be read from the SPI port`
	`Sys_SPI_WaitDone`

10.221 SYS_SPI_WRITE

Write the specified number of bits over the SPI port

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_SPI_Write( bits )`
Source File	sk3_sys_spi.cfi
Description	Write the specified number of bits over the SPI port. *Note:* Caller must wait until SPI operation is complete before starting another SPI operation.
Inputs	bits = Number of bits to send (Between 1-32) Optional: OR in SPI_CS_HIGH to force CS = 1
Outputs	None
Register Modifications	None
Assumptions	SPI port enabled
Clock Cycles	1

Example	`// Write 24 bits to the SPI port`
	`Sys_SPI_Write( 24 )`
	`// Block and wait for the 24 bits to be written to the SPI port`
	`Sys_SPI_WaitDone`

10.222 SYS_STACK_ALLOC

Allocate space on the stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Stack_Alloc( words )`
Source File	sk3_sys_utils.cfi
Description	Allocate space on the stack
Inputs	words = Number of words to allocate on stack
Outputs	None
Register Modifications	sp (sp = sp + words)
Assumptions	None
Clock Cycles	1
Example	`// Allocate 10 words on the stack` `Sys_Stack_Alloc( 10 )`

10.223 SYS_STACK_FREE

Free up previously allocated space on the stack

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Stack_Free( words )`
Source File	sk3_sys_utils.cfi
Description	Free up previously allocated space on the stack
Inputs	words = Number of words to free on stack
Outputs	None
Register Modifications	sp (sp = sp - words)
Assumptions	None
Clock Cycles	1
Example	`// Free 10 previously allocated words on the stack` `Sys_Stack_Free( 10 )`

10.224 SYS_TIMER1_CONFIG

Configure timer 1

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Timer1_Config( config )`
Source File	sk3_sys_timer.cfi
Description	Configure timer 1.
Inputs	config = Four-part parameter consisting of TIMER_[START | STOP], TIMER_[ONESHOT | FREERUN], Prescaler from SLOWCLK (TIMER_PRESCALE_*) and 12-bit delay (value between 0 and 4095)
Outputs	None
Register Modifications	y1
Assumptions	Current timer run state is no longer needed
Clock Cycles	2

Example	`// Configure the general-purpose timer 1 as a free-running timer,`
	`// triggering every second for a slow clock of 1.28MHz`
	`Sys_Timer1_Config( TIMER_FREERUN | TIMER_PRESCALE_32 | 40000 )`

10.225 SYS_TIMER1_START

Start timer 1

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Timer1_Start`
Source File	sk3_sys_timer.cfi
Description	Start timer 1
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Start only general-purpose timer 1` `Sys_Timer1_Start`

10.226 SYS_TIMER1_STOP

Stop timer 1

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Timer1_Stop`
Source File	sk3_sys_timer.cfi
Description	Stop timer 1
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	3
Example	`// Stop only general-purpose timer 1` `Sys_Timer1_Stop`

10.227 SYS_TIMER2_CONFIG

Configure timer 2

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Timer2_Config( config )`
Source File	sk3_sys_timer.cfi
Description	Configure timer 2.
Inputs	config = Four-part parameter consisting of TIMER_[START | STOP], TIMER_[ONESHOT | FREERUN], Prescaler from SLOWCLK (TIMER_PRESCALE_*) and 12-bit delay (value between 0 and 4095)
Outputs	None
Register Modifications	y1
Assumptions	Current timer run state is no longer needed
Clock Cycles	2

Example	`// Configure the general-purpose timer 2 as a free-running timer,`
	`// triggering every 10ms for a slow clock of 1.28MHz`
	`Sys_Timer2_Config( TIMER_FREERUN | TIMER_PRESCALE_32 | 400 )`

10.228 SYS_TIMER2_START

Start timer 2

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Timer2_Start`
Source File	sk3_sys_timer.cfi
Description	Start timer 2
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Start only general-purpose timer 2` `Sys_Timer2_Start`

10.229 SYS_TIMER2_STOP

Stop timer 2

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Timer2_Stop`
Source File	sk3_sys_timer.cfi
Description	Stop timer 2
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	3
Example	`// Stop only general-purpose timer 2` `Sys_Timer2_Stop`

10.230 SYS_TIMERS_START

Start both timers

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Timers_Start`
Source File	sk3_sys_timer.cfi
Description	Start both timers
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Start both general-purpose timers` `Sys_Timers_Start`

10.231 SYS_TIMERS_STOP

Stop both timers

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Timers_Stop`
Source File	sk3_sys_timer.cfi
Description	Stop both timers
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	6
Example	`// Stop both general-purpose timers` `Sys_Timers_Stop`

10.232 SYS_UART_DISABLE

Disable the UART

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_UART_Disable`
Source File	sk3_sys_uart.cfi
Description	Disable the UART
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Disable the UART interface` `Sys_UART_Disable`

10.233 SYS_UART_ENABLE

Enable the UART at the specified baud rate

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_UART_Enable( clk_speed, baud )`
Source File	sk3_sys_uart.cfi
Description	Enable the UART at the specified baud rate.
Inputs	clk_speed = SYSCLK clock speed baud = Baud rate to which the UART should be configured
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	2

Example	`// Configure the GPIO pads to use the UART functionality.`
	`Sys_GPIO_MUX_Select( GPIO0_SELECT_UART_TX | GPIO1_SELECT_UART_RX )`
	`Sys_GPIO_Configure( GPIO0_OUTPUT | GPIO1_INPUT )`
	
	`// Enable the UART interface for a communications baud rate of`
	`// 115,200 and a system clock speed of 5.12MHz`
	`Sys_UART_Enable( 5120000, 115200 )`

10.234 SYS_VALUE_XY

Define an XY value list that can be used with the .data directive and similar use cases

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Value_XY( high_bits, low_bits )`
Source File	sk3_sys_utils.cfi
Description	Define an XY value list that can be used with the .data directive and similar use cases
Inputs	high_bits = 24-bit value to be used for the Y portion of the XY data value low_bits = 24-bit value to be used for the X portion of the XY data value
Outputs	Data_List = Correctly ordered list that can be used to set the value of an XY data element
Register Modifications	None
Assumptions	None
Clock Cycles	0 (evaluated during compile)

Example	`// Reserve a block of 10 words in XY memory. Initialize the memory`
	`// to 0x123456ABCDEF.`
	`Sys_Mem_Start_Segment( Table_SegXY, SYS_MEMSPACE_XY )`
	`Sys_Mem_Reserve_Init( Table_InitializedXY, 10,`
	`Sys_Value_XY( 0x123456, 0xABCDEF ) )`

10.235 SYS_WATCHDOG_REFRESH

Refresh the watchdog timer count

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Watchdog_Refresh`
Source File	sk3_sys_watchdog.cfi
Description	Refresh the watchdog timer count
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`mainloop:`
	`  goto.1d mainloop`
	`  Sys_Watchdog_Refresh  // Refresh the watchdog in delay slot`

10.236 SYS_WATCHDOG_TIMEOUTCONFIG

Configure the watchdog timeout

Type	Macro
Syntax	`#include <sk3_sys.cfi>` `Sys_Watchdog_TimeoutConfig( timeout )`
Source File	sk3_sys_watchdog.cfi
Description	Configure the watchdog timeout
Inputs	timeout = Timeout value for watchdog; Use WATCHDOG_TIMEOUT_*
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1
Example	`// Set timeout to 3.2 seconds` `Sys_Watchdog_TimeoutConfig( WATCHDOG_TIMEOUT_3276M8 )`

11 System Library Reference

This reference chapter presents a detailed description of all the macros and functions in the system library, including calling parameters, modified registers and returned values.

11.1 SYS_CALIBRATEADCS

Calibrate the DC offsets for the ADCs

Type	Macro
Syntax	`#include <sk3_syslib.cfi>` `Sys_CalibrateADCs`
Source File	sk3_syslib.cfi
Description	Calibrate the DC offsets for the ADCs. Automatically calibrates the ADCs based on the calibration table and current ADCs preamp gain settings.
Inputs	None
Outputs	sr[SR_X_ZERO] = If 0, ADCs calibrated. If 1, could not calibrate.
Register Modifications	yp0, yp1, a0, a1, b1, x0, x1, y1, ystep1
Assumptions	Calibration table loaded
Clock Cycles	Not available

Example	`// Calibrate the ADCs`
	`// NOTE: Assumes a valid calibration table and that`
	`// Sys_Set_IN_GAIN_CTRL_NoCalibration or`
	`// Sys_SetReg_IN_GAIN_CTRL_NoCalibration has already been run`
	`Sys_CalibrateADCs`

11.2 SYS_DELAY

Delay the CFX for the specified amount of time

Type	Macro
Syntax	`#include <sk3_syslib.cfi>` `Sys_Delay( delay )`
Source File	sk3_syslib.cfi
Description	Delay the CFX for the specified amount of time. Uses SYSLIB_Delay.
Inputs	delay = Delay in milliseconds
Outputs	None
Register Modifications	b1, y1
Assumptions	None
Clock Cycles	Not applicable (delay is in milliseconds)

Example	`// Delay for 100 ms between calling function A and B`
	`call FunctionA`
	`Sys_Delay( 100 )`
	`call FunctionB`

11.3 SYS_DELAYREG

Delay the CFX for an amount of time specified in a register

Type	Macro
Syntax	`#include <sk3_syslib.cfi>` `Sys_DelayReg( reg )`
Source File	sk3_syslib.cfi
Description	Delay the CFX for an amount of time specified in a register. Uses SYSLIB_Delay.
Inputs	reg = Register containing delay (in milliseconds)
Outputs	None
Register Modifications	b1
Assumptions	None
Clock Cycles	Not applicable (delay is in milliseconds)

Example	`// Delay for 100 ms between calling function A and B`
	`load x0, 100`
	`call FunctionA`
	`Sys_DelayReg( x0 )`
	`call FunctionB`

11.4 SYS_RESET

Reset the system through a watchdog timeout

Type	Macro
Syntax	`#include <sk3_syslib.cfi>` `Sys_Reset`
Source File	sk3_syslib.cfi
Description	Reset the system through a watchdog timeout. Uses SYSLIB_Reset.
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	Not applicable (system is being reset)
Example	`// Reset the system` `Sys_Reset`

11.5 SYS_SETOPERATINGFREQUENCY

Set the operating frequency

Type	Macro
Syntax	`#include <sk3_syslib.cfi>` `Sys_SetOperatingFrequency( op_freq_index )`
Source File	sk3_syslib.cfi
Description	Set the operating frequency. Uses SYSLIB_SetOperatingFrequency.
Inputs	op_freq_index = Operating frequency index (Use SYS_FREQ_*)
Outputs	sr[SR_X_ZERO] = If 0, new operating frequency set. If 1, could not set frequency (no match).
Register Modifications	x1, x0, xp0, b1, b0
Assumptions	None
Clock Cycles	Not relevant (clock frequency is being adjusted)

Example	`// Switch the operating frequency to 2.56 MHz and check result.`
	`// Execute a breakpoint if no calibration value exists for`
	`// 2.56 MHz; otherwise continue with the application.`
	`Sys_SetOperatingFrequency( SYS_FREQ_2M56 )`
	`goto.0d ContinueWithApplication, not_zero`
	
	`// No calibration data. Perform some error handling.`
	`breakpoint`

11.6 SYS_SOFTPOWERDOWNDISABLE

Exit from soft power down mode

Type	Macro
Syntax	`#include <sk3_syslib.cfi>` `Sys_SoftPowerdownDisable`
Source File	sk3_syslib.cfi
Description	Exit from soft power down mode. Uses SYSLIB_SoftPowerdownDisable.
Inputs	None
Outputs	None
Register Modifications	y1, x1
Assumptions	System is in soft power down mode
Clock Cycles	Not relevant (clock source is being changed)
Example	`// Exit from soft power down mode` `Sys_SoftPowerdownDisable`

11.7 SYS_SOFTPOWERDOWNENABLE

Put the system in soft power down mode

Type	Macro
Syntax	`#include <sk3_syslib.cfi>` `Sys_SoftPowerdownEnable`
Source File	sk3_syslib.cfi
Description	Put the system in soft power down mode. In this mode analog blocks are disabled and the oscillator is slowed down to ~17 kHz. Uses SYSLIB_SoftPowerdownEnable.
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	Not relevant (clock source is being changed)
Example	`// Put the system into soft power down mode` `Sys_SoftPowerdownEnable`

11.8 SYS_SOFTPOWERDOWNENABLE_NOANALOG

Put the system in soft power down mode

Type	Macro
Syntax	`#include <sk3_syslib.cfi>` `Sys_SoftPowerdownEnable_NoAnalog`
Source File	sk3_syslib.cfi
Description	Put the system in soft power down mode. In this mode, the oscillator is slowed down to ~17 kHz; however, analog blocks remain enabled as recommended for VBAT voltages exceeding 1.3V. This scenario is described in the Hardware Errata item: high current consumption in soft power down mode. Uses SYSLIB_SoftPowerdownEnable_NoAnalog.
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	Not relevant (clock source is being changed)

Example	`// Put the system into soft power down mode without disabling`
	`// the analog blocks (recommended for VBAT > 1.3V and CIDs 101,`
	`// 102, 201).`
	`Sys_SoftPowerdownEnable_NoAnalog`

11.9 SYS_WAITFORSPIDONE

Wait for the SPI interface to become idle

Type	Macro
Syntax	`#include <sk3_syslib.cfi>` `Sys_WaitForSPIDone`
Source File	sk3_syslib.cfi
Description	Wait for the SPI interface to become idle. Uses SYSLIB_WaitForSPIDone.
Inputs	None
Outputs	None
Register Modifications	sr[SR_X_CARRY] (bit)
Assumptions	SPI interface enabled The stack has been initialized
Clock Cycles	Not applicable (arbitrary delay length)

Example	`// Transmit a word over the SPI interface in auto-mode and wait for`
	`// the SPI transmission to complete`
	`store p[D_SPI_DATA], b1`
	`Sys_WaitForSPIDone`

11.10 SYSLIB_CALIBRATEADCS

Calibrate the DC offsets for the ADCs

Type	Function
Syntax	`#include <sk3_syslib.cfi>` `call SYSLIB_CalibrateADCs`
Source File	sk3_syslib_cal_adcs.cfs
Description	Calibrate the DC offsets for the ADCs. Automatically calibrates the ADCs based on the calibration table and current ADCs preamp gain settings.
Inputs	yp0 = Pointer to ADC calibration table
Outputs	sr[SR_X_ZERO] = If 0, ADCs calibrated. If 1, could not calibrate.
Register Modifications	yp0, yp1, a0, a1, b1, x0, x1, y1, ystep1
Assumptions	Calibration table loaded
Clock Cycles	Not available

Example	`// Calibrate the ADCs`
	`// NOTE: Assumes a valid calibration table and that`
	`// Sys_Set_IN_GAIN_CTRL_NoCalibration or`
	`// Sys_SetReg_IN_GAIN_CTRL_NoCalibration has already been run`
	`call.1d SYSLIB_CalibrateADCs`
	`load yp0, SYSVAR_Y_ADCCAL_START // [ds] Address of calibration table`

11.11 SYSLIB_DELAY

Delay the core for the specified amount of time

Type	Function
Syntax	`#include <sk3_syslib.cfi>` `call SYSLIB_Delay`
Source File	sk3_syslib.cfs
Description	Delay the core for the specified amount of time
Inputs	b1 = Delay in ms - 1 (b1.l must be 0)
Outputs	None
Register Modifications	b1, y1
Assumptions	Valid SYSVAR_OPERATING_FREQ set by firmware The stack has been initialized
Clock Cycles	Not applicable (delay is in milliseconds)

Example	`// Delay for 100 ms`
	`call.1d SYSLIB_Delay`
	`load b1, 100 // Load delay value in delay slot`

11.12 SYSLIB_RESET

Reset the system with a watchdog timeout

Type	Function
Syntax	`#include <sk3_syslib.cfi>` `call SYSLIB_Reset`
Source File	sk3_syslib.cfs
Description	Reset the system with a watchdog timeout
Inputs	None
Outputs	None
Register Modifications	sr
Assumptions	None
Clock Cycles	Not applicable (system is being reset)

Example	`// Reset the system`
	`call.1d SYSLIB_Reset`
	`nop  // Delay slot`

11.13 SYSLIB_SETOPERATINGFREQUENCY

Set the operating frequency

Type	Function
Syntax	`#include <sk3_syslib.cfi>` `call SYSLIB_SetOperatingFrequency`
Source File	sk3_syslib.cfs
Description	Set the operating frequency
Inputs	b1.h = Operating frequency index (Use SYS_FREQ_* define)
Outputs	sr[SR_X_ZERO] = If 0, new operating frequency set. If 1, could not set frequency (no match).
Register Modifications	x1, x0, xp0, b1, b0
Assumptions	Calibration table loaded
Clock Cycles	Not relevant (clock frequency is being adjusted)

Example	`// Switch the operating frequency to 2.56 MHz and check result.`
	`// Execute a breakpoint if no calibration value exists for`
	`// 2.56 MHz; otherwise continue with the application.`
	`call.1d SYSLIB_SetOperatingFrequency`
	`load b1, SYS_FREQ_2M56 // Load frequency in delay slot`
	`goto.0d ContinueWithApplication, not_zero`
	
	`// No calibration data. Perform some error handling.`
	`breakpoint`

11.14 SYSLIB_SOFTPOWERDOWNDISABLE

Exit from soft power down mode

Type	Function
Syntax	`#include <sk3_syslib.cfi>` `call SYSLIB_SoftPowerdownDisable`
Source File	sk3_syslib.cfs
Description	Exit from soft power down mode.
Inputs	None
Outputs	None
Register Modifications	y1, x1
Assumptions	System is in soft power down mode (through previously called SYSLIB_SoftPowerdownEnable or SYSLIB_SoftPowerdownEnable_NoAnalog)
Clock Cycles	Not relevant (clock source is being changed)

Example	`// Restore the system from soft power down mode`
	`call.1d SYSLIB_SoftPowerdownDisable`
	`nop     // Delay slot`

11.15 SYSLIB_SOFTPOWERDOWNENABLE

Put the system in soft power down mode

Type	Function
Syntax	`#include <sk3_syslib.cfi>` `call SYSLIB_SoftPowerdownEnable`
Source File	sk3_syslib.cfs
Description	Put the system in soft power down mode. In this mode, analog blocks are disabled and the oscillator is slowed down to ~17 kHz.
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	Not relevant (clock source is being changed)

Example	`// Switch to soft power down mode`
	`call.1d SYSLIB_SoftPowerdownEnable`
	`nop     // Delay slot`

11.16 SYSLIB_SOFTPOWERDOWNENABLE_NOANALOG

Put the system in soft power down mode

Type	Function
Syntax	`#include <sk3_syslib.cfi>` `call SYSLIB_SoftPowerdownEnable_NoAnalog`
Source File	sk3_syslib_pdebl_noanalog.cfs
Description	Put the system in soft power down mode. In this mode, the oscillator is slowed down to ~17 kHz; however, analog blocks remain enabled as recommended for VBAT voltages exceeding 1.3V. This scenario is described in the Hardware Errata item for Ezairo devices with CID 1.01, 1.02 and BelaSigna 300 devices where this configuration causes high current consumption in soft power down mode [Function for Ezairo 5900, BelaSigna 300 only].
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	Not relevant (clock source is being changed)

Example	`// Switch to soft power down mode without disabling`
	`// the analog blocks (recommended for VBAT > 1.3V).`
	`call.1d SYSLIB_SoftPowerdownEnable_NoAnalog`
	`nop     // Delay slot`

11.17 SYSLIB_WAITFORSPIDONE

Wait for the SPI interface to become idle

Type	Function
Syntax	`#include <sk3_syslib.cfi>` `call SYSLIB_WaitForSPIDone`
Source File	sk3_syslib.cfs
Description	Wait for the SPI interface to become idle
Inputs	None
Outputs	None
Register Modifications	sr[SR_X_CARRY] (bit)
Assumptions	SPI interface enabled The stack has been initialized
Clock Cycles	Not applicable (arbitrary delay length)

Example	`// Read 8 bits from the SPI port and store the data in`
	`// a0. NOTE: SPI port must be enabled (not shown here)`
	`load  x0, (SPI_START | SPI_READ | (8 - 1))      // Read 8 bits`
	`call.1d SYSLIB_WaitForSPIDone`
	`store p[D_SPI_CTRL], x0                        // Delay slot`
	`load  a0, p[D_SPI_DATA]                         // Store data`

This page left blank for printing purposes.

12 Math Library Macro Reference

This reference chapter presents a detailed description of all the macros in the math library's include file, including calling parameters, modified registers, returned values and execution time.

Note: When using a ROM-based version of the math library, macro execution time is increased by 2 cycles per call into the ROM (as described in Section 2.4.1, “ROM-Based Libraries” on page 7).

12.1 MATHLIB_ATTACK_RELEASE

Calculate a first-order attack-release

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_ATTACK_RELEASE(XP_OUT, XP_COEFF)`
Source File	sk3_math.cfi
Description	Calculate a first-order attack-release. The outputs of the attack-release filter are calculated as: • y(n) = coeff * x(n) + (1-coeff) * y(n-1) = coeff * (x(n)-y(n-1)) + y(n-1) Where: • coeff = a if x(n) is larger or equal to y(n-1) • coeff = b if x(n) is smaller than y(n-1) • x(n) is the current input • y(n) is the current output • y(n-1) is the previous output • a is the coefficient for attack • b is the coefficient for release Coefficients a and b must be positive. It is assumed that (x(n)-y(n-1)) is in the range [-1, 1).
Inputs	a0.h = Input x(n) in user-defined 24-bit fixed-point format XP_OUT = Pointer to y(n-1) (24-bit fixed-point value) XP_COEFF = Pointer to coefficient a, followed by coefficient b (frac24 coefficients)
Outputs	a0.h = Output y(n) in user-defined 24-bit fixed-point format
Register Modifications	a0, x0, x1, y0
Assumptions	None
Clock Cycles	8
Example	See Section 5.6, “Examples” on page 59.

12.2 MATHLIB_ATTACK_RELEASE_VEC

Vector-wise first-order attack-release

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_ATTACK_RELEASE_VEC(XP_IN, XP_OUT, XP_COEFF, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Vector-wise first-order attack-release. See MATHLIB_ATTACK_RELEASE.
Inputs	XP_IN = Pointer to x(n) vector XP_OUT = Pointer to y(n-1) vector XP_COEFF = Pointer to a vector of coefficient-pairs VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Also the pointer to output vector y(n)
Register Modifications	a0, x0, x1, y0
Assumptions	None
Clock Cycles	3 + 9 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.3 MATHLIB_CONV_FL8N16_FRAC48

Convert fl8n16 format to frac48 format

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_CONV_fl8n16_frac48`
Source File	sk3_math.cfi
Description	Convert fl8n16 format to frac48 format.
Inputs	a0.h = Input value for conversion (fl8n16)
Outputs	a0 = Conversion result (frac48)
Register Modifications	a0, x0
Assumptions	None
Clock Cycles	3
Example	See Section 5.6, “Examples” on page 59.

12.4 MATHLIB_CONV_FL8N16_M24P24

Convert fl8n16 format to m24p24 format

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_CONV_fl8n16_m24p24`
Source File	sk3_math.cfi
Description	Convert fl8n16 format to m24p24 format.
Inputs	a0.h = Input value for conversion (fl8n16)
Outputs	a0 = Conversion result (m24p24)
Register Modifications	a0, x0
Assumptions	None
Clock Cycles	5
Example	See Section 5.6, “Examples” on page 59.

12.5 MATHLIB_CONV_FL8N16_M8P16

Convert fl8n16 format to m8p16 format

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_CONV_fl8n16_m8p16`
Source File	sk3_math.cfi
Description	Convert fl8n16 format to m8p16 format.
Inputs	a0.h = Input value for conversion (fl8n16)
Outputs	a0.h = Conversion result (m8p16)
Register Modifications	a0, x0
Assumptions	None
Clock Cycles	4
Example	See Section 5.6, “Examples” on page 59.

12.6 MATHLIB_CONV_FRAC24_FL8N16

Convert frac24 format to fl8n16 format

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_CONV_frac24_fl8n16`
Source File	sk3_math.cfi
Description	Convert frac24 format to fl8n16 format.
Inputs	a0.h = Input value for conversion (frac24)
Outputs	a0.h = Conversion result (fl8n16)
Register Modifications	a0, y0
Assumptions	None
Clock Cycles	6
Example	See Section 5.6, “Examples” on page 59.

12.7 MATHLIB_CONV_FRAC48_FL8N16

Convert frac48 format to fl8n16 format

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_CONV_frac48_fl8n16`
Source File	sk3_math.cfi
Description	Convert frac48 format to fl8n16 format.
Inputs	a0 = Input value for conversion (frac48)
Outputs	a0.h = Conversion result (fl8n16)
Register Modifications	a0, y0
Assumptions	None
Clock Cycles	5
Example	See Section 5.6, “Examples” on page 59.

12.8 MATHLIB_CONV_INT24_FL8N16

Convert int24 format to fl8n16 format

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_CONV_int24_fl8n16`
Source File	sk3_math.cfi
Description	Convert int24 format to fl8n16 format.
Inputs	a0.h = Input value for conversion (int24)
Outputs	a0.h = Conversion result (fl8n16)
Register Modifications	a0, x0, y0
Assumptions	Flag Updates mode is enabled
Clock Cycles	8
Example	See Section 5.6, “Examples” on page 59.

12.9 MATHLIB_CONV_M24P24_FL8N16

Convert m24p24 format to fl8n16 format

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_CONV_m24p24_fl8n16`
Source File	sk3_math.cfi
Description	Convert m24p24 format to fl8n16 format.
Inputs	a0 = Input value for conversion (m24p24)
Outputs	a0.h = Conversion result (fl8n16)
Register Modifications	a0, x0, y0
Assumptions	Flag Updates mode is enabled
Clock Cycles	7
Example	See Section 5.6, “Examples” on page 59.

12.10 MATHLIB_CONV_M8P16_FL8N16

Convert m8p16 format to fl8n16 format

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_CONV_m8p16_fl8n16`
Source File	sk3_math.cfi
Description	Convert m8p16 format to fl8n16 format.
Inputs	a0.h = Input value for conversion (m8p16)
Outputs	a0.h = Conversion result (fl8n16)
Register Modifications	a0, x0, y0
Assumptions	Flag Updates mode is enabled
Clock Cycles	7
Example	See Section 5.6, “Examples” on page 59.

12.11 MATHLIB_CONV_VEC_FL8N16_FRAC48

Convert vector from fl8n16 format to frac48 format

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_CONV_VEC_fl8n16_frac48(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Convert vector from fl8n16 format to frac48 format. See MATHLIB_CONV_fl8n16_frac48.
Inputs	XP_IN = Pointer to an input vector of fl8n16 elements in X memory VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of frac48 elements in XY memory.
Register Modifications	a0, x0
Assumptions	XP_IN != XP_OUT
Clock Cycles	3 + 5 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.12 MATHLIB_CONV_VEC_FL8N16_M24P24

Convert vector from fl8n16 format to m24p24 format

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_CONV_VEC_fl8n16_m24p24(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Convert vector from fl8n16 format to m24p24 format. See MATHLIB_CONV_fl8n16_m24p24.
Inputs	XP_IN = Pointer to an input vector of fl8n16 elements in X memory VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of m24p24 elements in XY memory.
Register Modifications	a0, x0
Assumptions	XP_IN != XP_OUT
Clock Cycles	3 + 7 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.13 MATHLIB_CONV_VEC_FL8N16_M8P16

Convert vector from fl8n16 format to m8p16 format

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_CONV_VEC_fl8n16_m8p16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Convert vector from fl8n16 format to m8p16 format. See MATHLIB_CONV_fl8n16_m8p16.
Inputs	XP_IN = Pointer to an input vector of fl8n16 elements in X memory VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of m8p16 elements in X memory.
Register Modifications	a0, x0
Assumptions	XP_IN != XP_OUT
Clock Cycles	3 + 6 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.14 MATHLIB_CONV_VEC_FRAC24_FL8N16

Convert vector from frac24 format to fl8n16 format

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_CONV_VEC_frac24_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Convert vector from frac24 format to fl8n16 format. See MATHLIB_CONV_frac24_fl8n16.
Inputs	XP_IN = Pointer to an input vector of frac24 elements in X memory VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of fl8n16 elements in X memory.
Register Modifications	a0, y0
Assumptions	XP_IN != XP_OUT
Clock Cycles	3 + 8 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.15 MATHLIB_CONV_VEC_FRAC48_FL8N16

Convert vector from frac48 format to fl8n16 format

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_CONV_VEC_frac48_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Convert vector from frac48 format to fl8n16 format. See MATHLIB_CONV_frac48_fl8n16.
Inputs	XP_IN = Pointer to an input vector of frac48 elements in XY memory VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of fl8n16 elements in X memory.
Register Modifications	a0, y0
Assumptions	XP_IN != XP_OUT
Clock Cycles	3 + 7 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.16 MATHLIB_CONV_VEC_INT24_FL8N16

Convert vector from int24 format to fl8n16 format

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_CONV_VEC_int24_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Convert vector from int24 format to fl8n16 format. See MATHLIB_CONV_int24_fl8n16.
Inputs	XP_IN = Pointer to an input vector of int24 elements in X memory VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of fl8n16 elements in X memory.
Register Modifications	a0, x0, y0
Assumptions	XP_IN != XP_OUT Flag Updates mode is enabled
Clock Cycles	3 + 10 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.17 MATHLIB_CONV_VEC_M24P24_FL8N16

Convert vector from m24p24 format to fl8n16 format

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_CONV_VEC_m24p24_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Convert vector from m24p24 format to fl8n16 format. See MATHLIB_CONV_m24p24_fl8n16.
Inputs	XP_IN = Pointer to input vector of m24p24 elements in XY memory. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of fl8n16 elements in X memory.
Register Modifications	a0, x0, y0
Assumptions	XP_IN != XP_OUT Flag Updates mode is enabled
Clock Cycles	3 + 9 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.18 MATHLIB_CONV_VEC_M8P16_FL8N16

Convert vector from m8p16 format to fl8n16 format

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_CONV_VEC_m8p16_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Convert vector from m8p16 format to fl8n16 format. See MATHLIB_CONV_m8p16_fl8n16.
Inputs	XP_IN = Pointer to input vector of m8p16 elements in X memory. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of fl8n16 elements in X memory.
Register Modifications	a0, x0, y0
Assumptions	XP_IN != XP_OUT Flag Updates mode is enabled
Clock Cycles	3 + 9 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.19 MATHLIB_COS_FRAC24_FRAC24

Approximate the cosine of a frac24 input

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_COS_frac24_frac24`
Source File	sk3_math.cfi
Description	Approximate the cosine of a frac24 input.
Inputs	a0.h = Normalized input x (frac24). [-1, 1) range of frac24 format number maps to [-PI, PI) range for input angle x
Outputs	a0.h = cos(x) (frac24)
Register Modifications	a0, xp0
Assumptions	None
Clock Cycles	5
Example	See Section 5.6, “Examples” on page 59.

12.20 MATHLIB_COS_VEC_FRAC24_FRAC24

Approximate the cosine of each frac24 element in a vector

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_COS_VEC_frac24_frac24(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Approximate the cosine of each frac24 element in a vector. See MATHLIB_COS_frac24_frac24.
Inputs	XP_IN = Pointer to an input vector of normalized frac24 angle values in X memory VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of frac24 elements in X memory.
Register Modifications	a0, xp0
Assumptions	None
Clock Cycles	3 + 7 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.21 MATHLIB_DB10_PWR_M8P16_FRAC48

Convert from dB10 unit to power value

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_DB10_PWR_m8p16_frac48`
Source File	sk3_math.cfi
Description	Convert from dB10 unit to power value. • y = 2^(x/(10*log(2)))
Inputs	a0.h = Input (m8p16), must be negative
Outputs	a0 = Conversion result (frac48)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	15
Example	See Section 5.6, “Examples” on page 59.

12.22 MATHLIB_DB10_PWR_VEC_M8P16_FRAC48

Convert vector of values from dB10 units to power values

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_DB10_PWR_VEC_m8p16_frac48(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Convert vector of values from dB10 units to power values. See MATHLIB_DB10_PWR_m8p16_frac48.
Inputs	XP_IN = Pointer to an input vector of m8p16 elements in X memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of frac48 elements in XY memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	6 + 12 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.23 MATHLIB_DB10_RMS_M8P16_FRAC48

Convert from dB10 unit to RMS value

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_DB10_RMS_m8p16_frac48`
Source File	sk3_math.cfi
Description	Convert from dB10 unit to RMS value. • y = 2^(x/(20*log(2)))
Inputs	a0.h = Input (m8p16), must be negative
Outputs	a0 = Conversion result (frac48)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	15
Example	See Section 5.6, “Examples” on page 59.

12.24 MATHLIB_DB10_RMS_VEC_M8P16_FRAC48

Convert vector of values from dB10 units to RMS values

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_DB10_RMS_VEC_m8p16_frac48(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Convert vector of values from dB10 units to RMS values. See MATHLIB_DB10_RMS_m8p16_frac48.
Inputs	XP_IN = Pointer to an input vector of m8p16 elements in X memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of frac48 elements in XY memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	6 + 12 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.25 MATHLIB_DB2_PWR_M8P16_FRAC48

Convert from dB2 unit to power value

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_DB2_PWR_m8p16_frac48`
Source File	sk3_math.cfi
Description	Convert from dB2 unit to power value. • y = 2^(x/10)
Inputs	a0.h = Input (m8p16), must be negative
Outputs	a0 = Conversion result (frac48)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	15
Example	See Section 5.6, “Examples” on page 59.

12.26 MATHLIB_DB2_PWR_VEC_M8P16_FRAC48

Convert vector of values from dB2 units to power values

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_DB2_PWR_VEC_m8p16_frac48(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Convert vector of values from dB2 units to power values. See MATHLIB_DB2_PWR_m8p16_frac48.
Inputs	XP_IN = Pointer to an input vector of m8p16 elements in X memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of frac48 elements in XY memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	6 + 12 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.27 MATHLIB_DB2_RMS_M8P16_FRAC48

Convert from dB2 unit to RMS value

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_DB2_RMS_m8p16_frac48`
Source File	sk3_math.cfi
Description	Convert from dB2 unit to RMS value. • y = 2^(x/20)
Inputs	a0.h = Input (m8p16), must be negative
Outputs	a0 = Conversion result (frac48)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	15
Example	See Section 5.6, “Examples” on page 59.

12.28 MATHLIB_DB2_RMS_VEC_M8P16_FRAC48

Convert vector of values from dB2 units to RMS values

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_DB2_RMS_VEC_m8p16_frac48(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Convert vector of values from dB2 units to RMS values. See MATHLIB_DB2_RMS_m8p16_frac48.
Inputs	XP_IN = Pointer to an input vector of m8p16 elements in X memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of frac48 elements in XY memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	6 + 12 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.29 MATHLIB_DIVIDE_FRAC24

Divide positive values and produce a frac24 quotient

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_DIVIDE_frac24`
Source File	sk3_math.cfi
Description	Divide positive values and produce a frac24 quotient. An undefined result is returned for dividend < 0 or divisor <= dividend.
Inputs	a0.h = Non-negative dividend (user-defined 24-bit fixed-point format) x0 = Strictly positive divisor (user-defined 24-bit fixed-point format)
Outputs	a0.l = Quotient (frac24)
Register Modifications	a0
Assumptions	The dividend is less than the divisor. Identical user-defined data formats are used for the dividend and divisor.
Clock Cycles	31
Example	See Section 5.6, “Examples” on page 59.

12.30 MATHLIB_DIVIDE_INT24

Divide positive values and produce an int24 quotient

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_DIVIDE_int24`
Source File	sk3_math.cfi
Description	Divide positive values and produce an int24 quotient. Upon completion, the remainder can be calculated in a0.h by executing the instruction "add a0, x0, lt". An undefined result is returned for dividend < 0 or divisor <= 0.
Inputs	a0.h = Non-negative dividend (user-defined 24-bit fixed-point format) x0 = Strictly positive divisor (user-defined 24-bit fixed-point format)
Outputs	a0.l = Quotient (int24)
Register Modifications	a0
Assumptions	Identical user-defined data formats are used for the dividend and divisor
Clock Cycles	32
Example	See Section 5.6, “Examples” on page 59.

12.31 MATHLIB_DIVIDE_M24P24

Divide positive values and produce an m24p24 quotient

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_DIVIDE_m24p24`
Source File	sk3_math.cfi
Description	Divide positive values and produce an m24p24 quotient. An undefined result is returned for dividend < 0 or divisor <= 0.
Inputs	a0.h = Non-negative dividend (user-defined 24-bit fixed-point format) x0 = Strictly positive divisor (user-defined 24-bit fixed-point format)
Outputs	a0 = Quotient (m24p24)
Register Modifications	a0, x1
Assumptions	Identical user-defined data formats are used for the dividend and divisor
Clock Cycles	61
Example	See Section 5.6, “Examples” on page 59.

12.32 MATHLIB_EXP2_M8P16_FRAC48

Approximate 2^x for m8p16 input x, with frac48 output

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_EXP2_m8p16_frac48`
Source File	sk3_math.cfi
Description	Approximate 2^x for m8p16 input x, with frac48 output. The input is represented as x = E + r, where E is the integer component of x and r is the fractional component. Two to the power x is approximated as: • 2^x = 2^E * 2^r = 2^(E+1) * 2^(r-1) The value of 2^(r-1) is approximated using a lookup table. Input values outside the range [-47, 0) are saturated to zero when x < -47 or to almost one (fractional full-scale value) when x >= 0.
Inputs	a0.h = Strictly negative input x (m8p16)
Outputs	a0 = Result 2^x (frac48)
Register Modifications	a0, x0, xp0
Assumptions	None
Clock Cycles	18
Example	See Section 5.6, “Examples” on page 59.

12.33 MATHLIB_EXP2_M8P16_M8P16

Approximate 2^x for m8p16 input x, with m8p16 output

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_EXP2_m8p16_m8p16`
Source File	sk3_math.cfi
Description	Approximate 2^x for m8p16 input x, with m8p16 output. The input is represented as x = E + r, where E is the integer component of x and r is the fractional component. Two to the power x is approximated as: • 2^x = 2^E * 2^r = 2^(E+1) * 2^(r-1) The value of 2^(r-1) is approximated using a lookup table. Input values outside the range [-40, 7) are saturated to zero when x < -40 or to almost 128 (m8p16 full-scale value) when x >= 7.
Inputs	a0.h = Input x (m8p16)
Outputs	a0 = Result 2^x (m8p16, but result also uses a0.l, as if output format was m8p40)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	19
Example	See Section 5.6, “Examples” on page 59.

12.34 MATHLIB_EXP2_VEC_M8P16_FRAC48

Approximate 2^x for each m8p16 element in a vector, with frac48 output values

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_EXP2_VEC_m8p16_frac48(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Approximate 2^x for each m8p16 element in a vector, with frac48 output values. See MATHLIB_EXP2_m8p16_frac48.
Inputs	XP_IN = Pointer to input vector of m8p16 elements in X memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of frac48 elements in XY memory. Cannot be xp0.
Register Modifications	a0, x0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	5 + 16 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.35 MATHLIB_EXP2_VEC_M8P16_M8P16

Approximate 2^x for each m8p16 element in a vector, with m8p16 output values

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_EXP2_VEC_m8p16_m8p16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Approximate 2^x for each m8p16 element in a vector, with m8p16 output values. See MATHLIB_EXP2_m8p16_m8p16.
Inputs	XP_IN = Pointer to input vector of m8p16 elements in X memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of m8p16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	6 + 16 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.36 MATHLIB_EXP_AVG

Calculate a first-order exponential average

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_EXP_AVG(XP_OUT, XP_COEFF)`
Source File	sk3_math.cfi
Description	Calculate a first-order exponential average. The outputs of the exponential average are calculated as: • y(n) = a * x(n) + (1-a) * y(n-1) = a * (x(n)-y(n-1)) + y(n-1) Where: • x(n) is the current input • y(n) is the current output • y(n-1) is the previous output • a is the coefficient Coefficient a must be positive. It is assumed that (x(n)-y(n-1)) is in the range [-1, 1).
Inputs	a0.h = Input x(n) in user-defined 24-bit fixed-point format XP_OUT = Pointer to y(n-1) (24-bit fixed-point value) XP_COEFF = Pointer to coefficient a (frac24)
Outputs	a0.h = Output y(n) in user-defined 24-bit fixed-point format
Register Modifications	a0, x0, y0
Assumptions	None
Clock Cycles	6
Example	See Section 5.6, “Examples” on page 59.

12.37 MATHLIB_EXP_AVG_VEC

Vector-wise first-order exponential average

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_EXP_AVG_VEC(XP_IN, XP_OUT, XP_COEFF, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Vector-wise first-order exponential average. See MATHLIB_EXP_AVG.
Inputs	XP_IN = Pointer to x(n) vector XP_OUT = Pointer to y(n-1) vector XP_COEFF = Pointer to a vector of coefficients VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Also the pointer to output vector y(n)
Register Modifications	a0, x0, y0
Assumptions	None
Clock Cycles	3 + 7 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.38 MATHLIB_LIN_INTERP

Linear interpolation

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_LIN_INTERP(XP_IN)`
Source File	sk3_math.cfi
Description	Linear interpolation. The interpolated result is calculated as: • y = t0 + x * (t1-t0) Where: • x is the index value, in range [-1, 1) • y is the interpolation result • t0 and t1 are the data points It is assumed that (t1-t0) is in the range [-1, 1).
Inputs	a0.h = Positive index value x (frac24) XP_IN = Pointer to data point t0, followed by data point t1
Outputs	a0.h = Interpolation result y (user-defined fixed-point format as defined by table data, t0 and t1)
Register Modifications	a0, x0, x1, y0
Assumptions	None
Clock Cycles	5
Example	See Section 5.6, “Examples” on page 59.

12.39 MATHLIB_LOG2_FL8N16_M8P16

Approximate the base 2 logarithm of a fl8n16 input

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_LOG2_fl8n16_m8p16`
Source File	sk3_math.cfi
Description	Approximate the base 2 logarithm of a fl8n16 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The base 2 logarithm of x is approximated as: • log2(x) = log2(m*2^E) = log2(m) + E The value of log2(m) is approximated using a lookup table. An undefined result is returned for input x <= 0. Overflow of the result returned in a0 occurs when x < 2^(-128).
Inputs	a0.h = Strictly positive input x (fl8n16)
Outputs	a0.h = Base 2 logarithm of x (m8p16)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	16
Example	See Section 5.6, “Examples” on page 59.

12.40 MATHLIB_LOG2_FRAC24_M8P16

Approximate the base 2 logarithm of a frac24 input

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_LOG2_frac24_m8p16`
Source File	sk3_math.cfi
Description	Approximate the base 2 logarithm of a frac24 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The base 2 logarithm of x is approximated as: • log2(x) = log2(m*2^E) = log2(m) + E The value of log2(m) is approximated using a lookup table. An undefined result is returned for input x <= 0.
Inputs	a0.h = Strictly positive input x (frac24)
Outputs	a0.h = Base 2 logarithm of x (m8p16)
Register Modifications	a0, x0, xp0
Assumptions	None
Clock Cycles	15
Example	See Section 5.6, “Examples” on page 59.

12.41 MATHLIB_LOG2_FRAC48_M8P16

Approximate the base 2 logarithm of a frac48 input

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_LOG2_frac48_m8p16`
Source File	sk3_math.cfi
Description	Approximate the base 2 logarithm of a frac48 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The base 2 logarithm of x is approximated as: • log2(x) = log2(m*2^E) = log2(m) + E The value of log2(m) is approximated using a lookup table. An undefined result is returned for input x <= 0.
Inputs	a0 = Strictly positive input x (frac48)
Outputs	a0.h = Base 2 logarithm of x (m8p16)
Register Modifications	a0, x0, xp0
Assumptions	None
Clock Cycles	15
Example	See Section 5.6, “Examples” on page 59.

12.42 MATHLIB_LOG2_M8P16_M8P16

Approximate the base 2 logarithm of an m8p16 input

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_LOG2_m8p16_m8p16`
Source File	sk3_math.cfi
Description	Approximate the base 2 logarithm of an m8p16 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The base 2 logarithm of x is approximated as: • log2(x) = log2(m*2^E) = log2(m) + E The value of log2(m) is approximated using a lookup table. An undefined result is returned for input x <= 0.
Inputs	a0.h = Strictly positive input x (m8p16)
Outputs	a0.h = Base 2 logarithm of x (m8p16)
Register Modifications	a0, x0, xp0
Assumptions	None
Clock Cycles	16
Example	See Section 5.6, “Examples” on page 59.

12.43 MATHLIB_LOG2_VEC_FL8N16_M8P16

Approximate log2 of each fl8n16 element in a vector

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_LOG2_VEC_fl8n16_m8p16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Approximate log2 of each fl8n16 element in a vector. See MATHLIB_LOG2_fl8n16_m8p16.
Inputs	XP_IN = Pointer to input vector of fl8n16 elements in X memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of m8p16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	5 + 16 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.44 MATHLIB_LOG2_VEC_FRAC24_M8P16

Approximate log2 of each frac24 element in a vector

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_LOG2_VEC_frac24_m8p16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Approximate log2 of each frac24 element in a vector. See MATHLIB_LOG2_frac24_m8p16.
Inputs	XP_IN = Pointer to input vector of frac24 elements in X memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of m8p16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	5 + 12 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.45 MATHLIB_LOG2_VEC_FRAC48_M8P16

Approximate log2 of each frac48 element in a vector

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_LOG2_VEC_frac48_m8p16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Approximate log2 of each frac48 element in a vector. See MATHLIB_LOG2_frac48_m8p16.
Inputs	XP_IN = Pointer to input vector of frac48 elements in XY memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of m8p16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	5 + 12 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.46 MATHLIB_LOG2_VEC_M8P16_M8P16

Approximate log2 of each m8p16 element in a vector

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_LOG2_VEC_m8p16_m8p16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Approximate log2 of each m8p16 element in a vector. See MATHLIB_LOG2_m8p16_m8p16.
Inputs	XP_IN = Pointer to input vector of m8p16 elements in X memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of m8p16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	6 + 13 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.47 MATHLIB_MAG_FRAC24_M2P46

Calculate the magnitude of a complex number

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_MAG_frac24_m2p46(XP_IN)`
Source File	sk3_math.cfi
Description	Calculate the magnitude of a complex number. • y = sqrt(real(x)^2 + imag(x)^2)
Inputs	XP_IN = Pointer to complex value x in X memory, with frac24 real part followed by frac24 imaginary part at next address
Outputs	a0 = Magnitude result (m2p46)
Register Modifications	a0, x0, xp0
Assumptions	None
Clock Cycles	19 (E even in SQRT) or 20 (E odd in SQRT)
Example	See Section 5.6, “Examples” on page 59.

12.48 MATHLIB_MAG_VEC_FRAC24_M2P46

Vector-wise magnitude of complex values

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_MAG_VEC_frac24_m2p46(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Vector-wise magnitude of complex values. See MATHLIB_MAG_frac24_m2p46.
Inputs	XP_IN = Pointer to an input vector of complex values in X memory. Real and complex components stored in interleaved manner. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of m2p46 elements in XY memory. Cannot be xp0.
Register Modifications	a0, x0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	In range [5 + 18 * VEC_LENGTH, 5 + 19 * VEC_LENGTH]
Example	See Section 5.6, “Examples” on page 59.

12.49 MATHLIB_PWR_DB10_FRAC48_M8P16

Convert from power value to dB10 unit

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_PWR_DB10_frac48_m8p16`
Source File	sk3_math.cfi
Description	Convert from power value to dB10 unit. • y = 10 * log10(2) * log2(x) Overflow of the result returned in a0.h will occur when the input is less than 10^(-128/10).
Inputs	a0 = Input (frac48)
Outputs	a0.h = Conversion result (m8p16)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	17
Example	See Section 5.6, “Examples” on page 59.

12.50 MATHLIB_PWR_DB10_VEC_FRAC48_M8P16

Convert vector of power values to dB10 units

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_PWR_DB10_VEC_frac48_m8p16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Convert vector of power values to dB10 units. See MATHLIB_PWR_DB10_frac48_m8p16.
Inputs	XP_IN = Pointer to an input vector of frac48 elements in XY memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of m8p16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	6 + 16 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.51 MATHLIB_PWR_DB2_FRAC48_M8P16

Convert from power value to dB2 unit

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_PWR_DB2_frac48_m8p16`
Source File	sk3_math.cfi
Description	Convert from power value to dB2 unit. • y = 10 * log2(x) Overflow of the result returned in a0.h will occur when the input is less than 2^(-128/10).
Inputs	a0 = Input (frac48)
Outputs	a0.h = Conversion result (m8p16)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	17
Example	See Section 5.6, “Examples” on page 59.

12.52 MATHLIB_PWR_DB2_VEC_FRAC48_M8P16

Convert vector of power values to dB2 units

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_PWR_DB2_VEC_frac48_m8p16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Convert vector of power values to dB2 units. See MATHLIB_PWR_DB2_frac48_m8p16.
Inputs	XP_IN = Pointer to an input vector of frac48 elements in XY memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of m8p16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	6 + 16 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.53 MATHLIB_RECIP_FL8N16_FL8N16

Approximate the reciprocal of a signed fl8n16 input

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_RECIP_fl8n16_fl8n16`
Source File	sk3_math.cfi
Description	Approximate the reciprocal of a signed fl8n16 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The reciprocal of x is approximated as: • 1/x = (1/m) * 2^(-E) = (1/(2m)) * (2^(-E+1)) 1/(2m) is approximated using a lookup table. An undefined result is returned for input x = 0. Overflow of the result returned in a0 occurs when the magnitude of the input x is less than 2^(-127).
Inputs	a0.h = Non-zero input x (fl8n16)
Outputs	a0.h = Reciprocal of x (fl8n16)
Register Modifications	a0, x0, x1, y0, y1, xp0
Assumptions	None
Clock Cycles	19
Example	See Section 5.6, “Examples” on page 59.

12.54 MATHLIB_RECIP_FRAC48_FL8N16

Approximate the reciprocal of a signed frac48 input

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_RECIP_frac48_fl8n16`
Source File	sk3_math.cfi
Description	Approximate the reciprocal of a signed frac48 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The reciprocal of x is approximated as: • 1/x = (1/m) * 2^(-E) = (1/(2m)) * (2^(-E+1)) 1/(2m) is approximated using a lookup table. An undefined result is returned for input x = 0.
Inputs	a0 = Non-zero input x (frac48)
Outputs	a0.h = Reciprocal of x (fl8n16)
Register Modifications	a0, x0, x1, y0, xp0
Assumptions	None
Clock Cycles	18
Example	See Section 5.6, “Examples” on page 59.

12.55 MATHLIB_RECIP_INT24_FL8N16

Approximate the reciprocal of a signed int24 input

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_RECIP_int24_fl8n16`
Source File	sk3_math.cfi
Description	Approximate the reciprocal of a signed int24 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The reciprocal of x is approximated as: • 1/x = (1/m) * 2^(-E) = (1/(2m)) * (2^(-E+1)) 1/(2m) is approximated using a lookup table. An undefined result is returned for input x = 0.
Inputs	a0.h = Non-zero input x (int24)
Outputs	a0.h = Reciprocal of x (fl8n16)
Register Modifications	a0, x0, x1, y0, xp0
Assumptions	None
Clock Cycles	19
Example	See Section 5.6, “Examples” on page 59.

12.56 MATHLIB_RECIP_M8P16_FL8N16

Approximate the reciprocal of a signed m8p16 input

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_RECIP_m8p16_fl8n16`
Source File	sk3_math.cfi
Description	Approximate the reciprocal of a signed m8p16 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The reciprocal of x is approximated as: • 1/x = (1/m) * 2^(-E) = (1/(2m)) * (2^(-E+1)) 1/(2m) is approximated using a lookup table. An undefined result is returned for input x = 0.
Inputs	a0.h = Non-zero input x (m8p16)
Outputs	a0.h = Reciprocal of x (fl8n16)
Register Modifications	a0, x0, x1, y0, xp0
Assumptions	None
Clock Cycles	19
Example	See Section 5.6, “Examples” on page 59.

12.57 MATHLIB_RECIP_POS_FL8N16_FL8N16

Approximate the reciprocal of a positive fl8n16 input

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_RECIP_POS_fl8n16_fl8n16`
Source File	sk3_math.cfi
Description	Approximate the reciprocal of a positive fl8n16 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The reciprocal of x is approximated as: • 1/x = (1/m) * 2^(-E) = (1/(2m)) * (2^(-E+1)) 1/(2m) is approximated using a lookup table. An undefined result is returned for input x <= 0. Overflow of the result returned in a0 occurs when x < 2^(-127).
Inputs	a0.h = Strictly positive input x (fl8n16)
Outputs	a0.h = Reciprocal of x (fl8n16)
Register Modifications	a0, x0, x1, y0, xp0
Assumptions	None
Clock Cycles	16
Example	See Section 5.6, “Examples” on page 59.

12.58 MATHLIB_RECIP_POS_FRAC48_FL8N16

Approximate the reciprocal of a positive frac48 input

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_RECIP_POS_frac48_fl8n16`
Source File	sk3_math.cfi
Description	Approximate the reciprocal of a positive frac48 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The reciprocal of x is approximated as: • 1/x = (1/m) * 2^(-E) = (1/(2m)) * (2^(-E+1)) 1/(2m) is approximated using a lookup table. An undefined result is returned for input x <= 0.
Inputs	a0 = Strictly positive input x (frac48)
Outputs	a0.h = Reciprocal of x (fl8n16)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	15
Example	See Section 5.6, “Examples” on page 59.

12.59 MATHLIB_RECIP_POS_INT24_FL8N16

Approximate the reciprocal of a positive int24 input

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_RECIP_POS_int24_fl8n16`
Source File	sk3_math.cfi
Description	Approximate the reciprocal of a positive int24 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The reciprocal of x is approximated as: • 1/x = (1/m) * 2^(-E) = (1/(2m)) * (2^(-E+1)) 1/(2m) is approximated using a lookup table. An undefined result is returned for input x <= 0.
Inputs	a0.h = Strictly positive input x (int24)
Outputs	a0.h = Reciprocal of x (fl8n16)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	16
Example	See Section 5.6, “Examples” on page 59.

12.60 MATHLIB_RECIP_POS_M8P16_FL8N16

Approximate the reciprocal of a positive m8p16 input

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_RECIP_POS_m8p16_fl8n16`
Source File	sk3_math.cfi
Description	Approximate the reciprocal of a positive m8p16 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The reciprocal of x is approximated as: • 1/x = (1/m) * 2^(-E) = (1/(2m)) * (2^(-E+1)) 1/(2m) is approximated using a lookup table. An undefined result is returned for input x <= 0.
Inputs	a0.h = Strictly positive input x (m8p16)
Outputs	a0.h = Reciprocal of x (fl8n16)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	16
Example	See Section 5.6, “Examples” on page 59.

12.61 MATHLIB_RECIP_POS_VEC_FL8N16_FL8N16

Approximate the reciprocal of each positive fl8n16 element in a vector

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_RECIP_POS_VEC_fl8n16_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Approximate the reciprocal of each positive fl8n16 element in a vector. See MATHLIB_RECIP_POS_fl8n16_fl8n16.
Inputs	XP_IN = Pointer to input vector of fl8n16 elements in X memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of fl8n16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, x1, y0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	5 + 16 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.62 MATHLIB_RECIP_POS_VEC_FRAC48_FL8N16

Approximate the reciprocal of each positive frac48 element in a vector

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_RECIP_POS_VEC_frac48_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Approximate the reciprocal of each positive frac48 element in a vector. See MATHLIB_RECIP_POS_frac48_fl8n16.
Inputs	XP_IN = Pointer to input vector of frac48 elements in XY memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of fl8n16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	5 + 12 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.63 MATHLIB_RECIP_POS_VEC_INT24_FL8N16

Approximate the reciprocal of each positive int24 element in a vector

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_RECIP_POS_VEC_int24_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Approximate the reciprocal of each positive int24 element in a vector. See MATHLIB_RECIP_POS_int24_fl8n16.
Inputs	XP_IN = Pointer to input vector of int24 elements in X memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of fl8n16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, x1, y0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	6 + 13 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.64 MATHLIB_RECIP_POS_VEC_M8P16_FL8N16

Approximate the reciprocal of each positive m8p16 element in a vector

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_RECIP_POS_VEC_m8p16_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Approximate the reciprocal of each positive m8p16 element in a vector. See MATHLIB_RECIP_POS_m8p16_fl8n16.
Inputs	XP_IN = Pointer to input vector of m8p16 elements in X memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of fl8n16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, x1, y0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	6 + 13 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.65 MATHLIB_RECIP_VEC_FL8N16_FL8N16

Approximate the reciprocal of each signed fl8n16 element in a vector

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_RECIP_VEC_fl8n16_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Approximate the reciprocal of each signed fl8n16 element in a vector. See MATHLIB_RECIP_fl8n16_fl8n16.
Inputs	XP_IN = Pointer to input vector of fl8n16 elements in X memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of fl8n16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, x1, y0, y1, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	5 + 19 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.66 MATHLIB_RECIP_VEC_FRAC48_FL8N16

Approximate the reciprocal of each signed frac48 element in a vector

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_RECIP_VEC_frac48_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Approximate the reciprocal of each signed frac48 element in a vector. See MATHLIB_RECIP_frac48_fl8n16.
Inputs	XP_IN = Pointer to input vector of frac48 elements in XY memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of fl8n16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, x1, y0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	5 + 15 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.67 MATHLIB_RECIP_VEC_INT24_FL8N16

Approximate the reciprocal of each signed int24 element in a vector

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_RECIP_VEC_int24_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Approximate the reciprocal of each signed int24 element in a vector. See MATHLIB_RECIP_int24_fl8n16.
Inputs	XP_IN = Pointer to input vector of int24 elements in X memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of fl8n16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, x1, y0, y1, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	6 + 16 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.68 MATHLIB_RECIP_VEC_M8P16_FL8N16

Approximate the reciprocal of each signed m8p16 element in a vector

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_RECIP_VEC_m8p16_fl8n16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Approximate the reciprocal of each signed m8p16 element in a vector. See MATHLIB_RECIP_m8p16_fl8n16.
Inputs	XP_IN = Pointer to input vector of m8p16 elements in X memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of fl8n16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, x1, y0, y1, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	6 + 16 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.69 MATHLIB_RMS_DB10_FRAC48_M8P16

Convert from RMS value to dB10 unit

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_RMS_DB10_frac48_m8p16`
Source File	sk3_math.cfi
Description	Convert from RMS value to dB10 unit. • y = 20 * log10(2) * log2(x) Overflow of the result returned in a0.h will occur when the input is less than 10^(-128/20).
Inputs	a0 = Input (frac48)
Outputs	a0.h = Conversion result (m8p16)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	17
Example	See Section 5.6, “Examples” on page 59.

12.70 MATHLIB_RMS_DB10_VEC_FRAC48_M8P16

Convert vector of RMS values to dB10 units

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_RMS_DB10_VEC_frac48_m8p16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Convert vector of RMS values to dB10 units. See MATHLIB_RMS_DB10_frac48_m8p16.
Inputs	XP_IN = Pointer to an input vector of frac48 elements in XY memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of m8p16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	6 + 16 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.71 MATHLIB_RMS_DB2_FRAC48_M8P16

Convert from RMS value to dB2 unit

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_RMS_DB2_frac48_m8p16`
Source File	sk3_math.cfi
Description	Convert from RMS value to dB2 unit. • y = 20 * log2(x) Overflow of the result returned in a0.h will occur when the input is less than 2^(-128/20).
Inputs	a0 = Input (frac48)
Outputs	a0.h = Conversion result (m8p16)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	17
Example	See Section 5.6, “Examples” on page 59.

12.72 MATHLIB_RMS_DB2_VEC_FRAC48_M8P16

Convert vector of RMS values to dB2 units

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_RMS_DB2_VEC_frac48_m8p16(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Convert vector of RMS values to dB2 units. See MATHLIB_RMS_DB2_frac48_m8p16.
Inputs	XP_IN = Pointer to an input vector of frac48 elements in XY memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of m8p16 elements in X memory. Cannot be xp0.
Register Modifications	a0, x0, y0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	6 + 16 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.73 MATHLIB_SIN_FRAC24_FRAC24

Approximate the sine of a frac24 input

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_SIN_frac24_frac24`
Source File	sk3_math.cfi
Description	Approximate the sine of a frac24 input.
Inputs	a0.h = Normalized input x (frac24). [-1, 1) range of frac24 format number maps to [-PI, PI) range for input angle x
Outputs	a0.h = sin(x) (frac24)
Register Modifications	a0, xp0
Assumptions	None
Clock Cycles	5
Example	See Section 5.6, “Examples” on page 59.

12.74 MATHLIB_SIN_VEC_FRAC24_FRAC24

Approximate the sine of each frac24 element in a vector

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_SIN_VEC_frac24_frac24(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Approximate the sine of each frac24 element in a vector. See MATHLIB_SIN_frac24_frac24.
Inputs	XP_IN = Pointer to an input vector of normalized frac24 angle values in X memory VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of frac24 elements in X memory.
Register Modifications	a0, xp0
Assumptions	None
Clock Cycles	3 + 7 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.75 MATHLIB_SQMAG_FRAC24_M2P46

Calculate the squared magnitude of complex number x

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_SQMAG_frac24_m2p46(XP_IN)`
Source File	sk3_math.cfi
Description	Calculate the squared magnitude of complex number x. • y = real(x)^2 + imag(x)^2 Saturation mode is assumed to be set.
Inputs	XP_IN = Pointer to complex value x in X memory, with frac24 real part followed by frac24 imaginary part at next address
Outputs	a0 = Squared magnitude result (m2p46)
Register Modifications	a0, x0
Assumptions	None
Clock Cycles	6
Example	See Section 5.6, “Examples” on page 59.

12.76 MATHLIB_SQMAG_VEC_FRAC24_M2P46

Vector-wise squared magnitude of complex values

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_SQMAG_VEC_frac24_m2p46(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Vector-wise squared magnitude of complex values. See MATHLIB_SQMAG_frac24_m2p46.
Inputs	XP_IN = Pointer to an input vector of complex values in X memory. Real and complex components stored in interleaved manner. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of m2p46 elements in XY memory. Cannot be xp0.
Register Modifications	a0, x0
Assumptions	XP_IN != XP_OUT
Clock Cycles	3 + 7 * VEC_LENGTH
Example	See Section 5.6, “Examples” on page 59.

12.77 MATHLIB_SQRT_FRAC24_FRAC48

Approximate the square root of a frac24 input

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_SQRT_frac24_frac48`
Source File	sk3_math.cfi
Description	Approximate the square root of a frac24 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The square root of x is approximated as either: • sqrt(x) = 2^(E/2) * sqrt(m) for even E • sqrt(x) = 2^((E+1)/2) * sqrt(m/2) for odd E The value of sqrt(m) or sqrt(m/2) is approximated using a lookup table. An undefined result is returned for input x <= 0.
Inputs	a0.h = Strictly positive input x (frac24)
Outputs	a0 = Square root of x (frac48)
Register Modifications	a0, x0, xp0
Assumptions	None
Clock Cycles	17 (E even) or 18 (E odd)
Example	See Section 5.6, “Examples” on page 59.

12.78 MATHLIB_SQRT_FRAC48_FRAC48

Approximate the square root of a frac48 input

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_SQRT_frac48_frac48`
Source File	sk3_math.cfi
Description	Approximate the square root of a frac48 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The square root of x is approximated as either: • sqrt(x) = 2^(E/2) * sqrt(m) for even E • sqrt(x) = 2^((E+1)/2) * sqrt(m/2) for odd E The value of sqrt(m) or sqrt(m/2) is approximated using a lookup table. An undefined result is returned for input x <= 0.
Inputs	a0 = Strictly positive input x (frac48)
Outputs	a0 = Square root of x (frac48)
Register Modifications	a0, x0, xp0
Assumptions	None
Clock Cycles	17 (E even) or 18 (E odd)
Example	See Section 5.6, “Examples” on page 59.

12.79 MATHLIB_SQRT_VEC_FRAC24_FRAC48

Approximate the square root of each frac24 element in a vector

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_SQRT_VEC_frac24_frac48(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Approximate the square root of each frac24 element in a vector. See MATHLIB_SQRT_frac24_frac48.
Inputs	XP_IN = Pointer to input vector of frac24 elements in X memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of frac48 elements in XY memory. Cannot be xp0.
Register Modifications	a0, x0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	In range [5 + 14 * VEC_LENGTH, 5 + 15 * VEC_LENGTH]
Example	See Section 5.6, “Examples” on page 59.

12.80 MATHLIB_SQRT_VEC_FRAC48_FRAC48

Approximate the square root of each frac48 element in a vector

Type	Macro
Syntax	`#include <sk3_math.cfi>` `MATHLIB_SQRT_VEC_frac48_frac48(XP_IN, XP_OUT, VEC_LENGTH)`
Source File	sk3_math.cfi
Description	Approximate the square root of each frac48 element in a vector. See MATHLIB_SQRT_frac48_frac48.
Inputs	XP_IN = Pointer to input vector of frac48 elements in XY memory. Cannot be xp0. VEC_LENGTH = Number of elements in the vector
Outputs	XP_OUT = Pointer to output vector of frac48 elements in XY memory. Cannot be xp0.
Register Modifications	a0, x0, xp0
Assumptions	XP_IN != XP_OUT
Clock Cycles	In range [5 + 14 * VEC_LENGTH, 5 + 15 * VEC_LENGTH]
Example	See Section 5.6, “Examples” on page 59.

13 Math Library Function Reference

This reference chapter presents a detailed description of all the functions in the math library, including calling parameters, modified registers, returned values and execution time.

Note: When using a ROM-based version of the math library, function execution time is increased by 2 cycles (as described in Section 2.4.1, “ROM-Based Libraries” on page 7).

13.1 _MATHLIB_DIVIDE

Divide a non-negative dividend by a positive divisor

Type	Function
Syntax	`#include <sk3_math.cfi>` `call _MATHLIB_DIVIDE`
Source File	sk3_mathlib.cfs
Description	Divide a non-negative dividend by a positive divisor. The data format of the dividend and divisor is user-defined but must be the same 24-bit fixed-point format. This function is used as a component of larger division routines implemented by the Math Library macros. An undefined result is returned for dividend < 0 or divisor <= 0.
Inputs	a0.h = Non-negative dividend (user-defined 24-bit fixed-point format) x0 = Strictly positive divisor (user-defined 24-bit fixed-point format)
Outputs	a0.l = Quotient (int24)
Register Modifications	a0
Assumptions	None
Clock Cycles	26
Example	See Section 5.6, “Examples” on page 59.

13.2 _MATHLIB_EXP2_M8P16_FRAC48

Approximate 2^x for m8p16 input x, with frac48 output

Type	Function
Syntax	`#include <sk3_math.cfi>` `call _MATHLIB_EXP2_m8p16_frac48`
Source File	sk3_mathlib.cfs
Description	Approximate 2^x for m8p16 input x, with frac48 output. The input is represented as x = E + r, where E is the integer component of x and r is the fractional component. Two to the power x is approximated as: • 2^x = 2^E * 2^r = 2^(E+1) * 2^(r-1) The value of 2^(r-1) is approximated using a lookup table. An undefined result is returned for input x >= 0.
Inputs	a0.h = Strictly negative input x (m8p16)
Outputs	a0 = Result 2^x (frac48)
Register Modifications	a0, x0, xp0
Assumptions	None
Clock Cycles	7
Example	See Section 5.6, “Examples” on page 59.

13.3 _MATHLIB_LOG2_FRAC48_M8P16

Approximate the base 2 logarithm of a frac48 input

Type	Function
Syntax	`#include <sk3_math.cfi>` `call _MATHLIB_LOG2_frac48_m8p16`
Source File	sk3_mathlib.cfs
Description	Approximate the base 2 logarithm of a frac48 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The base 2 logarithm of x is approximated as: • log2(x) = log2(m*2^E) = log2(m) + E The value of log2(m) is approximated using a lookup table. An undefined result is returned for input x <= 0.
Inputs	a0 = Strictly positive input x (frac48)
Outputs	a0.h = Base 2 logarithm of x (m8p16)
Register Modifications	a0, x0, xp0
Assumptions	None
Clock Cycles	9
Example	See Section 5.6, “Examples” on page 59.

13.4 _MATHLIB_RECIP_GEN_FRAC48_FL8N16

Approximate the reciprocal of a signed frac48 input

Type	Function
Syntax	`#include <sk3_math.cfi>` `call _MATHLIB_RECIP_GEN_frac48_fl8n16`
Source File	sk3_mathlib.cfs
Description	Approximate the reciprocal of a signed frac48 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The reciprocal of x is approximated as: • 1/x = (1/m) * 2^(-E) = (1/(2m)) * (2^(-E+1)) 1/(2m) is approximated using a lookup table. An undefined result is returned for input x = 0.
Inputs	a0 = Non-zero input x (frac48)
Outputs	a0.h = Reciprocal of x (fl8n16)
Register Modifications	a0, x0, x1, y0, xp0
Assumptions	None
Clock Cycles	12
Example	See Section 5.6, “Examples” on page 59.

13.5 _MATHLIB_RECIP_POS_FRAC48_FL8N16

Approximate the reciprocal of a positive frac48 input

Type	Function
Syntax	`#include <sk3_math.cfi>` `call _MATHLIB_RECIP_POS_frac48_fl8n16`
Source File	sk3_mathlib.cfs
Description	Approximate the reciprocal of a positive frac48 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The reciprocal of x is approximated as: • 1/x = (1/m) * 2^(-E) = (1/(2m)) * (2^(-E+1)) 1/(2m) is approximated using a lookup table. An undefined result is returned for input x <= 0.
Inputs	a0 = Strictly positive input x (frac48)
Outputs	a0.h = Reciprocal of x (fl8n16)
Register Modifications	a0, x0, y0, xp0
Assumptions	None
Clock Cycles	9
Example	See Section 5.6, “Examples” on page 59.

13.6 _MATHLIB_SQRT_FRAC48_FRAC48

Approximate the square root of a frac48 input

Type	Function
Syntax	`#include <sk3_math.cfi>` `call _MATHLIB_SQRT_frac48_frac48`
Source File	sk3_mathlib.cfs
Description	Approximate the square root of a frac48 input. The input is represented as x = m * 2^E, where E is the integer exponent of x and m is the mantissa in [0.5, 1). The square root of x is approximated as either: • sqrt(x) = 2^(E/2) * sqrt(m) for even E • sqrt(x) = 2^((E+1)/2) * sqrt(m/2) for odd E The value of sqrt(m) or sqrt(m/2) is approximated using a lookup table. An undefined result is returned for input x <= 0.
Inputs	a0 = Strictly positive input x (frac48)
Outputs	a0 = Square root of x (frac48)
Register Modifications	a0, x0, xp0
Assumptions	None
Clock Cycles	11 (E even) or 12 (E odd)
Example	See Section 5.6, “Examples” on page 59.

14 EEPROM Library Reference

This reference chapter presents a detailed description of all the macros and functions in the EEPROM library, including calling parameters, modified registers and returned values.

14.1 EELIB_CLOSE

Close the EEPROM library

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_Close`
Source File	sk3_eeprom.cfs
Description	Close the EEPROM library.
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	

Example	`// Close the EEPROM Library`
	`call.1d EELIB_Close`
	`nop`

14.2 EELIB_COMMIT

Commit EEPROM buffer to EEPROM non-volatile memory

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_Commit`
Source File	sk3_eeprom.cfs
Description	Commit EEPROM buffer to EEPROM non-volatile memory
Inputs	None
Outputs	sr[SR_X_ZERO] = Timeout status (0 - No timeout, 1 - Timeout)
Register Modifications	b1
Assumptions	None
Clock Cycles	

Example	`// Commit the current writes to the EEPROM library. Branch to`
	`// function A if the writes failed.`
	`call.1d EELIB_Commit`
	`nop`
	`goto.0d FunctionA, zero`

14.3 EELIB_INITIALIZE

Initialize the EEPROM library

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_Initialize`
Source File	sk3_eeprom.cfs
Description	Initialize the EEPROM library. Sets the default commit timeout and write disables the library.
Inputs	b1 = SPI speed (Use SPI_CLK_PRESCALE_* define)
Outputs	None
Register Modifications	b1, x1
Assumptions	None
Clock Cycles	

Example	`// Initialize the EEPROM Library`
	`call.1d EELIB_Initialize`
	`load b1, SPI_CLK_PRESCALE_2  // Load SPI prescaler in the delay slot`

14.4 EELIB_INITWRITEWORD

Initialize the EEPROM for writing at the specified address

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_InitWriteWord`
Source File	sk3_eeprom.cfs
Description	Initialize the EEPROM for writing at the specified address.
Inputs	b0.h = EEPROM Address
Outputs	None
Register Modifications	b1.l
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Setup to write a block of data at EEPROM_WRITE_ADDRESS`
	`load xp0, MEMORY_BUFFER_START`
	`call.1d EELIB_InitWriteWord`
	`load b0.h, EEPROM_WRITE_ADDRESS  // Load the address to write to`
	`  // in the delay slot`
	
	`// Write the 16-bit data from memory buffer to EEPROM`
	`Write_Loop:`
	`  call.1d EELIB_WriteNextWord16`
	`  nop ||| load b1.h, x[xp0++]  // Load the word to write in the`
	`  // delay slot`
	`  nop ||| move b1, xp0`
	`  cmp b1, MEMORY_BUFFER_SIZE ||| nop`
	`  goto.0d Write_Loop, ne`

14.5 EELIB_LIBWRITEDISABLE

Write disable the EEPROM library

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_LibWriteDisable`
Source File	sk3_eeprom.cfs
Description	Write disable the EEPROM library.
Inputs	None
Outputs	None
Register Modifications	b1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Write disable the EEPROM Library`
	`call.1d EELIB_LibWriteDisable`
	`nop`

14.6 EELIB_LIBWRITEENABLE

Write enable the EEPROM library

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_LibWriteEnable`
Source File	sk3_eeprom.cfs
Description	Write enable the EEPROM library.
Inputs	b1 = AT_WREN opcode
Outputs	None
Register Modifications	None
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Write enable the EEPROM Library`
	`call.1d EELIB_LibWriteEnable`
	`load b1, AT_WREN  // Load write enable code in the delay slot`

14.7 EELIB_POWERDOWN

Attempt to place the EEPROM in power down mode

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_PowerDown`
Source File	sk3_eeprom2.cfs
Description	Attempt to place the EEPROM in power down mode
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Power down the EEPROM to reduce the current consumed by this`
	`// device while it is not in use.`
	`call.1d EELIB_PowerDown`
	`nop`

14.8 EELIB_POWERUP

Place an EEPROM, that was in power down mode, into power up mode

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_PowerUp`
Source File	sk3_eeprom2.cfs
Description	Place an EEPROM, that was in power down mode, into power up mode
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Ensure the EEPROM is powered up before trying to read from it`
	`call.1d EELIB_PowerUp`
	`nop`

14.9 EELIB_READBLOCK_P

Read a block of data from EEPROM to PMEM

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_ReadBlock_P`
Source File	sk3_eeprom.cfs
Description	Read a block of data from EEPROM to PMEM. Leaves EEPROM queued.
Inputs	x0 = Size of block (in words) b0.h = EEPROM Address xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Read a block of MEMORY_BUFFER_SIZE words from EEPROM at`
	`// EEPROM_READ_ADDRESS, to P-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_READ_ADDRESS`
	`call.1d EELIB_ReadBlock_P`
	`load xp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`                                  // delay slot`

14.10 EELIB_READBLOCK_P16

Read a block of data from EEPROM to PMEM in 16-bit words

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_ReadBlock_P16`
Source File	sk3_eeprom.cfs
Description	Read a block of data from EEPROM to PMEM in 16-bit words. Leaves EEPROM queued.
Inputs	x0 = Size of block (in words) b0.h = EEPROM Address xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Read a block of MEMORY_BUFFER_SIZE 16-bit words from EEPROM at`
	`// EEPROM_READ_ADDRESS, to P-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_READ_ADDRESS`
	`call.1d EELIB_ReadBlock_P16`
	`load xp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`                                  // delay slot`

14.11 EELIB_READBLOCK_X

Read a block of data from EEPROM to XMEM

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_ReadBlock_X`
Source File	sk3_eeprom.cfs
Description	Read a block of data from EEPROM to XMEM. Leaves EEPROM queued.
Inputs	x0 = Size of block (in words) b0.h = EEPROM Address xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Read a block of MEMORY_BUFFER_SIZE words from EEPROM at`
	`// EEPROM_READ_ADDRESS, to X-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_READ_ADDRESS`
	`call.1d EELIB_ReadBlock_X`
	`load xp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`                                  // delay slot`

14.12 EELIB_READBLOCK_X16

Read a block of data from EEPROM to XMEM in 16-bit words

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_ReadBlock_X16`
Source File	sk3_eeprom.cfs
Description	Read a block of data from EEPROM to XMEM in 16-bit words. Leaves EEPROM queued.
Inputs	x0 = Size of block (in words) b0.h = EEPROM Address xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Read a block of MEMORY_BUFFER_SIZE 16-bit words from EEPROM at`
	`// EEPROM_READ_ADDRESS, to X-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_READ_ADDRESS`
	`call.1d EELIB_ReadBlock_X16`
	`load xp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`                                  // delay slot`

14.13 EELIB_READBLOCK_Y

Read a block of data from EEPROM to YMEM

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_ReadBlock_Y`
Source File	sk3_eeprom.cfs
Description	Read a block of data from EEPROM to YMEM. Leaves EEPROM queued.
Inputs	x0 = Size of block (in words) b0.h = EEPROM Address yp0 = Destination memory address
Outputs	None
Register Modifications	x0, yp0, b1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Read a block of MEMORY_BUFFER_SIZE words from EEPROM at`
	`// EEPROM_READ_ADDRESS, to Y-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_READ_ADDRESS`
	`call.1d EELIB_ReadBlock_Y`
	`load yp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`                                  // delay slot`

14.14 EELIB_READNEXTBLOCK_P

Read the next block of data from a queued EEPROM to PMEM

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_ReadNextBlock_P`
Source File	sk3_eeprom.cfs
Description	Read the next block of data from a queued EEPROM to PMEM.
Inputs	x0 = Size of block (in words) xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM queued for read
Clock Cycles	

Example	`// Read the next block of MEMORY_BUFFER_SIZE words from EEPROM to`
	`// P-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`call.1d EELIB_ReadNextBlock_P`
	`load xp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`// delay slot`

14.15 EELIB_READNEXTBLOCK_P16

Read the next block of data from a queued EEPROM to PMEM in 16-bit words

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_ReadNextBlock_P16`
Source File	sk3_eeprom.cfs
Description	Read the next block of data from a queued EEPROM to PMEM in 16-bit words
Inputs	x0 = Size of block (in words) xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM queued for read
Clock Cycles	

Example	`// Read the next block of MEMORY_BUFFER_SIZE 16-bit words from`
	`// EEPROM to P-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`call.1d EELIB_ReadNextBlock_P16`
	`load xp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`// delay slot`

14.16 EELIB_READNEXTBLOCK_X

Read the next block of data from a queued EEPROM to XMEM

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_ReadNextBlock_X`
Source File	sk3_eeprom.cfs
Description	Read the next block of data from a queued EEPROM to XMEM
Inputs	x0 = Size of block (in words) xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM queued for read
Clock Cycles	

Example	`// Read the next block of MEMORY_BUFFER_SIZE words from EEPROM to`
	`// X-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`call.1d EELIB_ReadNextBlock_X`
	`load xp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`// delay slot`

14.17 EELIB_READNEXTBLOCK_X16

Read the next block of data from a queued EEPROM to XMEM in 16-bit words

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_ReadNextBlock_X16`
Source File	sk3_eeprom.cfs
Description	Read the next block of data from a queued EEPROM to XMEM in 16-bit words
Inputs	x0 = Size of block (in words) xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM queued for read
Clock Cycles	

Example	`// Read the next block of MEMORY_BUFFER_SIZE 16-bit words from`
	`// EEPROM to X-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`call.1d EELIB_ReadNextBlock_X16`
	`load xp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`// delay slot`

14.18 EELIB_READNEXTBLOCK_Y

Read the next block of data from a queued EEPROM to YMEM

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_ReadNextBlock_Y`
Source File	sk3_eeprom.cfs
Description	Read the next block of data from a queued EEPROM to YMEM
Inputs	x0 = Size of block (in words) yp0 = Destination memory address
Outputs	None
Register Modifications	x0, yp0, b1
Assumptions	EEPROM queued for read
Clock Cycles	

Example	`// Read the next block of MEMORY_BUFFER_SIZE words from EEPROM to`
	`// Y-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`call.1d EELIB_ReadNextBlock_Y`
	`load yp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`// delay slot`

14.19 EELIB_READNEXTWORD16

Read a 16-bit word from the queued EEPROM

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_ReadNextWord16`
Source File	sk3_eeprom.cfs
Description	Read a 16-bit word from the queued EEPROM. Update the CRC value with the 16-bit word.
Inputs	None
Outputs	b1.h = Next word
Register Modifications	b1
Assumptions	EEPROM queued for read
Clock Cycles	

Example	`// Read the next word from EEPROM into the memory buffer`
	`call.1d EELIB_ReadNextWord16`
	`nop  // Delay slot`
	`nop ||| store x[xp0++], b1.h  // Store the word read to the buffer`

14.20 EELIB_READNEXTWORD24

Read a 24-bit word from the queued EEPROM

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_ReadNextWord24`
Source File	sk3_eeprom.cfs
Description	Read a 24-bit word from the queued EEPROM. Update the CRC value with the 24-bit word.
Inputs	None
Outputs	b1.h = Next word
Register Modifications	b1
Assumptions	EEPROM queued for read
Clock Cycles	

Example	`// Read the next word from EEPROM into the memory buffer`
	`call.1d EELIB_ReadNextWord24`
	`nop  // Delay slot`
	`nop ||| store x[xp0++], b1.h  // Store the word read to the buffer`

14.21 EELIB_READNEXTWORD32

Read a 32-bit word from the queued EEPROM

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_ReadNextWord32`
Source File	sk3_eeprom.cfs
Description	Read a 32-bit word from the queued EEPROM. Update the CRC value with the 32-bit word.
Inputs	None
Outputs	b1 = Next word
Register Modifications	b1
Assumptions	EEPROM queued for read
Clock Cycles	

Example	`// Read the next word from EEPROM into the memory buffer`
	`call.1d EELIB_ReadNextWord32`
	`nop // Delay slot`
	`store.ex p[xp0++], b1 // Store the word read to the buffer`

14.22 EELIB_READNEXTWORD8

Read an 8-bit word from the queued EEPROM

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_ReadNextWord8`
Source File	sk3_eeprom.cfs
Description	Read an 8-bit word from the queued EEPROM. Update the CRC value with the 8-bit word.
Inputs	None
Outputs	b1.h = Next word
Register Modifications	b1
Assumptions	EEPROM queued for read
Clock Cycles	

Example	`// Read the next byte from EEPROM into the memory buffer`
	`call.1d EELIB_ReadNextWord8`
	`nop  // Delay slot`
	`nop ||| store x[xp0++], b1.h  // Store the byte read to the buffer`

14.23 EELIB_READSTATUSREGISTER

Read the EEPROM status register

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_ReadStatusRegister`
Source File	sk3_eeprom.cfs
Description	Read the EEPROM status register.
Inputs	None
Outputs	b1.h = EEPROM status byte
Register Modifications	b1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Check the AT_WEN bit from the EEPROM status register and branch`
	`// to function A if not set`
	`call.1d EELIB_ReadStatusRegister`
	`nop`
	`bittst b1, (AT_WEN_POS + 24) ||| nop`
	`goto.0d FunctionA, no_carry`

14.24 EELIB_READWORD16

Initialize the EEPROM and read a 16-bit word

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_ReadWord16`
Source File	sk3_eeprom.cfs
Description	Initialize the EEPROM and read a 16-bit word. Update the CRC value with the 16-bit word.
Inputs	b0.h = EEPROM Address
Outputs	b1.h = Read word
Register Modifications	b1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Read a word from EEPROM into the memory buffer`
	`load xp0, MEMORY_BUFFER_START`
	`call.1d EELIB_ReadWord16`
	`load b0.h, EEPROM_READ_ADDRESS  // Load the address to read from in`
	`                                  // the delay slot`
	`nop ||| store x[xp0++], b1.h    // Store the word read to the buffer`

14.25 EELIB_READWORD24

Initialize the EEPROM and read a 24-bit word

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_ReadWord24`
Source File	sk3_eeprom.cfs
Description	Initialize the EEPROM and read a 24-bit word. Update the CRC value with the 24-bit word.
Inputs	b0.h = EEPROM Address
Outputs	b1.h = Read word
Register Modifications	b1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Read a word from EEPROM into the memory buffer`
	`load xp0, MEMORY_BUFFER_START`
	`call.1d EELIB_ReadWord24`
	`load b0.h, EEPROM_READ_ADDRESS  // Load the address to read from in`
	`                                  // the delay slot`
	`nop ||| store x[xp0++], b1.h    // Store the word read to the buffer`

14.26 EELIB_READWORD32

Initialize the EEPROM and read a 32-bit word

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_ReadWord32`
Source File	sk3_eeprom.cfs
Description	Initialize the EEPROM and read a 32-bit word. Update the CRC value with the 32-bit word.
Inputs	b0.h = EEPROM Address
Outputs	b1 = Read word
Register Modifications	b1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Read a word from EEPROM into the memory buffer`
	`load xp0, MEMORY_BUFFER_START`
	`call.1d EELIB_ReadWord32`
	`load b0.h, EEPROM_READ_ADDRESS  // Load the address to read from in`
	`                                  // the delay slot`
	`store.ex p[xp0++], b1    // Store the word read to the buffer`

14.27 EELIB_READWORD8

Initialize the EEPROM and read an 8-bit word

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_ReadWord8`
Source File	sk3_eeprom.cfs
Description	Initialize the EEPROM and read an 8-bit word. Updates the CRC value with the 8-bit word.
Inputs	b0.h = EEPROM Address
Outputs	b1.h = Read word
Register Modifications	b1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Read a byte from EEPROM into the memory buffer`
	`load xp0, MEMORY_BUFFER_START`
	`call.1d EELIB_ReadWord8`
	`load b0.h, EEPROM_READ_ADDRESS  // Load the address to read from in`
	`                                  // the delay slot`
	`nop ||| store x[xp0++], b1.h    // Store the byte read to the buffer`

14.28 EELIB_SELECTSEGMENT

Selects EEPROM segment 0 or 1 (address bit 17)

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_SelectSegment`
Source File	sk3_eeprom2.cfs
Description	Selects EEPROM segment 0 or 1 (address bit 17)
Inputs	b0.h = Segment number [0|1]
Outputs	None
Register Modifications	b1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Write EEPROM_DATA_VALUE to EEPROM_WRITE_ADDRESS in the high`
	`// segment of the custom E2M2 EEPROM memory.`
	`call.1d EELIB_SelectSegment`
	`load b0.h, 1`
	
	`load b1.h, EEPROM_DATA_VALUE`
	`call.1d EELIB_WriteWord16`
	`load b0.h, EEPROM_WRITE_ADDRESS  // Load the address to write to`
	`// in the delay slot`

14.29 EELIB_VERIFYEEPROMCONNECTION

Verify that an EEPROM is connected to the SPI port

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_VerifyEEPROMConnection`
Source File	sk3_eeprom.cfs
Description	Verify that an EEPROM is connected to the SPI port
Inputs	None
Outputs	b1.h = Connected status (0 - Connected; 1 - Not Connected)
Register Modifications	b0, b1
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Check if an EEPROM is connected, branching to function A if one`
	`// is detected`
	`call.1d EELIB_VerifyEEPROMConnection`
	`nop`
	`cmp b1, 0 ||| nop`
	`goto.0d FunctionA, eq`

14.30 EELIB_WRITEBLOCK_CONST32

Write a block of data from one constant PMEM word to EEPROM

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_WriteBlock_Const32`
Source File	sk3_eeprom.cfs
Description	Write a block of data from one constant PMEM word to EEPROM. Use for filling EEPROM block with single word pattern.
Inputs	x0 = Size of buffer (in words) b0.h = EEPROM Address xp0 = Pointer to source word (32-bit, in PMEM)
Outputs	sr[SR_X_ZERO] = Timeout status (0 - No timeout, 1 - Timeout)
Register Modifications	b1, b0
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Write a block of EEPROM at EEPROM_WRITE_ADDRESS, from a 32-bit`
	`// constant stored in P-memory at MEMORY_CONSTANT`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_WRITE_ADDRESS`
	`call.1d EELIB_WriteBlock_Const32`
	`load xp0, MEMORY_CONSTANT  // Load the buffer pointer in the`
	`  // delay slot`

14.31 EELIB_WRITEBLOCK_P

Write a block of data from PMEM to EEPROM

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_WriteBlock_P`
Source File	sk3_eeprom.cfs
Description	Write a block of data from PMEM to EEPROM
Inputs	x0 = Size of buffer (in words) b0.h = EEPROM Address xp0 = Source buffer address
Outputs	sr[SR_X_ZERO] = Timeout status (0 - No timeout, 1 - Timeout)
Register Modifications	b1, b0, xp0
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Write a block of EEPROM at EEPROM_WRITE_ADDRESS, from P-memory at`
	`// MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_WRITE_ADDRESS`
	`call.1d EELIB_WriteBlock_P`
	`load xp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`                                  // delay slot`

14.32 EELIB_WRITEBLOCK_X

Write a block of data from XMEM to EEPROM

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_WriteBlock_X`
Source File	sk3_eeprom.cfs
Description	Write a block of data from XMEM to EEPROM
Inputs	x0 = Size of buffer (in words) b0.h = EEPROM Address xp0 = Source buffer address
Outputs	sr[SR_X_ZERO] = Timeout status (0 - No timeout, 1 - Timeout)
Register Modifications	b1, b0, xp0
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Write a block of EEPROM at EEPROM_WRITE_ADDRESS, from X-memory at`
	`// MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_WRITE_ADDRESS`
	`call.1d EELIB_WriteBlock_X`
	`load xp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`// delay slot`

14.33 EELIB_WRITEBLOCK_Y

Write a block of data from YMEM to EEPROM

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_WriteBlock_Y`
Source File	sk3_eeprom.cfs
Description	Write a block of data from YMEM to EEPROM
Inputs	x0 = Size of buffer (in words) b0.h = EEPROM Address yp0 = Source buffer address
Outputs	sr[SR_X_ZERO] = Timeout status (0 - No timeout, 1 - Timeout)
Register Modifications	b1, b0, yp0
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Write a block of EEPROM at EEPROM_WRITE_ADDRESS, from Y-memory at`
	`// MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_WRITE_ADDRESS`
	`call.1d EELIB_WriteBlock_Y`
	`load yp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`// delay slot`

14.34 EELIB_WRITEDISABLE

Write disable the EEPROM

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_WriteDisable`
Source File	sk3_eeprom.cfs
Description	Write disable the EEPROM.
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Write disable the EEPROM`
	`call.1d EELIB_WriteDisable`
	`nop`

14.35 EELIB_WRITEENABLE

Write enable the EEPROM

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_WriteEnable`
Source File	sk3_eeprom.cfs
Description	Write enable the EEPROM
Inputs	None
Outputs	None
Register Modifications	None
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Write enable the EEPROM`
	`call.1d EELIB_WriteEnable`
	`nop`

14.36 EELIB_WRITENEXTWORD16

Write a 16-bit word to the EEPROM buffer

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_WriteNextWord16`
Source File	sk3_eeprom.cfs
Description	Write a 16-bit word to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 16-bit word.
Inputs	b1.h = Word to write
Outputs	None
Register Modifications	None
Assumptions	EEPROM queued for write
Clock Cycles	

Example	`// Continue writing to EEPROM`
	`call.1d EELIB_WriteNextWord16`
	`nop ||| load b1.h, x[xp0++]  // Load the next word to write in the`
	`// delay slot`

14.37 EELIB_WRITENEXTWORD24

Write a 24-bit word to the EEPROM buffer

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_WriteNextWord24`
Source File	sk3_eeprom.cfs
Description	Write a 24-bit word to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 24-bit word.
Inputs	b1.h = Word to write
Outputs	None
Register Modifications	None
Assumptions	EEPROM queued for write
Clock Cycles	

Example	`// Continue writing to EEPROM`
	`call.1d EELIB_WriteNextWord24`
	`nop ||| load b1.h, x[xp0++]  // Load the next word to write in the`
	`// delay slot`

14.38 EELIB_WRITENEXTWORD32

Write a 32-bit word to the EEPROM buffer

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_WriteNextWord32`
Source File	sk3_eeprom.cfs
Description	Write a 32-bit word to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 32-bit word.
Inputs	b1 = Word to write (Bits 23:0 from b1.h. Bits 31:24 from b1.e)
Outputs	None
Register Modifications	b1.l
Assumptions	EEPROM queued for write
Clock Cycles	

Example	`// Continue writing to EEPROM`
	`call.1d EELIB_WriteNextWord32`
	`load.ex b1, p[xp0++]  // Load the next word to write in the delay slot`

14.39 EELIB_WRITENEXTWORD8

Write a 8-bit word to the EEPROM buffer

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_WriteNextWord8`
Source File	sk3_eeprom.cfs
Description	Write a 8-bit word to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 8-bit word.
Inputs	b1.h = Word to write
Outputs	None
Register Modifications	None
Assumptions	EEPROM queued for write
Clock Cycles	

Example	`// Continue writing to EEPROM`
	`call.1d EELIB_WriteNextWord8`
	`nop ||| load b1.h, x[xp0++]  // Load the next byte to write in the`
	`// delay slot`

14.40 EELIB_WRITESTATUSREGISTER

Write the EEPROM status register

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_WriteStatusRegister`
Source File	sk3_eeprom.cfs
Description	Write the EEPROM status register.
Inputs	b0.h = New EEPROM status byte
Outputs	sr[SR_X_ZERO] = Timeout status (0 - No timeout, 1 - Timeout)
Register Modifications	b1
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Write the EEPROM status register, setting the block write protect`
	`// bits`
	`call.1d EELIB_WriteStatusRegister`
	`load b0.h, (3 << AT_BP0_POS)    // Load new value in the delay slot`

14.41 EELIB_WRITEWORD16

Initialize the EEPROM for writing and send a 16-bit word to be written to the EEPROM buffer

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_WriteWord16`
Source File	sk3_eeprom.cfs
Description	Initialize the EEPROM for writing and send a 16-bit word to be written to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 16-bit word.
Inputs	b0.h = EEPROM Address b1.h = Word to write
Outputs	None
Register Modifications	b1.l
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Write EEPROM_DATA_VALUE to EEPROM_WRITE_ADDRESS`
	`load b1.h, EEPROM_DATA_VALUE`
	`call.1d EELIB_WriteWord16`
	`load b0.h, EEPROM_WRITE_ADDRESS  // Load the address to write to`
	`// in the delay slot`

14.42 EELIB_WRITEWORD24

Initialize the EEPROM for writing and send a 24-bit word to be written to the EEPROM buffer

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_WriteWord24`
Source File	sk3_eeprom.cfs
Description	Initialize the EEPROM for writing and send a 24-bit word to be written to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 24-bit word.
Inputs	b0.h = EEPROM Address b1.h = Word to write
Outputs	None
Register Modifications	b1.l
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Write EEPROM_DATA_VALUE to EEPROM_WRITE_ADDRESS`
	`load b1.h, EEPROM_DATA_VALUE`
	`call.1d EELIB_WriteWord24`
	`load b0.h, EEPROM_WRITE_ADDRESS  // Load the address to write to`
	`// in the delay slot`

14.43 EELIB_WRITEWORD32

Initialize the EEPROM for writing and send a 32-bit word to be written to the EEPROM buffer

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_WriteWord32`
Source File	sk3_eeprom.cfs
Description	Initialize the EEPROM for writing and send a 32-bit word to be written to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 32-bit word.
Inputs	b0.h = EEPROM Address b1 = Word to write (Bits 23:0 from b1.h. Bits 31:24 from b1.e)
Outputs	None
Register Modifications	b1.l
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Write EEPROM_DATA_VALUE to EEPROM_WRITE_ADDRESS`
	`load b1, EEPROM_DATA_VALUE`
	`call.1d EELIB_WriteWord32`
	`load b0.h, EEPROM_WRITE_ADDRESS  // Load the address to write to`
	`// in the delay slot`

14.44 EELIB_WRITEWORD8

Initialize the EEPROM for writing and send a 8-bit word to be written to the EEPROM buffer

Type	Function
Syntax	`#include <sk3_eeprom.cfi>` `call EELIB_WriteWord8`
Source File	sk3_eeprom.cfs
Description	Initialize the EEPROM for writing and send a 8-bit word to be written to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 8-bit word.
Inputs	b0.h = EEPROM Address b1.h = Word to write
Outputs	None
Register Modifications	b1.l
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Write EEPROM_DATA_VALUE to EEPROM_WRITE_ADDRESS`
	`load b1.h, EEPROM_DATA_VALUE`
	`call.1d EELIB_WriteWord8`
	`load b0.h, EEPROM_WRITE_ADDRESS  // Load the address to write to`
	`// in the delay slot`

15 Dual EEPROM Library Reference

This reference chapter presents a detailed description of all the macros and functions in the dual EEPROM library (a library that supports the use of two EEPROMs), including calling parameters, modified registers and returned values.

15.1 EELIB_CLOSE

Close the EEPROM library

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_Close`
Source File	sk3_dual_eeprom.cfs
Description	Close the EEPROM library.
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	None
Clock Cycles	

Example	`// Close the EEPROM Library`
	`call.1d EELIB_Close`
	`nop`

15.2 EELIB_COMMIT

Commit EEPROM buffer to EEPROM non-volatile memory

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_Commit`
Source File	sk3_dual_eeprom.cfs
Description	Commit EEPROM buffer to EEPROM non-volatile memory
Inputs	None
Outputs	sr[SR_X_ZERO] = Timeout status (0 - No timeout, 1 - Timeout)
Register Modifications	b1, y1
Assumptions	None
Clock Cycles	

Example	`// Commit the current writes to the EEPROM library. Branch to`
	`// function A if the writes failed.`
	`call.1d EELIB_Commit`
	`nop`
	`goto.0d FunctionA, zero`

15.3 EELIB_INITIALIZE

Initialize the EEPROM library

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_Initialize`
Source File	sk3_dual_eeprom.cfs
Description	Initialize the EEPROM library. Sets the default commit timeout and write disables the library. Configures the selected EEPROM. If the data EEPROM is selected, configures the GPIO chip select pin as an output.
Inputs	b1 = SPI speed (Use SPI_CLK_PRESCALE_* define) : y1 - Selected EEPROM Use EE_SELECT_CODE_EEPROM or EE_SELECT_DATA_EEPROM
Outputs	None
Register Modifications	b1, x1, y1
Assumptions	None
Clock Cycles	

Example	`// Initialize the EEPROM Library`
	`call.1d EELIB_Initialize`
	`load b1, SPI_CLK_PRESCALE_2  // Load SPI prescaler in the delay slot`

15.4 EELIB_INITWRITEWORD

Initialize the EEPROM for writing at the specified address

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_InitWriteWord`
Source File	sk3_dual_eeprom.cfs
Description	Initialize the EEPROM for writing at the specified address.
Inputs	b0.h = EEPROM Address
Outputs	None
Register Modifications	b1.l
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Setup to write a block of data at EEPROM_WRITE_ADDRESS`
	`load xp0, MEMORY_BUFFER_START`
	`call.1d EELIB_InitWriteWord`
	`load b0.h, EEPROM_WRITE_ADDRESS  // Load the address to write to`
	`// in the delay slot`
	
	`// Write the 16-bit data from memory buffer to EEPROM`
	`Write_Loop:`
	`call.1d EELIB_WriteNextWord16`
	`nop ||| load b1.h, x[xp0++]  // Load the word to write in the`
	`// delay slot`
	`nop ||| move b1, xp0`
	`cmp b1, MEMORY_BUFFER_SIZE ||| nop`
	`goto.0d Write_Loop, ne`

15.5 EELIB_LIBWRITEDISABLE

Write disable the EEPROM library

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_LibWriteDisable`
Source File	sk3_dual_eeprom.cfs
Description	Write disable the EEPROM library.
Inputs	None
Outputs	None
Register Modifications	b1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Write disable the EEPROM Library`
	`call.1d EELIB_LibWriteDisable`
	`nop`

15.6 EELIB_LIBWRITEENABLE

Write enable the EEPROM library

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_LibWriteEnable`
Source File	sk3_dual_eeprom.cfs
Description	Write enable the EEPROM library.
Inputs	b1 = AT_WREN opcode
Outputs	None
Register Modifications	None
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Write enable the EEPROM Library`
	`call.1d EELIB_LibWriteEnable`
	`load b1, AT_WREN  // Load write enable code in the delay slot`

15.7 EELIB_POWERDOWN

Attempt to place the EEPROM in power down mode

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_PowerDown`
Source File	sk3_dual_eeprom2.cfs
Description	Attempt to place the EEPROM in power down mode
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Power down the EEPROM to reduce the current consumed by this`
	`// device while it is not in use.`
	`call.1d EELIB_PowerDown`
	`nop`

15.8 EELIB_POWERUP

Place an EEPROM, that was in power down mode, into power up mode

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_PowerUp`
Source File	sk3_dual_eeprom2.cfs
Description	Place an EEPROM, that was in power down mode, into power up mode
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Ensure the EEPROM is powered up before trying to read from it`
	`call.1d EELIB_PowerUp`
	`nop`

15.9 EELIB_READBLOCK_P

Read a block of data from EEPROM to PMEM

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_ReadBlock_P`
Source File	sk3_dual_eeprom.cfs
Description	Read a block of data from EEPROM to PMEM. Leaves EEPROM queued.
Inputs	x0 = Size of block (in words) b0.h = EEPROM Address xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Read a block of MEMORY_BUFFER_SIZE words from EEPROM at`
	`// EEPROM_READ_ADDRESS, to P-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_READ_ADDRESS`
	`call.1d EELIB_ReadBlock_P`
	`load xp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`                                  // delay slot`

15.10 EELIB_READBLOCK_P16

Read a block of data from EEPROM to PMEM in 16-bit words

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_ReadBlock_P16`
Source File	sk3_dual_eeprom.cfs
Description	Read a block of data from EEPROM to PMEM in 16-bit words. Leaves EEPROM queued.
Inputs	x0 = Size of block (in words) b0.h = EEPROM Address xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Read a block of MEMORY_BUFFER_SIZE 16-bit words from EEPROM at`
	`// EEPROM_READ_ADDRESS, to P-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_READ_ADDRESS`
	`call.1d EELIB_ReadBlock_P16`
	`load xp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`                                  // delay slot`

15.11 EELIB_READBLOCK_X

Read a block of data from EEPROM to XMEM

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_ReadBlock_X`
Source File	sk3_dual_eeprom.cfs
Description	Read a block of data from EEPROM to XMEM. Leaves EEPROM queued.
Inputs	x0 = Size of block (in words) b0.h = EEPROM Address xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Read a block of MEMORY_BUFFER_SIZE words from EEPROM at`
	`// EEPROM_READ_ADDRESS, to X-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_READ_ADDRESS`
	`call.1d EELIB_ReadBlock_X`
	`load xp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`                                  // delay slot`

15.12 EELIB_READBLOCK_X16

Read a block of data from EEPROM to XMEM in 16-bit words

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_ReadBlock_X16`
Source File	sk3_dual_eeprom.cfs
Description	Read a block of data from EEPROM to XMEM in 16-bit words. Leaves EEPROM queued.
Inputs	x0 = Size of block (in words) b0.h = EEPROM Address xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Read a block of MEMORY_BUFFER_SIZE 16-bit words from EEPROM at`
	`// EEPROM_READ_ADDRESS, to X-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_READ_ADDRESS`
	`call.1d EELIB_ReadBlock_X16`
	`load xp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`                                  // delay slot`

15.13 EELIB_READBLOCK_Y

Read a block of data from EEPROM to YMEM

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_ReadBlock_Y`
Source File	sk3_dual_eeprom.cfs
Description	Read a block of data from EEPROM to YMEM. Leaves EEPROM queued.
Inputs	x0 = Size of block (in words) b0.h = EEPROM Address yp0 = Destination memory address
Outputs	None
Register Modifications	x0, yp0, b1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Read a block of MEMORY_BUFFER_SIZE words from EEPROM at`
	`// EEPROM_READ_ADDRESS, to Y-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_READ_ADDRESS`
	`call.1d EELIB_ReadBlock_Y`
	`load yp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`                                  // delay slot`

15.14 EELIB_READNEXTBLOCK_P

Read the next block of data from a queued EEPROM to PMEM

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_ReadNextBlock_P`
Source File	sk3_dual_eeprom.cfs
Description	Read the next block of data from a queued EEPROM to PMEM.
Inputs	x0 = Size of block (in words) xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM queued for read
Clock Cycles	

Example	`// Read the next block of MEMORY_BUFFER_SIZE words from EEPROM to`
	`// P-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`call.1d EELIB_ReadNextBlock_P`
	`load xp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`                                  // delay slot`

15.15 EELIB_READNEXTBLOCK_P16

Read the next block of data from a queued EEPROM to PMEM in 16-bit words

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_ReadNextBlock_P16`
Source File	sk3_dual_eeprom.cfs
Description	Read the next block of data from a queued EEPROM to PMEM in 16-bit words
Inputs	x0 = Size of block (in words) xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM queued for read
Clock Cycles	

Example	`// Read the next block of MEMORY_BUFFER_SIZE 16-bit words from`
	`// EEPROM to P-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`call.1d EELIB_ReadNextBlock_P16`
	`load xp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`                                  // delay slot`

15.16 EELIB_READNEXTBLOCK_X

Read the next block of data from a queued EEPROM to XMEM

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_ReadNextBlock_X`
Source File	sk3_dual_eeprom.cfs
Description	Read the next block of data from a queued EEPROM to XMEM
Inputs	x0 = Size of block (in words) xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM queued for read
Clock Cycles	

Example	`// Read the next block of MEMORY_BUFFER_SIZE words from EEPROM to`
	`// X-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`call.1d EELIB_ReadNextBlock_X`
	`load xp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`                                  // delay slot`

15.17 EELIB_READNEXTBLOCK_X16

Read the next block of data from a queued EEPROM to XMEM in 16-bit words

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_ReadNextBlock_X16`
Source File	sk3_dual_eeprom.cfs
Description	Read the next block of data from a queued EEPROM to XMEM in 16-bit words
Inputs	x0 = Size of block (in words) xp0 = Destination memory address
Outputs	None
Register Modifications	x0, xp0, b1
Assumptions	EEPROM queued for read
Clock Cycles	

Example	`// Read the next block of MEMORY_BUFFER_SIZE 16-bit words from`
	`// EEPROM to X-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`call.1d EELIB_ReadNextBlock_X16`
	`load xp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`// delay slot`

15.18 EELIB_READNEXTBLOCK_Y

Read the next block of data from a queued EEPROM to YMEM

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_ReadNextBlock_Y`
Source File	sk3_dual_eeprom.cfs
Description	Read the next block of data from a queued EEPROM to YMEM
Inputs	x0 = Size of block (in words) yp0 = Destination memory address
Outputs	None
Register Modifications	x0, yp0, b1
Assumptions	EEPROM queued for read
Clock Cycles	

Example	`// Read the next block of MEMORY_BUFFER_SIZE words from EEPROM to`
	`// Y-memory at MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`call.1d EELIB_ReadNextBlock_Y`
	`load yp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`                                  // delay slot`

15.19 EELIB_READNEXTWORD16

Read a 16-bit word from the queued EEPROM

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_ReadNextWord16`
Source File	sk3_dual_eeprom.cfs
Description	Read a 16-bit word from the queued EEPROM. Update the CRC value with the 16-bit word.
Inputs	None
Outputs	b1.h = Next word
Register Modifications	b1, y1
Assumptions	EEPROM queued for read
Clock Cycles	

Example	`// Read the next word from EEPROM into the memory buffer`
	`call.1d EELIB_ReadNextWord16`
	`nop  // Delay slot`
	`nop ||| store x[xp0++], b1.h  // Store the word read to the buffer`

15.20 EELIB_READNEXTWORD24

Read a 24-bit word from the queued EEPROM

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_ReadNextWord24`
Source File	sk3_dual_eeprom.cfs
Description	Read a 24-bit word from the queued EEPROM. Update the CRC value with the 24-bit word.
Inputs	None
Outputs	b1.h = Next word
Register Modifications	b1
Assumptions	EEPROM queued for read
Clock Cycles	

Example	`// Read the next word from EEPROM into the memory buffer`
	`call.1d EELIB_ReadNextWord24`
	`nop  // Delay slot`
	`nop ||| store x[xp0++], b1.h  // Store the word read to the buffer`

15.21 EELIB_READNEXTWORD32

Read a 32-bit word from the queued EEPROM

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_ReadNextWord32`
Source File	sk3_dual_eeprom.cfs
Description	Read a 32-bit word from the queued EEPROM. Update the CRC value with the 32-bit word.
Inputs	None
Outputs	b1.h = Next word
Register Modifications	b1, y1
Assumptions	EEPROM queued for read
Clock Cycles	

Example	`// Read the next word from EEPROM into the memory buffer`
	`call.1d EELIB_ReadNextWord32`
	`nop // Delay slot`
	`store.ex p[xp0++], b1 // Store the word read to the buffer`

15.22 EELIB_READNEXTWORD8

Read an 8-bit word from the queued EEPROM

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_ReadNextWord8`
Source File	sk3_dual_eeprom.cfs
Description	Read an 8-bit word from the queued EEPROM. Update the CRC value with the 8-bit word.
Inputs	None
Outputs	b1.h = Next word
Register Modifications	b1, y1
Assumptions	EEPROM queued for read
Clock Cycles	

Example	`// Read the next byte from EEPROM into the memory buffer`
	`call.1d EELIB_ReadNextWord8`
	`nop  // Delay slot`
	`nop ||| store x[xp0++], b1.h  // Store the byte read to the buffer`

15.23 EELIB_READSTATUSREGISTER

Read the EEPROM status register

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_ReadStatusRegister`
Source File	sk3_dual_eeprom.cfs
Description	Read the EEPROM status register.
Inputs	None
Outputs	b1.h = EEPROM status byte
Register Modifications	b1, y1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Check the AT_WEN bit from the EEPROM status register and branch`
	`// to function A if not set`
	`call.1d EELIB_ReadStatusRegister`
	`nop`
	`bittst b1, (AT_WEN_POS + 24) ||| nop`
	`goto.0d FunctionA, no_carry`

15.24 EELIB_READWORD16

Initialize the EEPROM and read a 16-bit word

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_ReadWord16`
Source File	sk3_dual_eeprom.cfs
Description	Initialize the EEPROM and read a 16-bit word. Update the CRC value with the 16-bit word.
Inputs	b0.h = EEPROM Address
Outputs	b1.h = Read word
Register Modifications	b1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Read a word from EEPROM into the memory buffer`
	`load xp0, MEMORY_BUFFER_START`
	`call.1d EELIB_ReadWord16`
	`load b0.h, EEPROM_READ_ADDRESS  // Load the address to read from in`
	`                                  // the delay slot`
	`nop ||| store x[xp0++], b1.h    // Store the word read to the buffer`

15.25 EELIB_READWORD24

Initialize the EEPROM and read a 24-bit word

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_ReadWord24`
Source File	sk3_dual_eeprom.cfs
Description	Initialize the EEPROM and read a 24-bit word. Update the CRC value with the 24-bit word.
Inputs	b0.h = EEPROM Address
Outputs	b1.h = Read word
Register Modifications	b1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Read a word from EEPROM into the memory buffer`
	`load xp0, MEMORY_BUFFER_START`
	`call.1d EELIB_ReadWord24`
	`load b0.h, EEPROM_READ_ADDRESS  // Load the address to read from in`
	`                                  // the delay slot`
	`nop ||| store x[xp0++], b1.h    // Store the word read to the buffer`

15.26 EELIB_READWORD32

Initialize the EEPROM and read a 32-bit word

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_ReadWord32`
Source File	sk3_dual_eeprom.cfs
Description	Initialize the EEPROM and read a 32-bit word. Update the CRC value with the 32-bit word.
Inputs	b0.h = EEPROM Address
Outputs	b1.h = Read word
Register Modifications	b1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Read a word from EEPROM into the memory buffer`
	`load xp0, MEMORY_BUFFER_START`
	`call.1d EELIB_ReadWord32`
	`load b0.h, EEPROM_READ_ADDRESS  // Load the address to read from in`
	`                                  // the delay slot`
	`store.ex p[xp0++], b1    // Store the word read to the buffer`

15.27 EELIB_READWORD8

Initialize the EEPROM and read an 8-bit word

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_ReadWord8`
Source File	sk3_dual_eeprom.cfs
Description	Initialize the EEPROM and read an 8-bit word. Updates the CRC value with the 8-bit word.
Inputs	b0.h = EEPROM Address
Outputs	b1.h = Read word
Register Modifications	b1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Read a byte from EEPROM into the memory buffer`
	`load xp0, MEMORY_BUFFER_START`
	`call.1d EELIB_ReadWord8`
	`load b0.h, EEPROM_READ_ADDRESS  // Load the address to read from in`
	`                                  // the delay slot`
	`nop ||| store x[xp0++], b1.h    // Store the byte read to the buffer`

15.28 EELIB_SELECTSEGMENT

Selects EEPROM segment 0 or 1 (address bit 17)

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_SelectSegment`
Source File	sk3_dual_eeprom2.cfs
Description	Selects EEPROM segment 0 or 1 (address bit 17)
Inputs	b0.h = Segment number [0|1]
Outputs	None
Register Modifications	b1, y1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Write EEPROM_DATA_VALUE to EEPROM_WRITE_ADDRESS in the high`
	`// segment of the custom E2M2 EEPROM memory.`
	`call.1d EELIB_SelectSegment`
	`load b0.h, 1`
	
	`load b1.h, EEPROM_DATA_VALUE`
	`call.1d EELIB_WriteWord16`
	`load b0.h, EEPROM_WRITE_ADDRESS  // Load the address to write to`
	`// in the delay slot`

15.29 EELIB_VERIFYEEPROMCONNECTION

Verify that an EEPROM is connected to the SPI port

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_VerifyEEPROMConnection`
Source File	sk3_dual_eeprom.cfs
Description	Verify that an EEPROM is connected to the SPI port
Inputs	None
Outputs	b1.h = Connected status (0 - Connected; 1 - Not Connected)
Register Modifications	b0, b1
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Check if an EEPROM is connected, branching to function A if one`
	`// is detected`
	`call.1d EELIB_VerifyEEPROMConnection`
	`nop`
	`cmp b1, 0 ||| nop`
	`goto.0d FunctionA, eq`

15.30 EELIB_WRITEBLOCK_CONST32

Write a block of data from one constant PMEM word to EEPROM

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_WriteBlock_Const32`
Source File	sk3_dual_eeprom.cfs
Description	Write a block of data from one constant PMEM word to EEPROM. Use for filling EEPROM block with single word pattern.
Inputs	x0 = Size of buffer (in words) b0.h = EEPROM Address xp0 = Pointer to source word (32-bit, in PMEM)
Outputs	sr[SR_X_ZERO] = Timeout status (0 - No timeout, 1 - Timeout)
Register Modifications	b1, b0
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Write a block of EEPROM at EEPROM_WRITE_ADDRESS, from a 32-bit`
	`// constant stored in P-memory at MEMORY_CONSTANT`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_WRITE_ADDRESS`
	`call.1d EELIB_WriteBlock_Const32`
	`load xp0, MEMORY_CONSTANT  // Load the buffer pointer in the`
	`  // delay slot`

15.31 EELIB_WRITEBLOCK_P

Write a block of data from PMEM to EEPROM

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_WriteBlock_P`
Source File	sk3_dual_eeprom.cfs
Description	Write a block of data from PMEM to EEPROM
Inputs	x0 = Size of buffer (in words) b0.h = EEPROM Address xp0 = Source buffer address
Outputs	sr[SR_X_ZERO] = Timeout status (0 - No timeout, 1 - Timeout)
Register Modifications	b1, b0, xp0
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Write a block of EEPROM at EEPROM_WRITE_ADDRESS, from P-memory at`
	`// MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_WRITE_ADDRESS`
	`call.1d EELIB_WriteBlock_P`
	`load xp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`// delay slot`

15.32 EELIB_WRITEBLOCK_X

Write a block of data from XMEM to EEPROM

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_WriteBlock_X`
Source File	sk3_dual_eeprom.cfs
Description	Write a block of data from XMEM to EEPROM
Inputs	x0 = Size of buffer (in words) b0.h = EEPROM Address xp0 = Source buffer address
Outputs	sr[SR_X_ZERO] = Timeout status (0 - No timeout, 1 - Timeout)
Register Modifications	b1, b0, xp0
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Write a block of EEPROM at EEPROM_WRITE_ADDRESS, from X-memory at`
	`// MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_WRITE_ADDRESS`
	`call.1d EELIB_WriteBlock_X`
	`load xp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`// delay slot`

15.33 EELIB_WRITEBLOCK_Y

Write a block of data from YMEM to EEPROM

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_WriteBlock_Y`
Source File	sk3_dual_eeprom.cfs
Description	Write a block of data from YMEM to EEPROM
Inputs	x0 = Size of buffer (in words) b0.h = EEPROM Address yp0 = Source buffer address
Outputs	sr[SR_X_ZERO] = Timeout status (0 - No timeout, 1 - Timeout)
Register Modifications	b1, b0, yp0
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Write a block of EEPROM at EEPROM_WRITE_ADDRESS, from Y-memory at`
	`// MEMORY_BUFFER_START`
	`load x0, MEMORY_BUFFER_SIZE`
	`load b0.h, EEPROM_WRITE_ADDRESS`
	`call.1d EELIB_WriteBlock_Y`
	`load yp0, MEMORY_BUFFER_START  // Load the buffer pointer in the`
	`// delay slot`

15.34 EELIB_WRITEDISABLE

Write disable the EEPROM

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_WriteDisable`
Source File	sk3_dual_eeprom.cfs
Description	Write disable the EEPROM.
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	EEPROM not busy
Clock Cycles	

Example	`// Write disable the EEPROM`
	`call.1d EELIB_WriteDisable`
	`nop`

15.35 EELIB_WRITEENABLE

Write enable the EEPROM

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_WriteEnable`
Source File	sk3_dual_eeprom.cfs
Description	Write enable the EEPROM
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Write enable the EEPROM`
	`call.1d EELIB_WriteEnable`
	`nop`

15.36 EELIB_WRITENEXTWORD16

Write a 16-bit word to the EEPROM buffer

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_WriteNextWord16`
Source File	sk3_dual_eeprom.cfs
Description	Write a 16-bit word to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 16-bit word.
Inputs	b1.h = Word to write
Outputs	None
Register Modifications	y1
Assumptions	EEPROM queued for write
Clock Cycles	

Example	`// Continue writing to EEPROM`
	`call.1d EELIB_WriteNextWord16`
	`nop ||| load b1.h, x[xp0++]  // Load the next word to write in the`
	`// delay slot`

15.37 EELIB_WRITENEXTWORD24

Write a 24-bit word to the EEPROM buffer

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_WriteNextWord24`
Source File	sk3_dual_eeprom.cfs
Description	Write a 24-bit word to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 24-bit word.
Inputs	b1.h = Word to write
Outputs	None
Register Modifications	y1
Assumptions	EEPROM queued for write
Clock Cycles	

Example	`// Continue writing to EEPROM`
	`call.1d EELIB_WriteNextWord24`
	`nop ||| load b1.h, x[xp0++]  // Load the next word to write in the`
	`// delay slot`

15.38 EELIB_WRITENEXTWORD32

Write a 32-bit word to the EEPROM buffer

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_WriteNextWord32`
Source File	sk3_dual_eeprom.cfs
Description	Write a 32-bit word to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 32-bit word.
Inputs	b1 = Word to write (Bits 23:0 from b1.h. Bits 31:24 from b1.e)
Outputs	None
Register Modifications	y1, b1.l
Assumptions	EEPROM queued for write
Clock Cycles	

Example	`// Continue writing to EEPROM`
	`call.1d EELIB_WriteNextWord32`
	`load.ex b1, p[xp0++]  // Load the next word to write in the delay slot`

15.39 EELIB_WRITENEXTWORD8

Write a 8-bit word to the EEPROM buffer

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_WriteNextWord8`
Source File	sk3_dual_eeprom.cfs
Description	Write a 8-bit word to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 8-bit word.
Inputs	b1.h = Word to write
Outputs	None
Register Modifications	y1
Assumptions	EEPROM queued for write
Clock Cycles	

Example	`// Continue writing to EEPROM`
	`call.1d EELIB_WriteNextWord8`
	`nop ||| load b1.h, x[xp0++]  // Load the next byte to write in the`
	`// delay slot`

15.40 EELIB_WRITESTATUSREGISTER

Write the EEPROM status register

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_WriteStatusRegister`
Source File	sk3_dual_eeprom.cfs
Description	Write the EEPROM status register.
Inputs	b0.h = New EEPROM status byte
Outputs	sr[SR_X_ZERO] = Timeout status (0 - No timeout, 1 - Timeout)
Register Modifications	b1, y1
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Write the EEPROM status register, setting the block write protect`
	`// bits`
	`call.1d EELIB_WriteStatusRegister`
	`load b0.h, (3 << AT_BP0_POS)    // Load new value in the delay slot`

15.41 EELIB_WRITEWORD16

Initialize the EEPROM for writing and send a 16-bit word to be written to the EEPROM buffer

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_WriteWord16`
Source File	sk3_dual_eeprom.cfs
Description	Initialize the EEPROM for writing and send a 16-bit word to be written to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 16-bit word.
Inputs	b0.h = EEPROM Address b1.h = Word to write
Outputs	None
Register Modifications	b1.l
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Write EEPROM_DATA_VALUE to EEPROM_WRITE_ADDRESS`
	`load b1.h, EEPROM_DATA_VALUE`
	`call.1d EELIB_WriteWord16`
	`load b0.h, EEPROM_WRITE_ADDRESS  // Load the address to write to`
	`// in the delay slot`

15.42 EELIB_WRITEWORD24

Initialize the EEPROM for writing and send a 24-bit word to be written to the EEPROM buffer

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_WriteWord24`
Source File	sk3_dual_eeprom.cfs
Description	Initialize the EEPROM for writing and send a 24-bit word to be written to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 24-bit word.
Inputs	b0.h = EEPROM Address b1.h = Word to write
Outputs	None
Register Modifications	b1.l
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Write EEPROM_DATA_VALUE to EEPROM_WRITE_ADDRESS`
	`load b1.h, EEPROM_DATA_VALUE`
	`call.1d EELIB_WriteWord24`
	`load b0.h, EEPROM_WRITE_ADDRESS  // Load the address to write to`
	`// in the delay slot`

15.43 EELIB_WRITEWORD32

Initialize the EEPROM for writing and send a 32-bit word to be written to the EEPROM buffer

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_WriteWord32`
Source File	sk3_dual_eeprom.cfs
Description	Initialize the EEPROM for writing and send a 32-bit word to be written to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 32-bit word.
Inputs	b0.h = EEPROM Address b1 = Word to write (Bits 23:0 from b1.h. Bits 31:24 from b1.e)
Outputs	None
Register Modifications	b1.l
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Write EEPROM_DATA_VALUE to EEPROM_WRITE_ADDRESS`
	`load b1, EEPROM_DATA_VALUE`
	`call.1d EELIB_WriteWord32`
	`load b0.h, EEPROM_WRITE_ADDRESS  // Load the address to write to`
	`// in the delay slot`

15.44 EELIB_WRITEWORD8

Initialize the EEPROM for writing and send a 8-bit word to be written to the EEPROM buffer

Type	Function
Syntax	`#include <sk3_dual_eeprom.cfi>` `call EELIB_WriteWord8`
Source File	sk3_dual_eeprom.cfs
Description	Initialize the EEPROM for writing and send a 8-bit word to be written to the EEPROM buffer. Use EELIB_Commit to write the EEPROM buffer to the EEPROM non-volatile memory. Updates the CRC value with the 8-bit word.
Inputs	b0.h = EEPROM Address b1.h = Word to write
Outputs	None
Register Modifications	b1.l
Assumptions	EEPROM not busy EELIB_LibWriteEnable was called with the AT_WREN opcode
Clock Cycles	

Example	`// Write EEPROM_DATA_VALUE to EEPROM_WRITE_ADDRESS`
	`load b1.h, EEPROM_DATA_VALUE`
	`call.1d EELIB_WriteWord8`
	`load b0.h, EEPROM_WRITE_ADDRESS  // Load the address to write to`
	`// in the delay slot`

15.45 SYS_DUAL_EEPROM_CS_DATA_EEPROM

Set the GPIO chip select to low if SYSVAR_EE_SELECT == EE_SELECT_DATA_EEPROM (== 1)

Type	Macro
Syntax	`#include <sk3_dual_eeprom.cfi>` `Sys_Dual_EEPROM_CS_Data_EEPROM`
Source File	sk3_dual_eeprom.cfi
Description	Set the GPIO chip select to low if SYSVAR_EE_SELECT == EE_SELECT_DATA_EEPROM (== 1)
Inputs	None
Outputs	None
Register Modifications	y0, y1
Assumptions	None
Clock Cycles	5
Example	`// Deselect the data EEPROM` `Sys_Dual_EEPROM_CS_Data_EEPROM`

15.46 SYS_DUAL_EEPROM_CSS_HIGH

Set chip select high on both EEPROMs

Type	Macro
Syntax	`#include <sk3_dual_eeprom.cfi>` `Sys_Dual_EEPROM_CSs_High`
Source File	sk3_dual_eeprom.cfi
Description	Set chip select high on both EEPROMs.
Inputs	None
Outputs	None
Register Modifications	y1
Assumptions	SPI port enabled.
Clock Cycles	4
Example	`// Deselect both EEPROMs` `Sys_Dual_EEPROM_CSs_High`

15.47 SYS_SET_DUAL_EEPROM_SPI_CTRL

Clears the bit SPI_CTRL_CS_N_POS in D_SPI_CTRL if SYSVAR_EE_SELECT = EE_SELECT_CODE_EEPROM

Type	Macro
Syntax	`#include <sk3_dual_eeprom.cfi>` `Sys_Set_Dual_EEPROM_SPI_CTRL( setting )`
Source File	sk3_dual_eeprom.cfi
Description	Clears the bit SPI_CTRL_CS_N_POS in D_SPI_CTRL if SYSVAR_EE_SELECT = EE_SELECT_CODE_EEPROM. Sets this bit high if SYSVAR_EE_SELECT = EE_SELECT_DATA_EEPROM. Sets other bits in D_SPI_CTRL according to input.
Inputs	setting = Value to write to D_SPI_CTRL
Outputs	None
Register Modifications	y0, y1
Assumptions	EE_SELECT_CODE_EEPROM defined as 0
Clock Cycles	5
Example	`// Read a 16-bit word from the SPI interface from the selected EEPROM` `Sys_Set_Dual_EEPROM_SPI_CTRL(SPI_START | SPI_WRITE | (16-1))`

15.48 SYS_SETREG_DUAL_EEPROM_SEL_EEPROM

Set the selected EEPROM to value in reg

Type	Macro
Syntax	`#include <sk3_dual_eeprom.cfi>` `Sys_SetReg_Dual_EEPROM_Sel_EEPROM( reg )`
Source File	sk3_dual_eeprom.cfi
Description	Set the selected EEPROM to value in reg.
Inputs	reg = Register containing selected EEPROM type : Use EE_SELECT_CODE_EEPROM or EE_SELECT_DATA_EEPROM
Outputs	None
Register Modifications	None
Assumptions	None
Clock Cycles	1

Example	`// Setup the data EEPROM as the current EEPROM`
	`load y1, EE_SELECT_DATA_EEPROM`
	`Sys_SetReg_Dual_EEPROM_Sel_EEPROM(y1)`

15.49 SYS_SETREG_DUAL_EEPROM_SPI_CTRL

Clears the bit SPI_CTRL_CS_N_POS in D_SPI_CTRL if SYSVAR_EE_SELECT = EE_SELECT_CODE_EEPROM

Type	Macro
Syntax	`#include <sk3_dual_eeprom.cfi>` `Sys_SetReg_Dual_EEPROM_SPI_CTRL( reg )`
Source File	sk3_dual_eeprom.cfi
Description	Clears the bit SPI_CTRL_CS_N_POS in D_SPI_CTRL if SYSVAR_EE_SELECT = EE_SELECT_CODE_EEPROM. Sets this bit high if SYSVAR_EE_SELECT = EE_SELECT_DATA_EEPROM. Sets other bits in D_SPI_CTRL according to input.
Inputs	setting = Value to write to D_SPI_CTRL
Outputs	None
Register Modifications	y0, y1
Assumptions	EE_SELECT_CODE_EEPROM defined as 0
Clock Cycles	5

Example	`// Write a 16-bit word over the SPI interface`
	`load y1, (SPI_START | SPI_WRITE | (16-1))`
	`Sys_SetReg_Dual_EEPROM_SPI_CTRL(y1)`

Global Headquarters
ON Semiconductor
5005 East McDowell Road
Phoenix, Arizona 85008 USA
Tel: +1 602 244 6600

Website
www.onsemi.com

Copyright © 2011 Semiconductor Components Industries, LLC d/b/a ON Semiconductor. ON Semiconductor and the ON logo are registered trademarks of Semiconductor Components Industries, LLC (ON Semiconductor). BELASIGNA is a registered trademark of ON Semiconductor. All other brands, product names and company names mentioned herein may be trademarks or registered trademarks of their respective holders.